Diagnostic Manager

Technical Reference

MICROSAR Adaptive

Version master-25109

Authors	mschmidbauer, visaey, visbft, visbsz, vishrs, vislpp, vismhe, vissa, visstk, vistia, vistrh, viswsi, adey, vsarcmual
Status	Released

Document Information

History

The change history of this file is part of the component change history. It is documented in `ChangeHistory.txt` of the component folder.

Reference Documents

[1] ASAM. Service-Oriented Vehicle Diagnostics. Version 1.0.0.
[2] AUTOSAR. Adaptive Platform Release Overview. R20-11.
[3] AUTOSAR. Diagnostic Extract Template. 4.3.1.
[4] AUTOSAR. Specification of Diagnostics for Adaptive Platform. R19-03.
[5] AUTOSAR. Specification of Diagnostics for Adaptive Platform. R20-11.
[6] AUTOSAR. Specification of Diagnostics for Adaptive Platform. R21-11.
[7] AUTOSAR. Specification of Diagnostics for Adaptive Platform. R23-11.
[8] AUTOSAR. Specification of Execution Management for Adaptive Platform. R22-11.
[9] AUTOSAR. Specification of Intrusion Detection System Manager for Adaptive Platform. R22-11.
[10] AUTOSAR. Specification of Manifest. R20-11.
[11] AUTOSAR. Specification of Software Component Template. R20-11.
[12] Vector Informatik GmbH. Diagnostic Configuration User Manual.
[13] Vector Informatik GmbH. MICROSAR Adaptive User Manual.
[14] Vector Informatik GmbH. Technical Reference Adaptive Log and Trace Library.
[15] Vector Informatik GmbH. Technical Reference Application Base.
[16] Vector Informatik GmbH. Technical Reference Basic Communication Protocol.
[17] Vector Informatik GmbH. Technical Reference CRC.
[18] Vector Informatik GmbH. Technical Reference Crypto Stack.
[19] Vector Informatik GmbH. Technical Reference E2E Library.
[20] Vector Informatik GmbH. Technical Reference Execution Management.
[21] Vector Informatik GmbH. Technical Reference HttpCommon.
[22] Vector Informatik GmbH. Technical Reference HttpServer.
[23] Vector Informatik GmbH. Technical Reference Intrusion Detection System Manager.
[24] Vector Informatik GmbH. Technical Reference Modelled Data Type Generator.
[25] Vector Informatik GmbH. Technical Reference Msr4Base.
[26] Vector Informatik GmbH. Technical Reference OS Abstraction Library.
[27] Vector Informatik GmbH. Technical Reference Pdu-Header Transport Protocol.
[28] Vector Informatik GmbH. Technical Reference Persistency.
[29] Vector Informatik GmbH. Technical Reference Secure Communication.
[30] Vector Informatik GmbH. Technical Reference Vector Adaptive ASN1 Library.
[31] Vector Informatik GmbH. Technical Reference Vector Adaptive Common Library.

[32] Vector Informatik GmbH. Technical Reference Vector Adaptive Json Library.

[33] Vector Informatik GmbH. Technical Reference Vector Characterconversion.

[34] Vector Informatik GmbH. Technical Reference Vector Input/Output Stream Library.

[35] Vector Informatik GmbH. Technical Reference Vector Integrity Stream Library.

[36] Vector Informatik GmbH. Technical Reference Vector Thread Library.

[37] Ifconfig (io-sock). `https://www.qnx.com/developers/docs/7.1/index.html#com.qnx.doc.neutrino.io_sock/topic/ifconfig.html`.

[38] International Standardization Organization. ISO 13400-2:2012 Road vehicles – Diagnostic communication over Internet Protocol (DoIP) – Part 2: Transport protocol and network layer services.

[39] International Standardization Organization. ISO 13400-2:2019 Road vehicles – Diagnostic communication over Internet Protocol (DoIP) – Part 2: Transport protocol and network layer services.

[40] International Standardization Organization. ISO 14229-1: Road Vehicles – Unified Diagnostic Services (UDS) – Part 1: Specification and Requirements.

Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Introduction ... 56
1.1 Architectural Overview ... 56
1.1.1 Dependencies of the DM 56
1.1.2 Design of the DM 57
1.1.2.1 Interaction with the DM 58
1.1.2.2 Scope of Diagnostics: Software Cluster 58

2 Functional Description ... 59
2.1 Features ... 60
2.1.1 Transport Protocol 60
2.1.1.1 Transport Protocol Initialization 60
2.1.1.2 Start and Stop of a Transport Protocol handler 60
2.1.1.3 DoIP Transport Protocol 62
2.1.1.3.1 ISO Deviations 62
2.1.1.3.2 Socket Handling 63
2.1.1.3.3 Multiple Networks 64
2.1.1.3.4 Activation Line 64
2.1.1.3.5 DHCP 64
2.1.1.3.6 TLS 64
2.1.1.3.7 Alive Check 64
2.1.1.3.8 DoIP Configuration 65
2.1.1.3.9 Message Validation 73
2.1.1.3.10 Vehicle Identification 73
2.1.1.3.11 Node Information 73
2.1.1.3.12 Routing Activation 74
2.1.1.4 Custom Transport Protocol 74
2.1.1.5 Dispatching of UDS Requests 74
2.1.2 Diagnostic Server 74
2.1.3 Parallel Diagnostic Conversations 74
2.1.4 UDS Message Processing 75
2.1.4.1 Request Validation 75
2.1.4.1.1 ResponseOnAllRequestSids 75
2.1.4.1.2 Mapping of NRC codes and their names 75
2.1.4.2 Request Validation on Daemon Side 75
2.1.4.3 Response Behavior 75
2.1.4.4 Service Preconditions 75
2.1.4.4.1 Session and Security Dependency 76
2.1.4.4.2 Data Restrictions 77

based on template version 1.00.00

2.1.4.4.3 Authentication Restrictions 80
2.1.4.4.4 Manufacturer / Supplier Validation 82
2.1.4.5 Service Processing Notification 83
2.1.5 UDS Service Processing 83
2.1.5.1 Session Management 84
2.1.5.1.1 Diagnostic Session Control (0x10) 84
2.1.5.1.2 Tester Present (0x3E) 84
2.1.5.1.3 Session change through ECU reset cycles 85
2.1.5.2 ECU Reset (0x11) 85
2.1.5.2.1 Rejection of new requests during hard reset 86
2.1.5.2.2 Connections Stop 86
2.1.5.3 Fault Memory Services 86
2.1.5.3.1 ClearDiagnosticInformation (0x14) 86
2.1.5.3.2 Configuration 86
2.1.5.3.3 ReadDTCInformation (0x19) 86
2.1.5.3.4 ControlDTCSetting (0x85) 87
2.1.5.4 Data Identifier Access 87
2.1.5.4.1 ReadDataByIdentifier (0x22) 87
2.1.5.4.2 WriteDataByIdentifier (0x2E) 87
2.1.5.4.3 ReadDataByPeriodicIdentifier (0x2A) 87
2.1.5.4.4 DynamicallyDefineDataIdentifier (0x2C) 89
2.1.5.5 SecurityAccess (0x27) 89
2.1.5.5.1 Usage of brute-force-attack prevention 90
2.1.5.6 CommunicationControl (0x28) 90
2.1.5.7 Authentication (0x29) 90
2.1.5.8 RoutineControl (0x31) 91
2.1.5.9 Data Transfer Services 92
2.1.5.9.1 RequestDownload (0x34) 92
2.1.5.9.2 RequestUpload (0x35) 92
2.1.5.9.3 TransferData (0x36) 92
2.1.5.9.4 RequestTransferExit (0x37) 93
2.1.5.9.5 RequestFileTransfer (0x38) 94
2.1.5.9.6 Pipelined Data Transfer 94
2.1.5.10 SecuredDataTransmission (0x84) 96
2.1.5.10.1 Service Precondition Checks 98
2.1.5.10.2 Plug-In 98
2.1.5.11 ResponseOnEvent (0x86) 99
2.1.5.11.1 Diagnostic Session Change 100
2.1.5.12 Custom Service 101
2.1.5.13 Service handling through the generic UDS interface 102
2.1.5.14 Custom UDS NRC 102
2.1.5.15 UDS NRC Mapping 103

based on template version 1.00.00

2.1.6 SOVD Request Validation and Verification 103
2.1.6.1 SOVD Authorization 103
2.1.6.1.1 Token Validation 104
2.1.6.1.2 Token Revokation 105
2.1.6.1.3 Authorization failures 105
2.1.7 SOVD Request Processing 107
2.1.8 Diagnostic Data Management 107
2.1.8.1 Internal Diagnostic Data 107
2.1.8.2 External Diagnostic Data 108
2.1.9 Fault Memory 108
2.1.9.1 DTC Status Processing 108
2.1.9.2 File Handling 108
2.1.10 Diagnostic Event 109
2.1.10.1 Combined Events 109
2.1.10.2 Monitors 109
2.1.10.3 Event Notification 109
2.1.10.4 Event Debouncing 110
2.1.11 Operation Cycles 110
2.1.12 DTC Confirmation 110
2.1.13 Indicators 110
2.1.14 Snapshot Data Records 110
2.1.14.1 Usage of Snapshot Records 110
2.1.14.2 Configuration of Snapshot Records 111
2.1.15 Extended Data Records 112
2.1.15.1 Usage of Extended Data Records 112
2.1.15.2 Configuration of Extended Data Records 112
2.1.16 Clearing DTCs 113
2.1.16.1 Clear Conditions 114
2.1.17 DTC Aging 114
2.1.18 Enable Conditions 114
2.1.18.1 Initial State 114
2.1.19 Storage Conditions 115
2.1.20 Client Authentication (Authorization) 115
2.1.21 Security Event Log 116
2.1.21.1 Security Event Type 116
2.1.21.2 Security Event 116
2.1.22 Daemon Alive Check 116
2.1.23 Log Diaglibrary-Daemon interaction 116
2.1.24 Root Cause Event Associated Identification 118
2.1.25 Failure Event Associated Identification 118
2.1.26 Caching of application call 118
2.1.26.1 Activation Line State 119
2.1.26.2 Condition 119

based on template version 1.00.00

2.1.26.3 Conversation 119
2.1.26.3.1 Reset to default session 119
2.1.26.3.2 Overriding S3 timeout 119
2.1.26.4 DTC Information 119
2.1.26.5 Monitor Action 119
2.1.26.6 Operation Cycle 120
2.1.26.7 Security Event Log 120
2.1.26.7.1 Report Security Event 120
2.1.26.7.2 Add raw buffer entry to Security Event Types 120
2.1.27 Diagnostic RPC Usage Intensity Measurement 120
2.2 Deviations 122
2.3 Additions/Extensions 123
2.4 Limitations 124
2.4.1 DTC status processing 124
2.4.2 Event Debouncing 124
2.4.2.1 Counter-based debouncing 124
2.4.3 DTC confirmation 124
2.4.4 Indicators 124
2.4.5 DoIP 124
2.4.6 Single process instance 124
2.4.7 Endianness 124
2.4.8 Authentication 125
2.4.9 Security Event Log 125
2.4.10 Storage of Environmental Data 125
2.4.11 Limitations on QNX with an io-sock network stack 125
2.5 MetaInfo 127
2.6 States 128
2.6.1 Diagnostics Daemon 128
2.6.1.0.1 Not initialized 128
2.6.1.0.2 Initializing other stack components 128
2.6.1.0.3 Running 128
2.6.1.0.4 Shutting down 128
2.6.2 Diagnostics Library (ara::diag) 128
2.6.2.0.1 Not initialized 128
2.6.2.0.2 Initializing other stack components 128
2.6.2.0.3 Initialized 128
2.6.2.0.4 Disconnected 128
2.6.2.0.5 De-initialized 129
2.6.2.0.6 Observers 129
2.7 Handling of POSIX Signals 130

2.8 Error Handling ... 131
2.8.1 Exit Codes 131
2.8.2 Abnormal Terminations 131
2.8.2.1 Diagnostics Daemon 131
2.8.2.2 Applications using Diagnostics Library (ara::diag) 131
2.9 Thread Handling ... 132
2.9.1 Diagnostics Daemon 132
2.9.2 Diagnostics Library (ara::diag) 134
2.10 Steady Mode ... 136
2.10.1 Memory Demand 136
2.10.1.1 Measuring the memory demand 136
2.10.1.2 Steady and Non-Steady Functionality 136
2.10.2 Diagnostics Library (ara::diag) 136
2.10.2.1 AraDiag Memory Resource 137
2.10.2.2 RPC Memory Resource (Application Side) 137
2.10.3 Diagnostics Daemon 137

3 Integration ... 138
3.1 External Build Dependencies ... 138
3.2 Integrating Diagnostics Library (ara::diag) into an Application ... 138
3.2.1 Initialization and Deinitialization 138
3.2.2 Registering and Unregistering interaction with Diagnostic Ports 138
3.2.2.1 Provided Ports 138
3.2.2.2 Providing Notifiers to Diagnostics Library (ara::diag) 139
3.2.2.3 CancellationHandler 139
3.3 Diagnostic Manager Daemon Model ... 139
3.3.1 Integration of Transport Protocol 140
3.3.1.1 Integration of a Transport Protocol from the Vector Stack 141
3.3.1.2 Integration of use of TLS on channels 141
3.3.1.3 Integration of a Custom Transport Protocol 142
3.3.2 Integration of SecureDataTransmission PlugIn 143
3.3.2.1 Integration of a SecureDataTransmission PlugIn from the Vector Stack 144
3.3.2.2 Integration of a Custom SecureDataTransmission PlugIn 144
3.4 Runtime Dependencies ... 146
3.5 Files and Include Structure ... 146
3.6 CMake ... 147
3.6.1 Exported CMake targets 147
3.6.2 CMake Options 155
3.7 Installation ... 157

based on template version 1.00.00

3.8 Execution ... 157
3.8.1 Directory Structure of the Working Directory 157
3.8.2 Start-Up behavior 157
3.8.3 Measuring execution runtime 157
3.8.3.1 Watch Dog 157
3.8.3.2 Stop Watch 157
3.9 Application IDs ... 159
3.10 Context IDs ... 159

4 Diagnostic Library API Description ... 160
4.1 amsr::diag_utility::EventLoop ... 161
4.1.1 amsr::diag_utility::EventLoop::RegisterPeriodicTask 161
4.1.2 amsr::diag_utility::EventLoop::RegisterEvent 162
4.1.2.1 Register an event using specific name 162
4.1.2.2 Register an event 162
4.1.3 amsr::diag_utility::EventLoop::RaiseEvent 163
4.1.4 amsr::diag_utility::EventLoop::Unregister 163
4.1.5 amsr::diag_utility::EventLoop::DeferTask 164
4.1.6 amsr::diag_utility::EventLoop::Handle 164
4.1.6.1 amsr::diag_utility::EventLoop::Handle::Handle 165
4.1.6.2 amsr::diag_utility::EventLoop::Handle::~Handle 165
4.1.6.3 amsr::diag_utility::EventLoop::Handle::operator= 166
4.1.6.4 amsr::diag_utility::EventLoop::Handle::Unregister 166
4.1.7 amsr::diag_utility::EventLoop::Buzzer 167
4.1.7.1 amsr::diag_utility::EventLoop::Buzzer::Buzzer 167
4.1.7.2 amsr::diag_utility::EventLoop::Buzzer::~Buzzer 167
4.1.7.3 amsr::diag_utility::EventLoop::Buzzer::operator= 168
4.1.7.4 amsr::diag_utility::EventLoop::Buzzer::Unregister 168
4.1.7.5 amsr::diag_utility::EventLoop::Buzzer::RaiseEvent 169
4.2 amsr::diag_utility::RootSwc ... 170
4.2.1 amsr::diag_utility::RootSwc::GetAccess 170
4.2.2 amsr::diag_utility::RootSwc::GetEventLoop 170
4.3 amsr::diag::DiagUdsNrc ... 172
4.4 amsr::diag::NegativeResponseReason ... 174
4.5 amsr::diag::r21_11::DiagErrorDomain ... 175
4.5.1 amsr::diag::r21_11::DiagErrorDomain::DiagErrorDomain 175
4.6 amsr::diag::r21_11::DiagOfferErrorDomain ... 176
4.6.1 amsr::diag::r21_11::DiagOfferErrorDomain::DiagOfferErrorDomain 176
4.7 amsr::diag::r21_11::DiagReportingErrorDomain ... 177
4.7.1 amsr::diag::r21_11::DiagReportingErrorDomain::DiagReportingErrorDomain 177

based on template version 1.00.00

4.8 amsr::diag::r21_11::Monitor 178
4.8.1 amsr::diag::r21_11::Monitor::Monitor 178
4.8.2 amsr::diag::r21_11::Monitor::Monitor 179
4.8.3 amsr::diag::r21_11::Monitor::Monitor 179
4.8.4 amsr::diag::r21_11::Monitor::Monitor 180
4.8.5 amsr::diag::r21_11::Monitor::operator= 181
4.8.6 amsr::diag::r21_11::Monitor::InitMonitorReason 181
4.8.7 amsr::diag::r21_11::Monitor::CounterBased 182
4.8.8 amsr::diag::r21_11::Monitor::TimeBased 182
4.8.9 amsr::diag::r21_11::Monitor::MonitorAction 182
4.8.10 amsr::diag::r21_11::Monitor::IsValidInstanceSpecifier 183
4.8.11 amsr::diag::r21_11::Monitor::Offer 184
4.8.12 amsr::diag::r21_11::Monitor::ReportMonitorAction 184
4.8.13 amsr::diag::r21_11::Monitor::StopOffer 185
4.9 amsr::diag::r21_11::OperationCycle 186
4.9.1 amsr::diag::r21_11::OperationCycle::OperationCycle 186
4.9.2 amsr::diag::r21_11::OperationCycle::OperationCycle 186
4.9.3 amsr::diag::r21_11::OperationCycle::~OperationCycle 187
4.9.4 amsr::diag::r21_11::OperationCycle::operator= 187
4.9.5 amsr::diag::r21_11::OperationCycle::RestartOperationCycle 188
4.9.6 amsr::diag::r21_11::OperationCycle::SetNotifier 188
4.10 amsr::diag::ReleaseHandler 190
4.10.1 amsr::diag::ReleaseHandler::ReleaseHandler 190
4.10.2 amsr::diag::ReleaseHandler::operator= 190
4.10.3 amsr::diag::ReleaseHandler::MayRelease 191
4.10.4 amsr::diag::ReleaseHandler::SetNotifier 192
4.11 amsr::diag::SecurityEvent 193
4.11.1 amsr::diag::SecurityEvent::SecurityEvent 193
4.11.2 amsr::diag::SecurityEvent::SecurityEvent 193
4.11.3 amsr::diag::SecurityEvent::~SecurityEvent 194
4.11.4 amsr::diag::SecurityEvent::operator= 194
4.11.5 amsr::diag::SecurityEvent::Report 195
4.12 amsr::diag::SecurityEventLog 196
4.12.1 amsr::diag::SecurityEventLog::SecurityEventLog 196
4.12.2 amsr::diag::SecurityEventLog::SecurityEventLog 197
4.12.3 amsr::diag::SecurityEventLog::~SecurityEventLog 197
4.12.4 amsr::diag::SecurityEventLog::operator= 197
4.12.5 amsr::diag::SecurityEventLog::Event 198
4.12.5.1 amsr::diag::SecurityEventLog::Event::Event 198
4.12.5.2 amsr::diag::SecurityEventLog::Event::Event 199
4.12.5.3 amsr::diag::SecurityEventLog::Event::~Event 199

4.12.5.4 amsr::diag::SecurityEventLog::Event::operator= 200
4.12.5.5 amsr::diag::SecurityEventLog::Event::operator= 200
4.12.5.6 amsr::diag::SecurityEventLog::Event::GetCounter 201
4.12.5.7 amsr::diag::SecurityEventLog::Event::GetId 201
4.12.6 amsr::diag::SecurityEventLog::EventType 202
4.12.6.1 amsr::diag::SecurityEventLog::EventType::EventType 202
4.12.6.2 amsr::diag::SecurityEventLog::EventType::~EventType 203
4.12.6.3 amsr::diag::SecurityEventLog::EventType::operator= 203
4.12.6.4 amsr::diag::SecurityEventLog::EventType::GetBuffer 204
4.12.6.5 amsr::diag::SecurityEventLog::EventType::GetEvent 204
4.12.6.6 amsr::diag::SecurityEventLog::EventType::GetEvents 205
4.12.6.7 amsr::diag::SecurityEventLog::EventType::GetId 205
4.12.7 amsr::diag::SecurityEventLog::AddRawEntryToAll 206
4.12.8 amsr::diag::SecurityEventLog::GetEventLog 206
4.12.9 amsr::diag::SecurityEventLog::GetEventType 207
4.12.10 amsr::diag::SecurityEventLog::Report 207
4.12.11 amsr::diag::SecurityEventLog::SetNotifierNumberOfBufferEntriesChanged 208
4.12.12 amsr::diag::SecurityEventLog::SetNotifierSecurityEvent 209
4.13 amsr::diag::ServiceProcessingNotification 211
4.13.1 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification 211
4.13.2 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification 212
4.13.3 amsr::diag::ServiceProcessingNotification::~ServiceProcessingNotification 212
4.13.4 amsr::diag::ServiceProcessingNotification::operator= 212
4.13.5 amsr::diag::ServiceProcessingNotification::ConfirmationStatusType 213
4.13.6 amsr::diag::ServiceProcessingNotification::NegativeResponseDetails 213
4.13.7 amsr::diag::NegativeResponseDetails::GetNegativeResponseReason 214
4.13.8 amsr::diag::NegativeResponseDetails::GetRawNrc 214
4.13.9 amsr::diag::NegativeResponseDetails::GetUdsNrc 215
4.13.10 amsr::diag::ServiceProcessingNotification::Offer 215
4.13.11 amsr::diag::ServiceProcessingNotification::ProcessingEnd 216
4.13.12 amsr::diag::ServiceProcessingNotification::StopOffer 217
4.14 amsr::diag::EventStatusByte 218
4.14.1 amsr::diag::EventStatusByte::IsFailedAndTested 218
4.14.2 amsr::diag::EventStatusByte::IsNotSet 218
4.14.3 amsr::diag::EventStatusByte::IsNotSet 219
4.14.4 amsr::diag::EventStatusByte::IsPassedAndTested 220
4.14.5 amsr::diag::EventStatusByte::IsSet 220
4.14.6 amsr::diag::EventStatusByte::IsSet 221
4.15 amsr::diag::UdsDTCStatusByteType 222
4.15.1 amsr::diag::UdsDTCStatusByteType::operator== 222
4.15.2 amsr::diag::UdsDTCStatusByteType::IsFailedAndTested 222
4.15.3 amsr::diag::UdsDTCStatusByteType::IsNotSet 223

4.15.4 amsr::diag::UdsDTCStatusByteType::IsNotSet 223
4.15.5 amsr::diag::UdsDTCStatusByteType::IsPassedAndTested 224
4.15.6 amsr::diag::UdsDTCStatusByteType::IsSet 225
4.15.7 amsr::diag::UdsDTCStatusByteType::IsSet 225
4.16 amsr::diag::DataTransfer 227
4.16.1 amsr::diag::DataTransferReadByPullHandler 227
4.16.2 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler 227
4.16.3 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler 227
4.16.4 amsr::diag::DataTransferReadByPullHandler::~DataTransferReadByPullHandler 228
4.16.5 amsr::diag::DataTransferReadByPullHandler::operator= 228
4.16.6 amsr::diag::DataTransferReadByPullHandler::ExitRead 229
4.16.7 amsr::diag::DataTransferReadByPullHandler::Read 230
4.16.8 amsr::diag::DataTransferReadByPushHandler 231
4.16.9 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler 231
4.16.10 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler 232
4.16.11 amsr::diag::DataTransferReadByPushHandler::~DataTransferReadByPushHandler 232
4.16.12 amsr::diag::DataTransferReadByPushHandler::operator= 233
4.16.13 amsr::diag::DataTransferReadByPushHandler::ExitRead 233
4.16.14 amsr::diag::DataTransferReadByPushHandler::Read 234
4.16.15 amsr::diag::DataTransferReadSession 236
4.16.16 amsr::diag::DataTransferReadSession::DataTransferReadSession 236
4.16.17 amsr::diag::DataTransferReadSession::DataTransferReadSession 236
4.16.18 amsr::diag::DataTransferReadSession::DataTransferReadSession 237
4.16.19 amsr::diag::DataTransferReadSession::DataTransferReadSession 238
4.16.20 amsr::diag::DataTransferReadSession::~DataTransferReadSession 238
4.16.21 amsr::diag::DataTransferReadSession::operator= 239
4.16.22 amsr::diag::DataTransferReadSharedDataHandler 239
4.16.23 amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler 240
4.16.24 amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler 240
4.16.25 amsr::diag::DataTransferReadSharedDataHandler::~DataTransferReadSharedDataHandler 241
4.16.26 amsr::diag::DataTransferReadSharedDataHandler::operator= 241
4.16.27 amsr::diag::DataTransferReadSharedDataHandler::ExitRead 242
4.16.28 amsr::diag::DataTransferReadSharedDataHandler::Read 242
4.16.29 amsr::diag::DataTransferWriteHandler 244
4.16.30 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler 244
4.16.31 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler 244
4.16.32 amsr::diag::DataTransferWriteHandler::~DataTransferWriteHandler 245
4.16.33 amsr::diag::DataTransferWriteHandler::operator= 245

4.16.34 amsr::diag::DataTransferWriteHandler::ExitWrite 246
4.16.35 amsr::diag::DataTransferWriteHandler::Write 247
4.16.36 amsr::diag::DataTransferWriteSession 249
4.16.37 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 249
4.16.38 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 249
4.16.39 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 250
4.16.40 amsr::diag::DataTransferWriteSession::~DataTransferWriteSession 250
4.16.41 amsr::diag::DataTransferWriteSession::operator= 251

4.17 amsr::diag::FileTransferService 252
4.17.1 amsr::diag::FileTransferService::FileTransferService 252
4.17.2 amsr::diag::FileTransferService::FileTransferService 253
4.17.3 amsr::diag::FileTransferService::~FileTransferService 253
4.17.4 amsr::diag::FileTransferService::operator= 254
4.17.5 amsr::diag::FileTransferService::DeleteFile 254
4.17.6 amsr::diag::FileTransferService::Offer 255
4.17.7 amsr::diag::FileTransferService::RequestReadDirectory 255
4.17.8 amsr::diag::FileTransferService::RequestReadFile 256
4.17.9 amsr::diag::FileTransferService::RequestResumeWriteFile 257
4.17.10 amsr::diag::FileTransferService::RequestWriteFile 258
4.17.11 amsr::diag::FileTransferService::StopOffer 259

4.18 amsr::diag::DownloadService 261
4.18.1 amsr::diag::DownloadService::DownloadService 261
4.18.2 amsr::diag::DownloadService::DownloadService 261
4.18.3 amsr::diag::DownloadService::~DownloadService 262
4.18.4 amsr::diag::DownloadService::operator= 262
4.18.5 amsr::diag::DownloadService::DownloadData 263
4.18.6 amsr::diag::DownloadService::Offer 264
4.18.7 amsr::diag::DownloadService::RequestDownload 264
4.18.8 amsr::diag::DownloadService::RequestDownloadExit 265
4.18.9 amsr::diag::DownloadService::StopOffer 266

4.19 amsr::diag::SovdAuthorization 268
4.19.1 amsr::diag::SovdAuthorization::~SovdAuthorization 268
4.19.2 amsr::diag::SovdAuthorization::Offer 268
4.19.3 amsr::diag::SovdAuthorization::SovdAuthorization 269
4.19.4 amsr::diag::SovdAuthorization::SovdAuthorization 269
4.19.5 amsr::diag::SovdAuthorization::StopOffer 270
4.19.6 amsr::diag::SovdAuthorization::ValidateAuthorization 271
4.19.7 amsr::diag::SovdAuthorization::operator= 271

4.20 amsr::diag::SovdTokenAuthorization 273
4.20.1 amsr::diag::SovdTokenAuthorization::Authorize 273

based on template version 1.00.00

4.21 amsr::diag::SovdTokenAuthorizationHandle 274
4.21.1 amsr::diag::SovdTokenAuthorizationHandle::~SovdTokenAuthorizationHandle 274
4.21.2 amsr::diag::SovdTokenAuthorizationHandle::Revoke 274
4.21.3 amsr::diag::SovdTokenAuthorizationHandle::SetNotifier 275
4.21.4 amsr::diag::SovdTokenAuthorizationHandle::SovdTokenAuthorizationHandle 276
4.21.5 amsr::diag::SovdTokenAuthorizationHandle::operator= 276
4.22 amsr::diag::SovdLock 278
4.22.1 amsr::diag::SovdLock::~SovdLock 278
4.22.2 amsr::diag::SovdLock::GetAllLocks 278
4.22.3 amsr::diag::SovdLock::GetExpirationTime 279
4.22.4 amsr::diag::SovdLock::SetSovdLockStateHandler 279
4.22.5 amsr::diag::SovdLock::SovdLock 280
4.22.6 amsr::diag::SovdLock::SovdLock 281
4.22.7 amsr::diag::SovdLock::operator= 281
4.23 ara::diag::Authentication 282
4.23.1 ara::diag::Authentication::Authentication 282
4.23.2 ara::diag::Authentication::Authentication 283
4.23.3 ara::diag::Authentication::~Authentication 283
4.23.4 ara::diag::Authentication::operator= 284
4.23.5 ara::diag::Authentication::Offer 284
4.23.6 ara::diag::Authentication::StopOffer 285
4.23.7 ara::diag::Authentication::TransmitCertificate 285
4.23.8 ara::diag::Authentication::VerifyCertificateBidirectional 286
4.23.9 ara::diag::Authentication::VerifyCertificateUnidirectional 287
4.23.10 ara::diag::Authentication::VerifyOwnership 288
4.24 ara::diag::CancellationHandler 290
4.24.1 ara::diag::CancellationHandler::CancellationHandler 290
4.24.2 ara::diag::CancellationHandler::operator= 290
4.24.3 ara::diag::CancellationHandler::IsCanceled 291
4.24.4 ara::diag::CancellationHandler::SetNotifier 292
4.25 ara::diag::ClientAuthenticationHandle 293
4.25.1 ara::diag::ClientAuthenticationHandle::ClientAuthenticationHandle 293
4.25.2 ara::diag::ClientAuthenticationHandle::~ClientAuthenticationHandle 293
4.25.3 ara::diag::ClientAuthenticationHandle::operator= 294
4.25.4 ara::diag::ClientAuthenticationHandle::Append 294
4.25.5 ara::diag::ClientAuthenticationHandle::Refresh 295
4.25.6 ara::diag::ClientAuthenticationHandle::Revoke 296
4.25.7 ara::diag::ClientAuthenticationHandle::Set 296
4.26 ara::diag::ClientAuthentication 298
4.26.1 ara::diag::ClientAuthentication::ClientAuthentication 298
4.26.2 ara::diag::ClientAuthentication::~ClientAuthentication 299

4.26.3 ara::diag::ClientAuthentication::operator= 299
4.26.4 ara::diag::ClientAuthentication::DiagnosticAuthState 299
4.26.5 ara::diag::ClientAuthentication::DiagnosticAuthRole 300
4.26.6 ara::diag::ClientAuthentication::Authenticate 300
4.26.7 ara::diag::ClientAuthentication::GetState 301
4.26.8 ara::diag::ClientAuthentication::OverrideDefaultRoles 301
4.26.9 ara::diag::ClientAuthentication::SetNotifier 302

4.27 ara::diag::Condition 303
4.27.1 ara::diag::Condition::Condition 303
4.27.2 ara::diag::Condition::Condition 303
4.27.3 ara::diag::Condition::~Condition 304
4.27.4 ara::diag::Condition::operator= 304
4.27.5 ara::diag::Condition::ConditionType 305
4.27.6 ara::diag::Condition::GetCondition 305
4.27.7 ara::diag::Condition::SetCondition 306
4.27.8 ara::diag::Condition::SetNotifier 306

4.28 ara::diag::Conversation 308
4.28.1 ara::diag::Conversation::ActivityStatusType 308
4.28.2 ara::diag::Conversation::ConversationIdentifierType 309
4.28.3 ara::diag::Conversation::GetActivityStatus 309
4.28.4 ara::diag::Conversation::GetAllConversations 309
4.28.5 ara::diag::Conversation::GetConversation 310
4.28.6 ara::diag::Conversation::GetConversationIdentifier 311
4.28.7 ara::diag::Conversation::GetCurrentActiveConversations 311
4.28.8 ara::diag::Conversation::GetDiagnosticSecurityLevel 312
4.28.9 ara::diag::Conversation::GetDiagnosticSession 312
4.28.10 ara::diag::Conversation::ResetToDefaultSession 313
4.28.11 ara::diag::Conversation::SetActivityNotifier 313
4.28.12 ara::diag::Conversation::SetDiagnosticSessionNotifier 314
4.28.13 ara::diag::Conversation::SetSecurityLevelNotifier 315

4.29 ara::diag::DiagUdsNrcErrorDomain 317
4.29.1 ara::diag::DiagUdsNrcErrorDomain::DiagUdsNrcErrorDomain 317

4.30 ara::diag::DiagUdsNrcErrc 318

4.31 ara::diag::DiagnosticServiceDynamicAccessList 320
4.31.1 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList 320
4.31.2 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList 321
4.31.3 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList 321
4.31.4 ara::diag::DiagnosticServiceDynamicAccessList::~DiagnosticServiceDynamicAccessList 322

4.31.5 ara::diag::DiagnosticServiceDynamicAccessList::operator= 322
4.31.6 ara::diag::DiagnosticServiceDynamicAccessList::operator= 323
4.31.7 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder 323
4.31.8 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder 324
4.31.9 ara::diag::DiagnosticServiceDynamicAccessList::Reserve 325

4.32 ara::diag::DoIPActivationLine 326
4.32.1 ara::diag::DoIPActivationLine::DoIPActivationLine 326
4.32.2 ara::diag::DoIPActivationLine::DoIPActivationLine 326
4.32.3 ara::diag::DoIPActivationLine::~DoIPActivationLine 327
4.32.4 ara::diag::DoIPActivationLine::operator= 327
4.32.5 ara::diag::DoIPActivationLine::GetActivationLineState 328
4.32.6 ara::diag::DoIPActivationLine::GetNetworkInterfaceId 328
4.32.7 ara::diag::DoIPActivationLine::Offer 329
4.32.8 ara::diag::DoIPActivationLine::StopOffer 329
4.32.9 ara::diag::DoIPActivationLine::UpdateActivationLineState 330

4.33 ara::diag::DoIPGroupIdentification 332
4.33.1 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification 332
4.33.2 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification 332
4.33.3 ara::diag::DoIPGroupIdentification::~DoIPGroupIdentification 333
4.33.4 ara::diag::DoIPGroupIdentification::operator= 333
4.33.5 ara::diag::DoIPGroupIdentification::GidStatus 334
4.33.6 ara::diag::DoIPGroupIdentification::GetGidStatus 334
4.33.7 ara::diag::DoIPGroupIdentification::Offer 335
4.33.8 ara::diag::DoIPGroupIdentification::StopOffer 335

4.34 ara::diag::DoIPPowerMode 337
4.34.1 ara::diag::DoIPPowerMode::DoIPPowerMode 337
4.34.2 ara::diag::DoIPPowerMode::DoIPPowerMode 337
4.34.3 ara::diag::DoIPPowerMode::~DoIPPowerMode 338
4.34.4 ara::diag::DoIPPowerMode::operator= 338
4.34.5 ara::diag::DoIPPowerMode::PowerModeType 339
4.34.6 ara::diag::DoIPPowerMode::GetDoIPPowerMode 339
4.34.7 ara::diag::DoIPPowerMode::Offer 340
4.34.8 ara::diag::DoIPPowerMode::StopOffer 340

4.35 ara::diag::DownloadService 342
4.35.1 ara::diag::DownloadService::DownloadService 342
4.35.2 ara::diag::DownloadService::DownloadService 343
4.35.3 ara::diag::DownloadService::~DownloadService 343
4.35.4 ara::diag::DownloadService::operator= 343
4.35.5 ara::diag::DownloadService::OperationOutput 344
4.35.6 ara::diag::DownloadService::DownloadData 344
4.35.7 ara::diag::DownloadService::Offer 345
4.35.8 ara::diag::DownloadService::RequestDownload 346

based on template version 1.00.00

4.35.9 ara::diag::DownloadService::RequestDownloadExit 346
4.35.10 ara::diag::DownloadService::StopOffer 347

4.36 ara::diag::DTCInformation 349
4.36.1 ara::diag::DTCInformation::DTCInformation 349
4.36.2 ara::diag::DTCInformation::DTCInformation 350
4.36.3 ara::diag::DTCInformation::~DTCInformation 350
4.36.4 ara::diag::DTCInformation::operator= 351
4.36.5 ara::diag::DTCInformation::ControlDtcStatusType 351
4.36.6 ara::diag::DTCInformation::UdsStatusBit 352
4.36.7 ara::diag::DTCInformation::SnapshotDataIdentiferType 352
4.36.8 ara::diag::DTCInformation::SnapshotDataRecordType 352
4.36.9 ara::diag::DTCInformation::SnapshotRecordUpdatedType 353
4.36.10 ara::diag::DTCInformation::Clear 353
4.36.11 ara::diag::DTCInformation::EnableControlDtc 354
4.36.12 ara::diag::DTCInformation::GetControlDTCStatus 354
4.36.13 ara::diag::DTCInformation::GetCurrentStatus 355
4.36.14 ara::diag::DTCInformation::GetNumberOfStoredEntries 355
4.36.15 ara::diag::DTCInformation::SetControlDtcStatusNotifier 356
4.36.16 ara::diag::DTCInformation::SetDTCStatusChangedNotifier 357
4.36.17 ara::diag::DTCInformation::SetNumberOfStoredEntriesNotifier 358
4.36.18 ara::diag::DTCInformation::SetSnapshotRecordUpdatedNotifier 358

4.37 ara::diag::DynamicAccessListDiagServiceBuilder 360
4.37.1 ara::diag::DynamicAccessListDiagServiceBuilder::
~DynamicAccessListDiagServiceBuilder 360
4.37.2 ara::diag::DynamicAccessListDiagServiceBuilder::ByteString 361
4.37.3 ara::diag::DynamicAccessListDiagServiceBuilder::Byte 361
4.37.4 ara::diag::DynamicAccessListDiagServiceBuilder::Add 361
4.37.5 ara::diag::DynamicAccessListDiagServiceBuilder::Add 362
4.37.6 ara::diag::DynamicAccessListDiagServiceBuilder::Add 362
4.37.7 ara::diag::DynamicAccessListDiagServiceBuilder::Any 363
4.37.8 ara::diag::DynamicAccessListDiagServiceBuilder::Build 363
4.37.9 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith 364
4.37.10 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith 365

4.38 ara::diag::Event 366
4.38.1 ara::diag::Event::Event 366
4.38.2 ara::diag::Event::Event 367
4.38.3 ara::diag::Event::~Event 367
4.38.4 ara::diag::Event::operator= 368
4.38.5 ara::diag::Event::GetDTCNumber 368
4.38.6 ara::diag::Event::DTCFormatType 369
4.38.7 ara::diag::Event::EventStatusBit 369
4.38.8 ara::diag::Event::DebouncingState 369

4.38.9 ara::diag::Event::GetDebouncingStatus 370
4.38.10 ara::diag::Event::GetEventStatus 370
4.38.11 ara::diag::Event::GetFaultDetectionCounter 371
4.38.12 ara::diag::Event::GetLatchedWIRStatus 372
4.38.13 ara::diag::Event::GetTestComplete 372
4.38.14 ara::diag::Event::SetEventStatusChangedNotifier 373
4.38.15 ara::diag::Event::SetLatchedWIRStatus 374

4.39 ara::diag::ExternalAuthentication 375
4.39.1 ara::diag::ExternalAuthentication::ExternalAuthentication 375
4.39.2 ara::diag::ExternalAuthentication::ExternalAuthentication 375
4.39.3 ara::diag::ExternalAuthentication::~ExternalAuthentication 376
4.39.4 ara::diag::ExternalAuthentication::operator= 376
4.39.5 ara::diag::ExternalAuthentication::Address 377
4.39.6 ara::diag::ExternalAuthentication::Get 377
4.39.7 ara::diag::ExternalAuthentication::Get 378
4.39.8 ara::diag::ExternalAuthentication::GetAll 378

4.40 ara::diag::GenericDataIdentifier 380
4.40.1 ara::diag::GenericDataIdentifier::GenericDataIdentifier 380
4.40.2 ara::diag::GenericDataIdentifier::GenericDataIdentifier 381
4.40.3 ara::diag::GenericDataIdentifier::~GenericDataIdentifier 381
4.40.4 ara::diag::GenericDataIdentifier::operator= 381
4.40.5 ara::diag::GenericDataIdentifier::OperationOutput 382
4.40.6 ara::diag::GenericDataIdentifier::Offer 382
4.40.7 ara::diag::GenericDataIdentifier::Read 383
4.40.8 ara::diag::GenericDataIdentifier::StopOffer 383
4.40.9 ara::diag::GenericDataIdentifier::Write 384

4.41 ara::diag::GenericRoutine 386
4.41.1 ara::diag::GenericRoutine::GenericRoutine 386
4.41.2 ara::diag::GenericRoutine::GenericRoutine 387
4.41.3 ara::diag::GenericRoutine::~GenericRoutine 387
4.41.4 ara::diag::GenericRoutine::operator= 387
4.41.5 ara::diag::GenericRoutine::OperationOutput 388
4.41.6 ara::diag::GenericRoutine::Offer 388
4.41.7 ara::diag::GenericRoutine::RequestResults 389
4.41.8 ara::diag::GenericRoutine::Start 390
4.41.9 ara::diag::GenericRoutine::Stop 391
4.41.10 ara::diag::GenericRoutine::StopOffer 392

4.42 ara::diag::GenericUDSService 393
4.42.1 ara::diag::GenericUDSService::GenericUDSService 393
4.42.2 ara::diag::GenericUDSService::GenericUDSService 393
4.42.3 ara::diag::GenericUDSService::~GenericUDSService 394
4.42.4 ara::diag::GenericUDSService::operator= 394

4.42.5 ara::diag::GenericUDSService::OperationOutput 395
4.42.6 ara::diag::GenericUDSService::HandleMessage 395
4.42.7 ara::diag::GenericUDSService::Offer 396
4.42.8 ara::diag::GenericUDSService::StopOffer 397

4.43 ara::diag::Indicator 398
4.43.1 ara::diag::Indicator::Indicator 398
4.43.2 ara::diag::Indicator::Indicator 398
4.43.3 ara::diag::Indicator::~Indicator 399
4.43.4 ara::diag::Indicator::operator= 399
4.43.5 ara::diag::Indicator::IndicatorType 400
4.43.6 ara::diag::Indicator::GetIndicator 400
4.43.7 ara::diag::Indicator::SetNotifier 401

4.44 ara::diag::MetaInfo 403
4.44.1 ara::diag::MetaInfo::MetaInfo 403
4.44.2 ara::diag::MetaInfo::~MetaInfo 403
4.44.3 ara::diag::MetaInfo::operator= 404
4.44.4 ara::diag::MetaInfo::GetValue 404

4.45 ara::diag::ReentrancyType 406

4.46 ara::diag::SecurityAccess 407
4.46.1 ara::diag::SecurityAccess::SecurityAccess 407
4.46.2 ara::diag::SecurityAccess::SecurityAccess 408
4.46.3 ara::diag::SecurityAccess::~SecurityAccess 408
4.46.4 ara::diag::SecurityAccess::operator= 408
4.46.5 ara::diag::SecurityAccess::KeyCompareResultType 409
4.46.6 ara::diag::SecurityAccess::CompareKey 409
4.46.7 ara::diag::SecurityAccess::GetSeed 410
4.46.8 ara::diag::SecurityAccess::Offer 411
4.46.9 ara::diag::SecurityAccess::StopOffer 411

4.47 ara::diag::ServiceValidation 413
4.47.1 ara::diag::ServiceValidation::ServiceValidation 413
4.47.2 ara::diag::ServiceValidation::ServiceValidation 414
4.47.3 ara::diag::ServiceValidation::~ServiceValidation 414
4.47.4 ara::diag::ServiceValidation::operator= 414
4.47.5 ara::diag::ServiceValidation::ConfirmationStatusType 415
4.47.6 ara::diag::ServiceValidation::Confirmation 415
4.47.7 ara::diag::ServiceValidation::Offer 416
4.47.8 ara::diag::ServiceValidation::StopOffer 417
4.47.9 ara::diag::ServiceValidation::Validate 417

4.48 ara::diag::UploadService 419
4.48.1 ara::diag::UploadService::UploadService 419
4.48.2 ara::diag::UploadService::UploadService 420

based on template version 1.00.00

4.48.3 ara::diag::UploadService::~UploadService 420
4.48.4 ara::diag::UploadService::operator= 420
4.48.5 ara::diag::UploadService::OperationOutput 421
4.48.6 ara::diag::UploadService::Offer 421
4.48.7 ara::diag::UploadService::RequestUpload 422
4.48.8 ara::diag::UploadService::RequestUploadExit 423
4.48.9 ara::diag::UploadService::StopOffer 423
4.48.10 ara::diag::UploadService::UploadData 424

4.49 Free Functions 426
4.49.1 amsr::diag::AddIsReadyNotifier 426
4.49.2 amsr::diag::AddIsReadyNotifier 426
4.49.3 amsr::diag::IsReady 427
4.49.4 amsr::diag::SetIsReadyNotifier 428
4.49.5 amsr::diag::SetIsReadyNotifier 429
4.49.6 amsr::diag::AsUdsDtcStatusByte 429
4.49.7 amsr::diag::conversation::OverrideS3Timeout 430
4.49.8 amsr::diag::AsUdsNrc 431

4.50 Typed DataElement 432
4.50.1 <DataElementPortInterfaceName>::Output type 432
4.50.2 <DataElementPortInterfaceName> constructor 432
4.50.3 <DataElementPortInterfaceName> destructor 433
4.50.4 <DataElementPortInterfaceName>::Offer 433
4.50.5 <DataElementPortInterfaceName>::Read 434
4.50.6 <DataElementPortInterfaceName>::StopOffer 434

4.51 Typed DataIdentifier 436
4.51.1 <DataIdentifierPortInterfaceName>::Output type 436
4.51.2 <DataIdentifierPortInterfaceName> constructor 436
4.51.3 <DataIdentifierPortInterfaceName> destructor 437
4.51.4 <DataIdentifierPortInterfaceName>::Read 437
4.51.5 <DataIdentifierPortInterfaceName>::Write 438
4.51.6 <DataIdentifierPortInterfaceName>::Offer 439
4.51.7 <DataIdentifierPortInterfaceName>::StopOffer 439

4.52 Typed Routine 441
4.52.1 <RoutinePortInterfaceName>::StartOutput type 441
4.52.2 <RoutinePortInterfaceName>::StopOutput type 441
4.52.3 <RoutinePortInterfaceName>::RequestResultsOutput type 442
4.52.4 <RoutinePortInterfaceName> constructor 442
4.52.5 <RoutinePortInterfaceName> destructor 443
4.52.6 <RoutinePortInterfaceName>::Start 443
4.52.7 <RoutinePortInterfaceName>::Stop 444
4.52.8 <RoutinePortInterfaceName>::RequestResults 445
4.52.9 <RoutinePortInterfaceName>::Offer 446

based on template version 1.00.00

4.52.10 <RoutinePortInterfaceName>::StopOffer 446

5 UDS Transport Protocol API Description 447

5.1 amsr::uds_transport::SecuredDataTransmissionHandler 448

5.1.1 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler 448

5.1.2 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler 448

5.1.3 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler 449

5.1.4 amsr::uds_transport::SecuredDataTransmissionHandler::~SecuredDataTransmissionHandler 449

5.1.5 amsr::uds_transport::SecuredDataTransmissionHandler::GetServiceValidator 450

5.1.6 amsr::uds_transport::SecuredDataTransmissionHandler::HandleRequest 451

5.1.7 amsr::uds_transport::SecuredDataTransmissionHandler::HandleResponse 451

5.1.8 amsr::uds_transport::SecuredDataTransmissionHandler::Initialize 452

5.1.9 amsr::uds_transport::SecuredDataTransmissionHandler::Shutdown 453

5.2 amsr::uds_transport::AdministrativeParameter 454

5.2.1 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 454

5.2.2 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 455

5.2.3 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 455

5.2.4 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 456

5.2.5 amsr::uds_transport::AdministrativeParameter::~AdministrativeParameter 456

5.2.6 amsr::uds_transport::AdministrativeParameter::operator= 456

5.2.7 amsr::uds_transport::AdministrativeParameter::operator= 457

5.2.8 amsr::uds_transport::AdministrativeParameter::GetValue 458

5.2.9 amsr::uds_transport::AdministrativeParameter::IsMessageEncrypted 458

5.2.10 amsr::uds_transport::AdministrativeParameter::IsMessageSigned 459

5.2.11 amsr::uds_transport::AdministrativeParameter::IsRequestMessage 459

5.2.12 amsr::uds_transport::AdministrativeParameter::IsPreEstablishedKeyUsed 460

5.2.13 amsr::uds_transport::AdministrativeParameter::IsSignatureOnResponseRequested 460

5.2.14 amsr::uds_transport::AdministrativeParameter::SetMessageEncrypted 461

5.2.15 amsr::uds_transport::AdministrativeParameter::SetMessageSigned 461

5.2.16 amsr::uds_transport::AdministrativeParameter::SetRequestMessage 462

5.2.17 amsr::uds_transport::AdministrativeParameter::SetPreEstablishedKeyUsed 462

5.2.18 amsr::uds_transport::AdministrativeParameter::SetSignatureOnResponseRequested 463

5.2.19 amsr::uds_transport::AdministrativeParameter::SetValue 463

5.3 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage 465

5.3.1 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SecuredDataTransmissionEnclosedMessage 465

5.3.2 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SecuredDataTransmissionEnclosedMessage 466

based on template version 1.00.00

5.3.3 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::~SecuredDataTransmissionEnclosedMessage 466
5.3.4 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= 467
5.3.5 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= 467
5.3.6 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::Create 468
5.3.7 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetAntiReplayCounter 469
5.3.8 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage 469
5.3.9 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage 470
5.3.10 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload 470
5.3.11 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload 471
5.3.12 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature 471
5.3.13 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature 472
5.3.14 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SetAntiReplayCounter 473

5.4 amsr::uds_transport::SecuredDataTransmissionRequestMessage 474
5.4.1 amsr::uds_transport::SecuredDataTransmissionRequestMessage::SecuredDataTransmissionRequestMessage 474
5.4.2 amsr::uds_transport::SecuredDataTransmissionRequestMessage::~SecuredDataTransmissionRequestMessage 475
5.4.3 amsr::uds_transport::SecuredDataTransmissionRequestMessage::operator= 475
5.4.4 amsr::uds_transport::SecuredDataTransmissionRequestMessage::AddMetaInfoValue 475
5.4.5 amsr::uds_transport::SecuredDataTransmissionRequestMessage::Create 476
5.4.6 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetAdministrativeParameters 477
5.4.7 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetClientAddress 477
5.4.8 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMessage 478
5.4.9 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMetaInfoValue 478
5.4.10 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetPayload 479
5.4.11 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetSignEncryptCalculation 480

5.5 amsr::uds_transport::SecuredDataTransmissionResponseMessage 481
5.5.1 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SecuredDataTransmissionResponseMessage 481
5.5.2 amsr::uds_transport::SecuredDataTransmissionResponseMessage::~SecuredDataTransmissionResponseMessage 482
5.5.3 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AddMetaInfoValue 482
5.5.4 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage 483

5.5.5 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage 483
5.5.6 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Create 484
5.5.7 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Finalize 485
5.5.8 amsr::uds_transport::SecuredDataTransmissionResponseMessage ::GetAdministrativeParameters 485
5.5.9 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetClientAddress 486
5.5.10 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetMetaInfoValue 486
5.5.11 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: GetSignEncryptCalculation 487
5.5.12 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: SetAdministrativeParameters 488
5.5.13 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: SetSignEncryptCalculation 488
5.6 ara::diag::uds_transport::UdsTransportProtocolHandler 490
5.6.1 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 490
5.6.2 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 491
5.6.3 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 491
5.6.4 ara::diag::uds_transport::UdsTransportProtocolHandler::~UdsTransportProtocolHandler 492
5.6.5 ara::diag::uds_transport::UdsTransportProtocolHandler::GetHandlerID 492
5.6.6 ara::diag::uds_transport::UdsTransportProtocolHandler::GetPeriodicHandler 493
5.6.7 ara::diag::uds_transport::UdsTransportProtocolHandler::Initialize 493
5.6.8 ara::diag::uds_transport::UdsTransportProtocolHandler::NotifyReestablishment 494
5.6.9 ara::diag::uds_transport::UdsTransportProtocolHandler::Start 495
5.6.10 ara::diag::uds_transport::UdsTransportProtocolHandler::Stop 495
5.6.11 ara::diag::uds_transport::UdsTransportProtocolHandler::StopConnection 496
5.6.12 ara::diag::uds_transport::UdsTransportProtocolHandler::Transmit 497
5.7 ara::diag::uds_transport::UdsTransportProtocolMgr 499
5.7.1 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr 499
5.7.2 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr 500
5.7.3 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr 500
5.7.4 ara::diag::uds_transport::UdsTransportProtocolMgr::~UdsTransportProtocolMgr 501
5.7.5 ara::diag::uds_transport::UdsTransportProtocolMgr::operator= 501
5.7.6 ara::diag::uds_transport::UdsTransportProtocolMgr::operator= 501
5.7.7 ara::diag::uds_transport::UdsTransportProtocolMgr::ChannelReestablished 502
5.7.8 ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage 503
5.7.9 ara::diag::uds_transport::UdsTransportProtocolMgr::HandlerStopped 503
5.7.10 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage 504
5.7.11 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage 505

5.7.12 ara::diag::uds_transport::UdsTransportProtocolMgr::NotifyMessageFailure 506
5.7.13 ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation 507
5.7.14 ara::diag::uds_transport::UdsTransportProtocolMgr::TransmitConfirmation 507
5.8 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler 509
5.8.1 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler 509
5.8.2 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler 510
5.8.3 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler 510
5.8.4 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::~UdsTransportProtocolPeriodicHandler 511
5.8.5 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= 511
5.8.6 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= 512
5.8.7 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetMaxPayloadLength 512
5.8.8 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetNumberOfPeriodicMessages 513
5.8.9 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::PeriodicTransmit 513
5.9 ara::diag::uds_transport::UdsMessage 515
5.9.1 ara::diag::uds_transport::UdsMessage::UdsMessage 515
5.9.2 ara::diag::uds_transport::UdsMessage::UdsMessage 516
5.9.3 ara::diag::uds_transport::UdsMessage::~UdsMessage 516
5.9.4 ara::diag::uds_transport::UdsMessage::operator= 517
5.9.5 ara::diag::uds_transport::UdsMessage::operator= 517
5.9.6 ara::diag::uds_transport::UdsMessage::AddMetaInfo 518
5.9.7 ara::diag::uds_transport::UdsMessage::AddMetaInfo 518
5.9.8 ara::diag::uds_transport::UdsMessage::Capacity 519
5.9.9 ara::diag::uds_transport::UdsMessage::ChangePayloadView 519
5.9.10 ara::diag::uds_transport::UdsMessage::GetGlobalChannelIdentifier 520
5.9.11 ara::diag::uds_transport::UdsMessage::GetPayload 521
5.9.12 ara::diag::uds_transport::UdsMessage::GetPayload 521
5.9.13 ara::diag::uds_transport::UdsMessage::GetSa 522
5.9.14 ara::diag::uds_transport::UdsMessage::GetTa 522
5.9.15 ara::diag::uds_transport::UdsMessage::GetTaType 523
5.10 amsr::uds_transport::CancellationHandler 524
5.10.1 amsr::uds_transport::CancellationHandler::CancellationHandler 524
5.10.2 amsr::uds_transport::CancellationHandler::~CancellationHandler 524
5.10.3 amsr::uds_transport::CancellationHandler::operator= 525
5.10.4 amsr::uds_transport::CancellationHandler::IsCanceled 525
5.10.5 amsr::uds_transport::CancellationHandler::SetNotifier 526

5.11 amsr::uds_transport::MetaInfo 528
5.11.1 amsr::uds_transport::MetaInfo::MetaInfo 528
5.11.2 amsr::uds_transport::MetaInfo::~MetaInfo 528
5.11.3 amsr::uds_transport::MetaInfo::operator= 529
5.11.4 amsr::uds_transport::MetaInfo::GetValue 529
5.12 amsr::uds_transport::ServiceValidation 531
5.12.1 amsr::uds_transport::ServiceValidation::ServiceValidation 531
5.12.2 amsr::uds_transport::ServiceValidation::ServiceValidation 531
5.12.3 amsr::uds_transport::ServiceValidation::ServiceValidation 532
5.12.4 amsr::uds_transport::ServiceValidation::~ServiceValidation 532
5.12.5 amsr::uds_transport::ServiceValidation::Confirmation 533
5.12.6 amsr::uds_transport::ServiceValidation::Validate 533
5.13 amsr::uds_transport::ServiceValidation::ValidationError 535
5.13.1 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 535
5.13.2 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 535
5.13.3 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 536
5.13.4 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 536
5.13.5 amsr::uds_transport::ServiceValidation::ValidationError::~ValidationError 537
5.13.6 amsr::uds_transport::ServiceValidation::ValidationError::operator= 537
5.13.7 amsr::uds_transport::ServiceValidation::ValidationError::operator= 538
5.13.8 amsr::uds_transport::ServiceValidation::ValidationError::GetError 538

6 Configuration 540
6.1 Used Generators 540
6.2 MEX Configuration 540
6.2.1 MEX parameter in Diagnostics Library (ara::diag) 541
6.2.2 MEX parameter in Diagnostics Daemon 541
6.3 Compile Time Configuration 544
6.3.1 Diagnostic Server configuration: Software Cluster 544
6.3.1.1 Configuration Extensions 545
6.3.2 Modeling of Data Access 547
6.3.2.1 Application Interface Data Types 547
6.3.2.2 DEXT Data Types 548
6.3.2.3 Naming 548
6.3.2.4 Interface Selection 549
6.3.2.4.1 Interface DataIdentifier 549
6.3.2.4.2 Interface GenericUDS 549
6.3.2.4.3 Interface DataElement 550
6.3.2.5 Port Mapping 550
6.4 Runtime Configuration 551
6.4.1 Global Configuration 551

based on template version 1.00.00

6.4.2 Port Mappings 551
6.4.2.1 Event to Port Mapping 551
6.5 Service Validation Configuration 553
6.5.1 Service Validation Behavior Configuration 553
6.6 Logging Configuration 555
6.7 NvData Configuration 557
6.8 Fault Memory File Handling Configuration 558
6.8.1 File Handling Mode Multifile 559
6.8.2 File Handling Mode Singlefile 559
6.9 Persistency Redundancy Configuration 559
6.10 Dext Files Configuration 560
6.11 AraDiag Configuration 560
6.11.1 Request Execution Configuration 560
6.12 Authentication Configuration 561
6.12.1 Service 561
6.12.1.1 Authentication Service 561
6.12.1.2 Client Authorization 563
6.12.2 Ports 563
6.12.2.1 Application 563
6.12.2.2 Daemon 564
6.12.3 Service Mappings 564
6.12.3.1 Internal Service 564
6.12.3.2 External Service 564
6.13 Client Authentication Configuration 565
6.13.1 Diagnostic Client Authentication Identification 565
6.13.2 Suppress Precondition Evaluation 566
6.13.3 Ports 566
6.13.3.1 Application 566
6.13.3.2 Daemon 567
6.14 Failure event associated identification configuration 568
6.14.1 Failure Associated Identifier 568
6.14.2 Provided Data Mapping 568
6.14.3 Data Element Configuration 568
6.15 Security Event Log Configuration 569
6.15.1 Security Event Log type 569
6.15.2 Security Event type 569
6.15.3 Security Event 570
6.15.4 Security Event Log Port 571
6.15.5 Security Event Port 572

based on template version 1.00.00

6.16 Idsm Configuration ... 574
6.16.1 Feature activation 574
6.16.2 Number Of Asynchronous Jobs 574
6.16.3 IDSR Configuration 574
6.17 Service Processing Notification ... 576
6.18 Request File Transfer Configuration ... 577
6.18.1 Service 577
6.18.2 Ports 578
6.18.2.1 Application 578
6.18.2.2 Daemon 578
6.18.3 Service Mappings 579
6.18.3.1 Internal Service 579
6.18.3.2 External Service 579
6.19 Root cause event associated identification configuration ... 579
6.19.1 Root Cause Associated Identifier 579
6.19.2 Provided Data Mapping 580
6.19.3 Base Type Configuration 580
6.20 DTC confirmation configuration ... 580
6.21 Indicator configuration ... 581
6.22 DoIP activation line configuration ... 582
6.22.1 DoIP Activation Line Interface 582
6.22.2 DoIP Activation Line Port 582
6.22.3 DoIP activation line Mapping 582
6.23 DoIP group identification configuration ... 582
6.23.1 DoIP Group Identification Interface 582
6.23.2 DoIP Group Identification Port 582
6.23.3 DoIP Group Identification Mapping 583
6.24 DoIP vehicle identification number configuration ... 583
6.24.1 Data Identifier 583
6.24.2 Service Instance Mapping 584
6.24.3 Port Mapping 585
6.25 Secured Data Transmission Configuration ... 585
6.25.1 Secured Data Transmission properties configuration 585
6.25.2 Secured Data Transmission creator configuration 586
6.26 SOVD Configuration ... 586
6.26.1 Feature activation 586
6.26.2 SOVD module instantiation 586
6.26.3 SOVD server parameters 591

based on template version 1.00.00

6.26.4 SOVD resource collection 592
6.26.4.1 SOVD method 592
6.26.4.2 SOVD generic method 593
6.26.4.3 SOVD service instance 595
6.26.4.4 SOVD authorization 596
6.26.4.5 SOVD authorization port 597
6.26.4.6 SOVD lock 599
6.26.4.7 SOVD lock port 599
6.27 Data identifier configuration ... 601
6.28 Long Name Configuration ... 601
6.29 Steady Mode Configuration ... 601
6.29.1 Memory Resource Properties Configuration 602
6.29.2 Diagnostic Application Steady Mode Configuration 602
6.29.3 Diagnostics Daemon Steady Mode Configuration 603
6.30 UDS NRC Mapping Configuration ... 604
6.30.1 UDS NRC Mapping 604
6.30.2 UDS NRC Mapping Table 604
6.31 TSSR Configuration ... 604
6.31.1 Profile 604
6.31.2 DTC props configuration 605
6.32 Event memory configuration ... 607

7 Cybersecurity ... 608
7.1 DoIP Transport Protocol ... 608
7.1.1 Integration 608
7.1.2 Configuration 608
7.1.2.1 CMI-DoIP-TlsConfiguration 608
7.1.2.2 CMI-DoIP-UseTcpIfCertificatesAreUnavailable 608
7.1.3 Runtime Interfaces: Application 608
7.1.4 Runtime Interfaces: BSW 608

8 Glossary and Abbreviations ... 610
8.1 Glossary ... 610
8.2 Abbreviations ... 611

9 Contact ... 613

Illustrations

1-1 Architecture overview 58

2-1 General architecture and how the SDT plug-in fits into it. 97
2-2 Sequence diagram for the standard positive use case. 98
2-3 Sequence diagram for SOVD Authorization. 104
2-4 Sequence diagram for token Revokation. 106

3-1 Working Directory Structure of the Diagnostics Daemon 158

Tables

2-1 Supported AUTOSAR Conform Features 61
2-2 Supported DoIP Conform Features 62
2-3 Supported UDP Requests 63
2-4 Alive Check Features 65
2-5 DoIP Global configuration 67
2-6 DoIP Feature activation values 68
2-7 DoIP TLS configuration 69
2-8 DoIP Supported TLS version 1.2 cipher suits 69
2-9 DoIP Network configuration 71
2-10 DoIP TP Connections Object 72
2-11 DoIP UDP channel configuration 72
2-12 DoIP TCP channel configuration 73
2-13 DoIP configurable protocol versions 73
2-14 Negative response codes and the error they describe 76
2-15 NRC handling for service requests on DM side * not validated in case of Zero-copy 76
2-16 NRC handling for sub-functions requests on DM side 77
2-17 Supported UDS Services Overview 84
2-18 Value constraints of s3ServerTimeout parameter (in seconds) in Diagnostic Daemon 84
2-19 NRC handling for ReadDataByIdentifier request on DM side *checks done for every DID 87
2-20 NRC handling for WriteDataByIdentifier request on DM side 88

based on template version 1.00.00

2-21 NRC handling for ReadDataByPeriodicIdentifier request on DM side *checked for every periodic DID 88
2-22 NRC handling for SecurityAccess request on DM side 90
2-23 NRC handling for Authentication request on DM side 91
2-24 NRC handling for RoutineControl request on DM side *checks done for the RID 91
2-25 NRC handling for RequestDownload request on DM side 92
2-26 NRC handling for RequestUpload request on DM side 92
2-27 NRC handling for TransferData request on DM side 93
2-28 Additional validations if neither compression nor encoding is effective 93
2-29 NRC handling for RequestTransferExit request on DM side 93
2-30 NRC handling for FileTransfer request on DM side 94
2-31 Multi-tester strategy 100
2-32 Behavior of the setup events on diagnostic session transition 100
2-33 Description of Authorization failures 105
2-34 DTC status combination 109
2-35 NRC handling for ClearDiagnosticInformation request on DM side 114
2-36 Monitor behavior upon initialization with initial value of enable condition set to false 115
2-37 Monitor behavior upon initialization with initial value of enable condition set to true 115
2-38 Description of logging parameters 117
2-39 Application IDs of logging examples 117
2-40 RPC intensity keys for sample Application ID 9876 with two contexts 121
2-41 Features provided beyond the AUTOSAR standard 123
2-42 Limitations 126
2-43 Supported/unsupported MetaInfo keys 127
2-44 Vendor specific MetaInfo keys 127
2-45 Reactions to blockable POSIX Signals not blocked by the process 130
2-46 Threads spawned by Diagnostics Daemon executable 133
2-47 Thread order of Diagnostics Daemon executable 134
2-48 Threads of Diagnostics Library (ara::diag) 134
2-49 Thread order of Diagnostics Library (ara::diag) 135

3-1 amsr-vector-fs-udstransport cmake targets 147
3-2 amsr-vector-fs-aradiag cmake targets 148

based on template version 1.00.00

3-3 amsr-vector-fs-diagnosticrpc cmake targets 148
3-4 amsr-vector-fs-diagnosticrpcbinding cmake targets 149
3-5 amsr-vector-fs-diagnosticutility cmake targets 149
3-6 amsr-vector-fs-diagnosticmanager cmake targets 150
3-7 amsr-vector-fs-diagnosticmanager cmake targets 151
3-8 amsr-vector-fs-diagnosticmanager cmake targets 152
3-9 amsr-vector-fs-diagnosticmanager cmake targets 153
3-10 amsr-vector-fs-diagnosticmanager cmake targets 154
3-11 amsr-vector-fs-doipbinding cmake targets 155
3-12 CMake Options 156
3-13 Used Application ID of the DM 159
3-14 List of Context IDs used by the diagnostic manager 159

4-1 amsr::diag_utility::EventLoop 161
4-2 amsr::diag_utility::EventLoop::RegisterPeriodicTask 161
4-3 amsr::diag_utility::EventLoop::RegisterEventName 162
4-4 amsr::diag_utility::EventLoop::RegisterEvent 163
4-5 amsr::diag_utility::EventLoop::RaiseEvent 163
4-6 amsr::diag_utility::EventLoop::Unregister 164
4-7 amsr::diag_utility::EventLoop::DeferTask 164
4-8 amsr::diag_utility::EventLoop::Handle 165
4-9 amsr::diag_utility::EventLoop::Handle::Handle 165
4-10 amsr::diag_utility::EventLoop::Handle::~Handle 166
4-11 amsr::diag_utility::EventLoop::Handle::operator= 166
4-12 amsr::diag_utility::EventLoop::Handle::Unregister 167
4-13 amsr::diag_utility::EventLoop::Buzzer 167
4-14 amsr::diag_utility::EventLoop::Buzzer::Buzzer 167
4-15 amsr::diag_utility::EventLoop::Buzzer::~Buzzer 168
4-16 amsr::diag_utility::EventLoop::Buzzer::operator= 168
4-17 amsr::diag_utility::EventLoop::Buzzer::Unregister 169
4-18 amsr::diag_utility::EventLoop::Buzzer::RaiseEvent 169
4-19 amsr::diag_utility::RootSwc 170
4-20 amsr::diag_utility::RootSwc::GetAccess 170

based on template version 1.00.00

4-21 amsr::diag_utility::RootSwc::GetEventLoop ... 171
4-22 amsr::diag::DiagUdsNrc ... 173
4-23 amsr::diag::NegativeResponseReason ... 174
4-24 amsr::diag::r21_11::DiagErrorDomain ... 175
4-25 amsr::diag::r21_11::DiagErrorDomain::DiagErrorDomain ... 175
4-26 amsr::diag::r21_11::DiagOfferErrorDomain ... 176
4-27 amsr::diag::r21_11::DiagOfferErrorDomain::DiagOfferErrorDomain ... 176
4-28 amsr::diag::r21_11::DiagReportingErrorDomain ... 177
4-29 amsr::diag::r21_11::DiagReportingErrorDomain::DiagReportingErrorDomain ... 177
4-30 amsr::diag::r21_11::Monitor ... 178
4-31 amsr::diag::r21_11::Monitor::Monitor ... 179
4-32 amsr::diag::r21_11::Monitor::Monitor ... 179
4-33 amsr::diag::r21_11::Monitor::Monitor ... 180
4-34 amsr::diag::r21_11::Monitor::Monitor ... 181
4-35 amsr::diag::r21_11::Monitor::operator= ... 181
4-36 amsr::diag::r21_11::Monitor::InitMonitorReason ... 182
4-37 amsr::diag::r21_11::Monitor::CounterBased ... 182
4-38 amsr::diag::r21_11::Monitor::TimeBased ... 182
4-39 amsr::diag::r21_11::Monitor::MonitorAction ... 183
4-40 amsr::diag::r21_11::Monitor::IsValidInstanceSpecifier ... 184
4-41 amsr::diag::r21_11::Monitor::Offer ... 184
4-42 amsr::diag::r21_11::Monitor::ReportMonitorAction ... 185
4-43 amsr::diag::r21_11::Monitor::StopOffer ... 185
4-44 amsr::diag::r21_11::OperationCycle ... 186
4-45 amsr::diag::r21_11::OperationCycle::OperationCycle ... 186
4-46 amsr::diag::r21_11::OperationCycle::OperationCycle ... 187
4-47 amsr::diag::r21_11::OperationCycle::~OperationCycle ... 187
4-48 amsr::diag::r21_11::OperationCycle::operator= ... 188
4-49 amsr::diag::r21_11::OperationCycle::RestartOperationCycle ... 188
4-50 amsr::diag::r21_11::OperationCycle::SetNotifier ... 189
4-51 amsr::diag::ReleaseHandler ... 190
4-52 amsr::diag::ReleaseHandler::ReleaseHandler ... 190

4-53 amsr::diag::ReleaseHandler::operator= ... 191
4-54 amsr::diag::ReleaseHandler::MayRelease ... 191
4-55 amsr::diag::ReleaseHandler::SetNotifier ... 192
4-56 amsr::diag::SecurityEvent ... 193
4-57 amsr::diag::SecurityEvent::SecurityEvent ... 193
4-58 amsr::diag::SecurityEvent::SecurityEvent ... 194
4-59 amsr::diag::SecurityEvent::~SecurityEvent ... 194
4-60 amsr::diag::SecurityEvent::operator= ... 195
4-61 amsr::diag::SecurityEvent::Report ... 195
4-62 amsr::diag::SecurityEventLog ... 196
4-63 amsr::diag::SecurityEventLog::SecurityEventLog ... 197
4-64 amsr::diag::SecurityEventLog::SecurityEventLog ... 197
4-65 amsr::diag::SecurityEventLog::~SecurityEventLog ... 197
4-66 amsr::diag::SecurityEventLog::operator= ... 198
4-67 amsr::diag::SecurityEventLog::Event ... 198
4-68 amsr::diag::SecurityEventLog::Event::Event ... 199
4-69 amsr::diag::SecurityEventLog::Event::Event ... 199
4-70 amsr::diag::SecurityEventLog::Event::~Event ... 200
4-71 amsr::diag::SecurityEventLog::Event::operator= ... 200
4-72 amsr::diag::SecurityEventLog::Event::operator= ... 201
4-73 amsr::diag::SecurityEventLog::Event::GetCounter ... 201
4-74 amsr::diag::SecurityEventLog::Event::GetId ... 202
4-75 amsr::diag::SecurityEventLog::EventType ... 202
4-76 amsr::diag::SecurityEventLog::EventType::EventType ... 203
4-77 amsr::diag::SecurityEventLog::EventType::~EventType ... 203
4-78 amsr::diag::SecurityEventLog::EventType::operator= ... 204
4-79 amsr::diag::SecurityEventLog::EventType::GetBuffer ... 204
4-80 amsr::diag::SecurityEventLog::EventType::GetEvent ... 205
4-81 amsr::diag::SecurityEventLog::EventType::GetEvents ... 205
4-82 amsr::diag::SecurityEventLog::EventType::GetId ... 206
4-83 amsr::diag::SecurityEventLog::AddRawEntryToAll ... 206
4-84 amsr::diag::SecurityEventLog::GetEventLog ... 207

4-85 amsr::diag::SecurityEventLog::GetEventType 207
4-86 amsr::diag::SecurityEventLog::Report 208
4-87 amsr::diag::SecurityEventLog::SetNotifierNumberOfBufferEntriesChanged 209
4-88 amsr::diag::SecurityEventLog::SetNotifierSecurityEvent 210
4-89 amsr::diag::ServiceProcessingNotification 211
4-90 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification 212
4-91 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification 212
4-92 amsr::diag::ServiceProcessingNotification::~ServiceProcessingNotification 212
4-93 amsr::diag::ServiceProcessingNotification::operator= 213
4-94 amsr::diag::ServiceProcessingNotification::ConfirmationStatusType 213
4-95 amsr::diag::NegativeResponseDetails 214
4-96 amsr::diag::NegativeResponseDetails::GetNegativeResponseReason 214
4-97 amsr::diag::NegativeResponseDetails::GetRawNrc 215
4-98 amsr::diag::NegativeResponseDetails::GetUdsNrc 215
4-99 amsr::diag::ServiceProcessingNotification::Offer 216
4-100 amsr::diag::ServiceProcessingNotification::ProcessingEnd 217
4-101 amsr::diag::ServiceProcessingNotification::StopOffer 217
4-102 amsr::diag::EventStatusByte 218
4-103 amsr::diag::EventStatusByte::IsFailedAndTested 218
4-104 amsr::diag::EventStatusByte::IsNotSet 219
4-105 amsr::diag::EventStatusByte::IsNotSet 220
4-106 amsr::diag::EventStatusByte::IsPassedAndTested 220
4-107 amsr::diag::EventStatusByte::IsSet 221
4-108 amsr::diag::EventStatusByte::IsSet 221
4-109 amsr::diag::UdsDTCStatusByteType 222
4-110 amsr::diag::UdsDTCStatusByteType::operator== 222
4-111 amsr::diag::UdsDTCStatusByteType::IsFailedAndTested 223
4-112 amsr::diag::UdsDTCStatusByteType::IsNotSet 223
4-113 amsr::diag::UdsDTCStatusByteType::IsNotSet 224
4-114 amsr::diag::UdsDTCStatusByteType::IsPassedAndTested 225
4-115 amsr::diag::UdsDTCStatusByteType::IsSet 225
4-116 amsr::diag::UdsDTCStatusByteType::IsSet 226

based on template version 1.00.00

4-117 amsr::diag::DataTransferReadByPullHandler 227
4-118 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler . . . 227
4-119 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler . . . 228
4-120 amsr::diag::DataTransferReadByPullHandler::~DataTransferReadByPullHandler . . 228
4-121 amsr::diag::DataTransferReadByPullHandler::operator= 229
4-122 amsr::diag::DataTransferReadByPullHandler::ExitRead 230
4-123 amsr::diag::DataTransferReadByPullHandler::Read 231
4-124 amsr::diag::DataTransferReadByPushHandler 231
4-125 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler 232
4-126 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler 232
4-127 amsr::diag::DataTransferReadByPushHandler::~DataTransferReadByPushHandler 233
4-128 amsr::diag::DataTransferReadByPushHandler::operator= 233
4-129 amsr::diag::DataTransferReadByPushHandler::ExitRead 234
4-130 amsr::diag::DataTransferReadByPushHandler::Read 235
4-131 amsr::diag::DataTransferReadSession 236
4-132 amsr::diag::DataTransferReadSession::DataTransferReadSession 236
4-133 amsr::diag::DataTransferReadSession::DataTransferReadSession 237
4-134 amsr::diag::DataTransferReadSession::DataTransferReadSession 238
4-135 amsr::diag::DataTransferReadSession::DataTransferReadSession 238
4-136 amsr::diag::DataTransferReadSession::~DataTransferReadSession 239
4-137 amsr::diag::DataTransferReadSession::operator= 239
4-138 amsr::diag::DataTransferReadSharedDataHandler 240
4-139 amsr::diag::DataTransferReadSharedDataHandler::
DataTransferReadSharedDataHandler 240
4-140 amsr::diag::DataTransferReadSharedDataHandler::
DataTransferReadSharedDataHandler 240
4-141 amsr::diag::DataTransferReadSharedDataHandler::
~DataTransferReadSharedDataHandler 241
4-142 amsr::diag::DataTransferReadSharedDataHandler::operator= 241
4-143 amsr::diag::DataTransferReadSharedDataHandler::ExitRead 242
4-144 amsr::diag::DataTransferReadSharedDataHandler::Read 243
4-145 amsr::diag::DataTransferWriteHandler 244
4-146 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler 244

4-147 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler 245
4-148 amsr::diag::DataTransferWriteHandler::~DataTransferWriteHandler 245
4-149 amsr::diag::DataTransferWriteHandler::operator= 246
4-150 amsr::diag::DataTransferWriteHandler::ExitWrite 247
4-151 amsr::diag::DataTransferWriteHandler::Write 248
4-152 amsr::diag::DataTransferWriteSession 249
4-153 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 249
4-154 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 250
4-155 amsr::diag::DataTransferWriteSession::DataTransferWriteSession 250
4-156 amsr::diag::DataTransferWriteSession::~DataTransferWriteSession 251
4-157 amsr::diag::DataTransferWriteSession::operator= 251
4-158 amsr::diag::FileTransferService 252
4-159 amsr::diag::FileTransferService::FileTransferService 253
4-160 amsr::diag::FileTransferService::FileTransferService 253
4-161 amsr::diag::FileTransferService::~FileTransferService 253
4-162 amsr::diag::FileTransferService::operator= 254
4-163 amsr::diag::FileTransferService::DeleteFile 255
4-164 amsr::diag::FileTransferService::Offer 255
4-165 amsr::diag::FileTransferService::RequestReadDirectory 256
4-166 amsr::diag::FileTransferService::RequestReadFile 257
4-167 amsr::diag::FileTransferService::RequestResumeWriteFile 258
4-168 amsr::diag::FileTransferService::RequestWriteFile 259
4-169 amsr::diag::FileTransferService::StopOffer 260
4-170 amsr::diag::DownloadService 261
4-171 amsr::diag::DownloadService::DownloadService 261
4-172 amsr::diag::DownloadService::DownloadService 262
4-173 amsr::diag::DownloadService::~DownloadService 262
4-174 amsr::diag::DownloadService::operator= 263
4-175 amsr::diag::DownloadService::DownloadData 264
4-176 amsr::diag::DownloadService::Offer 264
4-177 amsr::diag::DownloadService::RequestDownload 265
4-178 amsr::diag::DownloadService::RequestDownloadExit 266

4-179 amsr::diag::DownloadService::StopOffer 267
4-180 amsr::diag::SovdAuthorization 268
4-181 amsr::diag::SovdAuthorization::~SovdAuthorization 268
4-182 amsr::diag::SovdAuthorization::Offer 269
4-183 amsr::diag::SovdAuthorization::SovdAuthorization 269
4-184 amsr::diag::SovdAuthorization::SovdAuthorization 270
4-185 amsr::diag::SovdAuthorization::StopOffer 270
4-186 amsr::diag::SovdAuthorization::ValidateAuthorization 271
4-187 amsr::diag::SovdAuthorization::operator= 272
4-188 amsr::diag::SovdTokenAuthorization 273
4-189 amsr::diag::SovdTokenAuthorization::Authorize 273
4-190 amsr::diag::SovdTokenAuthorizationHandle 274
4-191 amsr::diag::SovdTokenAuthorizationHandle::~SovdTokenAuthorizationHandle 274
4-192 amsr::diag::SovdTokenAuthorizationHandle::Revoke 275
4-193 amsr::diag::SovdTokenAuthorizationHandle::SetNotifier 276
4-194 amsr::diag::SovdTokenAuthorizationHandle::SovdTokenAuthorizationHandle 276
4-195 amsr::diag::SovdTokenAuthorizationHandle::operator= 277
4-196 amsr::diag::SovdLock 278
4-197 amsr::diag::SovdLock::~SovdLock 278
4-198 amsr::diag::SovdLock::GetAllLocks 279
4-199 amsr::diag::SovdLock::GetExpirationTime 279
4-200 amsr::diag::SovdLock::SetSovdLockStateHandler 280
4-201 amsr::diag::SovdLock::SovdLock 280
4-202 amsr::diag::SovdLock::SovdLock 281
4-203 amsr::diag::SovdLock::operator= 281
4-204 ara::diag::Authentication 282
4-205 ara::diag::Authentication::Authentication 283
4-206 ara::diag::Authentication::Authentication 283
4-207 ara::diag::Authentication::~Authentication 284
4-208 ara::diag::Authentication::operator= 284
4-209 ara::diag::Authentication::Offer 285
4-210 ara::diag::Authentication::StopOffer 285

based on template version 1.00.00

4-211 ara::diag::Authentication::TransmitCertificate 286
4-212 ara::diag::Authentication::VerifyCertificateBidirectional 287
4-213 ara::diag::Authentication::VerifyCertificateUnidirectional 288
4-214 ara::diag::Authentication::VerifyOwnership 289
4-215 ara::diag::CancellationHandler 290
4-216 ara::diag::CancellationHandler::CancellationHandler 290
4-217 ara::diag::CancellationHandler::operator= 291
4-218 ara::diag::CancellationHandler::IsCanceled 292
4-219 ara::diag::CancellationHandler::SetNotifier 292
4-220 ara::diag::ClientAuthenticationHandle 293
4-221 ara::diag::ClientAuthenticationHandle::ClientAuthenticationHandle 293
4-222 ara::diag::ClientAuthenticationHandle::~ClientAuthenticationHandle 294
4-223 ara::diag::ClientAuthenticationHandle::operator= 294
4-224 ara::diag::ClientAuthenticationHandle::Append 295
4-225 ara::diag::ClientAuthenticationHandle::Refresh 295
4-226 ara::diag::ClientAuthenticationHandle::Revoke 296
4-227 ara::diag::ClientAuthenticationHandle::Set 297
4-228 ara::diag::ClientAuthentication 298
4-229 ara::diag::ClientAuthentication::ClientAuthentication 298
4-230 ara::diag::ClientAuthentication::~ClientAuthentication 299
4-231 ara::diag::ClientAuthentication::operator= 299
4-232 ara::diag::ClientAuthentication::DiagnosticAuthState 300
4-233 ara::diag::ClientAuthentication::DiagnosticAuthRole 300
4-234 ara::diag::ClientAuthentication::Authenticate 300
4-235 ara::diag::ClientAuthentication::GetState 301
4-236 ara::diag::ClientAuthentication::OverrideDefaultRoles 302
4-237 ara::diag::ClientAuthentication::SetNotifier 302
4-238 ara::diag::Condition 303
4-239 ara::diag::Condition::Condition 303
4-240 ara::diag::Condition::Condition 304
4-241 ara::diag::Condition::~Condition 304
4-242 ara::diag::Condition::operator= 305

4-243 ara::diag::Condition::ConditionType 305
4-244 ara::diag::Condition::GetCondition 306
4-245 ara::diag::Condition::SetCondition 306
4-246 ara::diag::Condition::SetNotifier 307
4-247 ara::diag::Conversation 308
4-248 ara::diag::Conversation::ActivityStatusType 308
4-249 ara::diag::Conversation::ConversationIdentifierType 309
4-250 ara::diag::Conversation::GetActivityStatus 309
4-251 ara::diag::Conversation::GetAllConversations 310
4-252 ara::diag::Conversation::GetConversation 311
4-253 ara::diag::Conversation::GetConversationIdentifier 311
4-254 ara::diag::Conversation::GetCurrentActiveConversations 312
4-255 ara::diag::Conversation::GetDiagnosticSecurityLevel 312
4-256 ara::diag::Conversation::GetDiagnosticSession 313
4-257 ara::diag::Conversation::ResetToDefaultSession 313
4-258 ara::diag::Conversation::SetActivityNotifier 314
4-259 ara::diag::Conversation::SetDiagnosticSessionNotifier 315
4-260 ara::diag::Conversation::SetSecurityLevelNotifier 316
4-261 ara::diag::DiagUdsNrcErrorDomain 317
4-262 ara::diag::DiagUdsNrcErrorDomain::DiagUdsNrcErrorDomain 317
4-263 ara::diag::DiagUdsNrcErrc 319
4-264 ara::diag::DiagnosticServiceDynamicAccessList 320
4-265 ara::diag::DiagnosticServiceDynamicAccessList::
DiagnosticServiceDynamicAccessList 321
4-266 ara::diag::DiagnosticServiceDynamicAccessList::
DiagnosticServiceDynamicAccessList 321
4-267 ara::diag::DiagnosticServiceDynamicAccessList::
DiagnosticServiceDynamicAccessList 322
4-268 ara::diag::DiagnosticServiceDynamicAccessList::
~DiagnosticServiceDynamicAccessList 322
4-269 ara::diag::DiagnosticServiceDynamicAccessList::operator= 323
4-270 ara::diag::DiagnosticServiceDynamicAccessList::operator= 323
4-271 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder 324
4-272 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder 325

based on template version 1.00.00

4-273 ara::diag::DiagnosticServiceDynamicAccessList::Reserve 325
4-274 ara::diag::DoIPActivationLine 326
4-275 ara::diag::DoIPActivationLine::DoIPActivationLine 326
4-276 ara::diag::DoIPActivationLine::DoIPActivationLine 327
4-277 ara::diag::DoIPActivationLine::~DoIPActivationLine 327
4-278 ara::diag::DoIPActivationLine::operator= 328
4-279 ara::diag::DoIPActivationLine::GetActivationLineState 328
4-280 ara::diag::DoIPActivationLine::GetNetworkInterfaceId 329
4-281 ara::diag::DoIPActivationLine::Offer 329
4-282 ara::diag::DoIPActivationLine::StopOffer 330
4-283 ara::diag::DoIPActivationLine::UpdateActivationLineState 331
4-284 ara::diag::DoIPGroupIdentification 332
4-285 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification 332
4-286 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification 333
4-287 ara::diag::DoIPGroupIdentification::~DoIPGroupIdentification 333
4-288 ara::diag::DoIPGroupIdentification::operator= 334
4-289 ara::diag::DoIPGroupIdentification::GidStatus 334
4-290 ara::diag::DoIPGroupIdentification::GetGidStatus 335
4-291 ara::diag::DoIPGroupIdentification::Offer 335
4-292 ara::diag::DoIPGroupIdentification::StopOffer 336
4-293 ara::diag::DoIPPowerMode 337
4-294 ara::diag::DoIPPowerMode::DoIPPowerMode 337
4-295 ara::diag::DoIPPowerMode::DoIPPowerMode 338
4-296 ara::diag::DoIPPowerMode::~DoIPPowerMode 338
4-297 ara::diag::DoIPPowerMode::operator= 339
4-298 ara::diag::DoIPPowerMode::PowerModeType 339
4-299 ara::diag::DoIPPowerMode::GetDoIPPowerMode 340
4-300 ara::diag::DoIPPowerMode::Offer 340
4-301 ara::diag::DoIPPowerMode::StopOffer 341
4-302 ara::diag::DownloadService 342
4-303 ara::diag::DownloadService::DownloadService 342
4-304 ara::diag::DownloadService::DownloadService 343

4-305 ara::diag::DownloadService::~DownloadService 343
4-306 ara::diag::DownloadService::operator= 344
4-307 ara::diag::DownloadService::OperationOutput 344
4-308 ara::diag::DownloadService::DownloadData 345
4-309 ara::diag::DownloadService::Offer 346
4-310 ara::diag::DownloadService::RequestDownload 346
4-311 ara::diag::DownloadService::RequestDownloadExit 347
4-312 ara::diag::DownloadService::StopOffer 348
4-313 ara::diag::DTCInformation 349
4-314 ara::diag::DTCInformation::DTCInformation 350
4-315 ara::diag::DTCInformation::DTCInformation 350
4-316 ara::diag::DTCInformation::~DTCInformation 351
4-317 ara::diag::DTCInformation::operator= 351
4-318 ara::diag::DTCInformation::ControlDtcStatusType 352
4-319 ara::diag::DTCInformation::UdsStatusBit 352
4-320 ara::diag::DTCInformation::SnapshotDataIdentiferType 352
4-321 ara::diag::DTCInformation::SnapshotDataRecordType 353
4-322 ara::diag::DTCInformation::SnapshotRecordUpdatedType 353
4-323 ara::diag::DTCInformation::Clear 354
4-324 ara::diag::DTCInformation::EnableControlDtc 354
4-325 ara::diag::DTCInformation::GetControlDTCStatus 355
4-326 ara::diag::DTCInformation::GetCurrentStatus 355
4-327 ara::diag::DTCInformation::GetNumberOfStoredEntries 356
4-328 ara::diag::DTCInformation::SetControlDtcStatusNotifier 357
4-329 ara::diag::DTCInformation::SetDTCStatusChangedNotifier 358
4-330 ara::diag::DTCInformation::SetNumberOfStoredEntriesNotifier 358
4-331 ara::diag::DTCInformation::SetSnapshotRecordUpdatedNotifier 359
4-332 ara::diag::DynamicAccessListDiagServiceBuilder 360
4-333 ara::diag::DynamicAccessListDiagServiceBuilder::
~DynamicAccessListDiagServiceBuilder 360
4-334 ara::diag::DynamicAccessListDiagServiceBuilder::ByteString 361
4-335 ara::diag::DynamicAccessListDiagServiceBuilder::Byte 361
4-336 ara::diag::DynamicAccessListDiagServiceBuilder::Add 362

based on template version 1.00.00

4-337 ara::diag::DynamicAccessListDiagServiceBuilder::Add 362
4-338 ara::diag::DynamicAccessListDiagServiceBuilder::Add 363
4-339 ara::diag::DynamicAccessListDiagServiceBuilder::Any 363
4-340 ara::diag::DynamicAccessListDiagServiceBuilder::Build 364
4-341 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith 365
4-342 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith 365
4-343 ara::diag::Event 366
4-344 ara::diag::Event::Event 367
4-345 ara::diag::Event::Event 367
4-346 ara::diag::Event::~Event 368
4-347 ara::diag::Event::operator= 368
4-348 ara::diag::Event::GetDTCNumber 369
4-349 ara::diag::Event::DTCFormatType 369
4-350 ara::diag::Event::EventStatusBit 369
4-351 ara::diag::Event::DebouncingState 370
4-352 ara::diag::Event::GetDebouncingStatus 370
4-353 ara::diag::Event::GetEventStatus 371
4-354 ara::diag::Event::GetFaultDetectionCounter 372
4-355 ara::diag::Event::GetLatchedWIRStatus 372
4-356 ara::diag::Event::GetTestComplete 373
4-357 ara::diag::Event::SetEventStatusChangedNotifier 374
4-358 ara::diag::Event::SetLatchedWIRStatus 374
4-359 ara::diag::ExternalAuthentication 375
4-360 ara::diag::ExternalAuthentication::ExternalAuthentication 375
4-361 ara::diag::ExternalAuthentication::ExternalAuthentication 376
4-362 ara::diag::ExternalAuthentication::~ExternalAuthentication 376
4-363 ara::diag::ExternalAuthentication::operator= 377
4-364 ara::diag::ExternalAuthentication::Address 377
4-365 ara::diag::ExternalAuthentication::Get 378
4-366 ara::diag::ExternalAuthentication::Get 378
4-367 ara::diag::ExternalAuthentication::GetAll 379
4-368 ara::diag::GenericDataIdentifier 380

4-369 ara::diag::GenericDataIdentifier::GenericDataIdentifier 380
4-370 ara::diag::GenericDataIdentifier::GenericDataIdentifier 381
4-371 ara::diag::GenericDataIdentifier::~GenericDataIdentifier 381
4-372 ara::diag::GenericDataIdentifier::operator= 382
4-373 ara::diag::GenericDataIdentifier::OperationOutput 382
4-374 ara::diag::GenericDataIdentifier::Offer 383
4-375 ara::diag::GenericDataIdentifier::Read 383
4-376 ara::diag::GenericDataIdentifier::StopOffer 384
4-377 ara::diag::GenericDataIdentifier::Write 385
4-378 ara::diag::GenericRoutine 386
4-379 ara::diag::GenericRoutine::GenericRoutine 387
4-380 ara::diag::GenericRoutine::GenericRoutine 387
4-381 ara::diag::GenericRoutine::~GenericRoutine 387
4-382 ara::diag::GenericRoutine::operator= 388
4-383 ara::diag::GenericRoutine::OperationOutput 388
4-384 ara::diag::GenericRoutine::Offer 389
4-385 ara::diag::GenericRoutine::RequestResults 390
4-386 ara::diag::GenericRoutine::Start 391
4-387 ara::diag::GenericRoutine::Stop 392
4-388 ara::diag::GenericRoutine::StopOffer 392
4-389 ara::diag::GenericUDSService 393
4-390 ara::diag::GenericUDSService::GenericUDSService 393
4-391 ara::diag::GenericUDSService::GenericUDSService 394
4-392 ara::diag::GenericUDSService::~GenericUDSService 394
4-393 ara::diag::GenericUDSService::operator= 395
4-394 ara::diag::GenericUDSService::OperationOutput 395
4-395 ara::diag::GenericUDSService::HandleMessage 396
4-396 ara::diag::GenericUDSService::Offer 396
4-397 ara::diag::GenericUDSService::StopOffer 397
4-398 ara::diag::Indicator 398
4-399 ara::diag::Indicator::Indicator 398
4-400 ara::diag::Indicator::Indicator 399

based on template version 1.00.00

4-401 ara::diag::Indicator::~Indicator 399
4-402 ara::diag::Indicator::operator= 400
4-403 ara::diag::Indicator::IndicatorType 400
4-404 ara::diag::Indicator::GetIndicator 401
4-405 ara::diag::Indicator::SetNotifier 402
4-406 ara::diag::MetaInfo 403
4-407 ara::diag::MetaInfo::MetaInfo 403
4-408 ara::diag::MetaInfo::~MetaInfo 404
4-409 ara::diag::MetaInfo::operator= 404
4-410 ara::diag::MetaInfo::GetValue 405
4-411 ara::diag::ReentrancyType 406
4-412 ara::diag::SecurityAccess 407
4-413 ara::diag::SecurityAccess::SecurityAccess 407
4-414 ara::diag::SecurityAccess::SecurityAccess 408
4-415 ara::diag::SecurityAccess::~SecurityAccess 408
4-416 ara::diag::SecurityAccess::operator= 409
4-417 ara::diag::SecurityAccess::KeyCompareResultType 409
4-418 ara::diag::SecurityAccess::CompareKey 410
4-419 ara::diag::SecurityAccess::GetSeed 411
4-420 ara::diag::SecurityAccess::Offer 411
4-421 ara::diag::SecurityAccess::StopOffer 412
4-422 ara::diag::ServiceValidation 413
4-423 ara::diag::ServiceValidation::ServiceValidation 413
4-424 ara::diag::ServiceValidation::ServiceValidation 414
4-425 ara::diag::ServiceValidation::~ServiceValidation 414
4-426 ara::diag::ServiceValidation::operator= 415
4-427 ara::diag::ServiceValidation::ConfirmationStatusType 415
4-428 ara::diag::ServiceValidation::Confirmation 416
4-429 ara::diag::ServiceValidation::Offer 417
4-430 ara::diag::ServiceValidation::StopOffer 417
4-431 ara::diag::ServiceValidation::Validate 418
4-432 ara::diag::UploadService 419

based on template version 1.00.00

4-433 ara::diag::UploadService::UploadService 419
4-434 ara::diag::UploadService::UploadService 420
4-435 ara::diag::UploadService::~UploadService 420
4-436 ara::diag::UploadService::operator= 421
4-437 ara::diag::UploadService::OperationOutput 421
4-438 ara::diag::UploadService::Offer 422
4-439 ara::diag::UploadService::RequestUpload 422
4-440 ara::diag::UploadService::RequestUploadExit 423
4-441 ara::diag::UploadService::StopOffer 424
4-442 ara::diag::UploadService::UploadData 425
4-443 amsr::diag::AddIsReadyNotifier 426
4-444 amsr::diag::AddIsReadyNotifier 427
4-445 amsr::diag::IsReady 428
4-446 amsr::diag::SetIsReadyNotifier 429
4-447 amsr::diag::SetIsReadyNotifier 429
4-448 amsr::diag::AsUdsDtcStatusByte 430
4-449 amsr::diag::conversation::OverrideS3Timeout 431
4-450 431
4-451 <DataElementPortInterfaceName> 432
4-452 DataElement::Output 432
4-453 <DataElementPortInterfaceName>::<DataElementPortInterfaceName> 433
4-454 <DataElementPortInterfaceName>::~<DataElementPortInterfaceName> 433
4-455 <DataElementPortInterfaceName>::Offer 433
4-456 <DataElementPortInterfaceName>::Read 434
4-457 <DataElementPortInterfaceName>::StopOffer 435
4-458 <DataIdentifierPortInterfaceName> 436
4-459 <DataIdentifierPortInterfaceName>::Output 436
4-460 <DataIdentifierPortInterfaceName> constructor 437
4-461 <DataIdentifierPortInterfaceName>::~<DataIdentifierPortInterfaceName> 437
4-462 <DataIdentifierPortInterfaceName>::Read 438
4-463 <DataIdentifierPortInterfaceName>::Write 439
4-464 <DataIdentifierPortInterfaceName>::Offer 439

based on template version 1.00.00

4-465 <DataIdentifierPortInterfaceName>::StopOffer 440
4-466 <RoutinePortInterfaceName>::<RoutinePortInterfaceName> 441
4-467 <RoutinePortInterfaceName>::StartOutput 441
4-468 <RoutinePortInterfaceName>::StopOutput 442
4-469 <RoutinePortInterfaceName>::RequestResultsOutput 442
4-470 <RoutinePortInterfaceName>::<RoutinePortInterfaceName> 443
4-471 <RoutinePortInterfaceName>::~<RoutinePortInterfaceName> 443
4-472 <RoutinePortInterfaceName>::Start 444
4-473 <RoutinePortInterfaceName>::Stop 445
4-474 <RoutinePortInterfaceName>::RequestResults 445
4-475 <RoutinePortInterfaceName>::Offer 446
4-476 <RoutinePortInterfaceName>::StopOffer 446

5-1 amsr::uds_transport::SecuredDataTransmissionHandler 448
5-2 amsr::uds_transport::SecuredDataTransmissionHandler::
SecuredDataTransmissionHandler 448
5-3 amsr::uds_transport::SecuredDataTransmissionHandler::
SecuredDataTransmissionHandler 449
5-4 amsr::uds_transport::SecuredDataTransmissionHandler::
SecuredDataTransmissionHandler 449
5-5 amsr::uds_transport::SecuredDataTransmissionHandler::
~SecuredDataTransmissionHandler 450
5-6 amsr::uds_transport::SecuredDataTransmissionHandler::GetServiceValidator 451
5-7 amsr::uds_transport::SecuredDataTransmissionHandler::HandleRequest 451
5-8 amsr::uds_transport::SecuredDataTransmissionHandler::HandleResponse 452
5-9 amsr::uds_transport::SecuredDataTransmissionHandler::Initialize 453
5-10 amsr::uds_transport::SecuredDataTransmissionHandler::Shutdown 453
5-11 amsr::uds_transport::AdministrativeParameter 454
5-12 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 455
5-13 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 455
5-14 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 456
5-15 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter 456
5-16 amsr::uds_transport::AdministrativeParameter::~AdministrativeParameter 456
5-17 amsr::uds_transport::AdministrativeParameter::operator= 457

5-18 amsr::uds_transport::AdministrativeParameter::operator= 458
5-19 amsr::uds_transport::AdministrativeParameter::GetValue 458
5-20 amsr::uds_transport::AdministrativeParameter::IsMessageEncrypted 459
5-21 amsr::uds_transport::AdministrativeParameter::IsMessageSigned 459
5-22 amsr::uds_transport::AdministrativeParameter::IsRequestMessage 460
5-23 amsr::uds_transport::AdministrativeParameter::IsPreEstablishedKeyUsed 460
5-24 amsr::uds_transport::AdministrativeParameter::IsSignatureOnResponseRequested 461
5-25 amsr::uds_transport::AdministrativeParameter::SetMessageEncrypted 461
5-26 amsr::uds_transport::AdministrativeParameter::SetMessageSigned 462
5-27 amsr::uds_transport::AdministrativeParameter::SetRequestMessage 462
5-28 amsr::uds_transport::AdministrativeParameter::SetPreEstablishedKeyUsed 463
5-29 amsr::uds_transport::AdministrativeParameter::SetSignatureOnResponseRequested 463
5-30 amsr::uds_transport::AdministrativeParameter::SetValue 464
5-31 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage 465
5-32 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: SecuredDataTransmissionEnclosedMessage 466
5-33 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: SecuredDataTransmissionEnclosedMessage 466
5-34 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: ~SecuredDataTransmissionEnclosedMessage 467
5-35 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= 467
5-36 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= 468
5-37 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::Create 469
5-38 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: GetAntiReplayCounter 469
5-39 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: GetInternalMessage 470
5-40 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: GetInternalMessage 470
5-41 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload . . 471
5-42 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload . . 471
5-43 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature 472
5-44 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature 472

based on template version 1.00.00

5-45 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SetAntiReplayCounter 473
5-46 amsr::uds_transport::SecuredDataTransmissionRequestMessage 474
5-47 amsr::uds_transport::SecuredDataTransmissionRequestMessage::SecuredDataTransmissionRequestMessage 474
5-48 amsr::uds_transport::SecuredDataTransmissionRequestMessage::~SecuredDataTransmissionRequestMessage 475
5-49 amsr::uds_transport::SecuredDataTransmissionRequestMessage::operator= 475
5-50 amsr::uds_transport::SecuredDataTransmissionRequestMessage::AddMetaInfoValue 476
5-51 amsr::uds_transport::SecuredDataTransmissionRequestMessage::Create 477
5-52 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetAdministrativeParameters 477
5-53 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetClientAddress 478
5-54 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMessage . . 478
5-55 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMetaInfoValue 479
5-56 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetPayload . . . 479
5-57 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetSignEncryptCalculation 480
5-58 amsr::uds_transport::SecuredDataTransmissionResponseMessage 481
5-59 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SecuredDataTransmissionResponseMessage 482
5-60 amsr::uds_transport::SecuredDataTransmissionResponseMessage::~SecuredDataTransmissionResponseMessage 482
5-61 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AddMetaInfoValue 483
5-62 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage 483
5-63 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage 484
5-64 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Create 485
5-65 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Finalize 485
5-66 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetAdministrativeParameters 486
5-67 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetClientAddress 486

based on template version 1.00.00

5-68 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetMetaInfoValue 487
5-69 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: GetSignEncryptCalculation 488
5-70 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: SetAdministrativeParameters 488
5-71 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: SetSignEncryptCalculation 489
5-72 ara::diag::uds_transport::UdsTransportProtocolHandler 490
5-73 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 491
5-74 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 491
5-75 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler 492
5-76 ara::diag::uds_transport::UdsTransportProtocolHandler:: ~UdsTransportProtocolHandler 492
5-77 ara::diag::uds_transport::UdsTransportProtocolHandler::GetHandlerID 493
5-78 ara::diag::uds_transport::UdsTransportProtocolHandler::GetPeriodicHandler 493
5-79 ara::diag::uds_transport::UdsTransportProtocolHandler::Initialize 494
5-80 ara::diag::uds_transport::UdsTransportProtocolHandler::NotifyReestablishment . . 495
5-81 ara::diag::uds_transport::UdsTransportProtocolHandler::Start 495
5-82 ara::diag::uds_transport::UdsTransportProtocolHandler::Stop 496
5-83 ara::diag::uds_transport::UdsTransportProtocolHandler::StopConnection 497
5-84 ara::diag::uds_transport::UdsTransportProtocolHandler::Transmit 498
5-85 ara::diag::uds_transport::UdsTransportProtocolMgr 499
5-86 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr . . . 500
5-87 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr . . . 500
5-88 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr . . . 501
5-89 ara::diag::uds_transport::UdsTransportProtocolMgr::~UdsTransportProtocolMgr . . 501
5-90 ara::diag::uds_transport::UdsTransportProtocolMgr::operator= 501
5-91 ara::diag::uds_transport::UdsTransportProtocolMgr::operator= 502
5-92 ara::diag::uds_transport::UdsTransportProtocolMgr::ChannelReestablished 503
5-93 ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage 503
5-94 ara::diag::uds_transport::UdsTransportProtocolMgr::HandlerStopped 504

5-95 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage 505
5-96 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage 506
5-97 ara::diag::uds_transport::UdsTransportProtocolMgr::NotifyMessageFailure 507
5-98 ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation 507
5-99 ara::diag::uds_transport::UdsTransportProtocolMgr::TransmitConfirmation 508
5-100 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler 509
5-101 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
UdsTransportProtocolPeriodicHandler 510
5-102 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
UdsTransportProtocolPeriodicHandler 510
5-103 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
UdsTransportProtocolPeriodicHandler 511
5-104 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
~UdsTransportProtocolPeriodicHandler 511
5-105 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= 512
5-106 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= 512
5-107 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
GetMaxPayloadLength 513
5-108 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
GetNumberOfPeriodicMessages 513
5-109 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::PeriodicTransmit 514
5-110 ara::diag::uds_transport::UdsMessage 515
5-111 ara::diag::uds_transport::UdsMessage::UdsMessage 516
5-112 ara::diag::uds_transport::UdsMessage::UdsMessage 516
5-113 ara::diag::uds_transport::UdsMessage::~UdsMessage 517
5-114 ara::diag::uds_transport::UdsMessage::operator= 517
5-115 ara::diag::uds_transport::UdsMessage::operator= 518
5-116 ara::diag::uds_transport::UdsMessage::AddMetaInfo 518
5-117 ara::diag::uds_transport::UdsMessage::AddMetaInfo 519
5-118 ara::diag::uds_transport::UdsMessage::Capacity 519
5-119 ara::diag::uds_transport::UdsMessage::ChangePayloadView 520
5-120 ara::diag::uds_transport::UdsMessage::GetGlobalChannelIdentifier 521
5-121 ara::diag::uds_transport::UdsMessage::GetPayload 521
5-122 ara::diag::uds_transport::UdsMessage::GetPayload 522

based on template version 1.00.00

5-123 ara::diag::uds_transport::UdsMessage::GetSa 522
5-124 ara::diag::uds_transport::UdsMessage::GetTa 523
5-125 ara::diag::uds_transport::UdsMessage::GetTaType 523
5-126 amsr::uds_transport::CancellationHandler 524
5-127 amsr::uds_transport::CancellationHandler::CancellationHandler 524
5-128 amsr::uds_transport::CancellationHandler::~CancellationHandler 525
5-129 amsr::uds_transport::CancellationHandler::operator= 525
5-130 amsr::uds_transport::CancellationHandler::IsCanceled 526
5-131 amsr::uds_transport::CancellationHandler::SetNotifier 527
5-132 amsr::uds_transport::MetaInfo 528
5-133 amsr::uds_transport::MetaInfo::MetaInfo 528
5-134 amsr::uds_transport::MetaInfo::~MetaInfo 529
5-135 amsr::uds_transport::MetaInfo::operator= 529
5-136 amsr::uds_transport::MetaInfo::GetValue 530
5-137 amsr::uds_transport::ServiceValidation 531
5-138 amsr::uds_transport::ServiceValidation::ServiceValidation 531
5-139 amsr::uds_transport::ServiceValidation::ServiceValidation 532
5-140 amsr::uds_transport::ServiceValidation::ServiceValidation 532
5-141 amsr::uds_transport::ServiceValidation::~ServiceValidation 533
5-142 amsr::uds_transport::ServiceValidation::Confirmation 533
5-143 amsr::uds_transport::ServiceValidation::Validate 534
5-144 amsr::uds_transport::ServiceValidation::ValidationError 535
5-145 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 535
5-146 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 536
5-147 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 536
5-148 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError 537
5-149 amsr::uds_transport::ServiceValidation::ValidationError::~ValidationError 537
5-150 amsr::uds_transport::ServiceValidation::ValidationError::operator= 538
5-151 amsr::uds_transport::ServiceValidation::ValidationError::operator= 538
5-152 amsr::uds_transport::ServiceValidation::ValidationError::GetError 539

6-1 Description of Diagnostics Library (ara::diag) executable's MEX parameter 541

based on template version 1.00.00

6-2 Description of Diagnostics Library (ara::diag) executable's memory management MEX parameter ... 541
6-3 Description of MEX parameter in Diagnostics Daemon ... 544
6-4 Supported Data Types ... 547
6-5 Supported DiagnosticSwMappings ... 552
6-6 ExecutionMode JSON parameter description ... 553
6-7 Logging JSON parameter description ... 555
6-8 NvData JSON parameter description ... 557
6-9 Fault Memory File Handling JSON parameter description ... 558
6-10 DextFiles JSON parameter description ... 560
6-11 Value constraints of request execution related parameters ... 561
6-12 SOVD module instantiation MEX parameters ... 587
6-13 Supported SOVD Resource Collections ... 597

7-1 Cybersecurity-relevant Dependencies for Runtime Interfaces of DoIP ... 609

Listings

2-1 Examples of DoIP array configuration ... 68
2-2 AccessPermissions ... 77
2-3 No Access Permissions ... 77
2-4 Data-Driven Preconditions ... 78
2-5 Definition of an Environmental Condition ... 79
2-6 Definition of a Data Identifier ... 80
2-7 Extending Diagnostics Daemon model with Authentication Roles ... 80
2-8 Extending DIAGNOSTIC-ACCESS-PERMISSION with references to Authentication Roles ... 81
2-9 Extending DIAGNOSTIC-ACCESS-PERMISSION with authentication relevancy ... 81
2-10 Configuring the s3ServerTimeout in seconds ... 84
2-11 Definition of a DiagnosticSession ... 85
2-12 Extending Diagnostics Daemon model with maximum number change of data identifier events ... 101
2-13 `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` as a sub-function of a custom service ... 102
2-14 Internal DataMapping ... 107

2-15 Snapshot Record Enumeration ... 111
2-16 Snapshot Content ... 111
2-17 Maximum number of calculated Snapshot Record ... 112
2-18 Configured Snapshot Record ... 112
2-19 Extended Data Record ... 113
2-20 Extended Data Record Property ... 113
2-21 Example to enable RPC intensity measurement ... 121
2-22 Configuration of a base type byte order ... 124
3-1 AdminData for Generation of Code to Initialize and Deinitialize ARA Libraries ... 140
3-2 Diagnostics Daemon Port Prototype Configuration ... 140
3-3 Configuring DM Executable to use DoIP Transport Protocol ... 141
3-4 Configuring DM Executable to use TLS ... 142
3-5 Signature of constructor of a Transport Protocol handler ... 142
3-6 Signature of a custom function to create a Transport Protocol Handler ... 143
3-7 Extending the DM daemon model with Transport Protocol information ... 143
3-8 Signature of constructor of a SecureDataTransmission handler ... 144
3-9 Signature of a custom function to create a SecureDataTransmission Handler ... 145
3-10 Secured Data Transmission creator ... 145
3-11 Secured Data Transmission executable reference ... 146
6-1 Software Cluster ... 545
6-2 MEX data in SoftwareCluster ... 546
6-3 MEX data in SoftwareCluster (continued) ... 547
6-4 DataIdentifier Mapping ... 549
6-5 GenericUDS Mapping ... 550
6-6 Internal DataElement Mapping ... 550
6-7 Example definition of ExecutionMode where service validation will return NRC `0x10` if the responsible application is not connected yet ... 554
6-8 Example definition of logging for Diagnostics Daemon ... 556
6-9 Request execution related AdminData on AraDiag Executable ... 561
6-10 `DIAGNOSTIC-CUSTOM-SERVICE-CLASS` as Authentication class ... 562
6-11 `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` as Authentication sub-function ... 562
6-12 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with `P-PORT`s for authentication service ... 563

6-13 Extending SWC-SERVICE-DEPENDENCY with reference to P-Port extension 564
6-14 Extending SOFTWARE-CLUSTER with client authentication identifications 565
6-15 Suppress precondition evaluation configuration 566
6-16 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPorts for external client authentication 567
6-17 Extending SWC-SERVICE-DEPENDENCY with rPortRef for external client authentication 567
6-18 Failure event associated identification 568
6-19 Failure event associated identification mapping 568
6-20 Failure associated identification data element type 569
6-21 `SECURITY-EVENT-DEFINITION` as security event type 570
6-22 `SECURITY-EVENT-CONTEXT-PROPS` as security event 571
6-23 Secuirty event log `SWC-SERVICE-DEPENDENCY` configuration 572
6-24 Security event log `R-PORT-PROTOTYPE` configuration 572
6-25 Security event `SECURITY-EVENT-MAPPING` configuration 573
6-26 Security event `SWC-SERVICE-DEPENDENCY` configuration 573
6-27 Security event `R-PORT-PROTOTYPE` configuration 574
6-28 IDSR local IP address credential configuration 574
6-29 Extending SOFTWARE-CLUSTER with client authentication identifications 575
6-30 IDSR remote endpoint configuration - Network endpoint 575
6-31 Service processing notification `SWC-SERVICE-DEPENDENCY` configuration 576
6-32 Service processing notification `P-PORT-PROTOTYPE` configuration 577
6-33 `DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS` configuration 577
6-34 `DIAGNOSTIC-REQUEST-FILE-TRANSFER` configuration 577
6-35 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with `P-PORT`s for `DIAGNOSTIC-REQUEST-FILE-TRANSFER` 578
6-36 SWC-SERVICE-DEPENDENCY configuration for `DIAGNOSTIC-REQUEST-FILE-TRANSFER` 579
6-37 Root cause event associated identification 579
6-38 Root cause event associated identification mapping 580
6-39 Root cause event associated identification base type 580
6-40 Indicator Configuration 581
6-41 Indicator Configuration 581
6-42 DoIP activation line port 582

based on template version 1.00.00

6-43 DoIP activation line mapping 582
6-44 DoIP group identification port 583
6-45 DoIP group identification mapping 583
6-46 DoIP vehicle group identification data identifier 584
6-47 Secured Data Transmission properties 585
6-48 Example of SOVD module instantiation 587
6-49 Example of transport layer security configuration 588
6-50 Extending SOFTWARE-CLUSTER with client authentication identifications 589
6-51 Example of `TlsSecureComProps` configuration 589
6-52 Example of key/certificate configuration 590
6-53 IDSR remote endpoint configuration - Network endpoint 591
6-54 Example of SOVD server parameters 592
6-55 Example of SOVD method 592
6-56 Example of SOVD generic method 593
6-57 Example of SOVD generic method mapping 594
6-58 Example of SOVD generic method SWC service dependency 594
6-59 Example of SOVD generic method port 595
6-60 Example of SOVD service instance 596
6-61 Example of SOVD generic service instance 596
6-62 Example of SOVD authorization 597
6-63 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with PPort for SOVD authorization 598
6-64 Extending SWC-SERVICE-DEPENDENCY with sovdPPortRef for SOVD Authorization 598
6-65 Example of SOVD lock 599
6-66 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPort for SOVD lock 600
6-67 Extending SWC-SERVICE-DEPENDENCY with sovdRPortRef for SOVD lock 600
6-68 Extened data identifier parameters 601
6-69 Long name configuration 601
6-70 Memory Resource Props configuration 602
6-71 Memory Resource configuration 603
6-72 UDS NRC Mapping Configuration 604

based on template version 1.00.00

6-73 TSSR profile configuration 605
6-74 TSSR configuration of DTC 606
6-75 Configuration of snapshot overflow update strategy in primary memory 607
6-76 Configuration of snapshot overflow update strategy in user defined memory 607

based on template version 1.00.00

1 Introduction

This document describes the functionality, API and configuration of the Adaptive AUTOSAR BSW module DM as specified in [5].

> Latest analyzed AUTOSAR Release: **R20-11**
> AUTOSAR Schema Compatibility: **AUTOSAR_00049.xsd**

The DM is responsible for all aspects related to diagnostic functionality. It provides a self-contained Adaptive Platform Application, whose start should be triggered by the Execution Manager (ara::em).

1.1 Architectural Overview

This document contains information on the BSW module DM. For full diagnostic functionality, at least the following components (which are all described in this document) are needed:

> amsr-vector-fs-aradiag
> amsr-vector-fs-diagnosticmanager
> amsr-vector-fs-udstransport

In addition to that, at least one component that represents an instance of the UdsTransport layer is required. One such component that is also described in this document is:

> amsr-vector-fs-doipbinding

This component, however, can also be replaced by a custom implementation of the UdsTransport-API. AMSR provides in addition to the **amsr-vector-fs-doipbinding** also the following alternatives:

> amsr-vector-fs-pduhdrtpbinding - enables communication over TCP/IP (for details please refer to [27])

1.1.1 Dependencies of the DM

In addition to the inherent dependencies between the different components that the DM consists of as described in 1.1, there is a logical dependency of the DM on the the AMSR Basic Communication Protocol which is used to implement the communication between the DM and the Adaptive Platform Applications.

The components included in listings 1.1 have direct dependencies to the following AUTOSAR and AMSR modules (for dependencies related to Idsm, see [23]):

> **amsr-vector-fs-applicationbase**:
Provides the global initialization and shutdown functions (please refer to [15]).
> **amsr-vector-fs-aracrypto**:
Used for secure communication (please refer to [18], consider the note below about runtime dependencies).
> **amsr-vector-fs-araidsm**:
Used for system intrusion detection (please refer to [23]).
> **amsr-vector-fs-characterconversion**:
Provides text-to-number and number-to-text conversions (please refer to [33]).
> **amsr-vector-fs-crc**:
Used to calculate a cyclic redundancy check (crc) sum when communication through http (please refer to [17]).
> **amsr-vector-fs-diagnosticrpc**:
Provides diagnostic internal communication if the communication between DM with Adaptive Platform Applications happens via the Basic Communication Protocol.

based on template version 1.00.00

- **amsr-vector-fs-diagnosticrpcbcpbinding**:
  Provides diagnostic internal communication if the communication between DM with Adaptive Platform Applications happens via the Basic Communication Protocol.
- **amsr-vector-fs-diagnosticutility**:
  Provides diagnostic internal utilities.
- **amsr-vector-fs-em-executionmanager**:
  The Diagnostics Daemon reports the current execution state to the Execution Manager (ara::em) (please refer to [20]).
- **amsr-vector-fs-iointegritystream**:
  Provides input/output operations on stream objects (please refer to [35]).
- **amsr-vector-fs-libiostream**:
  Provides functionality for using input and output streams (please refer to [34]).
- **amsr-vector-fs-libosabstraction**:
  Platform abstraction layer (please refer to [26]).
- **amsr-vector-fs-libvac**:
  Implementation of containers and helper classes that are used by all parts of AMSR (please refer to [31]).
- **amsr-vector-fs-log-api**:
  Used for logging debug information (please refer to [14]).
- **amsr-vector-fs-per-libpersistency**:
  Used to store persistent information such as fault data (please refer to [28]).
- **amsr-vector-fs-sec-cryptostack**:
  Used for secure communication (please refer to [18]).
- **amsr-vector-fs-sec-libseccom**:
  Used to secure messages with the TLS and DTLS protocols (please refer to [29]).
- **amsr-vector-fs-thread**:
  Provides thread functionality (please refer to [36]).
- **amsr-vector-fs-vajson**:
  Used for parsing JSON structures (please refer to [32]).
- **amsr-vector-fs-zerocopycommon**:
  Provides communication between two processes on the same processor without copying.
- **vmsr-httpcommon**:
  Used for validation, parsing and normalization of URL and HTTP (please refer to [21]).
- **vmsr-httpserver**:
  Used for receiving http requests in the context of SOVD (please refer to [22]).

To learn more about runtime dependencies, please refer to 3.4.

1.1.2 Design of the DM

The DM is a composition of the following parts:

- A Diagnostics Daemon handling the UDS protocol and SOVD requests for the whole machine
- The Diagnostics Library (ara::diag). Application code can only interact with diagnostics using the API provided here.
- The SOVD. Application code can only interact with diagnostics using the API provided here.
- The Intrusion Detection System Manager Library (ara::idsm). Application code can only interact with DM using the API in [23]. Intrusion Detection System Manager Library (ara::idsm) can either be run in the same process as Diagnostics Library (ara::diag) or separately.

The Diagnostics Daemon proivides a standardized extension API to support arbitrary communication

based on template version 1.00.00

protocols (see [5, chapter 8.1] for further details). We provide the following protocol implementations:

> DoIP - ISO13400 compatible protocol (see 2.1.1.3)
> PDU Header TP - Communication protocol based on Classic AUTOSAR PduID Header (see [27])

1.1.2.1 Interaction with the DM

A tester application that is connected to an ECU running the Adaptive AUTOSAR stack including a Diagnostics Daemon will be able to retrieve diagnostic information e.g. by using DoIP [38]. Figure 1-1 shows a possible setup.

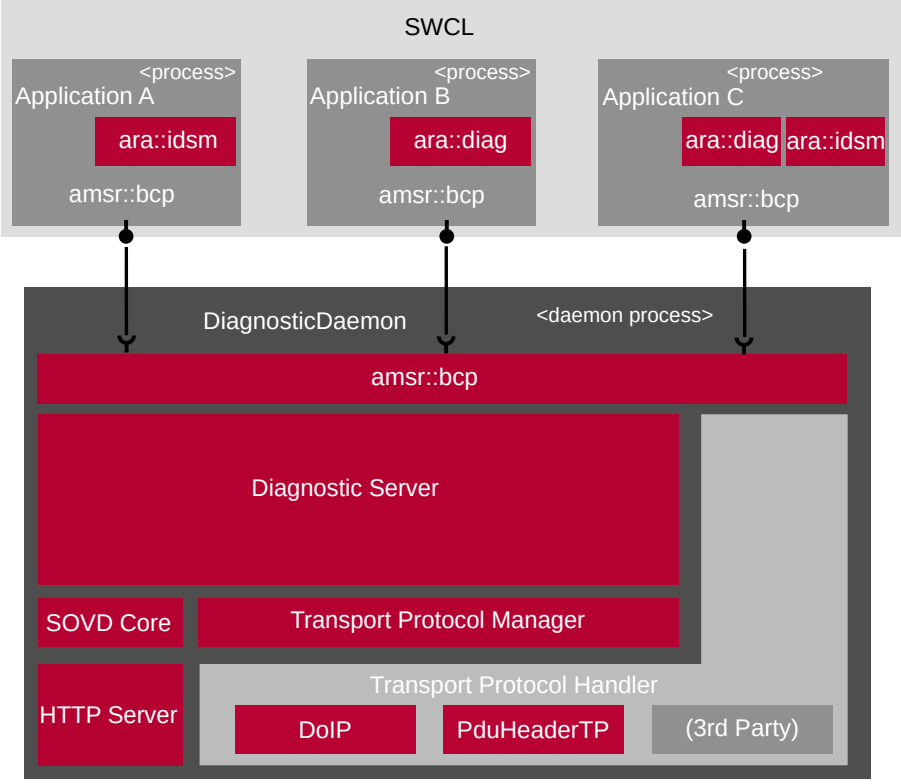

Figure 1-1 Architecture overview

Some diagnostic functionality is handled internally inside the Diagnostics Daemon, e.g. Session Control. For other functionality, the Diagnostics Daemon communicates with adaptive platform diagnostic applications via the Basic Communication Protocol (for details see [5]). For Idsm functionality, the Diagnostics Daemon interacts with other Idsm related Adaptive Platform Applications. The communication takes place via Basic Communication Protocol.

Generally speaking, standardized extension points have to be implemented by the DM and Adaptive Platform Applications.

1.1.2.2 Scope of Diagnostics: Software Cluster

Adaptive Applications are grouped into so called Software Clusters. Each Software Cluster represents an independent diagnostic entity with its own diagnostic address. Diagnostics of one Software Cluster is logically independent of diagnostics of other Software Clusters. This applies to the configuration of diagnostics as well as to the processing of UDS requests and handling of the diagnostic fault memory. The Diagnostics Daemon provides a dedicated Diagnostic Server Instance per installed Software Cluster. All Diagnostic Server Instances share a single Transport Layer instance (e.g. DoIP on TCP/IP port 13400). The Diagnostics Daemon dispatches incoming UDS requests towards Software Clusters according to the received target address, and forwards the request to the responsible Diagnostic Server Instance.

based on template version 1.00.00

2 Functional Description

MICROSAR DM is responsible for all aspects related to diagnostic functionality, especially diagnostic service handling and event memory management.

Referring to the AUTOSAR standard functionality as specified in [5], section 2.1 provides a list of the implemented features. Deviations of the implementation and the AUTOSAR standard are described in section 2.2. The corresponding features are listed in the following tables:

> For supported AUTOSAR Features (see table 2-1)

> For unsupported AUTOSAR Features (see table 2-42)

Vector Informatik provides further DM functionality that extends the AUTOSAR standard. A list of these features is given in section 2.3.

Known limitations of the current implementation of the module are listed in section 2.4.

MICROSAR DM is also responsible for all aspects related to the Idsm security events, especially their reporting, qualification and management. In the Idsm technical reference [23], the supported and unsupported features specified in AUTOSAR Idsm specifications [9] are listed. In the same documentation, the deviations and addional Idsm features are also described.

based on template version 1.00.00

2.1 Features

The features listed in this chapter cover the complete functionality supported by the DM. Table 2-1 provides an overview of supported features that are specified by [5].

For further information about not supported features or deviations see also section 2.4.

2.1.1 Transport Protocol

This section describes the transport protocols that are supported by the DM. As built-in feature, Diagnostics Daemon supports the DoIP protocol – see section 2.1.1.3 for details. Alternatively, the Diagnostics Daemon supports linking to a custom transport protocol based on the transport protocol API described in section 2.1.1.4.

How to select between the different transport protocols is described in section 2.1.1.1.

2.1.1.1 Transport Protocol Initialization

Diagnostics Daemon can support multiple Transport Protocols. The instantiation of the Transport Protocols is initiated by Diagnostics Daemon. To perform this task, Diagnostics Daemon requires a list of creator functions which instantiate the Transport Protocol handlers. This list of creator functions of Transport Protocol handlers should be modeled as AdminData configuration of the Transport Protocols (for details see section 3.3.1). DM will create and initialize the configured Transport Protocol handlers during the initialization phase.

2.1.1.2 Start and Stop of a Transport Protocol handler

Beginning with the call of the `Start` method on any Transport Protocol handler, the Diagnostics Daemon is prepared to receive requests from diagnostic clients.

Caution

The `Start` and `Stop` methods will be called even though the prior `Initialization` call of the Transport Protocol handler returns a failing response. All other transactions will be blocked for this handler by the Transport Protocol manager.

Caution

Prior to the `Start` call, a Transport Protocol handler is not allowed to call any of the AUTOSAR specified APIs of the Transport Protocol manager.

On shutdown, the Diagnostics Daemon calls the `Stop` method on each Transport Protocol handler. With this call, the Diagnostics Daemon guarantees that no further transmission is triggered on the respective Transport Protocol handler. The Diagnostics Daemon expects the Transport Protocol handlers to call the `HandlerStopped` method on the Transport Protocol manager when the Transport Protocol handler has completed its shut down triggered by the `Stop` call. Before calling `HandlerStopped` the Transport Protocol handler must ensure that:

> all UdsMessage variables that were handed over to the corresponding `ara::diag::uds_transport::UdsTransportProtocolHandler` via the `IndicateMessage(...)` have been destructed or handed back to DM by calling either `ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage` or `ara::diag::uds_transport::NotifyMessageFailure`

> all UdsMessage variables that were handed over to the corresponding `ara::diag::uds_transport::UdsTransportProtocolHandler` via the `Transmit(...)` method have been destructed or handed back to DM by calling `ara::diag::uds_transport::UdsTransportProtocolMgr::Tran`

based on template version 1.00.00

Feature	Description
DoIP Transport Layer	See section 2.1.1.3
Custom Transport Layer	See section 2.1.1.4
Dispatching of UDS Requests	See section 2.1.1.5
Diagnostic Servers	See section 2.1.2
Parallel Diagnostic Conversations	See section 2.1.3
UDS Message Processing	See section 2.1.4
SOVD Request Validation and Verification	See section 2.1.6
SOVD Request Processing	See section 2.1.7
Diagnostic Data Management	See sections 2.1.8.1 and 2.1.8.2
Memory Destinations	See section 6.3
Event Reporting	Via monitors, see section 2.1.10.2
DTC Status Processing	See section 2.1.9.1
Monitors	See section 2.1.10.2
Operation Cycles	See section 2.1.11
Indicators	See section 2.1.13
Event Debouncing	See section 2.1.10.4
DTC ExtendedDataRecords	See section 2.1.15
DTC SnapshotDataRecords	See section 2.1.14
DTC Aging	See section 2.1.17
Clearing DTCs	See section 2.1.16
Enable Conditions	See section 2.1.18
Storage Conditions	See section 2.1.19
Clear Conditions	See section 2.1.16.1
Client Authentication (Authorization)	See section 2.1.20
Security Event Log	See section 2.1.21
Daemon alive check	See section 2.1.22
Log Api Daemon interaction	See section 2.1.23
Root Cause Event Associated Identification	See section 2.1.24
Caching of application call	See section 2.1.26

Table 2-1 Supported AUTOSAR Conform Features

based on template version 1.00.00

smitConfirmation

> all UdsMessage variables that were handed over to the corresponding `ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler` via the `PeriodicTransmit(...)` method have been destructed or handed back to DM by calling `ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation`

2.1.1.3 DoIP Transport Protocol

The DoIP transport protocol is a protocol to transport diagnostics data over Ethernet from the diagnostic client to the ECU and vice versa. According to [5], the DoIP transport protocol is implemented in accordance with [39]. The respective subcomponent of the Diagnostics Daemon is further referred to as DoIP subcomponent. Table 2-2 lists the supported features.

Feature	Description
Socket Handling	Configurable resources for communication via UDP and TCP sockets, see section 2.1.1.3.2
Multiple Networks	Support for handling connections coming from multiple networks, see section 2.1.1.3.3
Activation Line	Support network activation with ara::diag::DoIPActivationLine, see section 2.1.1.3.4
Message Validation	Validation of incoming DoIP messages
Vehicle Identification	Procedure to detect DoIP entities
Node Information	Requests the current status of a DoIP entity
Routing Activation	Procedure to register a diagnostic client to a DoIP entity
Diagnostic Messages	Forwarding diagnostic messages from a diagnostic client to a ECU and vice versa
DHCP	Support for dynamically assigning IP addresses to configured networks, see section 2.1.1.3.5
TLS	Establishing an authenticated and encrypted communication channel, see section 2.1.1.3.6
Alive Check	Support replacing dead connections if conflicts in source address or max socket count arise, see section 2.1.1.3.7

Table 2-2 Supported DoIP Conform Features

2.1.1.3.1 ISO Deviations

The deviations from [39] are:

> The features of a DoIP gateway or a DoIP edge node are not supported.

> Features for Authentication and Confirmation are not supported.
Authentication and Confirmation checks for the Routing Activation Handler as specified in [39, figure 22] are treated as if 'No Authentication/Confirmation is required'.

> Memory handling of DoIP generic header handler differs from what is specified in [39, figure 16] and by [39, DoIP-044].
If pre-allocated memory is insufficient for the payload of a DoIP message, memory will be allocated after the payload type specific length has been checked. If the payload length is valid for the payload type, but sufficient memory cannot be allocated, the program will abort without sending a NACK. Note that this is different from [39, DoIP-072], which will always be fulfilled.

> For incoming DoIP Server UDP/TCP messages, the server shall ignore messages with payload

based on template version 1.00.00

type `0x0000` (NACK) and `0x0004` (VAM), regardless if the protocol version given in the DoIP header is supported by the server.
This behavior is meant to avoid undesirable NACK responses to a DoIP entity that uses a different protocol version in order to prevent NACK responses from being bounced back and forth.

2.1.1.3.2 Socket Handling

Communication between a diagnostic client and the DoIP subcomponent is handled via UDP and TCP sockets. On startup, the DoIP subcomponent opens a socket for UDP communication and a TCP socket listening for connection requests from one or multiple diagnostic clients. The IP addresses used for these sockets are configurable in the doip configuration file (`diagnosticmanager_doip.json`) or configure them with DHCP, See section 2.1.1.3.5. In accordance with [38], the port number is fixed to port 13400. The DoIP subcomponent supports selection of internet protocol versions IPv4 or IPv6.

UDP communication The DoIP subcomponent supports the UDP requests given in table 2-3.

Payload type value	Payload type name	Description
`0x0001`	Vehicle identification request message	A request without EID or VIN see [38]
`0x0002`	Vehicle identification request message with EID	For payload type value 0x0001 see [38]
`0x0003`	Vehicle identification request message with VIN	For payload type value 0x0001 see [38]
`0x4001`	DoIP entity status request	see [38]
`0x4003`	Power mode information request	see [38], will report the value returned by the application. It will report the initial value configured in Table 2-5 when the application is not deployed or still not connected. If no application is deployed and no initial value is configured, then NotSupported is sent. If application is deployed but yet not connected and no initial value is configured, then NotReady will be sent.

Table 2-3 Supported UDP Requests

TCP communication The listening TCP socket accepts one connection for each channel (configured by the value of `MaxChannels` in the DoIP JSON file) plus one additional connection. After establishing a connection to the DoIP subcomponent, the Diagnostic Client is expected to send a Routing Activation request (see [38]) within the Initial Inactivity Time of 2 seconds.

After successful Routing Activation, the Diagnostic Client is able to send Diagnostic Messages within the General Inactivity Time of 5 minutes.

If Routing Activation fails or either of the Inactivity Timeouts occurs, the TCP socket connection is closed.

based on template version 1.00.00

2.1.1.3.3 Multiple Networks

DoIP supports the capability of handling connections from multiple clients across multiple network interfaces, allowing for diagnostic communication via onboard and offboard testers as specified in [38]. Each network will handle connections as specified in table 2-3, however the total number of available channels is a global setting. One internal network (the primary network) is always available and required during ECU startup, any other networks (the temporary networks) are optional. Refer to section 2.1.1.3.8 for configuration instructions.

2.1.1.3.4 Activation Line

DoIP supports notification from ara::diag::DoIPActivationLine that will activate or shut down networks [5].

The Activation Line feature is activated in the DoIP configuration file with the keyword `ActivationLine` inside the `FeatureFlags` array. Only then will the configuration parameters `NetworkInterfaceId` and `IsActivationLineDependent` be required for every temporary network configuration. The primary network is always active and thus does not require any of the Activation Line settings.

Any networks configured with `IsActivationLineDependent = true` will wait until notified by the ara::diag API to create sockets and send out vehicle announcement messages.

2.1.1.3.5 DHCP

DoIP provides the mechanism for allocating the IP addresses to the network. If the network is configured as dynamic then use the interface based on the configured `InterfaceName` otherwise use the statically configured IP address.

The DHCP feature is activated in the DoIP configuration file with the keyword `DHCP` inside the `FeatureFlags` array. Only then will the configuration parameters `IpAddressType`, `IpVersion` and `InterfaceName` be required for every network configuration. Similarly the configuration parameters `IpVersion` and `InterfaceName` are only supposed to be present when `IpAddressType = kDynamic`.

2.1.1.3.6 TLS

DoIP supports secure diagnostic communication channel between the client DoIP entity and the server DoIP entity.

The TLS feature is activated in the DoIP configuration file with the keyword `TLS` inside the `FeatureFlags` array. Only then will the configuration parameters `TlsMode` and `TransportSecurity` be required for TLS configuration.

2.1.1.3.7 Alive Check

The Alive Check has a set of features described in table 2-4:

based on template version 1.00.00

Features	Description
Source address conflict check	When a routing activation request is received with a source address that is already active on a different socket, send an alive check request to that other socket. If an Alive Check Reponse (specified by [39, table 28]) is received within a set amount of time (500ms), the routing activation is denied, keeping the existing connection. Otherwise, that socket is closed, and the routing activation is allowed to proceed, replacing the dead connection.
Socket capacity check	When a routing activation request is processed and there are no more available TCP sockets, send an alive check to all active TCP sockets (see figure [39, figure 12] for more details). Close any sockets that do not respond with an Alive Check Reponse (specified by [39, table 28]) within a set amount of time (500ms). If no sockets were closed, deny the routing activation. Otherwise, allow the routing activation to proceed.
Concurrent checks	Alive checks can run concurrently. If a new routing activation request shares its source address with an ongoing request, it fails immediately; otherwise, it triggers a concurrent alive check.
Early death	When a client sends a routing activation request and triggers an alive check, if the checked channel is then closed for an external reason before the alive check is finished, routing activation is allowed to continue immediately.
One request per socket	When a client sends a routing activation request and triggers an alive check, if an alive check response is already awaited from the socket to check, do not send a new request. The result of the routing activation request will instead depend on the result of the existing check.
Second request forbidden	When a client sends a second Routing Activation Request after having triggered an alive check, it will be closed.

Table 2-4 Alive Check Features

2.1.1.3.8 DoIP Configuration

DoIP configuration is done by modifying the `diagnosticmanager_doip.json` file. Table 2-5 illustrates the possible configuration settings in the config file. The order of the fields in the config must exactly match the order in the tables, however some fields are optional and need certain conditions to be met in order for the config parser to expect them, these prerequisites are stated in the description of the config field. Every field not containing a prerequisite condition in the description is mandatory.

Setting name	Description	Condition
FeatureFlags	A list of activated DoIP features. See table 2-6 for possible features and their respective values.	
ProtocolVersion	DoIP Protocol version, see table 2-13 for accepted values and supported versions. Refer to [39, table 16] for more details.	

based on template version 1.00.00

Setting name	Description	Condition
NodeType	The DoIP node type. `0x00` for DoIP gateway and `0x01` for DoIP node. Refer to [39, table 11] for more details.	
EID	The DoIP entity identifier for vehicle identification requests. Use value of "MAC" in order to default to the primary network's MAC address.	
MaxChannels	The maximum amount of concurrent TCP diagnostic sessions. Must be at least 1.	
MaxMessageLength	The maximum allowed message size.	
MaxPreviousDiagnosticMessageDataLength	The maximum size of Diagnostic message to append to a Diagnostic ACK/-NACK.	Optional (Default = 0)
MaxLengthInEntityStatusResponse	Whether to report the maximum message length in the entity status response.	
UseTcpIfCertificatesAreUnavailable	Allow the usage of TCP communication in case the TLS certificates are still unavailable. Once the crypto provider successfully load any of the certificates from the configured labels, TLS are enforced again for new connections during routing activation	false
PrimaryTargetAddress	The primary address of the DoIP entity.	
TargetFuncAddressArray	An array containing the functional target addresses for DoIP communication.	Mandatory
TargetPhysAddressArray	An array containing the physical target addresses for DoIP communication.	Mandatory
TlsMode	The TLS modes, `psk`, `certificate` and `both` are supported modes for TLS communication.	`TLS`
TransportSecurity	The transport security as described in table 2-7.	`TLS`
PrimaryNetwork	The primary network configuration as described in table 2-9.	
TemporaryNetworks	A list of optional network configurations as described in table 2-9.	

based on template version 1.00.00

Setting name	Description	Condition
PowerModeInitialValue	The initial power mode to use when no application providing the power mode is deployed or connected yet. It can have one of the possible values (Ready, NotReady, NotSupported). If this field is missing, NotReady is sent for a power mode request when application is not connected and NotSupported when the application is not deployed	Optional

Table 2-5 DoIP Global configuration

based on template version 1.00.00

Note: Some DoIP configurations are of an JSON array type. The very first element indicate the size of the array, followed by the actual elements. See listing 2-1 below.

DOIP JSON Array Examples

```
{
  "PrimaryTargetAddress": 1,
  "TargetFuncAddressArray": [
    1,
    58368
  ],
  "TargetPhysAddressArray": [
    3,
    5,
    12,
    90
  ]
}
```

Listing 2-1 Examples of DoIP array configuration

Table 2-6 describes the additional features supported for DoIP, certain fields in the config will be required based on whether their respective flags are provided as part of the `FeatureFlags` field.

Feature value	Description
ActivationLine	Turn on Activation Line feature. See section 2.1.1.3.4
DHCP	Turn on DHCP feature. See section 2.1.1.3.5
TLS	Turn on TLS feature. See section 2.1.1.3.6

Table 2-6 DoIP Feature activation values

Table 2-7 contains all the possible settings to configure secure diagnostic communication using `TLS` globally for every configured network in the `TransportSecurity` field as described in Table 2-5. These settings are used to provide enough information to the TLS provider in order to configure secure TCP communication using the requested cipher suites. The `CipherSuiteList` needs to reflect the `TlsMode` settings, i.e. if using certificates only, make sure the cipher suites are certificate-based cipher suites and the same applies for PSK cipher suites. This field is only required when the `TLS` feature is enabled, but the contents vary depending on the desired `TlsMode` setting.

Table 2-8 illustrates TLS version 1.2 cipher suits both supported and unsupported by the TLS server. The table contains all TLS 1.2 cipher suits accepted according to [38] and some additional cipher suits not mentioned in [38]. The cipher suits not mentioned in [38] may not be secure so should generally not be used, but could ease testing.

based on template version 1.00.00

Setting name	Description	Condition
PskIdentityHint	The pre shared key identity hint for psk mode configuration.	TlsMode=`psk` or `both`
PskIdentity	The pre shared key identity for psk mode configuration.	TlsMode=`psk` or `both`
PskUuid	The pre shared key universal unique identifier for psk mode configuration.	TlsMode=`psk` or `both`
PrivateKeyUuid	The private key universal unique identifier for certificate mode configuration.	TlsMode=`certificate` or `both`
CertificateLabelList	A list of certificate chain labels for certificate mode configuration.	TlsMode=`certificate` or `both`
CryptoProviderUuid	The crypto provider universal unique identifier for TLS configuration.	
CipherSuiteList	A list of desired cipher suite codes for TLS configuration. Possible values shown in Table 2-8.	
TlsBufferSize	Number of bytes to allocate for the TLS rx buffer for each connections. Optional. Default: 1024 Bytes	

Table 2-7 DoIP TLS configuration

Code	TLS 1.2 version cipher suites	ISO 13400-2	Support
`0xC02B`	TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256	✓	✓
`0xC02C`	TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384	✓	
`0xCCA9`	TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256	✓	
`0xC0AC`	TLS_ECDHE_ECDSA_WITH_AES_128_CCM	✓	
`0xC0AE`	TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8	✓	
`0xC023`	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256	✓	✓
`0xC024`	TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA384	✓	
`0xC006`	TLS_ECDHE_ECDSA_WITH_NULL_SHA	✓	✓
`0x0000`	TLS_NULL_WITH_NULL_NULL		✓
`0x00B0`	TLS_PSK_WITH_NULL_SHA256		✓
`0x00A8`	TLS_PSK_WITH_AES_128_GCM_SHA256		✓

Table 2-8 DoIP Supported TLS version 1.2 cipher suits

based on template version 1.00.00

Table 2-9 describes all the possible fields used for configuring a network. These settings are used to provide enough information for packet transmission and reception for each configured network.

based on template version 1.00.00

Setting name	Description	Condition
IpAddressType	The address assignment mode for the network. It specifies whether the address for the network is statically or dynamically configured. Supported values include: `static` and `dynamic`.	`DHCP`
IPAddress	The address of the interface used for this network, both TCP and UDP.	`DHCP` disabled or IpAddressType=`static`
EnableTls	Whether to enable TLS. If this field is missing, TLS will be enabled by default.	Optional
IpVersion	The protocol family, supported values include: `IPv4` and `IPv6`.	`DHCP` and IpAddressType=`dynamic`
InterfaceName	The name of the interface used by the network.	`DHCP` and IpAddressType=`dynamic`
NetworkInterfaceId	A unique network interface ID to use for the temporary network.	`ActivationLine` and not primary network
IsActivationLineDependent	Whether Activation Line is used to activate the temporary network.	`ActivationLine` and not primary network
StartAnnouncementMessageDelay	Time in milliseconds after which the first vehicle announcement message will be broadcasted.	
TpConnections	A list of TransportProtocol connection source address ranges that are accepted during routing activation of connections on the interface. The individual configuration objects are described in table 2-10.	Optional. Shall not be empty omit it otherwise, if one network is configured then all networks shall also be configured. Default: `[[0x0001, 0x0DFF], [0x0x0E80, 0x0x7FFF]]`
UDPChannel	The UDP channel configuration as described in table 2-11.	
TCPChannel	The TCP channel configuration as described in table 2-12.	
TcpGeneralInactivityTime	This timeout specifies the maximum time of inactivity on a TCP_DATA socket (no data received or sent) before it is closed. If omitted, the default value is 300 seconds(5 min)	

Table 2-9 DoIP Network configuration

based on template version 1.00.00

Table 2-10 describes an accepted source address range that would be allowed connection to DoIP Server.

Setting name	Description
SourceAddressRangeStart	The start range value in hex (if preceded by 0x) or decimal. Must be between `[0x0001, 0x7FFF]`
SourceAddressRangeEnd	The end range value in hex (if preceded by 0x) or decimal. Must be between `[0x0001, 0x7FFF]` and not less than SourceAddressRangeStart

Table 2-10 DoIP TP Connections Object

Table 2-11 describes all the possible fields used for UDP transmission and reception on a configured network. These settings are used to provide enough information for to the network to configure the required data structures and sockets in order to transmit and receive UDP messages.

Setting name	Description	Condition
IPAddress	Same as IPAddress in table 2-9	Deprecated. Network::IPAddress is used instead
BroadcastAddress	The address used for listening for broadcast/multicast messages. For IPv4 it is the network's broadcast address and for IPv6 it is the multicast address to subscribe to.	`DHCP` disabled or IpAddressType=`static` or (IpAddressType=`dynamic` and IpVersion=`IPv6`)
TTL	The time to live for the UDP packets. Value of `0` uses the default OS TTL value.	

Table 2-11 DoIP UDP channel configuration

based on template version 1.00.00

Table 2-12 describes all the possible fields used for TCP transmission and reception on a configured network. These settings are used to provide enough information for the network to configure the required data structures. As well as accept incoming TCP socket connections in order to transmit and receive TCP messages.

Setting name	Description	Condition
IPAddress	Same as IPAddress in table 2-9	Deprecated. Network::IPAddress is used instead
TTL	The time to live for the TCP packets. Value of `0` uses the default OS TTL value.	

Table 2-12 DoIP TCP channel configuration

Caution
The IPAddress in table 2-12 and table 2-11 is deprecated and IPAddress in table 2-9 should be used instead. If DHCP isn't enabled or static addressing is used at least one instance of the keys has to exist. If multiple exists they all have to contain the same value.

Table 2-13 describes the accepted protocol versions, this is the protocol version used in the header when DoIPBinding sends messages. Default value is ISO 13400-2:2019 if the Json `ProtocolVersion` field is omitted.

Protocol Version	Json field value	Supported
ISO/DIS 13400-2:2012	`0x02`	✓
ISO/DIS 13400-2:2019	`0x03`	✓(default)

Table 2-13 DoIP configurable protocol versions

2.1.1.3.9 Message Validation

Each incoming DoIP message is subject to the following DoIP header checks:

> Check header for correct version
> Check for Supported Payload type
> Check for valid payload length

2.1.1.3.10 Vehicle Identification

DoIP supports the following Vehicle Identification requests.

> Vehicle Identification Requests
> Vehicle Identification Requests with EID
> Vehicle Identification Requests with VIN

2.1.1.3.11 Node Information

DoIP supports diagnostic power mode information requests. The response for example can indicate if it is ready to perform reliable diagnostics. It also supports entity status requests indicating the following:

> Whether the DoIP instance is a DoIP node or a gateway
> Maximum number of concurrent TCP_DATA sockets allowed

based on template version 1.00.00

> Number of currently established sockets
> Maximum size of one logical request that this DoIP entity can process

2.1.1.3.12 Routing Activation

DoIP supports Routing Activation. It supports the Default Routing Activation type only.

2.1.1.4 Custom Transport Protocol

The custom transport protocol layers shall implement the APIs described in the adaptive AUTOSAR specification. If a TP layer is supposed to support periodic transmission, it shall also implement the APIs for periodic transmission defined in adaptive AUTOSAR R20-11.

Caution
The transport protocol layer is responsible to synchronize transmission, its related confirmation and the indication of a new messages to/from the same diagnostic client. As consequence, if transmission and transmission confirmation is interleaved by an indication of a new message, this new request will be rejected by the Diagnostic Manager. This is then unexpected from point of view of the diagnostic client, that sends in a new request right after having received the final response of a previous request.

2.1.1.5 Dispatching of UDS Requests

2.1.2 Diagnostic Server

According to [5], the Diagnostics Daemon shall support multiple Diagnostic Servers working in parallel and sharing the same Transport Layer. A Diagnostic Server is a transport layer independent entity to process diagnostic tasks, like UDS messages or fault memory related events.

Caution
Currently, the Diagnostics Daemon only supports a single Diagnostic Server.

2.1.3 Parallel Diagnostic Conversations

A Diagnostic Conversation is an abstract term for the one-to-one communication between a Diagnostic Client and a Diagnostic Server.

The Diagnostic Server supports handling of multiple Diagnostic Conversations in parallel, see section 6.3 for configuration details.

Diagnostics Library (ara::diag) exposes an interface to get information about Diagnostic Conversation to the Adaptive Application via the Conversation API. The constructor to this Conversation object (that exposes the API) is of custom signature and not defined by AUTOSAR. The full description of AUTOSAR Diagnostic Conversation can be found in [5].

Conversation object offers the following operations defined by AUTOSAR:

> `GetConversation(meta_info)` - Gets a specific Diagnostic Conversation
> `GetAllConversations()` - Gets all Diagnostic Conversations
> `GetCurrentActiveConversations()` - Gets all active Diagnostic Conversations
> `SetActivityNotifier(activity_status_notifier)` - Sets Activity Status Notifier
> `GetActivityStatus()` - Gets the Activity Status (keeps track of the request processing of a conversation)
> `SetDiagnosticSessionNotifier()` - Sets Diagnostic Session Notifier

> `GetDiagnosticSession()` - Gets Diagnostic Session
> `SetSecurityLevelNotifier(security_level_notifier)` - Sets Security Level Notifier
> `GetDiagnosticSecurityLevel()` - Gets Diagnostic Security Level

Caution
To use Diagnostic Conversation the corresponding Adaptive Application requires at least one diagnostic port.

2.1.4 UDS Message Processing

2.1.4.1 Request Validation

This section describes the validation of the incoming service requests done by the Diagnostics Daemon.

2.1.4.1.1 ResponseOnAllRequestSids

Diagnostics Daemon shall optionally ignore diagnostic service requests with SID within the response SID range.

If the configuration parameter `RESPONSE-ON-ALL-REQUEST-SIDS` is set to true, all the requests are handed over for further processing, irrespective of whether its SID falls in the range of response SIDs or of request SIDs. In this case, if the SID is found to be unsupported on further processing, the Diagnostics Daemon will respond with an NRC.

If this configuration parameter is set to false, Diagnostics Daemon silently ignores an incoming request, if the SID belongs to the range of the response SIDs: [0x40,0x7F]∪[0xC0,0xFF].

2.1.4.1.2 Mapping of NRC codes and their names

Table 2-14 shows a list of the error codes sent out in the context of validating service requests and maps them to the corresponding errors.

2.1.4.2 Request Validation on Daemon Side

The DM conducts the validation steps listed in table 2-15. If a service has sub-functions, the validation steps listed in table 2-16 are executed. For UDS service requests involving any diagnostic service port other than the `ara::diag::ServiceValidation` one, implemented by an external application, the DM also checks whether these required services are already accessible. If this is not the case, the DM will respond with NRC `0x94`.

Different service requests might also require specific additional validation steps. Those are specified in the sections on the respective services.

A complete overview of all validation steps that have to be applied for each service request (without or with sub-functions) can be found in [40]. Validations required for all services and sub-services can be found in [40] in figures 5 and 6, respectively.

All validation steps that are specified in [40] and not implemented by the DM should be performed by the application. This is valid for the validations that should be conducted for all service requests as well as for the validations specific to individual services.

2.1.4.3 Response Behavior

2.1.4.4 Service Preconditions

The DM provides the following mechanisms to control service execution. These differ in the NRC returned, and the exact time when they are applied. Since the exact timing differs based on the UDS service, this document will not provide a flow chart. Refer to [40] for details.

based on template version 1.00.00

NRC	Respective Error that the NRC described
NRC `0x11`	serviceNotSupported
NRC `0x12`	subFunctionNotSupported
NRC `0x13`	incorrectMessageLengthOrInvalidFormat
NRC `0x14`	responseTooLong
NRC `0x21`	busyRepeatRequest
NRC `0x22`	conditionsNotCorrect
NRC `0x24`	requestSequenceError
NRC `0x31`	requestOutOfRange
NRC `0x33`	securityAccessDenied
NRC `0x34`	authenticationRequired
NRC `0x35`	invalidKey
NRC `0x36`	exceedNumberOfAttempts
NRC `0x37`	requiredTimeDelayNotExpired
NRC `0x70`	uploadDownloadNotAccepted
NRC `0x71`	transferDataSuspended
NRC `0x72`	generalProgrammingFailure
NRC `0x73`	wrongBlockSequenceCounter
NRC `0x7E`	subFunctionNotSupportedInActiveSession
NRC `0x94`	resourceTemporarilyNotAvailable

Table 2-14 Negative response codes and the error they describe

Validations	NRC sent if unsuccessful
Manufacturer validation	NRC returned by application
SID supported	NRC `0x11`
Authentication*	NRC `0x34`
Diagnostic session*	NRC `0x7F`
Security access*	NRC `0x33`
Environmental preconditions*	NRC as configured for condition
Supplier validation	NRC returned by application

Table 2-15 NRC handling for service requests on DM side * not validated in case of Zero-copy

Also, please refer to the DEXT specification for details on how to model all aspects of service preconditions. This section only focuses on the `DIAGNOSTIC-ACCESS-PERMISSION` model, not on its correct usage.

2.1.4.4.1 Session and Security Dependency

[40] defines two possible preconditions: The active session and the security level. Both are directly supported in the DEXT model and can be checked by the Diagnostics Daemon.

In the diagnostic model, a `DIAGNOSTIC-ACCESS-PERMISSION` lists all sessions and security levels in

based on template version 1.00.00

Validations	NRC sent if unsuccessful
Minimum length	NRC 0x13
Sub-function supported	NRC 0x12
Authentication	NRC 0x34
Diagnostic session	NRC 0x7E
Security access	NRC 0x33
Environmental preconditions	NRC as configured for condition

Table 2-16 NRC handling for sub-functions requests on DM side

which the service is allowed to be executed. The DIAGNOSTIC-ACCESS-PERMISSION element can be reused on all instances that share the same permission requirements.

AccessPermissions

```
<DIAGNOSTIC-ACCESS-PERMISSION>
  <SHORT-NAME>SecurityRestriction</SHORT-NAME>
  <DIAGNOSTIC-SESSION-REFS>
    <DIAGNOSTIC-SESSION-REF DEST="DIAGNOSTIC-SESSION">/DEXT/ProgrammingSession</DIAGNOSTIC-
        SESSION-REF>
  </DIAGNOSTIC-SESSION-REFS>
  <SECURITY-LEVEL-REFS>
    <SECURITY-LEVEL-REF DEST="DIAGNOSTIC-SECURITY-LEVEL">/DEXT/
        SecurityLevel_FlashProgramming</SECURITY-LEVEL-REF>
  </SECURITY-LEVEL-REFS>
</DIAGNOSTIC-ACCESS-PERMISSION>
```

Listing 2-2 AccessPermissions

If a reference list is omitted entirely, all possible states are allowed. The example in listing 2-3 shows a DIAGNOSTIC-ACCESS-PERMISSION that allows service execution in all sessions and security levels.

NoAccessPermissions

```
<DIAGNOSTIC-ACCESS-PERMISSION>
  <SHORT-NAME>NoRestrictions</SHORT-NAME>
</DIAGNOSTIC-ACCESS-PERMISSION>
```

Listing 2-3 No Access Permissions

2.1.4.4.2 Data Restrictions

There can be additional permission checks for certain services, e.g. to model that service execution is only allowed while the engine is idle, or ignition is off (see listing 2-4). Classic AUTOSAR also allows services to depend on MODES which are currently not supported in AUTOSAR AP.

Note: It is not possible to list multiple data dependent permissions in one DIAGNOSTIC-ACCES↲S-PERMISSION. The combination of multiple conditions has to be performed by the model of the DIAGNOSTIC-ENVIRONMENTAL-CONDITION.

DataDrivenPrecondition

```
<DIAGNOSTIC-ACCESS-PERMISSION>
  <SHORT-NAME>ConditionRestriction_Basic</SHORT-NAME>
  <ENVIRONMENTAL-CONDITION-REF DEST="DIAGNOSTIC-ENVIRONMENTAL-CONDITION">/DEXT/
      ActiveRole_Basic</ENVIRONMENTAL-CONDITION-REF>
</DIAGNOSTIC-ACCESS-PERMISSION>
```

Listing 2-4 Data-Driven Preconditions

The supported feature set allows to compare a live value with a constant. This further extends the permission checks to also allow checks that can be reduced to a comparison of two values. The configuration model allows to define the concrete `NRC-VALUE` to return if a check fails. The check consists of multiple terms combined with a boolean operation `OP` (`LOGICAL-OR` or `LOGICAL-AND`), and the terms to combine.

The example in listing 2-5 shows a data condition that allows service execution if a specific data element (`ActiveRoleData`) is 2, 3 or 4 – otherwise NRC `0x33` is sent.

based on template version 1.00.00

EnvironmentalCondition

```
<DIAGNOSTIC-ENVIRONMENTAL-CONDITION>
  <SHORT-NAME>ActiveRole_Basic</SHORT-NAME>
  <FORMULA>
    <NRC-VALUE>0x33</NRC-VALUE>
    <OP>LOGICAL-OR</OP>
    <PARTS>
      <DIAGNOSTIC-ENV-DATA-CONDITION>
        <COMPARE-TYPE>IS-EQUAL</COMPARE-TYPE>
        <COMPARE-VALUE>
          <NUMERICAL-VALUE-SPECIFICATION>
            <VALUE>2</VALUE>
          </NUMERICAL-VALUE-SPECIFICATION>
        </COMPARE-VALUE>
        <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/DEXT/ActiveRole/ActiveRoleData</
            DATA-ELEMENT-REF>
      </DIAGNOSTIC-ENV-DATA-CONDITION>
      <DIAGNOSTIC-ENV-DATA-CONDITION>
        <COMPARE-TYPE>IS-EQUAL</COMPARE-TYPE>
        <COMPARE-VALUE>
          <NUMERICAL-VALUE-SPECIFICATION>
            <VALUE>3</VALUE>
          </NUMERICAL-VALUE-SPECIFICATION>
        </COMPARE-VALUE>
        <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/DEXT/ActiveRole/ActiveRoleData</
            DATA-ELEMENT-REF>
      </DIAGNOSTIC-ENV-DATA-CONDITION>
      <DIAGNOSTIC-ENV-DATA-CONDITION>
        <COMPARE-TYPE>IS-EQUAL</COMPARE-TYPE>
        <COMPARE-VALUE>
          <NUMERICAL-VALUE-SPECIFICATION>
            <VALUE>4</VALUE>
          </NUMERICAL-VALUE-SPECIFICATION>
        </COMPARE-VALUE>
        <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/DEXT/ActiveRole/ActiveRoleData</
            DATA-ELEMENT-REF>
      </DIAGNOSTIC-ENV-DATA-CONDITION>
    </PARTS>
  </FORMULA>
</DIAGNOSTIC-ENVIRONMENTAL-CONDITION>
```

Listing 2-5 Definition of an Environmental Condition

The current value to compare against is retrieved from the application. The only way to model this data element is as a `DIAGNOSTIC-DATA-ELEMENT` which cannot exist outside a DID definition. Therefore, to use this restriction type, a `DIAGNOSTIC-DATA-IDENTIFIER` including the necessary data element must be provided in the model.

Note: A `DIAGNOSTIC-DATA-IDENTIFIER` does not imply tester access. To have tester access, the DID must also be referenced by a read or write service instance.

based on template version 1.00.00

DataIdentifier

```
<DIAGNOSTIC-DATA-IDENTIFIER>
  <SHORT-NAME>ActiveRole</SHORT-NAME>
  <ID>0x0174</ID>
  <DATA-ELEMENTS>
    <DIAGNOSTIC-PARAMETER>
      <BIT-OFFSET>0</BIT-OFFSET>
      <DATA-ELEMENTS>
        <DIAGNOSTIC-DATA-ELEMENT>
          <SHORT-NAME>ActiveRoleData</SHORT-NAME>
          <SW-DATA-DEF-PROPS>
            <SW-DATA-DEF-PROPS-VARIANTS>
              <SW-DATA-DEF-PROPS-CONDITIONAL>
                <BASE-TYPE-REF DEST="SW-BASE-TYPE">/BaseTypes/uint8</BASE-TYPE-REF>
              </SW-DATA-DEF-PROPS-CONDITIONAL>
            </SW-DATA-DEF-PROPS-VARIANTS>
          </SW-DATA-DEF-PROPS>
        </DIAGNOSTIC-DATA-ELEMENT>
      </DATA-ELEMENTS>
    </DIAGNOSTIC-PARAMETER>
  </DATA-ELEMENTS>
</DIAGNOSTIC-DATA-IDENTIFIER>
```

Listing 2-6 Definition of a Data Identifier

2.1.4.4.3 Authentication Restrictions

`AR-PACKAGE` in DEXT is extended to hold the configuration of authentication roles as illustrated in 2-7.

Extending AR_PACKAGE with Authentication Roles

```
    <AR-PACKAGES>
      <AR-PACKAGE>
        <SHORT-NAME>AuthenticationRoles</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:DiagnosticAccessPermissionArPackageExtensions">
              <SDG GID="diagnosticAuthRole">
                <SDG GID="DiagnosticAuthRole">
                  <SDG-CAPTION>
                    <SHORT-NAME>production</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="isDefault">true</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
      </AR-PACKAGE>
    </AR-PACKAGES>
```

Listing 2-7 Extending Diagnostics Daemon model with Authentication Roles

A default authentication role can be set by setting the parameter `isDefault` to `true`. Upon the server startup, a default authentication role is activated.

`DIAGNOSTIC-ACCESS-PERMISSION` is extended to hold references to authentication roles as illustrated

based on template version 1.00.00

in 2-8.

Extending DIAGNOSTIC-ACCESS-PERMISSION with references to Authentication Roles

```
<DIAGNOSTIC-ACCESS-PERMISSION>
        <SHORT-NAME>AP_Authentication</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:DiagnosticAccessPermissionExtensions">
              <SDG GID="diagnosticAuthRoleRef">
                <SDX-REF DEST="SDG-CAPTION">/Project/Dcm/AuthenticationRoles/production
                    </SDX-REF>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
</DIAGNOSTIC-ACCESS-PERMISSION>
```

Listing 2-8 Extending DIAGNOSTIC-ACCESS-PERMISSION with references to Authentication Roles

`DIAGNOSTIC-ACCESS-PERMISSION` is extended to hold configuration of `authenticationRelevant` as illustrated in 2-9.

Extending DIAGNOSTIC-ACCESS-PERMISSION with authentication relevancy

```
<DIAGNOSTIC-ACCESS-PERMISSION>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticAccessPermissionExtensions">
        <SD GID="authenticationRelevant">true</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-ACCESS-PERMISSION>
```

Listing 2-9 Extending DIAGNOSTIC-ACCESS-PERMISSION with authentication relevancy

`authenticationRelevant` defines relevancy of `authenticationRole` pre-conditions. True, indicates the necessity to use authentication and therefore all modelled references to `authenticationRole`s are relevant. False, indicates that authentication is not required for a service.

Info

Empty `authenticationRole`s access permission is interpreted depending on authentication activation and authentication relevancy. If authentication is activated and authentication is relevant (`authenticationRelevant=true`), empty `authenticationRole`s access permission is interpreted as total denial. On the other hand, if authentication is deactivated regardless whether authentication is relevant, empty `authenticationRole`s access permission is interpreted as no restrictions. The criteria to activate authentication is described in 6.13.1.

based on template version 1.00.00

Info

Authentication restrictions evaluation can be disabled globally for all services via the configuration of MEX parameter `ignorePreconditions` to a boolean true value. 6.13.2 explains the configuration methodology.

2.1.4.4.4 Manufacturer / Supplier Validation

The most generic approach is to define a hook function that is called with the complete request data received from the bus. Using such a hook, an Adaptive Application or in-library component (restricted to SDT plug-in) can implement arbitrary check to restrict globally the processing of all diagnostic services.

DM offers two kinds of service validation mechanism:

> Manufacturer diagnostic service validation
> System supplier diagnostic service validation

The difference between manufacturer and supplier validation is the exact time during service processing at which the hook is called. See [40, Figure 5] for details.

DM supports service validations to be executed within an Adaptive Application or SDT plug-in. (see 2.1.5.10.2)

The full description of the `ara::diag::ServiceValidation` service interfaces can be found in [5]. For configuration aspects of the service validation behavior see section 6.5.1.

Caution

If the Adaptive Application responsible for service validation is not yet connected to Diagnostics Daemon, Diagnostics Daemon will:

> skip the service validation
> or reject each diagnostic service request with NRC `0x10`

depending on the configuration of the service validation behavior.

Caution

The required order of execution of the `ServiceValidation` service interface methods in the corresponding Adaptive Applications is:

1. Invoke `Validate()`, prior executing the requested diagnostic service
2. Invoke `Confirmation()` once the same diagnostic service execution has finished

In general, on subsequent diagnostic requests the `Validate()` methods will be executed only **after** the `Confirmation()` of the preceding request has been invoked.

Caution

The service validation polling is done until response is received or cancellation is requested. A warning log message will be created every two seconds if the application needs more time for the service validation.

2.1.4.5 Service Processing Notification

The DM provides notifications at the end of each processed service request. Applications can use the `amsr::diag::ServiceProcessingNotification` service to receive these notifications. The first eight bytes contain information about the service request and the confirmation status.

In case of a negative response the notification contains details about the failure. These details can be accessed via the `amsr::diag::NegativeResponseDetails` class which grants access to the NRC as well as the reason (`amsr::diag::NegativeResponseReason`) for the failed processing.

2.1.5 UDS Service Processing

DM handles the UDS services defined by [40]. The following chapters describe each UDS service and information about its configuration. Table 2-17 provides an overview of the currently supported UDS services and a link to the section that describes the respective service further. The column "custom implementable" specifies for each service whether it is possible or required to implement the complete UDS message processing in the application. We differentiate as follows:

> not allowed: the handling of the UDS message is always done by DM - the application might still be required to provide specific functionality (e.g. reading DID)

> allowed: the handling of the UDS message can be implemented completely in the application

> required: the handling of the UDS message must be implemented completely in the application

SID	Service Description	Custom implementation	Section
0x10	DiagnosticSessionControl	not allowed	2.1.5.1.1
0x11	ECUReset	required	2.1.5.2
0x14	ClearDiagnosticInformation	allowed	2.1.5.3.1
0x19	ReadDTCInformation	not allowed	2.1.5.3.3
0x22	ReadDataByIdentifier	not allowed	2.1.5.4.1
0x27	SecurityAccess	allowed	2.1.5.5
0x28	CommunicationControl	required	2.1.5.6
0x29	Authentication	allowed	2.1.5.7
0x2A	ReadDataByPeriodicIdentifier	not allowed	2.1.5.4.3
0x2C	DynamicallyDefineDataIdentifier	not allowed	2.1.5.4.4
0x2E	WriteDataByIdentifier	not allowed	2.1.5.4.2
0x31	RoutineControl	not allowed	2.1.5.8
0x34	RequestDownload	allowed	2.1.5.9.1
0x35	RequestUpload	allowed	2.1.5.9.2
0x36	TransferData	allowed	2.1.5.9.3
0x37	RequestTransferExit	allowed	2.1.5.9.4
0x38	RequestFileTransfer	allowed	2.1.5.9.5
0x3E	TesterPresent	not allowed	2.1.5.1.2
0x84	SecuredDataTransmission	not allowed	2.1.5.10
0x85	ControlDTCSetting	not allowed	2.1.5.3.4
0x86	ResponseOnEvent	not allowed	2.1.5.11

based on template version 1.00.00

SID	Service Description	Custom implementation	Section
Custom	Implementation Defined	required	2.1.5.12

Table 2-17 Supported UDS Services Overview

2.1.5.1 Session Management

2.1.5.1.1 Diagnostic Session Control (`0x10`)

The Diagnostic Session Control service is used to enable different diagnostic sessions in the server. A diagnostic session enables a specific set of diagnostic services and/or functionality in the server. The UDS Specification [40] defines, that the $P2$-Time and the $P2^*$-Time shall be returned as part of the positive response. The Diagnostics Daemon uses the values configured for the sessions from the configuration (see section 6.4 configuration). When the requested session change is not supported, a negative response is returned.

Configuration

The general configuration of the service is described in [12].

If the "Switch to Default session after S3-timeout" is enabled (see 6.2.2), the value for the timeout itself can also be configured as shown in 2-10 (please take the permitted range of values described in 2-18 into consideration).

Extending DIAGNOSTIC_SESSION_CONTROL_CLASS with s3ServerTimeout

```
<DIAGNOSTIC-SESSION-CONTROL-CLASS>
  <SHORT-NAME>DiagnosticSessionControlClass</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSessionControlClassExtensions">
        <SD GID="s3ServerTimeout">7.00</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <...>
</DIAGNOSTIC-SESSION-CONTROL-CLASS>
```

Listing 2-10 Configuring the s3ServerTimeout in seconds

MEX Parameter	Valid range (inclusive)	Default value	Important note
`s3ServerTimeout`	5.0 . . . 65.535	5.0 (s)	Please be aware that huge S3 timeouts can introduce cyber security attack enlightments.

Table 2-18 Value constraints of s3ServerTimeout parameter (in seconds) in Diagnostic Daemon

2.1.5.1.2 Tester Present (`0x3E`)

This service is used to indicate to a server (or multiple servers) that a client is still connected to the vehicle and that previously activated diagnostic services and/or communications are to remain active.

Configuration

The service is handled by the Diagnostics Daemon and does not require any configuration.

2.1.5.1.3 Session change through ECU reset cycles

The Diagnostic Session Control service is often also used as a switch into a boot loader context. The Diagnostics Daemon supports to do this switch through an ECU reset, or a restart of the diagnostic manager daemon.

If the Session setting `JUMP-TO-BOOT-LOADER` in the DEXT is set to something other than NO-BOOT, the Diagnostics Daemon will persist the current session in a file. On the next startup, the DM will then restore this session. The application code is responsible to perform any additional action, especially the reset/restart.

> `OEM-BOOT` or `SYSTEM-SUPPLIER-BOOT` The positive response is delayed to the restart. Once the daemon restarts, it will send out the positive response to the session change once the tester reconnects. The application can check that the service was received e.g. using a ServiceValidation. *This approach is not recommended since there is no dedicated trigger to perform the restart.* The Diagnostics Daemon will continue sending out ResponsePending responses until the restart occurs.

> `OEM-BOOT-RESP-APP` or `SYSTEM-SUPPLIER-BOOT-RESP-APP` The positive response is sent to the tester immediately. Restarting the ECU or daemon can then be handled in a ServiceConfirmation (section 2.1.4.4.4) callout.

Caution

The ECU Reset service (`0x11`) configured with response after reset (section 2.1.5.2), cannot be used in combination with the jump-to-boot feature. If an ECU reset request is received while a session is active that has the `JUMP-TO-BOOT-LOADER` attribute set to something other than `NO-BOOT`, then neither the preservation of the session after the restart nor the response after reset of the ECU Reset is guaranteed.

DiagnosticSession

```
<DIAGNOSTIC-SESSION>
  <SHORT-NAME>Programming</SHORT-NAME>
  <ID>2</ID>
  <JUMP-TO-BOOT-LOADER>OEM-BOOT</JUMP-TO-BOOT-LOADER>
  <P-2-SERVER-MAX>0.05</P-2-SERVER-MAX>
  <P-2-STAR-SERVER-MAX>5</P-2-STAR-SERVER-MAX>
</DIAGNOSTIC-SESSION>
```

Listing 2-11 Definition of a DiagnosticSession

2.1.5.2 ECU Reset (`0x11`)

This service implementation provides the reset functionality within the ECU.

Caution

Diagnostics Daemon does not perform the reset by itself.
It just evaluates the diagnostic request and delegates the requested mode type and the trigger time point to the application.

based on template version 1.00.00

The kind of the response behavior (before/after reset) is configurable:

> RESPOND-AFTER-RESET – Answer to Ecu Reset service should come after the reset
> RESPOND-BEFORE-RESET – Answer to Ecu Reset service should come before the reset

2.1.5.2.1 Rejection of new requests during hard reset

If the ECU Reset service is configured with RESPOND-BEFORE-RESET and the service processing of a hard reset request ends successfully, the DM will reject new requests for the diagnostic server until the glsDM is restarted. If instead RESPONSE-AFTER-RESET is configured, the DM will reject new requests while the application handles the ECU Reset.

2.1.5.2.2 Connections Stop

In case the ECU Reset service is configured with RESPOND-BEFORE-RESET and a hard reset request is positively responded, the diagnostic manager daemon informs the TP handlers to stop the connections with the testers using the API defined in 5.6.11. The connections stop is executed during post handling of the request, before sending the service confirmation to the adaptive diagnostic applications.

The following aspects are not according to AUTOSAR:

> Running state is reported to Execution Manager (ara::em) before the initialization is finished by the DM. This does not properly meet the specified requirement in [8] Chapter 7.4.1. There is a cmake variable AMSR_DIAG_DAEMON_REPORT_RUNNING_AFTER_INIT to change that behaviour (see table 3-12).

2.1.5.3 Fault Memory Services

2.1.5.3.1 ClearDiagnosticInformation (0x14)

This service implementation clears the stored fault memory content for primary/user memory. By adding the optional UserMemoryIdentifier to the service request Diagnostics Daemon will clear the stored fault memory content for the corresponding user memory.

2.1.5.3.2 Configuration

This service can be configured as described in [12] as a service fully handled by Diagnostics Daemon. However, the service can also be configured as a custom service. In that case, it needs to be configured like any other custom service (see 2.1.5.12 and the respective section in [12] , but here the hint about the id chosen for the service should be ignored).

2.1.5.3.3 ReadDTCInformation (0x19)

This service implementation provides read access to the stored fault memory information.

The Diagnostics Daemon internally provides the following sub-functions:

> ReportNumberOfDTCByStatusMask (0x01)
> ReportDTCByStatusMask (0x02)
> ReportDTCSnapshotRecordByDTCNumber (0x04)
> ReportDTCExtDataRecordByDTCNumber (0x06)
> ReportSupportedDTC (0x0A)
> ReportUserDefMemoryDTCByStatusMask (0x17)
> ReportUserDefMemoryDTCSnapshotRecordByDTCNumber (0x18)
> ReportUserDefMemoryDTCExtDataRecordByDTCNumber (0x19)

Limitations and Deviations

> The following sub-functions are NOT supported:
  > ReportNumberOfDTCBySeverityMaskRecord (0x07)
  > ReportDTCFaultDetectionCounter (0x14)

based on template version 1.00.00

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.3.4 ControlDTCSetting (0x85)

This service implementation manipulates the setting of the DTC in the ECU memory entries (i.e. while the communication is disabled).

The Diagnostics Daemon internally provides the following sub-functions:

> ControlDTCSetting on (0x01)
> ControlDTCSetting off (0x02)

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.4 Data Identifier Access

2.1.5.4.1 ReadDataByIdentifier (0x22)

The ReadDataByIdentifier service allows the client to request data record values, identified by one or more DataIdentifiers composed of two bytes, from the server. The support for DIDs is implemented by a collaboration of the Diagnostics Daemon and other Adaptive Platform Applications that provide DIDs via the Communication Middleware.

Table 2-19 shows the checks and NRC handling on the DM side. Figure 20 in [40] provides further information on all checks needed in the context of UDS Service 0x22.

Validations	NRC sent if unsuccessful
Message length acc. to [40]	NRC 0x13
Authentication*	NRC 0x34
Security access*	NRC 0x33
Environmental preconditions*	NRC 0x22
Total response length	NRC 0x14
At least one DID supported (in active diagnostic session)	NRC 0x31

Table 2-19 NRC handling for ReadDataByIdentifier request on DM side *checks done for every DID

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.4.2 WriteDataByIdentifier (0x2E)

The WriteDataByIdentifier service allows the client to write information into the server at an internal location specified by the provided data identifier.

Table 2-20 shows the validation steps and NRC handling on the DM side. Figure 20 in [40] provides further information on all checks needed in the context of UDS Service 0x22.

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.4.3 ReadDataByPeriodicIdentifier (0x2A)

The ReadDataByPeriodicIdentifier service allows a client to request periodic transmission of data from a server identified by one or more data identifier(s). According to [40], the data-identifier range 0xF2XX

based on template version 1.00.00

Validations	NRC sent if unsuccessful
Minimum message length	NRC `0x13`
Authentication	NRC `0x34`
Security access	NRC `0x33`
Environmental preconditions	NRC `0x22`
Total response length	NRC `0x14`
DID supported (in active diagnostic session)	NRC `0x31`

Table 2-20 NRC handling for WriteDataByIdentifier request on DM side

is reserved for periodic transmissions. Only the low byte is required in a periodic transmission request. E.g. PDID `0xF200` is recognized in a periodic transmission request as `0x00`.

The ReadDataByPeriodicIdentifier service can be used by only one client at a time as defined per [40] and any `0x2A` request received from another client would be declined.

In order to be supported, a PDID shall be first supported by the ReadDataByIdentifier service (see 2.1.5.4.1).

This service also relies on the transport protocol (see 2.1.1) which shall support periodic transmissions.

Note

When a client requests a periodic update, the diagnostic applications are triggered to periodically send data records to Diagnostics Daemon for the corresponding PDIDs mentioned in the request. These records are then cached. Later, Diagnostics Daemon scheduler sends the cached periodic updates to the client according to the configured read mode (See "pdidReadMode" in table 6-3).

Table 2-21 shows the validation steps and NRC handling on the DM side. Figure 23 in [40] provides further information on all checks needed in the context of UDS Service `0x2A`.

Validations	NRC sent if unsuccessful
Message length [40]	NRC `0x13`
Transmission mode	NRC `0x31`
Free room in scheduler for new DIDs	NRC `0x31`
Existance of environmental rules*	NRC `0x22`
At least one DID supported (in active diagnostic session)	NRC `0x31`
Authentication*	NRC `0x34`
Security access*	NRC `0x33`
Environmental preconditions*	NRC `0x22`
Periodic response message fits supported payload size	NRC `0x14`

Table 2-21 NRC handling for ReadDataByPeriodicIdentifier request on DM side *checked for every periodic DID

based on template version 1.00.00

Limitations and Deviations

- The periodic transmission of internal data elements is not supported e.g. diagnostic session, VIN, environmental data ...
- If there is an ongoing periodic transmission and one of the diagnostic application providing one of the PDID is restarted the behavior is undefined.
- In case application PDID Read fails with an NRC, the `0x2A` Service behavior is as follow:
  - For the first read attempt of a PDID Diagnostics Daemon waits and blocks all other PDIDs
  - For the subsequent failure (i.e. one valid value has been read), it depends on the configuration:
    - If values are reported only on update, Diagnostics Daemon blocks the transmission until a valid value is available
    - If any value is reported (not only on update), Diagnostics Daemon reports the last known good value
- The ISO 14229-1:2020 [40] defines two types of scheduling but only the scheduler type 1 is supported

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.4.4 DynamicallyDefineDataIdentifier (`0x2C`)

The DynamicallyDefineDataIdentifier service allows the client to define a DDDID during runtime that can be requested via the ReadDataByIdentifier/ReadDataByPeriodicIdentifier service at a later time.

The intention of this service is to provide the client the ability to group one or more data elements into a data superset that can be requested via the ReadDataByIdentifier or ReadDataByPeriodicIdentifier service.

The Diagnostics Daemon provides the following sub-functions:

- DefineByIdentifier (`0x01`)
- ClearDynamicallyDefinedDataIdentifier (`0x03`)

If a DDDID is defined, then following parameters must be configured:

- Check source elements precondition is defined globally for all DDDIDs `(CHECK-PER-SOURCE-ID)`
- Maximum number of source elements has to be defined for each DDDID `(MAX-SOURCE-ELEMENT)`

Limitations and Deviations

- Persistency will not be available if DDDID is defined
- A DDDID can only be used by the tester which has defined it
- If a periodically read DDDID is cleared and then redefined after a really short time, it may happen that NRC `0x31` (RequestOutOfRange) is returned
- Sub-function DefineByMemoryAddress (`0x02`) is not supported (see limitations listed in [5])

Configuration

For details about the configuration of this service, please consult the respective section in [12].

2.1.5.5 SecurityAccess (`0x27`)

This service implementation manages the security level of the ECU used to constrain the diagnostic access to critical services, like writing data in restricted areas.

Table 2-22 shows the validation steps and NRC handling on the DM sideexecuted in the context of UDS Service `0x29`.

based on template version 1.00.00

Validations	NRC sent if unsuccessful
Key length	NRC 0x13
Execution sequence of key sub-functions	NRC 0x24, NRC 0x35, NRC 0x36, NRC 0x37

Table 2-22 NRC handling for SecurityAccess request on DM side

Configuration

For details about the configuration of this service, please consult the respective section in [12]. This service can also be configured as a custom service with or without subservices (see 2.1.5.12).

2.1.5.5.1 Usage of brute-force-attack prevention

In case a brute-force-attack prevention is needed (i.e. delay time and attempt counter(s)), Diagnostics Daemon implements the algorithm internally.

Configuration of brute-force-attack prevention

See listing 6-2 for an example configuration, and a description of the parameters in table 6-3.

2.1.5.6 CommunicationControl (0x28)

Limitations and Deviations

Communication Control is a service that needs to be handled by the application. The DM only dispatches the services and sub-functions and checks the preconditions, but the actual handling of the service needs to be implemented on application side.

Configuration

Please consider that the AUTOSAR interface currently is not supported and that the service therefore needs to be implemented as a service handled by the application. For details refer to the respective section of this diagnostic service in [12].
This service can also be configured as a custom service with or without subservices (see 2.1.5.12).

2.1.5.7 Authentication (0x29)

The Authentication service is used by the client to authenticate itself. The possible interactions for authentication are the following:

- DeAuthenticate (0x00): Deauthenticate the client
- VerifyCertificateUnidirectional (0x01): Verify the certificate of the client and get a challenge from the server for unidirectional authentication
- VerifyCertificateBidirectional (0x02): Verify the certificate of the client and get a challenge, certificate and proof of ownership from the server for bidirectional authentication
- ProofOfOwnership (0x03): Send proof of ownership for the received challenge from the server
- TransmitCertificate (0x04): Present the server a certificate that qualifies the client for further processing
- AuthenticationConfiguration (0x08): Returns identifier for the used authentication configuration which is always APCE (Authentication with PKI Certificate Exchange)

For the supported subfunctions VerifyCertificateUnidirectional, VerifyCertificateBidirectional, ProofOfOwnership and TransmitCertificate the respective functions of the ara::diag::Authentication (4.23) class are invoked to handle the request.

Clients with the same client identification are handled as the same client for the authentication. Each call to ProofOfOwnership must be preceded by a call to VerifyCertificateUnidirectional or to VerifyCertificateBidirectional from a client with the same client identification.

based on template version 1.00.00

Table 2-23 shows the validation steps and NRC handling on the DM side. Figure 11 in [40] provides further information on all checks needed in the context of UDS Service `0x29`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`, NRC `0x14`
Execution sequence of proofOfOwnership and verifyCertificateUnidirectional/-Bidirectional	NRC `0x24`
Certificate data values	NRC returned by application

Table 2-23 NRC handling for Authentication request on DM side

Configuration

For details about the configuration of this service, please consult the respective section in [12]. This service can also be implemented as a custom service with or without sub-functions (see 2.1.5.12). Please refer to 6.12.3.2 when configuring a custom service mapping for this service implemented as a custom service.

2.1.5.8 RoutineControl (`0x31`)

The RoutineControl service is used by the client to execute a defined sequence of steps and obtain any relevant results. A routine is referenced by a 2-byte routineIdentifier and the interaction with a routine comprises the following actions:

> Start a routine (`0x01`)
> Stop a routine (`0x02`)
> Request routine results (`0x03`)

Table 2-24 shows the validation steps and NRC handling on DM side for RoutineControl requests. Figure 30 in [40] provides further information on all validation steps needed in the context of UDS Service `0x31`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`
Diagnostic session*	NRC `0x31`
Authentication*	NRC `0x34`
Security access*	NRC `0x33`
Environmental preconditions*	NRC `0x22`
Sub-function supported*	NRC `0x12`
Sub-function supported in active diagnostic session	NRC `0x7E`

Table 2-24 NRC handling for RoutineControl request on DM side *checks done for the RID

Conclusively, the following parameters should be validated by the application code:

> `routine_control_option_record` - UDS routineControlOptionRecord

Configuration

For details about the configuration of this service, please consult the respective section in [12].

based on template version 1.00.00

2.1.5.9 Data Transfer Services

Configuration

For details about the configuration of these services, please consult the respective section in [12]. These services can also be implemented as custom services without sub-functions (see 2.1.5.12).

2.1.5.9.1 RequestDownload (`0x34`)

This service is used to request a download of data. It provides the functionality to initiate a download data transfer from the client to the server.

Diagnostics Library (ara::diag) provides an interface for **RequestDownload** defined by AUTOSAR: `RequestDownload()`

Table 2-25 shows the checks and NRC handling on DM side. Figure 31 in [40] provides further information on all checks needed in the context of UDS Service `0x34`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`
Ongoing transfer	NRC `0x22`

Table 2-25 NRC handling for RequestDownload request on DM side

Conclusively, the following parameters should be validated by the application code:

> `data_format_identifier` - UDS dataFormatIdentifier (DFI)
> `address_and_length_format_identifier` - UDS addressAndLengthFormatIdentifier (ALFID)
> `memory_address_and_size`

The full description of the AUTOSAR API `RequestDownload()` can be found in [5].

2.1.5.9.2 RequestUpload (`0x35`)

This service is used to request an upload of data.

Diagnostics Library (ara::diag) provides an interface for **RequestUpload** defined by AUTOSAR: `RequestUpload()`

Table 2-26 shows the checks and NRC handling on DM side. Figure 32 in [40] provides further information on all checks needed in the context of UDS Service `0x35`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`
Ongoing transfer	NRC `0x22`

Table 2-26 NRC handling for RequestUpload request on DM side

Consequently, the following parameters should be validated by the application code:

> `data_format_identifier` - UDS dataFormatIdentifier (DFI)
> `address_and_length_format_identifier` - UDS addressAndLengthFormatIdentifier (ALFID)
> `memory_address_and_size`

The full description of the AUTOSAR API `RequestUpload()` can be found in [5].

2.1.5.9.3 TransferData (`0x36`)

This service is used to request a transfer of data (download/upload/file transfer). It provides the functionality to transfer data from the client to the server. The parameters (not the data content) will be validated within the Diagnostics Daemon.

based on template version 1.00.00

For **TransferData**, Diagnostics Library (ara::diag) provides two interfaces defined by AUTOSAR: `DownloadData()`, `UploadData()` and two interfaces defined by AMSR: `Read()`, `Write()`

> `DownloadData()` - Called for TransferData following a previous RequestDownload
> `UploadData()` - Called for TransferData following a previous RequestUpload
> `Read()` - Called for TransferData following a previous RequestFileTransfer with a read mode of operation
> `Write()` - Called for TransferData following a previous RequestFileTransfer with a write mode of operation

If DFI `0x0` (NoCompressionNoEncryption) is used, the DM verifies that the data transfer will accept no more than the requested `memorySize` amount of data bytes. For any other, customer specific value of the DFI parameter, the diagnostic application must verify that the transfered amount of data corresponds to the expected total size.

Table 2-27 shows the checks and NRC handling on the DM side. Figure 33 in [40] provides further information on all checks needed in the context of UDS Service `0x36`.

Validations	NRC sent if unsuccessful
Sequence respected for SID	NRC `0x24`
Minimum message length	NRC `0x13`
Block sequence counter	NRC `0x73`

Table 2-27 NRC handling for TransferData request on DM side

Validations	NRC sent if unsuccessful
Transfered data size	NRC `0x71`

Table 2-28 Additional validations if neither compression nor encoding is effective

Conclusively, the following parameters should be validated by the application code:

> `transfer_request_parameter_record` - UDS transferRequestParameterRecord

If compression or encoding are active, the app should also do the memory size checks.

The full description of the AUTOSAR APIs `DownloadData()` and `UploadData()` can be found in [5]. The full description of the AMSR APIs `Read()` and `Write()` can be found in 4.16.

2.1.5.9.4 RequestTransferExit (`0x37`)

This service is used to request a transfer exit to stop the data transfer (download/upload/file transfer). It provides the functionality to terminate a data transfer between the client and the server.

For **RequestTransferExit**, Diagnostics Library (ara::diag) provides two interfaces defined by AUTOSAR: `RequestDownloadExit()`, `RequestUploadExit()` and two interfaces defined by AMSR: `ExitRead()`, `ExitWrite()`

Table 2-29 shows the checks and NRC handling on the DM side. Figure 34 in [40] provides further information on all checks needed in the context of UDS Service `0x37`.

Validations	NRC sent if unsuccessful
Request sequence	NRC `0x24`

Table 2-29 NRC handling for RequestTransferExit request on DM side

For `RequestDownloadExit()` the following parameters have to be validated by the application code:

> `transfer_request_parameter_record`

For `RequestUploadExit()` the following parameters have to be validated by the application code:

> `transferRequestParameterRecord`

The full description of the AUTOSAR APIs `RequestDownloadExit()` and `RequestUploadExit()` can be found in [5]. The full description of the AMSR APIs `ExitRead()` and `ExitWrite()` can be found in 4.16.

2.1.5.9.5 RequestFileTransfer (`0x38`)

This service is used to request a file transfer.

Diagnostics Library (ara::diag) provides interfaces for **FileTransfer** defined by AMSR:

> `RequestReadFile()` - Called for RequestFileTransfer with ModeOfOperation ReadFile
> `RequestReadDirectory()` - Called for RequestFileTransfer with ModeOfOperation ReadDir
> `RequestWriteFile()` - Called for RequestFileTransfer with ModeOfOperation WriteFile
> `RequestResumeWriteFile()` - Called for RequestFileTransfer with ModeOfOperation ResumeFile
> `DeleteFile()` - Called for RequestFileTransfer with ModeOfOperation DeleteFile

The full description of the AMSR API can be found in 4.17.

Table 2-30 shows the checks and NRC handling on the DM side. Figure 35 in [40] provides further information on all checks needed in the context of UDS Service `0x38`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`
Supported by DM mode of operation	NRC `0x31`
Ongoing transfer	NRC `0x22`

Table 2-30 NRC handling for FileTransfer request on DM side

Consecutively, the following parameters should be validated by the application code:

> `file_size_uncompressed` - UDS fileSizeUncompressed
> `file_size_compressed` - UDS fileSizeCompressed
> `file_path_and_name` - UDS filePathAndName

Note
Currently only the MOOPs ReadFile, AddFile and ReplaceFile are supported.

2.1.5.9.6 Pipelined Data Transfer

For transferring data to an application using service `0x34` and `0x38`, there exists the option to use a pipelined behavior.

Note
The pipelined behavior can only be used to transfer data towards the application. Thus for service `0x38` only the MOOPs AddFile and ReplaceFile will use the pipelined behavior.

When this pipelined behavior is configured, the service `0x36` will deviate from the ISO14229-1 [40] in the following aspects:

> For service `0x36` (TransferData) no service validation is performed and no session preconditions

based on template version 1.00.00

are validated
- For service `0x36` positive service responses will be sent as soon as the data is forwarded to the application without waiting for a response from the application
- For service `0x36` the response parameter transferResponseParameterRecord in the UDS message will always be empty

The application side implementation must consider the following aspects:
- For service `0x34`
  - Only the `amsr::diag::DownloadService` interface will be used (`ara::diag::DownloadService` is not supported for pipelined data transfer)
  - The MetaInfo passed to `amsr::diag::DownloadService::DownloadData` is the same one that was used for `amsr::diag::DownloadService::RequestDownload`
  - The application implementing the `amsr::diag::DownloadService::DownloadData` shall not return any response Data. In case such Data is returned, it will not reach the diagnostic client.
  - Errors returned from `amsr::diag::DownloadService::DownloadData` will not be directly sent to the UDS-client in the response to the request. To signal that the download was not successful the response for service `0x37` (RequestTransferExit) can be used. Additionally, errors returned from `amsr::diag::DownloadService::DownloadData` will eventually (after some time) lead to negative responses for further service `0x36` request (The NRC `0x72` will be returned in this case).
- For service `0x38`:
  - The MetaInfo passed to `amsr::diag::DataTransferWriteHandler::Write` is the same one that was used for `amsr::diag::FileTransferService::RequestWriteFile`
  - Errors returned from `amsr::diag::DataTransferWriteHandler::Write` will not be directly sent to the UDS-client in the response to the request. To signal that the file transfer write was not successful the response for service `0x37` (RequestTransferExit) can be used. Additionally, errors returned from `amsr::diag::DataTransferWriteHandler::Write` will eventually (after some time) lead to negative responses for further service `0x36` request (The NRC `0x72` will be returned in this case).

The pipelined data transfer uses pre-allocated buffers to receive the data from the Transport Protocol handler and sends it to the application. The number of these buffers can be configured with the MEX-parameter `numberOfZeroCopySlots`. If this number is 0 (default value) then the pipelined behavior is not active and for any other number the pipelined behavior is active.

Info

Consideration for the configured amount of `numberOfZeroCopySlots`: The `numberOfZeroCopySlots` parameter determines how many requests for SID `0x36` can be sent by the UDS-client without a response from the `amsr::diag::DownloadService::DownloadData` or `amsr::diag::DataTransferWriteHandler::Write` interface. Once all the slots are utilized, the DM will "pause" the pipeline flow and will send response-pending messages (NRC `0x78`) until at least one slot gets free again. Having this in mind, depending on how fast application data processing speed is, a configuration with very few slots can cause scenarios where the application processes all slots before the DM ends his "pause" and sends new data. Increasing the amount of slots will keep the application busy processing without a need to wait for new diagnostic client data and hence having maximum processing throughput. Too many slots would increase the RAM usage, with no significant performance boost. The optimal value shall be calculated based on measurement with the real data consuming part of the pipeline and desired diagnostic client throughput.

Caution

pipelined data transfer has the following requirements/limitations:

> the Transport Protocol handler must pass at least the first byte of the UDS message in the `payload_info` to the `ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage` method
> it cannot be used together with SecuredDataTransmission (`0x84`) (see 2.1.5.10)
> at any time at most one UDS-client can have a running download data transfer

2.1.5.10 SecuredDataTransmission (`0x84`)

SDT is used as a security sub-layer between the UDS server and the transport layer that encapsulates a normal UDS Request/Response Message in order to guarantee the payload authenticity. It is referred to by [40] as service `0x84`.

In contrast to all other UDS services, the `0x84` is not a regular UDS service that executes a specific diagnostic job and can have any execution pre-conditions, such as Session, SecurityAccess etc., but rather has only a secure transport task. Due to this fact, it needs the help of an external plug-in, that handles all the specific cryptography tasks such as message encryption/signing resp. decryption/verification based on OEM specific algorithms.

If service `0x84` is not enabled in the DM configuration (6.25), there is no plug-in required and no run-time overhead is applied for receiving and processing a UDS request. Any SDT request will be treated as a normal UDS one and since not supported by the DM configuration, will result in a NRC `0x11`.

If service `0x84` is enabled in the DM configuration (6.25), a plug-in (see 2.1.5.10.2) will be required to be configured as specified in 6.25.2 and linked to the DM executable. The plug-in shall implement the API defined in 5.1.

Here is the architectured overview on the SDT integration:

Caution

Due to the nature of the following UDS service, it is not possible to use them over SDT.

> Diagnostic Session Control (`0x10`) configured with JUMP-TO-BOOT-LOADER as OEM-BOOT-RESP-APP or SYSTEM-SUPPLIER-BOOT-RESP-APP (see 2.1.5.1.1)
> ECU Reset (`0x11`) configured with RESPOND-TO-RESET as RESPOND-AFTER-RESET (see 2.1.5.2)
> ReadDataByPeriodicIdentifier (`0x2A`) (see 2.1.5.4.3)
> ResponseOnEvent (`0x86`) (see 2.1.5.11)

These services send

> responses after a restart (`0x10` and `0x11`)
> additionally to the request/response communication unsolicited responses (`0x2A` and `0x86`)

which will always be sent in plain-text, i.e. without SDT encapsulation.

Configuration

For details about the configuration of this service, please consult the respective section in [12].

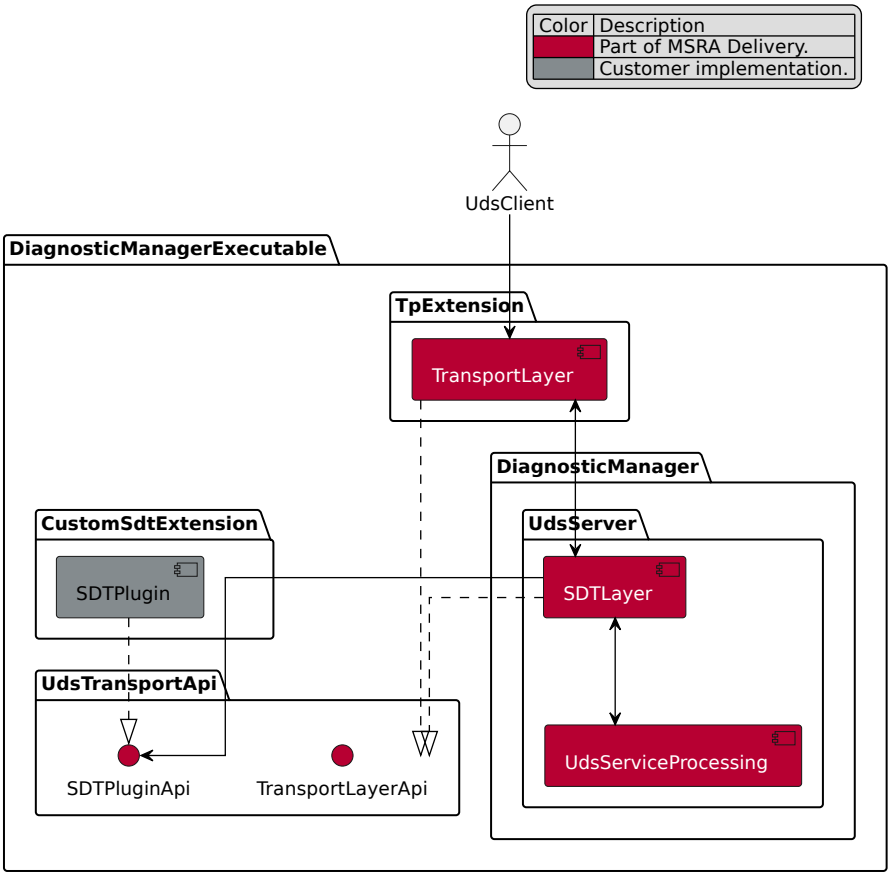

Figure 2-1 General architecture and how the SDT plug-in fits into it.

based on template version 1.00.00

2.1.5.10.1 Service Precondition Checks

According to the [40, Figure 39], upon reception of a diagnostic request, the server shall verify in addition to the other service execution preconditions also whether the requested service is allowed to be sent over SDT. This validation must be performed within the diagnostic application using a ServiceValidation (see 2.1.4.4.4) service port. Even though the ServiceValidation port will be presented with the internal UDS message of the service `0x84` request, it will be able to identify that the request has been sent via SDT by means of the provided MetaInfo (see 2.5, where a specific key has been added for SDT).

2.1.5.10.2 Plug-In

The SDT plug-in is a DM module extension point. It handles all the cryptography-specific tasks, external service validation and in some cases can handle some OEM specific behavior on `0x84`. Here is a simplified interaction with the plug-in (positive case only):

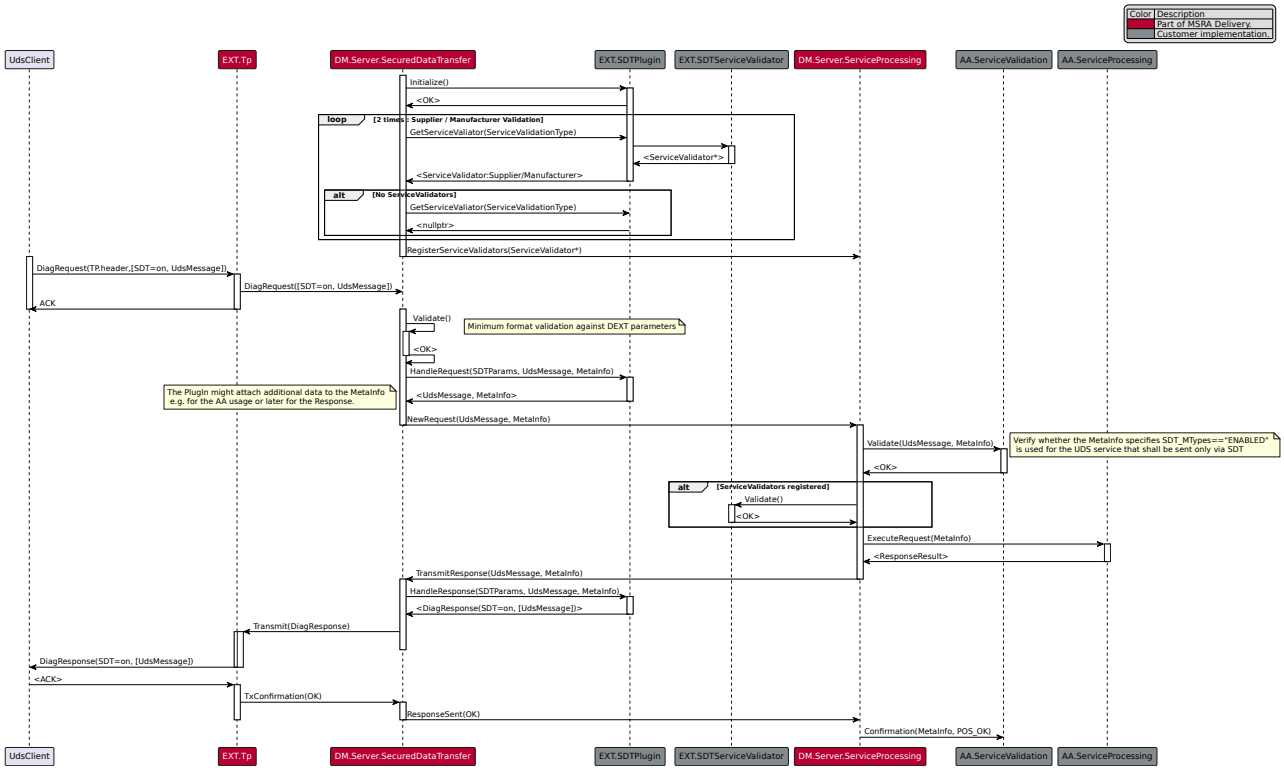

Figure 2-2 Sequence diagram for the standard positive use case.

On startup the DM will ask the plug-in for supplier and manufacturer service validators, which will be considered when executing services. If no external service validation is desired then `nullptr` shall be returned instead. If such service validation is desired, then one has to provide an implementation of 5.12 and forward it via the `GetServiceValidator()` API (see 5.1.5) towards the DM. Those service validations will then be executed in respective order as [40] describes, but with higher priority than the configured and implemented `ara::diag::ServiceValidation` interfaces.

On reception of a `0x84` diagnostic request, the security sub-layer in DM will perform a few request format checks, such as a minimum length check, and it will verify that the administrative parameter value is permitted by the configuration (6.25). Only if those checks are passed, the plug-in's HandleRequest() API (see 5.1.6) will be invoked. If any validation error was detected, a UDS NRC will be sent back to the client. In latter case, the TesterPresent (S3) timer will be restarted.

The plug-in may then verify the SDT parameters and the request signature. It is expected to decrypt

the internal UDS request message and provide it as a result. The plug-in may also attach a custom MetaInfo to the request message. This MetaInfo will then be attached to the internal UDS message during the following request processing (also during service validation).

Service processing continues for the internal UDS request as usual until the final response is determined. At this point the HandleResponse() API (see 5.1.7) of the plug-in is invoked. The pre-built SDT response message that is passed to the plug-in already has the same SDT parameters (administrative parameter, signature/encryption calculation, anti-replay counter) set that the request message had (except that the request-message flag of the administrative parameter is deactivated), but the plug-in is free to alter them as needed. However, the pre-built response message does not yet have a payload buffer allocated. Therefore, the plug-in must call Allocate() (see 5.5.5) on the response message once it knows the final length of the SDT response. The plug-in is then expected to provide the encrypted internal response message and a signature (if needed) to finalize the response.

Info

In cases where the plug-in needs to have full control over the positive response message payload located after the `0xC4` SID byte, the generic Allocate() method (see 5.5.4) can be used to reserve the whole message length of the SDT response. The plug-in is then expected to provide not only the encrypted internal response message and a signature (if needed), but also all the response parameters as per specification (either UDS or customer) to finalize the response.

Note that the plug-in will be triggered only for responses belonging to a request sent via service `0x84`. If service processing of the internal UDS message triggers a response-pending message (NRC `0x78`), this response-pending message will also be passed to the plug-in, and here again it will contain the same SDT parameters from the request, as it is the case for the final response.

If the plug-in ever returns an NRC during request or response handling, this NRC will be sent to the UDS client as a response to the service `0x84` request (e. g. `0x7F 0x84 0x10` for NRC `0x10`).

To ease the implementation of the SDT plug-in, another set of classes (see API chapters starting with 5.2) is provided that expose both structured and low-level access to the SDT request and response message contents.

2.1.5.11 ResponseOnEvent (`0x86`)

RoE is used to execute a diagnostic request upon an event trigger. It is referred to by [40] as service `0x86`.

In order to request RoE from DM, two phases of preparation are required. Setup phase, In which the type of trigger and STRT are specified. Controlling phase, in which the setup events can be started, stopped, or cleared.

Note

In the control phase, if start, stop, or clear events is requested, it shall be applied on all the setup events. If new events were setup while other events had been started already, the new events shall be started automatically without explicit start request

Caution

Only one tester at a time is allowed to request RoE service. If this is violated, the strategy illustrated in 2-31 is enforced.

based on template version 1.00.00

Tester A / Tester B	Started	Stopped (Setup)	Cleared
Setup	Clear Events for both contexts and setup new event	Clear Events for both contexts and setup new event	Setups new event
Start	Send NRC `0x24`	Send NRC `0x24`	Send NRC `0x24`
Stop	Clear Events for both contexts	Clear Events for both contexts	Clear Events for both contexts
Clear	Clear Events for both contexts	Clear Events for both contexts	Clear Events for both contexts

Table 2-31 Multi-tester strategy

2.1.5.11.1 Diagnostic Session Change

On changing the diagnostic session, the setup events are stopped or/and cleared, depending on the type of the previous and new session. In the special case of a transition from the default session to the default session nothing changes. The behavior on the different transitions is illustrated in 2-32.

New / Previous	Default Session	Non-Default Session (may be the same as the previous)
Default Session	No changes	Stop Events
Non-Default Session	Clear Events and reactivate previously in default session active Events	Clear Events

Table 2-32 Behavior of the setup events on diagnostic session transition

The following sub-functions are supported:

- StopResponseOnEvent (`0x00`)
- OnChangeOfDataIdentifier (`0x03`)
- StartResponseOnEvent (`0x05`)
- ClearResponseOnEvent (`0x06`)

Limitations and Deviations

- The following sub-functions are not supported:
  - OnDTCStatusChange (`0x01`)
  - ReportActivatedEvents (`0x04`)
  - OnComparisionOfValues (`0x07`)
  - ReportMostRecentDtcOnStatusChange (`0x08`)
  - ReportDTCRecordInformationOnDtcStatusChange (`0x08`)
- Event storage is not supported.
- Only infinite window time is supported. However, any type of event window time is accepted knowing that the treatment of the event windows is the same from the point of view of the Diagnostics Daemon as the storage flag is not currently evaluated

Configuration

The general configuration of the service is described in [12].

Response on Event Class Configuration Extension `DIAGNOSTIC-RESPONSE-ON-EVENT-CLASS` in DEXT is extended to hold the configuration of maximum number change of data identifier events as illustrated in 2-12.

Extending DIAGNOSTIC_RESPONSE_ON_EVENT_CLASS with maximum number change of data identifier events

```
<DIAGNOSTIC-RESPONSE-ON-EVENT-CLASS>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticResponseOnEventClassExtensions">
        <SD GID="maxNumChangeOfDataIdentifierEvents">5</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-RESPONSE-ON-EVENT-CLASS>
```

Listing 2-12 Extending Diagnostics Daemon model with maximum number change of data identifier events

2.1.5.12 Custom Service

For a custom service, the DM forwards the complete request to the mapped GenericUDSService port 2.1.5.13. There, the application can process the request and return the response that is sent back to the UdsClient. Like for all other services, the DM checks the configured preconditions (e.g. environmental preconditions, sessions, authentication roles).
For a custom service with sub-services, each sub-service can be configured with its own set of preconditions.

Caution
The suppress response message functionality of a custom sub-function service (defined in [40]) is not even given when configured additionally.

Configuration

For details about the configuration of a custom service, please consult the respective section in [12]. To configure a custom service with sub-functions, the `DIAGNOSTIC-CUSTOM-SERVICE-CLASS` shall be extended with the MEX `DvMex:DiagnosticCustomServiceClassExtensions` and the `subServiceKind` shall be set to `SUB_FUNCTION_ID`. The sub-function shall then also hold a MEX `DvMex:DiagnosticCustomServiceClassExtensions` with the `subServiceId` defining the sub-function id. An example configuration can be seen in 2-13.

based on template version 1.00.00

Configuration of a sub-function for a custom service

```
<DIAGNOSTIC-CUSTOM-SERVICE-CLASS>
  <SHORT-NAME>CustomServiceClass</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticCustomServiceClassExtensions">
        <SD GID="subServiceKind">SUB_FUNCTION_ID</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <ACCESS-PERMISSION-REF DEST="DIAGNOSTIC-ACCESS-PERMISSION">/Test_SWCL/Dcm/AP_All</
      ACCESS-PERMISSION-REF>
  <ACCESS-PERMISSION-VALIDITY>ACCESS-PERMISSION-SERVICE-CLASS</ACCESS-PERMISSION-VALIDITY
      >
  <CUSTOM-SERVICE-ID>0xXX</CUSTOM-SERVICE-ID>
</DIAGNOSTIC-CUSTOM-SERVICE-CLASS>
<DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE>
  <SHORT-NAME>CustomSubService</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticCustomServiceInstanceExtensions">
        <SD GID="subServiceId">0</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <CUSTOM-SERVICE-CLASS-REF DEST="DIAGNOSTIC-CUSTOM-SERVICE-CLASS">/Test_SWCL/Dcm/
      ServiceClasses/CustomServiceClass</CUSTOM-SERVICE-CLASS-REF>
</DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE>
```

Listing 2-13 `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` as a sub-function of a custom service

2.1.5.13 Service handling through the generic UDS interface

A `ara::diag::GenericUDSService` (4.42) implementation handles the external service request as it was sent by the tester. In case the requested service has defined sub-functions, the SPRMIB is always evaluated (according to [40]).

Caution

If the SPRMIB of the sub-function request is set, the Diagnostics Daemon will suppress a positive response by the Adaptive Application to the tester if no response pending was sent before.

The Diagnostics Daemon will always send a positive response to the tester if it is waiting for an answer due to a response pending message. Therefore the Adaptive Application should not handle the SPRMIB on its own by making use of the internal UDS NRC `0xFF kNoProcessingNoResponse`.

It is required that the Adaptive Application returns the response with the corresponding UDS service protocol header but without the SID.

2.1.5.14 Custom UDS NRC

On application side the NRCs defined in `ara::diag::DiagUdsNrcErrc` (4.30) can be used to report NRCs for different error conditions. In case a NRC that is not defined in `ara::diag::DiagUdsNrcErrc` shall be reported, the `amsr::diag::AsUdsNrc` (4.49.8) function can be used to create such an error.

Note

While `static_cast<ara::diag::DiagUdsNrcErrc>(nrc_value)` also works, the usage of `amsr::diag::AsUdsNrc(nrc_value)` is recommended because it avoids creating a value of the enum type for which no enumerator is defined.

2.1.5.15 UDS NRC Mapping

For any UDS NRC resulted in a diagnostic request processing, the diagnostic manager can map it to another one, prior sending the final response to the UDS client. This mapping is configured as outlined in 6.30. The business logic of the diagnostic manager remains unaffected; only the final response sent to the UDS client is modified. Consequently, any service processing end notification will still contain the initial NRC (source NRC).

NRC value configuration limitations

It is important to note that NRC mappings should not be transitive, i.e. an element of the target NRC value set cannot be included in the source NRC value set. In addition each source NRC value must be unique.

Important NRC mapping impacts

The DM supports the mapping of any UDS and non-UDS-reserved NRC values, i.e. custom NRCs, with some limitations:

> Mapping NRC values can affect negative response message content according to ISO [40], for example mapping NRC `0x78` (source) to another value (target) will affect Request Correctly Received - Response Pending (RCR-RP) messages, as they will include the target NRC value.

> Mapping the UDS NRC `0xFF` `kNoProcessingNoResponse` as source to another target value will have no effect, as this value is reserved for no response transmission.

For more details regarding custom NRCs, see 2.1.5.14.

2.1.6 SOVD Request Validation and Verification

DM validates and verifies the SOVD requests as specified in [7]. This chapter describes the different validation and verification steps and explains how it operates.

2.1.6.1 SOVD Authorization

The SOVD server shall protect all resources against unauthorized access. Only if the SOVD client has the appropriate access rights, the SOVD server shall grant the access to the requested resources.

As specified in [1], the process of authentication and authorization of a SOVD client is composed of following steps:

1. Prior to be able to request a token, the SOVD client needs to be authenticated
   This step requires an online backend user administration or an offline vehicle authorization server
2. Once authenticated, the SOVD client requests a token
3. The SOVD client issues a request to the SOVD server containing the token as part of the authorization header
4. The SOVD server validates the token and the access can be granted

From the above listed steps, the DM is responsible only for the token validation with the help of a

diagnostic application.

2.1.6.1.1 Token Validation

Before executing a request the SOVD server checks if the SOVD client has sent an authorization token within the headers of the http request. If the token is available and not authorized yet, the server requires the adaptive application to validate the token and then authorize the SOVD client with the appropriate ID, roles and validity time.
To validate the token the custom implementation of the SovdAuthorization port shall be instantiated and offered (See section 4.19) and then the validation is done by calling the `ValidateAuthorization` API specified in section 4.19.6. The `ValidateAuthorization` shall recover the information encoded within the token and then authorize the token by calling `Authorize` API specified in section 4.20.1 of `SovdTokenAuthorization` (See section 4.20). For configuration of `SovdAuthorization`, please refer to section 6.26.4.4.

If the token was authorized in a previous request or has just been validated and authorized, the SOVD server checks if the SOVD client roles allow the execution of the request. The roles allowed for a request are configured as part of the `<DIAGNOSTIC-ACCESS-PERMISSION>` as specified in section 2.1.4.4.3.

The figure 2-3 shows the validation sequence of an authorization token.

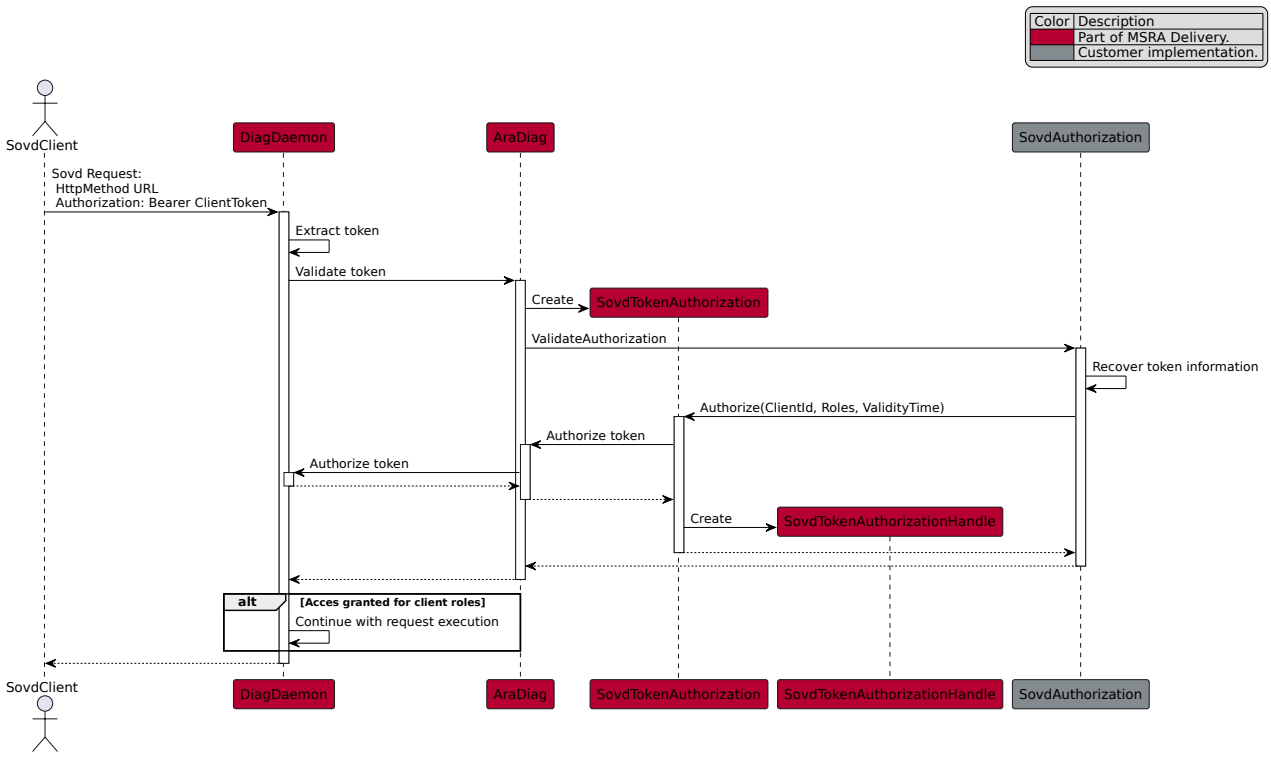

Figure 2-3 Sequence diagram for SOVD Authorization.

based on template version 1.00.00

Info

Only the Bearer authentication scheme is supported. Authorization header shall have following syntax:

```
Authorization:  Bearer ClientToken
```

Tokens may be implemented e.g., using JSON Web Tokens

2.1.6.1.2 Token Revokation

Once a token has been authorized, the token may be revoked. This means that a new SOVD request containing the revoked token will trigger the token validation process again. The revokation can happen in two different manners:

> Validity time expiration: The validity time specified during authorization has expired.
> Manual revokation: By calling `Revoke` on the `SovdTokenAuthorizationHandle`, the token can be manually revoked (See section 4.21).

Once the diagnostic daemon has revoked a token, a notification is sent to the diagnostic application. If a notifier has been previously registerd using `SetNotifier` on the `SovdTokenAuthorizationHandle` (See section 4.21), the notifier is executed.

The figure 2-4 shows the revokation sequence of an authorized token.

2.1.6.1.3 Authorization failures

The following table 2-33 describes the different scenarios of authorization failures on SOVD request reception and the associated responses:

Description	Http response code	Http response headers	Http response body
SOVD request doesn't contain authorization header, but request execution requires authorized client	401	WWW-Authenticate: Bearer realm="SOVD"	N/A
SOVD request contains authorization header, but it could not be validated (e.g. Not Bearer schema, diagnostic application rejected the token ...)	401	WWW-Authenticate: Bearer error="invalid_token"	N/A
SOVD request contains authorization header with valid toeken but acces rights are insufficient	403	WWW-Authenticate: Bearer error="insufficient_scope"	`GenericError` with `error_code`="insufficient-access-rights" (See [1])
SOVD request contains authorization header but no entry for token validation is currently free	500	N/A	`GenericError` with `error_code`="sovd-server-failure" (See [1])

Table 2-33 Description of Authorization failures

based on template version 1.00.00

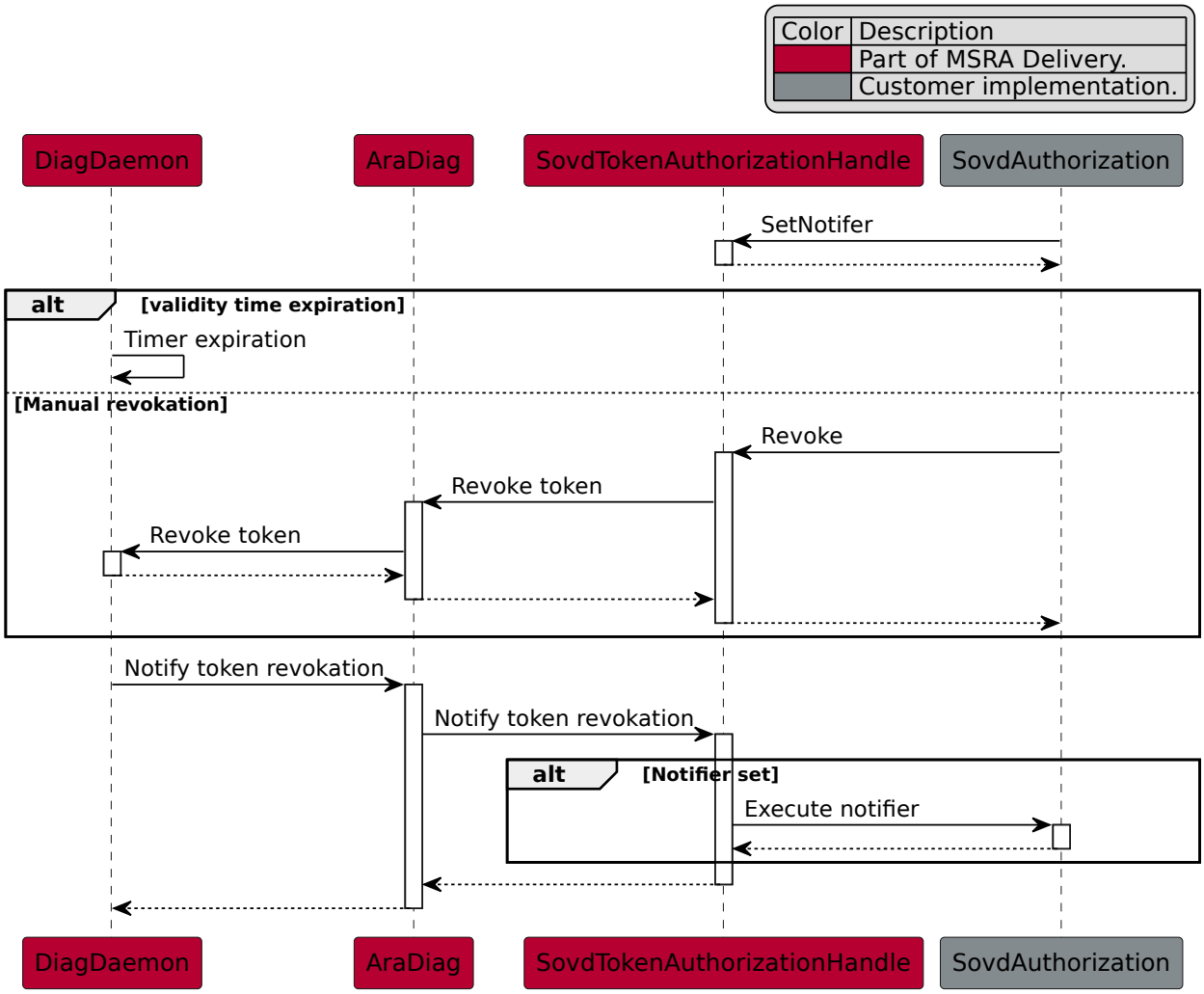

Figure 2-4 Sequence diagram for token Revokation.

based on template version 1.00.00

2.1.7 SOVD Request Processing

The following SOVD REST API are supported with some restrictions and deviations from [1]:

- Data resource read/write access
- Control of operations
- Fault handling

The following additionnal SOVD REST API are supported :

- Generic SOVD request

For more details please contact Vector Informatik GmbH.

2.1.8 Diagnostic Data Management

2.1.8.1 Internal Diagnostic Data

There are two scenarios for the data provision with regard to the `DIAGNOSTIC-PROVIDED-DATA-MAPPING` setting:

- With provided data mapping (e. g. 2-14): The supported values for `DATA-PROVIDER` settings are specified in [5]. If such a data mapping exists, there is no application interaction necessary to store the DTC beyond the reporting of the monitor status.
- Without a provided data mapping: The data is fetched from the application using the `DataElementInterface`. These interface types have limited amount of variants determined by the supported implementation data types (see 6.3.2.1), still they are completely defined by the `DiagnosticPortInterface` in the application model. As a result the application model defines `DiagnosticPortInterface` namespace and class name to be generated. If no application has offered the port at the time a DTC needs to store the extended data, the Diagnostics Daemon substitutes the data with a pattern of `0xFF`.

ExtendedDataRecords are always collected by data element. There is no interface to fetch a more complex record structure at once in the manner of DataIdentifiers.

InternalDataMapping

```
<DIAGNOSTIC-PROVIDED-DATA-MAPPING>
  <SHORT-NAME>PriorityMapping</SHORT-NAME>
  <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/ExtendedDataRecords/InternalDataRecord/
      priority</DATA-ELEMENT-REF>
  <DATA-PROVIDER>DEM_PRIORITY</DATA-PROVIDER>
</DIAGNOSTIC-PROVIDED-DATA-MAPPING>
```

Listing 2-14 Internal DataMapping

Caution

The following DATA-PROVIDER are not supported

- DEM_CURRENT_FDC
- DEM_CYCLES_SINCE_FIRST_FAILED
- DEM_CYCLES_SINCE_LAST_FAILED
- DEM_FAILED_CYCLES
- DEM_MAX_FDC_DURING_CURRENT_CYCLE
- DEM_MAX_FDC_SINCE_LAST_CLEAR
- DCM_SECURITY_LEVEL
- EVENT_ASSOCIATED_IDENTIFICATION

2.1.8.2 External Diagnostic Data

2.1.9 Fault Memory

The Fault Memory manages and persists fault memory related data. In the context of ISO [40], Fault Memory is needed for supporting UDS services 0x14, 0x19, and 0x85. Refer to section 2.1.5.3.

A Fault Memory contains Event Memories. Each Event Memory contains an independent subset of DTCs, such that each DTC belongs to exactly one Event Memory.

There are multiple types of additional data for DTCs, the most common ones are Snapshot Data, and Extended Data Records. Both data types are supported by AUTOSAR and can be used in parallel for single DTCs.

The default behavior of an Extended Data Record is to store application defined data. This data is retrieved when the DTC status changes according to the TRIGGER. Usually, Extended Data Records provide access to statistics computed within the fault manager. This can be achieved in a similar manner as for DIDs, using DIAGNOSTIC-PROVIDED-DATA-MAPPING.

2.1.9.1 DTC Status Processing

A DTC is an externally visible data set, which is identified by an identifier unique to an Event Memory. This identifier is used to access data logically belonging to a DTC.

The DTC Status aggregates all status information about a DTC. It contains the information known as DTC status from UDS [40].

Since the DTC Status is calculated from all sorts of triggers and signal flanks, calculating it is distributed across the Fault Memory implementation as reaction on the triggers and flanks notified to the Diagnostics Daemon.

Triggers can be every application Monitor that reports a current qualified test result for the supervised symptom.

All incoming triggers are queued and processed on the Threads of the Fault Memory. See table 2-46. This reduces the possible sources of race conditions, and allows a defined order of calculation for deterministic behavior.

If the DTC Status has changed the changes are persisted, see section 2.1.9.2.

2.1.9.2 File Handling

The Fault Memory stores its non-volatile data as binary files in a file storage of ara::per [28]. On startup, the Diagnostics Daemon reads the persisted data into memory.

During runtime, the Diagnostics Daemon periodically checks the Fault Memory for updates and synchronizes these updates with the persistent storage. The update intervall can be configured in millisecconds, see section 6.8. Additionally, the content of the Fault Memory is synchronized with

based on template version 1.00.00

persistency on shutdown of the Diagnostics Daemon.

Different file handling modes can be selected for persisting the Fault Memory into the file storage. The mode can be selected based on the requirements for storage capacity or performance. Supported modes are:

> `Multifile`: Each DTC is stored in a seperate file (default)
> `Singlefile`: All DTCs are stored in one single file.

Detailed considerations for configuring the file handling are shown in section 6.8.

2.1.10 Diagnostic Event

A diagnostic event stores the result of a monitor which can be located in an application or a BSW module. These monitors can report system observations to the event – either directly or through debouncing (see section 2.1.10.4).

2.1.10.1 Combined Events

With combined events it is possible to combine the results of multiple monitors to a single DTC. For combined events DM extends[1] the AUTOSAR SWS [5].

Since the DTC status is combined from multiple event status, Diagnostics Daemon will derive the actual DTC status from the status of multiple events. The combined status is calculated according to table 2-34.

Combined UDS DTC status bit	Logical equation
Bit 0 - TestFailed	OR (Event[i].Bit0)
Bit 1 - Test Failed This Operation Cycle	OR (Event[i].Bit1)
Bit 4 - Test not Completed Since Last Clear	latched version of Bit6 to 0 (until clear)
Bit 5 - Test Failed Since Last Clear	latched version of Bit1 to 1 (until clear)
Bit 6 - Test not Completed This Operation Cycle	OR (Event[i].Bit6) AND NOT Bit1

Table 2-34 DTC status combination

Caution
In AUTOSAR SWS [5], SinceLastClear bits are not available in the event status. Therefore the calculation contains latched versions.

Basically, the DTC status is an OR combination of all events, with the resulting status byte modified by an additional combination term. This is done to make sure that a failed result will also reset the ‘test not completed’ bits, even if not all contributing monitors have completed their test cycle.

2.1.10.2 Monitors

Monitors shall be able to report a qualified passed and qualified failed status. Diagnostics Daemon will calculate the DTC status from these reports. See 2.1.9.1.

2.1.10.3 Event Notification

The application can be notified from the Diagnostics Daemon about event status changes, or query them directly.

[1] Implementation according to AUTOSAR Classic module Dem (SWS_Dem_00441) with deviations, refer to table 2-34

2.1.10.4 Event Debouncing

Debouncing is triggered by a monitor state update which can be performed both by the application and by the Diagnostics Library (ara::diag). The request will reach Diagnostics Library (ara::diag) and be forwarded to the Debouncing module. This trigger may, as a result, lead to an event status change, which would be then reported to the Diagnostics Daemon via the monitor.

Available debouncing algorithms are:

> Counter Based Debouncing (debouncing handled by Diagnostics Library (ara::diag))
> Time Based Debouncing (debouncing handled by Diagnostics Library (ara::diag))
> Monitor internal debouncing (debouncing handled by application)

2.1.11 Operation Cycles

Each event is assigned to an operation cycle, e.g. ignition cycle.

Operation cycles:

> Trigger Dtc Status transitions on flanks of the operation cycle
> Act as a 'Timer' - DTC age timed by aging cycle periods

2.1.12 DTC Confirmation

Confirmation threshold is configured per DiagnosticEvent in the DEXT but they are managed on DTC level in the DM. This leads to the constraints described in section 2.4.3.

Every time the operation cycle associated to a DTC is changed:

> If the DTC is failed this operation cycle, the internal DTC trip counter is increment
> If the DTC is currently passed, the internal DTC trip counter is reset

During a failed report, if the confirmation threshold value is equal to the DTC trip counter + 1, the DTC confirmed bit is set. The confirmed bit is reset on DTC aging or on DTC clear.

See section 6.20 for how to configure confirmation.

2.1.13 Indicators

Indicators can be configured per DiagnosticEvent in the DEXT but they are managed on DTC level in the DM. This leads to the constraints described in section 2.4.4.

In case a DTC is confirmed or the DTC trip counter has reached the confirmation threshold again (2.1.12), it votes for activation of all connected Indicators with the configured behavioral type (e.g blinking, continuous etc.).

After a configurable number of operation cycles where the DTC was tested without failure the vote of the DTC is withdrawn. Is the number of this operation cycles configured to zero so is the vote withdrawn when the DTC is qualified the first time as passed.

See section 6.21 for how to configure Indicators.

2.1.14 Snapshot Data Records

2.1.14.1 Usage of Snapshot Records

Snapshot Data is additional data stored for the cause of a DTC. It is mainly used to provide details about the cause of a failure, such as the current voltage at the time of an implausible sensor reading. In the context of DEXT, a Snapshot Record is called FreezeFrame.

The DM supports two different algorithms for the storage of Snapshot Data:

> The first option is the 'Calculated Snapshot Record', for which snapshot records are captured with each passed-failed transition of an event. The snapshot records are enumerated from one upto the maximum number of snapshot records configured for each DTC.

based on template version 1.00.00

> The second option is the ‘Configured Snapshot Record’, which allows defining for each snapshot record in detail the storage trigger (when the snapshot record shall be stored/updated) and the update trigger (whether the snapshot record shall be updated with each new trigger). Currently available trigger options include ‘Failed’ and ‘Confirmed’.

In a common usecase, multiple causes could provoke the same failure symptom. Therefore, separate instances (called records) of this data can be stored for fault occurrences. An example would be to store one data record for the first, and another data record for most recent occurrence of a failure.

2.1.14.2 Configuration of Snapshot Records

The snapshot record enumeration is configured for each Event Memory separately.

FreezeFrameRecordEnumeration

```
<DIAGNOSTIC-MEMORY-DESTINATION-PRIMARY>
  <SHORT-NAME>ExamplePrimaryMemory</SHORT-NAME>
  <TYPE-OF-FREEZE-FRAME-RECORD-NUMERATION>CALCULATED</TYPE-OF-FREEZE-FRAME-RECORD-
      NUMERATION>
</DIAGNOSTIC-MEMORY-DESTINATION-PRIMARY>
<DIAGNOSTIC-MEMORY-DESTINATION-USER-DEFINED>
  <SHORT-NAME>ExampleUserDefinedMemory</SHORT-NAME>
  <TYPE-OF-FREEZE-FRAME-RECORD-NUMERATION>CONFIGURED</TYPE-OF-FREEZE-FRAME-RECORD-
      NUMERATION>
</DIAGNOSTIC-MEMORY-DESTINATION-USER-DEFINED>
```

Listing 2-15 Snapshot Record Enumeration

Since possible reasons for a failure of a DTC don’t change over time, the data stored as snapshot is defined once for each DTC:

DataIdentifierList

```
<DIAGNOSTIC-DATA-IDENTIFIER-SET>
  <SHORT-NAME>SnapshotContents</SHORT-NAME>
  <DATA-IDENTIFIER-REFS>
    <DATA-IDENTIFIER-REF DEST="DIAGNOSTIC-DATA-IDENTIFIER">/DEXT/ExampleDid</DATA-
        IDENTIFIER-REF>
    <DATA-IDENTIFIER-REF DEST="DIAGNOSTIC-DATA-IDENTIFIER">/DEXT/GenericDid</DATA-
        IDENTIFIER-REF>
  </DATA-IDENTIFIER-REFS>
</DIAGNOSTIC-DATA-IDENTIFIER-SET>

<DIAGNOSTIC-TROUBLE-CODE-PROPS>
  <SHORT-NAME>ExampleDTCProperties</SHORT-NAME>
  <FREEZE-FRAME-CONTENT-REF DEST="DIAGNOSTIC-DATA-IDENTIFIER-SET">/DEXT/SnapshotContents</
      FREEZE-FRAME-CONTENT-REF>
</DIAGNOSTIC-TROUBLE-CODE-PROPS>
```

Listing 2-16 Snapshot Content

For calculated snapshot records, the maximum number of snapshot records to be stored is configured for each DTC separately:

based on template version 1.00.00

MaxNumCalculatedSnapshotRecords

```
<DIAGNOSTIC-TROUBLE-CODE-PROPS>
<SHORT-NAME>ExampleDtcProperties</SHORT-NAME>
<MAX-NUMBER-FREEZE-FRAME-RECORDS>2</MAX-NUMBER-FREEZE-FRAME-RECORDS>
</DIAGNOSTIC-TROUBLE-CODE-PROPS>
```

Listing 2-17 Maximum number of calculated Snapshot Record

Whereas for configured snapshot records, record number of a data set, the point in time of its storage and its update configuration is then defined separately:

ConfiguredSnapshot

```
<DIAGNOSTIC-FREEZE-FRAME>
  <SHORT-NAME>ExampleSnapshot</SHORT-NAME>
  <RECORD-NUMBER>0x10</RECORD-NUMBER>
  <TRIGGER>TEST-FAILED</TRIGGER>
  <UPDATE>true</UPDATE>
</DIAGNOSTIC-FREEZE-FRAME>
```

Listing 2-18 Configured Snapshot Record

2.1.15 Extended Data Records

2.1.15.1 Usage of Extended Data Records

Extended data is additional data stored for the DTC itself, not its cause. It is mainly used to provide additional information related to every DTC, such as the current fault detection counter or the number of cycles until DTC aging.

2.1.15.2 Configuration of Extended Data Records

Since the data provided by an Extended Data Record relates to all DTCs, there is no DTC specific configuration of these records. The meaning is global:

based on template version 1.00.00

ExtendedDataRecord

```
<DIAGNOSTIC-EXTENDED-DATA-RECORD>
  <SHORT-NAME>ExampleExtendedRecord</SHORT-NAME>
  <RECORD-ELEMENTS>
    <DIAGNOSTIC-PARAMETER>
      <BIT-OFFSET>0</BIT-OFFSET>
      <DATA-ELEMENTS>
        <DIAGNOSTIC-DATA-ELEMENT>
          <SHORT-NAME>FaultDetectionCounter</SHORT-NAME>
          <SW-DATA-DEF-PROPS>
            <SW-DATA-DEF-PROPS-VARIANTS>
              <SW-DATA-DEF-PROPS-CONDITIONAL>
                <BASE-TYPE-REF DEST="SW-BASE-TYPE">/BaseTypes/uint8</BASE-TYPE-REF>
              </SW-DATA-DEF-PROPS-CONDITIONAL>
            </SW-DATA-DEF-PROPS-VARIANTS>
          </SW-DATA-DEF-PROPS>
        </DIAGNOSTIC-DATA-ELEMENT>
      </DATA-ELEMENTS>
    </DIAGNOSTIC-PARAMETER>
  </RECORD-ELEMENTS>
  <RECORD-NUMBER>0x10</RECORD-NUMBER>
  <TRIGGER>TEST-FAILED</TRIGGER>
  <UPDATE>true</UPDATE>
</DIAGNOSTIC-EXTENDED-DATA-RECORD>
```

Listing 2-19 Extended Data Record

Note

For all extended data records, irrespective of the configuration in 2-19, the storage trigger is always "Test-Failed" and they are updated with each new trigger.

DTCs can support a subset of available Extended Data Records, for example to restrict WWH-OBD counters to emission related DTCs.

ExtendedDataRecordProperty

```
<DIAGNOSTIC-TROUBLE-CODE-PROPS>
  <SHORT-NAME>ExampleDTCProperties</SHORT-NAME>
  <EXTENDED-DATA-RECORDS>
    <DIAGNOSTIC-EXTENDED-DATA-RECORD-REF-CONDITIONAL>
      <DIAGNOSTIC-EXTENDED-DATA-RECORD-REF DEST="DIAGNOSTIC-EXTENDED-DATA-RECORD">/DEXT/
          ExampleExtendedRecord</DIAGNOSTIC-EXTENDED-DATA-RECORD-REF>
    </DIAGNOSTIC-EXTENDED-DATA-RECORD-REF-CONDITIONAL>
  </EXTENDED-DATA-RECORDS>
</DIAGNOSTIC-TROUBLE-CODE-PROPS>
```

Listing 2-20 Extended Data Record Property

2.1.16 Clearing DTCs

Clearing DTCs from the tester is done with UDS service ClearDiagnosticInformation (`0x14`), refer to 2.1.5.3.1.

Clearing DTCs usually is a bulk operation, affecting all DTCs in an event memory. There can be

multiple sources (multiple testers, multiple applications) involved that this process has to be serialized.

Caution
If a clear job is pending, Diagnostics Daemon will reject new clear requests with NRC `0x22` (ConditionsNotCorrect).

Table 2-35 shows the checks and NRC handling on the DM side. Figure 28 in [40] provides further information on all checks needed in the context of UDS Service `0x14`.

Validations	NRC sent if unsuccessful
Message length	NRC `0x13`
Memory selection supported and valid	NRC `0x31`
Requested DTC (group) supported	NRC `0x31`
Clear DTC result	NRC `0x22`, NRC `0x21`

Table 2-35 NRC handling for ClearDiagnosticInformation request on DM side

Diagnostics Library (ara::diag) provides a method `ara::diag::DTCInformation::Clear()` for Clearing DTCs.

The full description of the AUTOSAR API `ara::diag::DTCInformation` and the `Clear()` method can be found in [5].

2.1.16.1 Clear Conditions

Diagnostics Library (ara::diag) provides an interface for **Clear Conditions** defined by AUTOSAR: `ara::diag::Condition`

The full description of the AUTOSAR API `ara::diag::Condition` can be found in [5].

2.1.17 DTC Aging

Aging or unlearning is the process of removing the **ConfirmedDTC** status from a DTC. As a consequence of unlearning, stored data concerning the fault occurrence are discarded as well. This includes all Snapshot Data, and Extended Data Records captured to file. See 2.1.15 for different types of extended data.

Trigger for aging a DTC is a period (operation cycle, see 2.1.11) in which the DTC was confirmed to be no longer defect.

2.1.18 Enable Conditions

Diagnostics Library (ara::diag) provides an interface for **Enable Conditions** defined by AUTOSAR: `ara::diag::Condition`

The full description of the AUTOSAR API `ara::diag::Condition` can be found in [5].

2.1.18.1 Initial State

By default, the initial state of an enable condition is `ConditionTrue`. However, it is possible to configure an enable condition initial value explicitly as well. For more details, please see [5].

The expected result whether the monitor is reported is illustrated in 2-36 and 2-37.

T_{EC} represents the time, from ECU startup, required for the daemon to be updated by the enable-condition-setter application with the incoming condition state.

T_M represents the time, from ECU startup, required for the daemon to update the monitoring applica-

based on template version 1.00.00

tion with the enable condition state.

Enable condition to monitor application connection time \ Requested enable condition state	kConditionTrue	kConditionFalse
$T_{EC} > T_M$	No Report (False Negative)	No Report
$T_{EC} < T_M$	Report	No Report

Table 2-36 Monitor behavior upon initialization with initial value of enable condition set to false

Enable condition to monitor application connection time \ Requested enable condition state	kConditionTrue	kConditionFalse
$T_{EC} > T_M$	Report	Report (False Positive)
$T_{EC} < T_M$	Report	No Report

Table 2-37 Monitor behavior upon initialization with initial value of enable condition set to true

2.1.19 Storage Conditions

Are not supported.

2.1.20 Client Authentication (Authorization)

Client authorization grants the application control over which clients shall receive specific access rights to execute services. The application should first check the trustworthy of the client. This can be done via UDS-service `0x29` (see 2.1.5.7) or any other authorization mechanism.

The client source addresses that need to be authorized must be known beforehand and pre-configured in the DEXT. A **default role** is set via DEXT configuration and can be changed at run-time by an already authorized instance. Authentication roles are treated as part of the diagnostic pre-conditions.

In order to authorize a client the application has to call `ara::diag::ExternalAuthentication::Get` (4.39) first to get access to the client specific authentication interface. The received `ara::diag::ClientAuthentication` (4.26) object can then be used to authorize the associated client with a specific **authentication role** or temporarily override the default roles without authorizing the client.

The authorization has a configured **timeout**. If the authorization was successful a `ara::diag::ClientAuthenticationHandle` (4.25) is returned for further interaction with the authorized client's session. Since the authorization has a configured **timeout**, the corresponding timer is started (see the configuration chapter for Authentication Service in [12]. which timeout modes are available). This handle object allows to override the existing dynamic access list while being in authorized state. Additionally the handle can refresh the running authorization timer or could de-authorize the client. If the authorization- or S3-timer expires, the corresponding client will be de-authorized. Upon de-authorization the client is reset to the **default role**.

A already authorized instance can be modified with a **dynamic access list**. It is used to override the authorization restrictions of a service for a particular diagnostic client. If no dynamic access list is set,

based on template version 1.00.00

this does not necessarily mean that the service is not executable.

A dynamic access list consists of group of so-called service heads, each of them represents a pattern-match for a UDS diagnostic request header. Any received UDS diagnostic service request(s) that matches such a pattern, will be allowed for execution regardless of their configured authentication pre-condition. Examples:

> If the service head addresses the UDS SID like `0x22`, according to authentication, read of any DID is allowed.
> If the service head addresses specific sub-service parameters like `0x221234`, only one service is allowed to be executed which is reading DID `0x1234`.

See 2.1.4.2 for the all service validation steps including the client authorization.

2.1.21 Security Event Log

The Security Event Log allows logging of security events with context information. Each Security Event (see section 2.1.21.2) is part of a group named Security Event Type (see section 2.1.21.1). The content of a Security Event Log can be retrieved by the application as a snapshot (see section 4.12.8). To be notified when a new Security Event has been reported to the Security Event Log, notifiers can be registered. Both for notification that the Security Event itself has been reported and that the content of a Security Event Log has changed (see section 4.12.12 and section 4.12.11). Furthermore, it is possible to provide the Security Event Log with a buffer entry (to all Security Event Types), each of which has a definable one-byte pattern (see section 4.12.7). The full Security Event Log API description can be found in section 4.12. The configuration is described in section 6.15.

2.1.21.1 Security Event Type

A Security Event Type contains multiple Security Events (see section 2.1.21.2) and when a Security Event is reported with context data, the data is stored into a buffer, which is common for all Security Events within this Security Event Type. The associated buffer has an overflow strategy, i.e. when the configured buffer size of a Security Event Type is reached, always the first stored buffer entry will be dropped so the new reported Security Event with context data or the new raw buffer entry is stored (circular buffer strategy). When a Security Event without context data is reported, only its counter is incremented, nothing will be stored in the associated Security Event Type's buffer.

2.1.21.2 Security Event

A Security Event contains a counter, which is incremented each time an event is reported. This counter is not incremented anymore once a configurable upper limit has been reached (overflow strategy). Reporting a Security Event is done using the interface described in section 4.11.

Persisted data

NOTE: The buffers and counters are also stored into persistency memory. After restart of the Diagnostics Daemon the values are reloaded from persistency memory.

2.1.22 Daemon Alive Check

Diagnostics Library (ara::diag) shall detect a disconnection of the Diagnostics Daemon to notify the diagnostic application that interfaces cannot currently be used. This is done by an alive check mechanism.

2.1.23 Log Diaglibrary-Daemon interaction

To easily trace the interactions between Daemon and the diagnotic library in the log, the following template is used:

based on template version 1.00.00

```
[JobId XX][Direction][Type][App XXXXXX] Some additional information
```

Throughout the different communication layers and across the communicating processes the same JobId and direction are used, this also makes the analysis of interactions easier.

The parameters in the template are all optional and described in table 2-38:

Parameter	Description
JobId	An interaction requiring a response has a unique Id to identify it
Direction	> ALRq: Request sent from diag library to the Daemon > DmRs: Response sent from Daemon to a diagnostic library request > DmRq: Request sent from Daemon to the library > ALRs: Response sent from diag library to a Daemon request > ALEv: Event sent from the library to the Daemon > DmEv: Event sent from Daemon to Diag library
Type	The transaction type (For example: ServiceValidation, ReadData ...)
App	In case of a daemon request, it's necessary to identify to which application the request is sent. The application handle is logged in decimal form.

Table 2-38 Description of logging parameters

Below some examples can be found. Additionally to the above mentioned information, the application ids and context ids are also logged according to section 3.9 and section 3.10. The applications IDs used in the examples are described in table 2-39.

Application ID	Description
VDMd	The vector diagnostic manager daemon ID
APPL	The Adaptive Application ID

Table 2-39 Application IDs of logging examples

Log daemon request

```
[VDMd vdrd][DEBUG] [JobId 27][DmRq] Pending request added to the list
[VDMd vdrd][INFO] [JobId 27][DmRq][ReadData][App 1591435491] Request data from library
[APPL vdal][INFO] [JobId 27][DmRq][ReadData] Received request with data=0x4E_1F
[APPL vdal][INFO] [JobId 27][DmRq] Execute job and convert response
[APPL vdal][INFO] [JobId 27][ALRs] Send positive response with data 0xXX_XX...
[APPL vdra][INFO] [JobId 27][ALRs][PositiveResponse] Sending response to Daemon
[VDMd vdrd][INFO] [JobId 27][ALRs][PositiveResponse] Report positive response
```


based on template version 1.00.00

Log diag library request

[APPL vdal][VERBOSE] Client authentication found for source address = 0x0E80
[APPL vdra][INFO] [JobId 5][ALRq][Authenticate] Send request to Daemeon with [...]
[APPL vdra][VERBOSE] [JobId 5][ALRq] Add pending request to queue
[APPL vdra][DEBUG] [JobId 5][ALRq] Send event on DM_IPC.
[VDMd vdrd][INFO] [JobId 5][ALRq][Authenticate][App 1591435491] Received a lib request with [...]
[VDMd vdrd][INFO] [JobId 5][DmRs][App 1591435491] Send positive response to library
[APPL vdra][INFO] [JobId 5][DmRs] Response for daemon request received
[APPL vdra][VERBOSE] [JobId 5][DmRs] Removing request to daemon queue

2.1.24 Root Cause Event Associated Identification

This feature allows a tester to identify the root cause of DTC failure. Please refer to section 6.19 for more information on how to configure the elements of this feature.

The root cause event associated identification can be read as a part of an extended data record or a part of a snapshot record via service ReadDTCInformation (section 2.1.5.3.3).

Caution

Services ReadDataByIdentifier (section 2.1.5.4.1), ReadDataByPeriodicIdentifier (section 2.1.5.4.3), and DynamicallyDefineDataIdentifier (section 2.1.5.4.4) are not used to read the root cause associated identification. Therefore, only internal DIDs can be used to store the root cause event associated identification.

Note

In case of combined events DTC, the root cause event associated identification shall correspond to the event which triggered the DTC failure (Transition of 0 to 1 of the testFailed-Bit in the DTC Status byte).

2.1.25 Failure Event Associated Identification

This feature allows to report the status and associated id of all the events that are correlated to any DTC. Please refer to section 6.14 for more information on how to configure the elements of this feature.

The failure event associated identification is read as a part of an extended data record via service ReadDTCInformation (section 2.1.5.3.3).

If the total number of requested events is less than `MAX-NUMBER-OF-ELEMENTS` in `DIAGNOSTIC-DATA-ELEMENT` configuration of internal data element, then unused bytes are padded with value "0xFF". Also, additional events are dropped when total number of requested events is more than `MAX-NUMBER-OF-ELEMENTS`.

2.1.26 Caching of application call

The application may use the Diagnostics Library (ara::diag) to call Diagnostics Daemon functions at any point in time. For the application calling such a function it is unknown if the DM is actually currently available. Typical examples are:

> At startup the application is up and running and reporting information but the DM is not yet started and likely be up shortly after the call

based on template version 1.00.00

> The communication between the Diagnostics Library (ara::diag) interface and the Diagnostics Daemon itself is temporarily interrupted

To avoid each application to implement a complex strategy reacting on the availability of the Diagnostics Daemon, the Diagnostics Library (ara::diag) is providing a caching mechanism, which caches the information and forward it to the Diagnostics Daemon once it is available.

2.1.26.1 Activation Line State

The Diagnostics Library (ara::diag) is caching the latest update of the ActivationLineState (see section 4.32.9), when the Diagnostics Daemon is not available. Once the connection is (re-)established the last reported ActivationLineState is sent to the Diagnostics Daemon and evaluated.

2.1.26.2 Condition

The Diagnostics Library (ara::diag) is caching the latest reported condition described in section 2.1.18, when the Diagnostics Daemon is not available. Once the connection is (re-)established the last reported condition is sent to the Diagnostics Daemon and evaluated.

2.1.26.3 Conversation

2.1.26.3.1 Reset to default session

The Diagnostics Library (ara::diag) is caching the latest reset to default session notification (see section 4.28.10), when the Diagnostics Daemon is not available. Once the connection is reestablished the cached reset to default session notification is sent to the Diagnostics Daemon and evaluated. If a new connection is detected, the cached reset to default session notification is discarded.

2.1.26.3.2 Overriding S3 timeout

The S3 timer can be set to a specific value for the current session through the use of the respective API (see 4.49.7). Otherwise the value configured is used and if no S3 timeout value is configured, it is set to 5 ms.

Note
The overridden S3 timer value is not persisted alongside with the session to be started after ECU reset.

2.1.26.4 DTC Information

The Diagnostics Library (ara::diag) is caching calls to EnableControlDtc, when the Diagnostics Daemon is not available. Once the connection is (re-)established the EnableControlDtc will be forwarded to the Diagnostics Daemon.

2.1.26.5 Monitor Action

The Diagnostics Library (ara::diag) is caching calls to the ReportMonitorAction method of a diagnostic monitor (see 2.1.10.2), when the Diagnostics Daemon is not available. Once the connection is (re-)established the cached monitor actions will be forwarded to the Diagnostics Daemon.

Note

The reporting of monitor actions depends on the current state of the enable condition. If the enable condition is disabled, the reported monitor actions will be ignored. Therefore, during periods of disconnection the last known states will be used. Upon (re-)connection the new states will be received and cached monitor actions may be discarded before being sent to the Diagnostics Daemon if it turns out that in the meantime

> the associated operation cycle was restarted
> the associated enable condition was disabled or
> the event was cleared.

2.1.26.6 Operation Cycle

When the Diagnostics Daemon is not available and the operation cycle is restarted, the Diagnostics Library (ara::diag) caches the request to restart the operation cycle (if the operation cycle is restarted multiple times only one restart request is cached). Once the connection is (re-)established the request is sent to the Diagnostics Daemon and evaluated.

2.1.26.7 Security Event Log

Caching of Security Event Log

NOTE: The different API calls are not synchronized with each other. This means that the order of security events/buffer entries reported through different APIs is not necessarily conserved during periods of disconnection.

2.1.26.7.1 Report Security Event

The Diagnostics Library (ara::diag) is caching all reported SecurityEvents (see section 2.1.21.1) accumulated, when the Diagnostics Daemon is not available. Once the connection is (re-)established, all cached reported SecurityEvents are then send to the Diagnostics Daemon. This applies when reporting a SecurityEvent directly (see section 4.11.5), as well as through the generic SecurityEvent reporting API (see section 4.12.10).

2.1.26.7.2 Add raw buffer entry to Security Event Types

The Diagnostics Library (ara::diag) is caching all added raw buffer entries (see section 2.1.21) accumulated, when the Diagnostics Daemon is not available. Once the connection is (re-)established, all cached added raw buffer entries are then send to the Diagnostics Daemon.

2.1.27 Diagnostic RPC Usage Intensity Measurement

Caution

This is a MICROSAR product internal feature and thereby the usage of this feature is restricted.

Caution

This feature is not steady (see 2.10 for more details regarding Steady Mode).

Caution

This feature shall not be enabled in a production environment.

DM could be configured to log usage intensity of diagnostic RPC. To enable the measurement, the DM process shall be indicated with the environment variable "MECA_SHMEM_PATH". The environment variable must be to set to a system path which is writeable. If the environment variable is not set, the feature remains turned off by default. An example of the same is provided in listing 2-21

Enabling RPC usage intensity measurement

```
MECA_SHMEM_PATH="\tmp\diagnostic_rpc_shmem"
```

Listing 2-21 Example to enable RPC intensity measurement

If this feature is enabled, DM writes the statistics of transmitted and received message lengths into the specified shared memory file. The minimum, maximum, most recent and average values of message lengths are logged along with the number of samples. The message lengths are logged for all applications communicating with the DM, for each diagnostic context (either Diagnostics Library (ara::diag) or Intrusion Detection System Manager Library (ara::idsm)). The context id of Diagnostics Library (ara::diag) is indicated as 0 and that for Intrusion Detection System Manager Library (ara::idsm) is indicated as 1. An example of the keys that will be logged is provided in the table 2-40.

Type	Context	Key
Tx	Diagnostics Library (ara::diag)	amsr/diag/rpc/9876_0_TxParam
Tx	Intrusion Detection System Manager Library (ara::idsm)	amsr/diag/rpc/9876_1_TxParam
Rx	Diagnostics Library (ara::diag)	amsr/diag/rpc/9876_0_RxParam
Rx	Intrusion Detection System Manager Library (ara::idsm)	amsr/diag/rpc/9876_1_RxParam

Table 2-40 RPC intensity keys for sample Application ID 9876 with two contexts

These values can be consumed by another process running in parallel.

based on template version 1.00.00

2.2 Deviations

Currently, deviations from features specified in [5] only consist of limitations as listed in table 2-42.

The following aspects are not according to AUTOSAR:

> Running state is reported to Execution Manager (ara::em) before the initialization is finished by the DM. This does not properly meet the specified requirement in [8] Chapter 7.4.1. There is a cmake variable `AMSR_DIAG_DAEMON_REPORT_RUNNING_AFTER_INIT` to change that behaviour (see table 3-12).

based on template version 1.00.00

2.3 Additions/Extensions

The following features are provided beyond the AUTOSAR standard:

Feature	Description
Service Processing Notification	see 2.1.4.5
Security Event Log	see 2.1.21
Root Cause Event Associated Identification	see 2.1.24
Observation of the connection state	see 2.6.2.0.6
Event status as a UDS DTC status byte	see 4-448
Ignore authentication preconditions	see 2.1.4.4.3
Supported UDS services and their configuration	see 2.1.5
Pipelined Data Transfer	see 2.1.5.9.6
Configuration and overriding of S3 timer duration	see 6.2.2 and 2.1.26.3.2
UDS NRC mapping	see 2.1.5.15

Table 2-41 Features provided beyond the AUTOSAR standard

based on template version 1.00.00

2.4 Limitations

Table 2-42 lists the features specified in [5] which are supported with deviations:

2.4.1 DTC status processing

> DTC storage is unlimited, DTC displacement never applies.

2.4.2 Event Debouncing

2.4.2.1 Counter-based debouncing

> Internal debounce counter is always initialized with `0x0`
> Persistency of internal debounce counter is not supported

2.4.3 DTC confirmation

> For limitations only applying on configuration see 6.20
> Occurrence Counter increment strategy 'confirmedDtcBit' is not supported, only 'testFailed'-only strategy is supported: On DTC transition to test failed, the occurence counter will always be increment regardless the confirmed DTC bit value.

2.4.4 Indicators

> The DTC operation cycle is always used as healing operation cycle
> For event combinations all events must have the same indicator configuration, i.e. same indicators involved, same behavior on the corresponding indicator, same number of healing cycles etc.
> User controlled WarningIndicatorRequested-bit is not supported
> Only the configuration parameters shown in section 6.21 are supported

2.4.5 DoIP

> Not all features of DoIP according to [39] are supported, see section 2.1.1.3.1.
> CAP_NET_RAW capability is needed for kernel version lower than 5.7-rc1 in Linux.

2.4.6 Single process instance

Applications that contribute to Diagnostics shall be instantiated as a process only once. This is reflected by the constraint in configuration that the `Executable` representing the given application shall be referenced only by a single `ProcessDesign` element and respectively by a single `Process` element.

2.4.7 Endianness

> For Data types in DEXT, as illustated in 2-22, referenced by data elements of DIDs/RIDs, only Motorola (MostSignificantByteFirst) byte order is internally supported. If Intel (MostSignificantByte-Last) byte order is required, DID/RIDs must be configured as a generic interface where the byte order is handled by the application.

Configuration of a base type byte order

```
<SW-BASE-TYPE>
  <SHORT-NAME>BaseType</SHORT-NAME>
  <BYTE-ORDER>MOST-SIGNIFICANT-BYTE-FIRST</BYTE-ORDER>
</SW-BASE-TYPE>
```

Listing 2-22 Configuration of a base type byte order

based on template version 1.00.00

> For Data types in DEXT referenced by data elements of extended data records, Only Motorola byte order is supported. Intel byte order is not supported.

2.4.8 Authentication

> User defined event memory is not restricted by Authentication

2.4.9 Security Event Log

> Security Event Log and Idsm functionalities are not supported at the same time. The selection of the functionality to support is done using ARXML configuraiton (See section 6.15.1).
> IDSR communications are limited only via PDU protocol

2.4.10 Storage of Environmental Data

In order to optimize memory usage and performance, the size of environmental data collection queue for a DTC is limited internally to 128. So, if a DTC toggles more than 128 times within a short span of time, it could lead to environmental data (Snapshot Records, Extended Data Records, ..) not being collected for the last toggles.

2.4.11 Limitations on QNX with an io-sock network stack

> Diagnostic manager will not work correctly with io-sock network stack in QNX in the following situations.
> When the network is IPv4 and there are multiple network interfaces which share same directed broadcast address. (eg: interface 1 having IP Address 192.168.0.1 with directed broadcast address 192.168.0.255 and interface 2 having IP Address 192.168.0.2 with directed broadcast address 192.168.0.255) and there is a need to send a packet to limited broadcast address 255.255.255.255 on interface 1, the message might be sent on the wrong interface, ie. interface 2. Workaround is to avoid having single host with different IP Addresses belonging to same network.
> The io-sock network stack does not support VLAN priority of a socket. This means that the customer shall not try to configure a VLAN priority in ARXML. Instead the customer shall use QNX provided means to have VLAN priority (eg: `ifconfig vlanpcp`). See [37]

based on template version 1.00.00

Limitation	Description
Configuration of multiple Software Clusters	Diagnostics Daemon supports one Software Cluster
DTC status processing	See 2.4.1
Custom DTC groups	DM only supports group ALL DTCs (`0xFFFFFF`)
Monitors	Monitor initialization for storage condition re-enabling reason is not supported
Event debouncing	See 2.4.2
DTC Confirmation	See 2.4.3
Indicators	See 2.4.4
DynamicAccessListDiagServiceBuilder::ByteRange	The ByteRange is not supported
ReentrancyType	ReentrancyType is not supported
Storage Conditions	Storage conditions are not supported
DoIP	See 2.4.5
Prioritization	Prioritization of Diagnostic Conversations is not supported
UDS Services	For limitations regarding UDS services, please check the sections of the respective services in 2.1.5
SOVD Request Validation and Verification	For limitations regarding SOVD Request Validation and Verification, please check the sections of the respective validation steps in section 2.1.6
SOVD Request Processing	For limitations regarding SOVD Request Processing, please check the sections of the respective SOVD REST API in section 2.1.7
SOVD Lock ports	For limitations regarding SOVD lock ports please refere to section 6.26.4.7
API: Re-Entrant PortInterfaces	Port interfaces do not support configurable reentrancy
API: Single process instance	See 2.4.6
API: PortInterfaces Notification	A registered notifier shall not actively wait for another PortInterfaces Notification to prevent dead lock.
Routing Activation	Only Default routing activation is supported.
IDSM	EventId is currently restricted to 2 bytes for Security Events.
S3 timeout	The overriden S3 timer value is not persisted alongside with the session to be started after ECU reset.

Table 2-42 Limitations

based on template version 1.00.00

2.5 MetaInfo

The following MetaInfo keys specified in [5] are supported/unsupported:

Key	Supported
kSA	yes
kTA	yes
kTAType	yes
kRequestType	yes
kSuppPosResponse	yes
kSecurityTimerElapsed	no
kRequestHandle	yes
kLocalIP	only for DoIP
kLocalPort	only for DoIP
kRemoteIP	only for DoIP
kRemotePort	only for DoIP
kDtc	yes

Table 2-43 Supported/unsupported MetaInfo keys

The following MetaInfo key(s) are supported in addition to [5]:

Key	Values	Description	Constraints
SDT_MTypes	ENABLED; DISABLED	Specifies whether the current diagnostic request is sent over SecuredData-Transmission.	Supported only if UDS Service SecuredDataTransmission (`0x84`)is enabled

Table 2-44 Vendor specific MetaInfo keys

based on template version 1.00.00

2.6 States

2.6.1 Diagnostics Daemon

Diagnostics Daemon has four states:

> Not initialized
> Initializing other stack components
> Running
> Shutting down

2.6.1.0.1 Not initialized

When the ECU is started, Diagnostics Daemon is in a not initialized state. In this state no aspects related to diagnostic functionality can be performed. Diagnostics Daemon can be initialized using `ara::core::Initialize()`.

2.6.1.0.2 Initializing other stack components

Diagnostics Daemon initializes the AUTOSAR runtime environment (ARA) within `ara::core::Initialize()`. Refer to section 2.9.1.

2.6.1.0.3 Running

After initialization of other stack components, Diagnostics Daemon is able to handle all aspects related to diagnostic functionality and to react to POSIX exit signals.

2.6.1.0.4 Shutting down

Diagnostics Daemon shutdown is triggered by exit signal, see 2.7. Diagnostics Daemon de-initializes the AUTOSAR runtime environment and other stack components (ARA) within `ara::core::Deinitialize()`. Refer to section 2.9.1.

2.6.2 Diagnostics Library (ara::diag)

Diagnostics Library (ara::diag) has five states:

> Not initialized
> Initializing other stack components
> Initialized
> Disconnected
> De-initialized

2.6.2.0.1 Not initialized

In this state, the Diagnostics Library (ara::diag) does not operate. Diagnostics Library (ara::diag) can be initialized using `ara::core::Initialize()`.

2.6.2.0.2 Initializing other stack components

Diagnostics Library (ara::diag) initializes the AUTOSAR runtime environment (ARA) within `ara::core::Initialize()`. Refer to section 2.9.2.

2.6.2.0.3 Initialized

In this state, the Diagnostics Library (ara::diag) is fully operational. Requests to all APIs are accepted.

2.6.2.0.4 Disconnected

In this state, Diagnostics Library (ara::diag) is disconnected from Diagnostics Daemon. Requests to all APIs is not transfered to the Diagnostics Daemon.

based on template version 1.00.00

2.6.2.0.5 De-initialized

Diagnostics Library (ara::diag) de-initialization is triggered by calling `ara::core::Deinitialize()`. Diagnostics Library (ara::diag) de-initializes the AUTOSAR runtime (ARA) within `ara::core::Deinitialize()`. Refer to section 2.9.2.

2.6.2.0.6 Observers

It is possible to register an observer to get notified when Diagnostics Library (ara::diag) switches to the Initialized or Disconnected state. The observer takes `amsr::diag::NotificationReason` as a parameter and can react accordingly. To register an observer two methods are available in `amsr/diag/lib_ara_diag.h`:

> `void AddIsReadyNotifier(LibraryStateNotifier notifier) noexcept` This method adds an addiotional notifier.

> `void SetIsReadyNotifier(LibraryStateNotifier notifier) noexcept` This method removes already-registered notifiers and adds a new single notifier.

If a new observer is added or set when the library is already initialized, the observer is directly called with initialized as notification reason.

based on template version 1.00.00

2.7 Handling of POSIX Signals

All code of the component must be protected against the concurrent reception of POSIX Signals. If the code of the component cannot be protected against the concurrent reception of POSIX Signals, the reaction to the reception of a Signal must be the immediate termination of the process without further processing of AMSR code. Please see the AMSR product manuals for details how to achieve this in customer applications.

Library parts of this component do not interact with POSIX Signals.

To protect AMSR code against the concurrent reception of POSIX Signals in the Application Diagnostics Daemon, Diagnostics Daemon blocks the reception of all POSIX Signals with the exception of the POSIX Signals listed below. For some of the listed Signals, an explicit handling is implemented. For other POSIX Signals, the Signal is not blocked but the default handling of the underlying POSIX System is used. In this case, the expectation on the default handling is documented. Note that the actual actions performed may be dependent on the configuration of the underlying POSIX System (e.g., whether Core Dump files are written to disk when a POSIX Signal is received). Non-blockable signals are processed as specified by the underlying POSIX environment.

POSIX Signal	Reaction
SIGABRT	System Default Behavior. The process is expected to terminate without further processing of AMSR code.
SIGBUS	System Default Behavior. The process is expected to terminate without further processing of AMSR code.
SIGFPE	System Default Behavior. The process is expected to terminate without further processing of AMSR code.
SIGILL	System Default Behavior. The process is expected to terminate without further processing of AMSR code.
SIGINT	Explicitly handled by component implementation. Causes an orderly shutdown of the process.
SIGSEGV	System Default Behavior. The process is expected to terminate without further processing of AMSR code.
SIGTERM	Explicitly handled by component implementation. Causes an orderly shutdown of the process.

Table 2-45 Reactions to blockable POSIX Signals not blocked by the process

based on template version 1.00.00

2.8 Error Handling

For a detailed description of error handling refer to chapter 4 Diagnostic Library API Description.

2.8.1 Exit Codes

In case of a successful normal termination after receiving a SIGTERM signal, a process returns a zero exit code. Diagnostics Daemon does not return non-zero codes after termination in response to errors.

Diagnostics Library (ara::diag) is not an executable but a library, therefore it does not return any exit codes.

Caution

In case of an abort a non-zero exit code is returned

2.8.2 Abnormal Terminations

2.8.2.1 Diagnostics Daemon

Unexpected errors will terminate the Diagnostics Daemon process. Please see [13, chapter “Abnormal Terminations”, section “Application Components and Daemons”] for details about event notification.

2.8.2.2 Applications using Diagnostics Library (ara::diag)

Any unexpected error will either be propagated to the caller or terminate the process. Please see [13, chapter “Abnormal Terminations”, section “Library Components”] for the mechanism used and details about event notification.

based on template version 1.00.00

2.9 Thread Handling

This chapter describes the following aspects of multithreading for the communication of Diagnostics Daemon, and Diagnostics Library (ara::diag):

- The creation of threads
- The number of threads created
- The execution phase in which each thread is created
- The purpose of the created threads
- The configuration of the thread settings
- The order of creation and termination of threads

For threads configuration concerning Intrusion Detection System Manager Library (ara::idsm), please refer to [23]. For communication of Diagnostics Daemon, and a diagnostic tester, please refer to the respective technical references accordingly.

The configuration of thread settings (e.g., scheduling priority) are supported for Diagnostics Library (ara::diag), and for Diagnostics Daemon for each of its created threads. Detailed explanation on available settings and how to configure them can be found in [36].

Caution

Other components that DM depends on, like logging may also create their own threads at runtime.

2.9.1 Diagnostics Daemon

For each SoftwareCluster the Diagnostics Daemon executable spawns threads in different execution phases of the component lifetime (see Table 2-46). The number of spawned threads depends ultimately on the Diagnostics Daemon configuration. To configure the settings for threads in the Diagnostics Daemon use the identifier "DiagDefault". If the settings for the DiagDefault identifier are not specified and for threads that have no identifier specified, the settings of the parent thread are applied.

based on template version 1.00.00

Thread Name	Thread Identifier	Mult.	Phase	Description
vDiagRpcBcpDaem	N/A	1	Init	Server thread for communication towards Idsm
vDMRun	N/A	1	Init	Main Diagnostics Daemon thread. Starts all other threads
vDMS*EM*	DiagDefault	(0..n) · 2	Running	Two threads per Event Memory, see Model parameter `DIAGNOSTIC-MEMORY-DESTINATION-PRIMARY` and `DIAGNOSTIC-MEMORY-DESTINATION-USER-DEFI`
vDMS*FMUp	DiagDefault	1	Running	One Fault Memory Update thread
vDMS_CTx*	DiagDefault	n	Init	Thread which handles the transmission of messages, number of threads equals the Model parameter `SD GID="numberOfConversations"`
vDmsMh*	DiagDefault	n	Running	Thread that handles tester communication, number of threads equals the Model parameter `SD GID="numberOfConversations"`
vDMSNtf	DiagDefault	1	Running	One Notification thread
vDMSPerUp	DiagDefault	1	Running	One Persistency Update thread
vDMWdg_EvLp	N/A	(0..1)	Init	Thread spawned if watch dog is enabled and measures the execution time of added tasks section 3.8.3 and section 3.8.3.1
vIdsm	DiagDefault	(0..1)	Init	If Idsm is configured, this thread is spawned for requests and notifications toward Intrusion Detection System Manager Library (ara::idsm)
vPDIDHandlerThr	DiagDefault	1	Init	Thread which handles the periodic DIDs
vRootSwc	DiagDefault	1	Running	General purpose thread for synchronization offered by Diagnostics Daemon. For usage, see 4.2 and 4.1
vRoEExecThr	DiagDefault	1	Init	Thread which handles the RoE execution
vSDT*	DiagDefault	(0..n)	Init	"NumberOfConversations" threads if SDT is enabled

Table 2-46 Threads spawned by Diagnostics Daemon executable

All threads are joined during shutdown of the Diagnostics Daemon. Table 2-47 lists the startup and shutdown order of threads in Diagnostics Daemon.

based on template version 1.00.00

Startup order	Shutdown order
vDMRun	vDiagRpcBcpDaem
vDMWdg_EvLp	vDMS*FMUp
vDiagRpcBcpDaem	vDMS*EM*D*
vDMS_CTx*	vIdsm
vDmsMh*	vDMSNtf
vSDT*	vDMSPerUp
vIdsm	vRoEExecThr
vPDIDHandlerThr	vDmsMh*
vRoEExecThr	vDMS_CTx*
vRootSwc	vPDIDHandlerThr
vDMSPerUp	vSDT*
vDMSNtf	vRootSwc
vDMS*FMUp	vDMWdg_EvLp
vDMS*EM*D*	vDMRun

Table 2-47 Thread order of Diagnostics Daemon executable

2.9.2 Diagnostics Library (ara::diag)

The Diagnostics Library (ara::diag) spawns threads in the initialization phase of the component (see Table 2-48).

For thread setting configuration in Diagnostics Library (ara::diag), Table 2-48 shows the configurable Thread Identifier(*BswThreadConfiguration SHORT-NAME*) element for each thread.

Thread Name	Thread Identifier	Mult.	Phase	Description
vAdiagRpcRctr	N/A	(0..1)	Init	Reactor driver for communication from Application to Daemon
vAdiagFastJob	ADFastTask	1	Init	Thread that handles cancellation of jobs and connection related notifications
vAdiagLowJob	ADProcessingTask	1	Init	Thread that handle background jobs like ROE, Periodic Data Polling from Daemon
vAdiagMedJob*	ADProcessingTask	n	Init	Threads that handle requests from Daemon see Model parameter `numberOfParallelRequestExecutions` in 6.11
vAdiagPeriodic	ADPeriodicDataScheduler	1	Init	Thread which handles the polling of various data elements
vAdiagRctr	ADTimerTask	1	Init	Thread that works as a timer driver

Table 2-48 Threads of Diagnostics Library (ara::diag)

based on template version 1.00.00

Thread Setting Configuration

Suggested thread setting relations in regard to performance and "real-time" like-behavior for Thread Identifier mentioned in table 2-48:

ADTimerTask == ARpcConMon >= ADFastTask >= ADPeriodicDataScheduler >= ADProcessingTask

All threads are joined during shutdown of the Diagnostics Library (ara::diag). Table 2-49 lists the startup and shutdown order of threads in Diagnostics Library (ara::diag).

Startup order	Shutdown order
vAdiagRctr	vAdiagRpcRctr
vAdiagLowJob	vAdiagPeriodic
vAdiagMedJob*	vAdiagRctr
vAdiagFastJob	vAdiagFastJob
vAdiagPeriodic	vAdiagLowJob
vAdiagRpcRctr	vAdiagMedJob*

Table 2-49 Thread order of Diagnostics Library (ara::diag)

based on template version 1.00.00

2.10 Steady Mode

The diagnostic cluster supports Steady Mode: Reserving memory to avoid (de-)allocations on the operating system heap during runtime to the extent feasible. If no memory resources are reserved (i.e. not configured at all), no functionality will use Steady Mode (i.e. to not cause (de-)allocations at runtime) and the process-wide Default Memory Resource is used instead.

Reserved memory exhaustion

The program aborts if memory resources are reserved (i.e. properly configured) and the associated memory is exhausted, unless the resources are configured to acquire additional memory e.g. from the operating system heap and the request can be served.

2.10.1 Memory Demand

2.10.1.1 Measuring the memory demand

The size of the diagnostic memory resources is to be measured by the user. It is the responsibility of the user to define a suitable test to reliably determine the maximum memory demanded. The maximum size of the diagnostic memory resources is reported as a log message upon their destruction (see 2.10.2).

2.10.1.2 Steady and Non-Steady Functionality

The internal diagnostic implementation may not differentiate between steady and non-steady functionalities. Therefore, the use of a non-steady functionality can additionally increase the memory need for a resource. This should be taken into account when measuring the size of the memory resources.

Reduce memory demand for non-steady functionality

For diagnostic service handling `0x34` and `0x38` we suggest to use the pipelined data transfer in order to omit additional memory demand for IPC (see 2.1.5.9.6).

2.10.2 Diagnostics Library (ara::diag)

The Diagnostics Library (ara::diag) provides APIs annotated as `steady : true`, which avoid (de-)allocations in the system memory to the extent feasible. For this purpose, the dedicated memory resources “AraDiag” and “DiagRPC-App” are reserved during initialization (using the system memory). The Steady Mode is active when these resources are reserved, i.e. properly configured (see 6.29.2). During de-initialization, these resources are destroyed and all allocated memory is freed.

Memory is obtained from the Default Memory Resource in the following events / scenarios:

- (De-)Initialization
- Using non-steady public API
- (Re-)connect to and disconnect from the Diagnostics Daemon

Log memory resource size on de-initialization

The maximum memory utilization of a diagnostic application’s resource is printed via an info log message during de-initialization with the following content: “Maximum memory utilization of destroyed diagnostic resource”. The associated logging context is “vDRS”.

based on template version 1.00.00

2.10.2.1 AraDiag Memory Resource

The AraDiag Memory Resource is used exclusively by the internal implementation, which covers the dynamic memory requirements of the Diagnostics Library (ara::diag) API. Since specific APIs require the transfer of data ownership allocated in the AraDiag Memory Resource to the user, it must be ensured that all such data is released before Diagnostics Library (ara::diag) is de-initialized.

Provided data ownership by Diagnostics Library (ara::diag)

Storing data provided by Diagnostics Library (ara::diag) within external implementation is not recommended in the context of Steady Mode, as this would prevent the associated memory from being returned to Diagnostics Library (ara::diag) at runtime, potentially increasing the overall memory demand of AraDiag Memory Resource. If data is still stored by the application, it must be destroyed before Diagnostics Library (ara::diag) is de-initialized.

2.10.2.2 RPC Memory Resource (Application Side)

A diagnostic application uses IPC to exchange data with the Diagnostics Daemon. The DiagRPC-App memory resource is designed in such a way that the communication, where applicable, avoids (de-)allocations in the system memory. Since both Diagnostics Library (ara::diag) and ldsm use the same channel for communication, both libraries have access to this resource, which must be considered when measuring the size of this memory resource.

2.10.3 Diagnostics Daemon

The diagnostic daemon process uses the Default Memory Resource to handle diagnostic services, fault memory management and diagnostic application requests that are specified as steady. Configure the Default Memory Resource as documented in [15].

The following diagnostic services are steady:

- DiagnosticSessionControl (`0x10`)
- ReadDtcInformation (`0x19`)
- ReadDataByIdentifier (`0x22`)
- WriteDataByIdentifier (`0x2E`)
- RoutineControl (`0x31`)
- TesterPresent (`0x3E`)

The following events / scenarios may require dynamic (de-)allocations in the system memory:

- (De-)Initialization
- NV-data handling
- (Re-)connect to and disconnect from a diagnostic application / tester
- Using TLS for DoIP

based on template version 1.00.00

3 Integration

Note
For Idsm application integration, please refer to Idsm technical reference [23].

3.1 External Build Dependencies

There are no external build dependencies.

3.2 Integrating Diagnostics Library (ara::diag) into an Application

3.2.1 Initialization and Deinitialization

Before usage, Diagnostics Library (ara::diag) and all its dependencies (see section 3.1) need to be initialized. Conversely, on shutdown, Diagnostics Library (ara::diag) and its dependencies need to be deinitialized, and after deinitialization there shall be no interaction with any API of Diagnostics Library (ara::diag). The AUTOSAR defined APIs `ara::core::Initialize` and `ara::core::Deinitialize` can be used for this purpose.

Caution
Any interaction with Diagnostics Library (ara::diag) shall be restricted to the time period between initialization and deinitialization of Diagnostics Library (ara::diag). This in particular also includes the construction and destruction of diagnostic port interface instances.

3.2.2 Registering and Unregistering interaction with Diagnostic Ports

For all methods provided by the application to be called by the Diagnostics Library (ara::diag) it must be ensured that they are unregistered in a proper way before calling them becomes invalid.

3.2.2.1 Provided Ports

If you instantiate a provided port and call `Offer()` you must finally call `StopOffer()` to inform Diagnostics Library (ara::diag) about end of the interaction in order to ensure safe destruction of the port.

Caution
If `Offer()` is called for a port and it's destructor runs before `StopOffer()` is called, Diagnostics Library (ara::diag) cannot guarantee safe application object destruction. In this Situation Diagnostics Library (ara::diag) will issue a warning containing the message: "StopOffer was not called before destructor of offered API port".

Note
If a port is not offered and a request is received on that port, UDS NRC `ResourceTemporarilyNotAvailable` is issued.
An exception to that would be service validation port. UDS NRC `kGeneralReject` is issued.

3.2.2.2 Providing Notifiers to Diagnostics Library (ara::diag)

For notifiers passed as `std::function` to ports, the functions might be called as long as the port is not destructed.

A notifier that is passed as `std::function` to a Conversation (4.28) or a ClientAuthentication (4.26) might get called until the Diagnostics Library (ara::diag) is completely deinitialized (i. e. the call to `ara::core::Deinitialize()` has returned).

A notifier that is passed as `std::function` to a CancellationHandler (4.24) might get called immediately if it's already canceled and must be valid to be called until the CancellationHandler is destroyed (if CancellationHandler was not moved). Since the CancellationHandler is always received as a function parameter, it will be destroyed only after the local variables of the function body where the CancellationHandler was received. So, care should be taken to make sure the notifier function does not hold a reference to any local variable.

A notifier that is passed as `std::function` to a ReleaseHandler (4.10) will get called exactly once, and it must be valid to be called until that point in time (or until it is replaced by another notifier).

A notifier that is passed to the Diagnostics Library (ara::diag) as a readiness notifier (see 4.49.1 or 4.49.4 for example) might get called until the Diagnostics Library (ara::diag) is completely deinitialized (i. e. the call to `ara::core::Deinitialize()` has returned).

In all the above cases, it's guaranteed that the `std::function` will no longer get called if it is replaced by another notifier.

3.2.2.3 CancellationHandler

Some APIs provide a CancellationHandler (4.24) to get notified about the cancellation of the corresponding diagnostic service.

Returning to library

Nevertheless the application has to return any pending promise in case of cancellation. The returned value of the promise will be ignored in the end.

Caution

The CancellationHandler shall not be stored or accessed beyond the lifetime of the currently ongoing operation.

3.3 Diagnostic Manager Daemon Model

The Diagnostics Daemon needs to be modeled within ARXML as an executable of category `PLATFORM_LEVEL`. The qualified shortname of the executable can be freely chosen, as well as the namings of the `ROOT-SW-COMPONENT-PROTOTYPE` and its referenced `ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE`.

To identify the executable as the Diagnostics Daemon, a model extension needs to be added to the executable configuration. This extension is evaluated by the generators that create model dependent code for initialization and deinitialization as well as the Diagnostics Daemon runtime configuration files.

based on template version 1.00.00

Caution

Without the model extension that identifies the Diagnostics Daemon, the Diagnostics Daemon source code will not compile successfully, and the diagnostic generator will not create the runtime configuration files, see section 6.1.

AdminData for Generation of Code to Initialize and Deinitialize ARA Libraries

```
<EXECUTABLE>
  <SHORT-NAME>my_diagnostic_daemon</SHORT-NAME>
  <CATEGORY>PLATFORM_LEVEL</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:BswExecutable">
        <SD GID="bswExecutableType">DIAGNOSTIC_MANAGER</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 3-1 AdminData for Generation of Code to Initialize and Deinitialize ARA Libraries

In addition, the `ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE` needs to include port definitions as shown in listing 3-2.

Diagnostics Daemon Port Prototype Configuration

```
    <PR-PORT-PROTOTYPE>
        <SHORT-NAME>Persistency</SHORT-NAME>
        <PROVIDED-REQUIRED-INTERFACE-TREF DEST="PERSISTENCY-FILE-STORAGE-INTERFACE">/amsr/
            diag/persistency/PersistencyInterface</PROVIDED-REQUIRED-INTERFACE-TREF>
    </PR-PORT-PROTOTYPE>
```

Listing 3-2 Diagnostics Daemon Port Prototype Configuration

The persistency port is used for the storage of diagnostic NV data. The port prototype must reference a persistency file storage of category 'binary'. Please refer to the documentation of ara::per ([28]) for details and for the deployment of this port.

Caution

The port prototype for persistency has to reference a persistency file storage with following name /amsr/diag/persistency/PersistencyInterface. If the name is not correct, the `amsr_diagnosticmanager_daemon` generator will issue an error.

3.3.1 Integration of Transport Protocol

DM requires a list of creator functions which instantiate the Transport Protocol handlers. This list of creator functions of Transport Protocol handlers should be modeled as AdminData configuration of the Transport Protocols. The AdminData configuration of the Diagnostics Daemon Executable

based on template version 1.00.00

references the Transport Protocol admin data configuration. Diagnostics Daemon uses this information to determine the Transport Protocols to instantiate at start-up.

Caution

The Diagnostics Daemon executable should ideally be configured with at least one Transport Protocol. If no Transport Protocols are configured, Diagnostics Daemon cannot communicate with an external client.

3.3.1.1 Integration of a Transport Protocol from the Vector Stack

Each Transport Protocol provided out of the box from the Vector Stack includes an integration ARXML model under its respective model directory. To integrate a Transport Protocol into the Diagnostics Daemon, this ARXML document has to be included into the project configuration, and the Diagnostics Daemon model needs to be extended with a reference to its integration model.

The `DvMex:transportProtocol` attribute of `DvMex:DiagnosticManagerExecutableExtensions` should point to the shortname (fully qualified) of the Transport Protocol creator configuration. An example on how to configure DoIP is shown in 3-3.

Configuring DM Executable to use DoIP Transport Protocol

```
<EXECUTABLE>
  <SHORT-NAME>amsr_diag_example_daemon</SHORT-NAME>
  ...
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticManagerExecutableExtensions">
        <SDG GID="transportProtocol">
          <SDX-REF DEST="SDG-CAPTION">/amsr/doip_binding/DoIP</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 3-3 Configuring DM Executable to use DoIP Transport Protocol

A list of the creator functions of the modeled Transport Protocols will be generated for the Diagnostics Daemon executable. The handlers returned from the instantiation of a Transport Protocol are later used by the Diagnostics Daemon to initialize, start and stop the corresponding Transport Protocols.

3.3.1.2 Integration of use of TLS on channels

Having a channel that uses TLS requires Crypostack daemon to be run and `ara::crypto` initialized. This requires diagnostic manager daemon to be modelled such that `ara::crypto` gets initialized. This can be done by adding the following in the `<EXECUTABLE>` level as shown in 3-4

based on template version 1.00.00

Configuring DM Executable to use TLS

```
<EXECUTABLE>
  <SHORT-NAME>amsr_diag_example_daemon</SHORT-NAME>
  ...
  <CATEGORY>PLATFORM_LEVEL</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:CryptoExecutableExtension">
        <SD GID="useCryptoApi">true</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 3-4 Configuring DM Executable to use TLS

3.3.1.3 Integration of a Custom Transport Protocol

The information about a custom Transport Protocol creator has to be modeled first before it can be referenced by the Diagnostics Daemon Executable. An `AR-PACKAGE` model extension has been created for this purpose. Any `AR-PACKAGE` can be extended with the details of the Transport Protocol as shown in 3-7. The following information about the Transport Protocol has to be modeled:

- Name of the Transport Protocol as short name
- Fully qualified function to create the Transport Protocol handler
- Header file to be included for declarations of the creator function
- The type of the creator function which can either be **CONSTRUCTOR** or **CUSTOM_FUNCTION**.

The function to create the Transport Protocol can be a constructor of the TransportProtocol, whose signature confirms to the standard signature as shown in 3-5. The constructor should ideally be `noexcept` qualified.

Signature of constructor of a Transport Protocol handler

```
ara::diag::uds_transport::UdsTransportProtocolHandler TransportProtocolHandler(ara::diag::
    uds_transport::UdsTransportProtocolHandlerID handler_id, ara::diag::uds_transport::
    UdsTransportProtocolMgr &tp_manager) noexcept
```

Listing 3-5 Signature of constructor of a Transport Protocol handler

Alternatively a custom function can be provided whose signature confirms to that of 3-9 in the two scenarios mentioned below.

- The constructor of the Transport Protocol handler cannot be `noexcept` qualified.
- Additional steps are involved in the creation of a Transport Protocol Handler other than constructor invocation.

If the constructor cannot be `noexcept` qualified, a custom function should be used instead. The constructor can be invoked from within the custom function. The custom function should handle the probable exceptions from the constructor. If construction of the Transport Protocol handler fails, this custom function should return a null-pointer, in which case the Diagnostics Daemon will log a respective error and ignore this Transport Protocol.

If some additional steps are involved before or after the construction of the Transport Protocol handler, a custom function should be used.

Signature of a custom function to create a Transport Protocol Handler

```
auto CreateTransportProtocolHandler(ara::diag::uds_transport::
    UdsTransportProtocolHandlerID handler_id, ara::diag::uds_transport::
    UdsTransportProtocolMgr &tp_manager) noexcept -> std::unique_ptr<ara::diag::
    uds_transport::UdsTransportProtocolHandler>
```

Listing 3-6 Signature of a custom function to create a Transport Protocol Handler

A model of the integration parameters for Transport Protocols should be created with the information on available Transport Protocols as shown in 3-7.

Extending an AR-PACKAGE with Transport Protocol information

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>custom</SHORT-NAME>
     <AR-PACKAGES>
       <AR-PACKAGE>
         <SHORT-NAME>transport_protocol</SHORT-NAME>
         <ADMIN-DATA>
           <SDGS>
             <SDG GID="DvMex:DiagnosticManagerArPackageExtensions">
               <SDG GID="transportProtocolCreator">
                 <SDG GID="TransportProtocolCreator">
                   <SDG-CAPTION>
                     <SHORT-NAME>CustomTP</SHORT-NAME>
                   </SDG-CAPTION>
                   <SD GID="fullyQualifiedCreatorFunction">amsr::custom::CustomTP</SD>
                   <SD GID="creatorHeaderFilePath">amsr/custom/custom_tp.h</SD>
                   <SD GID="creatorFunctionType">CONSTRUCTOR</SD>
                 </SDG>
               </SDG>
             </SDG>
           </SDGS>
         </ADMIN-DATA>
       </AR-PACKAGE>
     </AR-PACKAGES>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 3-7 Extending the DM daemon model with Transport Protocol information

The AUTOSAR path of the custom Transport Protocol creator configuration can now be referenced by the Executable as in the case of DoIP as shown in 3-3.

3.3.2 Integration of SecureDataTransmission PlugIn

As already described in 2.1.5.10, the SDT feature requires the help of a plug-in to handle the crypto and parameter verification tasks. Such a plug-in must be part of the model 3-10 and associated with the Diagnostics Daemon Executable 3-11 in order to be used by the Diagnostics Daemon at runtime.

based on template version 1.00.00

The creator element represents the factory method that will be called by Diagnostics Daemon at runtime in order to instantiate the modeled plug-in at start-up.

3.3.2.1 Integration of a SecureDataTransmission PlugIn from the Vector Stack

Currently no pre-defined implementation available.

3.3.2.2 Integration of a Custom SecureDataTransmission PlugIn

The information about a custom SecureDataTransmission creator has to be modeled first before it can be referenced by the Diagnostics Daemon Executable:

An `AR-PACKAGE` model extension has been created for this purpose. Any `AR-PACKAGE` can be extended with the details of the Transport Protocol as shown in 3-7. The following information about the Transport Protocol has to be modeled:

> Name of the SecureDataTransmission plug-in as short name
> Fully qualified function to create the SecureDataTransmission handler
> Header file to be included for declarations of the creator function
> The type of the creator function which can either be **CONSTRUCTOR** or **CUSTOM_FUNCTION**.

The function to create the SecureDataTransmission PlugIn can be a constructor of the SecureDataTransmissionHandler interface implementation, whose signature conforms to the one as shown in 3-8. The constructor should ideally be `noexcept` qualified.

Signature of constructor of a SecureDataTransmission handler

```
SecuredDataTransmissionHandler() noexcept
```

Listing 3-8 Signature of constructor of a SecureDataTransmission handler

Alternatively a custom function can be provided whose signature confirms to that of 3-9 in the two scenarios mentioned below.

> The constructor of the SecureDataTransmission handler cannot be `noexcept` qualified.
> Additional steps are involved in the creation of a SecureDataTransmission Handler other than constructor invocation.

If the constructor cannot be `noexcept` qualified, a custom function should be used instead. The constructor can be invoked from within the custom function. The custom function should handle the probable exceptions from the constructor.

Caution

If construction of the SecureDataTransmission handler fails, this custom function should return a null-pointer, in which case the Diagnostics Daemon will log a respective fatal-error and immediatelly abort!

If some additional steps are involved before or after the construction of the SecureDataTransmission handler, a custom function should be used.

Signature of a custom function to create a SecureDataTransmissionHandler

```
auto CreateSecureDataTransmissionHandler() noexcept -> std::unique_ptr<amsr::uds_transport
    ::SecuredDataTransmissionHandler>
```

Listing 3-9 Signature of a custom function to create a SecureDataTransmission Handler

AR-PACKAGE in DEXT is extended to hold the configuration of secured data transmission creator as illustrated in 3-10.

EXECUTABLE in DEXT is extended to hold the configuration of a reference to secured data transmission creator as illustrated in 3-11.

Secured Data Transmission creator

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>custom</SHORT-NAME>
    <AR-PACKAGES>
      <AR-PACKAGE>
        <SHORT-NAME>secured_data_transmission</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:DiagnosticManagerArPackageExtensions">
              <SDG GID="securedDataTransferCreator">
                <SDG GID="SecuredDataTransferCreator">
                  <SDG-CAPTION>
                    <SHORT-NAME>SecuredDataTransmissionHandler</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="fullyQualifiedCreatorFunction">path::
                      StubSecuredDataTransmissionHandler</SD>
                  <SD GID="creatorHeaderFilePath">path/
                      stub_secured_data_transmission_handler.h</SD>
                  <SD GID="creatorFunctionType">CONSTRUCTOR</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
      </AR-PACKAGE>
    </AR-PACKAGES>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 3-10 Secured Data Transmission creator

based on template version 1.00.00

Secured Data Transmission executable reference

```
<EXECUTABLE>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticManagerExecutableExtensions">
        <SDG GID="securedDataTransfer">
          <SDX-REF DEST="SDG-CAPTION">path/SecuredDataTransmissionHandler</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</EXECUTABLE>
```

Listing 3-11 Secured Data Transmission executable reference

In the following, the parameters mentioned in 3-10 are defined:

- `fullyQualifiedCreatorFunction` represents the function that creates the secured data transmission handler based on the function type.
- `creatorHeaderFilePath` represents the location of the header file where the fully qualified creator function is declared.
- `creatorFunctionType` represent the type of the fully qualified creator function. It can be either `CUSTOM_FUNCTION` or `CONSTRUCTOR`.
  - If the function type is `CUSTOM_FUNCTION`, the function returns the created secured data transmission handler.
  - If the function type is `CONSTRUCTOR`, It refers to the handler constructor.

3.4 Runtime Dependencies

To provide diagnostic functionality, the following components are required:

- amsr-vector-fs-em-executionmanager
- Diagnostics Daemon
- Crypostack daemon if tls is enabled in configuration.
- Secure communication through the use of the cryptostack introduces a runtime dependency on the amsr-vector-fs-sec-cryptostack daemon. For further details, please refer to[18].

Please also note figure 1-1 .

3.5 Files and Include Structure

- **Use Diagnostics Library (ara::diag):**
  amsr-vector-fs-aradiag is the Diagnostics Library (ara::diag).
  Header files can be included like this: `#include "ara/diag/<header>"`
  The header files are specified in [5].
  Special stub abstraction of the Diagnostics Library (ara::diag) is found under `testing/StubAraDiag`.
  This stub is only intended to be used for testing purpose.

- **Compile the Application:**
  Fulfill all dependencies to Diagnostics Library (ara::diag) listed in section 3.1
- **Link the Application:**
  All source files of Diagnostics Library (ara::diag) that must be compiled to link an application.
- **Deploy the Application:**

based on template version 1.00.00

No additional files are necessary to deploy Diagnostics Library (ara::diag) within the application.

> **Runtime Files:**
Diagnostics Library (ara::diag) reads the following configuration file(s) on startup in the `etc` folder of the application:
> `aradiag.json`: Used to configure the Diagnostics Library (ara::diag)-specific values

Diagnostics Daemon creates/reads/writes files during runtime via the persistency port referenced in the "NvData" tag of the Global Configuration, see section 6.7 for details and figure 3-1 for an exemplary folder structure. Following files will be placed in the `nv_data` folder:
> `dm_response_after_reset`: Used for Response After Reset
> `dm_security_access_faam`: Used for brute-force-attack prevention
> `doip_response_after_reset_00`: Used for Response After Reset
> `security_event_type_`0 `-` `security_event_type_`k: Used for Security Event Log
> If file handling mode is set to Multifile (see 2.1.9.2):
  > `PrimaryMemory`0 `-` `PrimaryMemory`n: Used for non-volatile data of the primary memory
  > `<UserMemoryShortName>`$n+1$ `-` `<UserMemoryShortName>`m: Used for non-volatile data of the user-defined memory
> If file handling mode is set to Singlefile (see 2.1.9.2):
  > `PrimaryMemory.singlefile`: Used for non-volatile data of the primary memory
  > `<UserMemoryShortName>.singlefile`: Used for non-volatile data of the user-defined memory

Diagnostics Daemon reads the following configuration files on startup in the `etc` folder (for details see figure 3-1)
> `dm_server_config_software_cluster.json`: Used to configure the DEXT for Diagnostics Daemon (actual name and path is defined in `diagnosticmanager.json`, for details see section 6.4.1)
> `diagnosticmanager_doip.json`: Optionally, used to configure DoIP
> `logging_config.json`: Used to configure log-output (for details see [14])
> `per_file_storage_config.json`: Used to configure FileStorage (for details see [28])

3.6 CMake

For detailed usage of CMake see documentation available at cmake.org/documentation.

3.6.1 Exported CMake targets

Exported cmake targets are listed in following tables.
> amsr-vector-fs-udstransport: table 3-1
> amsr-vector-fs-aradiag: table 3-2
> amsr-vector-fs-diagnosticrpc: table 3-3
> amsr-vector-fs-diagnosticrpcbinding: table 3-4
> amsr-vector-fs-diagnosticutility: table 3-5
> amsr-vector-fs-diagnosticmanager: table 3-6, table 3-7, table 3-8, table 3-9
> amsr-vector-fs-doipbinding: table 3-11

CMake exported target	Description	TYPE	VISIBILITY
`amsr::udstransport`	Contains the APIs for UDS tranport	STATIC	PUBLIC

Table 3-1 amsr-vector-fs-udstransport cmake targets

CMake exported target	Description	TYPE	VISIBILITY
`amsr::aradiag`	Library used to implement a diagnostic Adaptive Application	STATIC	PUBLIC
`amsr::aradiag_stub`	Stubs the AraDiag library for tests	STATIC	PUBLIC
`amsr::aradiag_uds_core_static`	Library responsible for UDS related interaction	STATIC	PRIVATE
`amsr::aradiag_sovd_core_static`	Library responsible for SOVD related interaction	STATIC	PRIVATE
`amsr::aradiag_core_public_api_static`	Library for public APIs relying on the core	STATIC	PRIVATE
`amsr::aradiag_public_api_static`	Library containing common public types and APIs visible to the user and internal libraries	STATIC	PRIVATE
`amsr::aradiag_core_static`	Library containing base functionality for diagnostic interactions which is only visible to internal libraries	STATIC	PRIVATE

Table 3-2 amsr-vector-fs-aradiag cmake targets

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diagnosticrpc_app_side`	Contains all parts relevant for the AraDiag library to communicate with the DiagnosticManager	STATIC	PRIVATE
`amsr::diagnosticrpc_daemon_side`	Contains all parts relevant for the DiagnosticManager to communicate with the AraDiag library	STATIC	PRIVATE
`amsr::diagnosticrpc_types`	Common parts between App and Daemon side libraries	STATIC	PRIVATE

Table 3-3 amsr-vector-fs-diagnosticrpc cmake targets

based on template version 1.00.00

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diagnosticrpcbinding_app_side`	Contains all parts relevant for the AraDiag library to communicate with the DiagnosticManager over RPC	STATIC	PRIVATE
`amsr::diagnosticrpcbinding_daemon_side`	Contains all parts relevant for the DiagnosticManager to communicate with the AraDiag library over RPC	STATIC	PRIVATE
`amsr::diagnosticrpcbinding_types`	Common parts between App and Daemon side libraries	STATIC	PRIVATE

Table 3-4 amsr-vector-fs-diagnosticrpcbinding cmake targets

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diagnosticutility`	Contains utilities for development of diagnostic features	STATIC	PRIVATE
`amsr::diagnosticutility_lg_qm`	Contains utilities for development of diagnostic features with QM qualification only	STATIC	PRIVATE

Table 3-5 amsr-vector-fs-diagnosticutility cmake targets

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diag_daemon`	The diagnostic-manager daemon with runtime handling and main.cpp implementation	INTERFACE	PUBLIC
`amsr::diag_daemon_impl`	The diagnostic-manager daemon with runtime handling but without main.cpp implementation	INTERFACE	PUBLIC
`amsr::diag_daemon_impl_static_sovd_xor_null`	Static part of `amsr::diag_daemon_impl` without SOVD	STATIC	PRIVATE
`amsr::diag_daemon_impl_static_sovd_xor_impl`	Static part of `amsr::diag_daemon_impl` with SOVD	STATIC	PRIVATE
`amsr::diag_daemon_impl_sovd_xor_null`	The diagnostic-manager daemon with runtime handling without SOVD	INTERFACE	PRIVATE
`amsr::diag_daemon_impl_sovd_xor_impl`	The diagnostic-manager daemon with runtime handling with SOVD	INTERFACE	PRIVATE
`amsr::diag_daemon_static`	Static part of `amsr::diag_daemon`	STATIC	PRIVATE
`amsr::diag_daemon_impl_static`	Static part of `amsr::diag_daemon_impl`	STATIC	PRIVATE
`amsr::diag_daemon_app_communication`	Necessary types for communication with diagnostic Adaptive Application	STATIC	PRIVATE

Table 3-6 amsr-vector-fs-diagnosticmanager cmake targets

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diag_daemon_app_services`	Provides access to the diagnostic Adaptive Application services	STATIC	PRIVATE
`amsr::diag_daemon_authentication_base`	Base functionalities of client authentication	STATIC	PRIVATE
`amsr::diag_daemon_client_authentication`	Client authentication functionalities	STATIC	PRIVATE
`amsr::diag_daemon_common`	Shared functionalities between daemon libraries	STATIC	PRIVATE
`amsr::diag_daemon_conversation`	Contains implementation of UDS conversation	STATIC	PRIVATE
`amsr::diag_daemon_core`	Core functionality of diagnostic manager daemon	STATIC	PRIVATE
`amsr::diag_daemon_daemon_services`	Provides access to the diagnostic manager daemon services	STATIC	PRIVATE
`amsr::diag_daemon_data_element`	Data element functionalities	STATIC	PRIVATE
`amsr::diag_daemon_data_management`	Manages the diagnostic data	STATIC	PRIVATE
`amsr::diag_daemon_data_transfer`	Data transfer functionalities	STATIC	PRIVATE

Table 3-7 amsr-vector-fs-diagnosticmanager cmake targets

based on template version 1.00.00

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diag_daemon_ecu_reset`	ECU reset functionalities	STATIC	PRIVATE
`amsr::diag_daemon_fault_memory`	fault memory functionalities	STATIC	PRIVATE
`amsr::diag_daemon_idsm`	Idsm functionalities	STATIC	PRIVATE
`amsr::diag_daemon_message_handling`	UDS message handling	STATIC	PRIVATE
`amsr::diag_daemon_periodic`	Periodic data handling	STATIC	PRIVATE
`amsr::diag_daemon_persistency`	Methods to store data into non-volatile memory	STATIC	PRIVATE
`amsr::diag_daemon_persistency_binding`	Binding for daemon persistency library	INTERFACE	PRIVATE
`amsr::diag_daemon_persistency_binding_headers`	Headers for persistency binding	INTERFACE	PRIVATE
`amsr::diag_daemon_preconditions`	Preconditions of diagnostic service execution	STATIC	PRIVATE
`amsr::diag_daemon_response_on_event`	Implement response on event service functionality	STATIC	PRIVATE

Table 3-8 amsr-vector-fs-diagnosticmanager cmake targets

based on template version 1.00.00

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diag_daemon_routine`	Implement UDS routine functionalities	STATIC	PRIVATE
`amsr::diag_daemon_rpc_binding_creator`	Rpc binding creator	STATIC	PRIVATE
`amsr::diag_daemon_secured_data_transmission`	UDS secured data transmission functionalities	STATIC	PRIVATE
`amsr::diag_daemon_secured_data_transmission_gen_headers`	Contains the headers required for the generated part of the Sdt library	INTERFACE	PRIVATE
`amsr::diag_daemon_security_access`	UDS security access functionalities	STATIC	PRIVATE
`amsr::diag_daemon_sevl`	Security event log functionalities	STATIC	PRIVATE
`amsr::diag_daemon_service_processing`	Processing of UDS services	STATIC	PRIVATE
`amsr::diag_daemon_service_validation`	UDS service validation	STATIC	PRIVATE
`amsr::diag_daemon_session_control`	UDS session control	STATIC	PRIVATE
`amsr::diag_daemon_system`	The System library interfaces with the ECU	STATIC	PRIVATE
`amsr::diag_daemon_system_sovd_xor_null`	The System library interfaces with the ECU without SOVD	STATIC	PRIVATE
`amsr::diag_daemon_system_sovd_xor_impl`	The System library interfaces with the ECU with SOVD	STATIC	PRIVATE

Table 3-9 amsr-vector-fs-diagnosticmanager cmake targets

based on template version 1.00.00

CMake exported target	Description	TYPE	VISIBILITY
`amsr::diag_daemon_uds_common`	Shared utilities without functional components for UDS	STATIC	PRIVATE
`amsr::diag_daemon_uds_services`	Implementation of the UDS services	STATIC	PRIVATE
`amsr::diag_daemon_uds_transport`	Implementation of the UdsTransport-ProtocolMgr and UdsMessage provider	STATIC	PRIVATE
`amsr::diag_daemon_uds_transport_gen_headers`	Contains the headers required for the generated part of the UdsTransport library	INTERFACE	PRIVATE
`amsr::diag_daemon_zero_copy`	Holds necessary information to coordinate the large data transfer without a large memory effort	STATIC	PRIVATE
`amsr::diag_daemon_sovd_common`	Common functionalities for SOVD libraries	STATIC	PRIVATE
`amsr::diag_daemon_sovd_core`	Process the SOVD requests and dispatchs it to the correct extension	STATIC	PRIVATE
`amsr::diag_daemon_sovd_extension`	Hanldes SOVD requests data	STATIC	PRIVATE
`amsr::diag_daemon_sovd_wrapper`	Wrapper for SOVD	STATIC	PRIVATE
`amsr::diag_daemon_sovd_wrapper_xor_null`	Implementation of wrapper when SOVD is disabled	STATIC	PRIVATE
`amsr::diag_daemon_sovd_wrapper_xor_impl`	Implementation of wrapper when SOVD is enabled	STATIC	PRIVATE

Table 3-10 amsr-vector-fs-diagnosticmanager cmake targets

based on template version 1.00.00

CMake exported target	Description	TYPE	VISIBILITY
`amsr::doipbinding`	DoIP extension according to `AMSR_DOIP_EXTENSION` cmake variable	INTERFACE	PUBLIC
`amsr::doipbinding_common`	Contains common implementation for DoIP libraries	STATIC	PRIVATE
`amsr::doipbinding_null_extension`	DoIP null extension	INTERFACE	PRIVATE

Table 3-11 amsr-vector-fs-doipbinding cmake targets

These targets with PUBLIC visivility can be used inside your CMake project as shown in the following example:

CMake usage example

```
set(AMSR_STATIC_LIBRARIES TRUE)
find_package(AMSR
        COMPONENTS amsr::<exported-target-name>)

target_link_libraries(${TARGET_NAME}
        PUBLIC amsr::<exported-target-name>)
```

The call to find_package() is necessary only if AMSR is built and installed as a separate CMake project.

The targets with PRIVATE visivility are only used internnally and shall not be linked against.

3.6.2 CMake Options

The CMake project provided with this component has options as specified in table 3-12.

based on template version 1.00.00

CMake Option	Descrip	Default Value
-DAMSR_ENABLE_EXEC_MANAGER	Enable Execution Manager (ara::em) support.	ON
-DAMSR_DOIP_EXTENSION	Enable Doipbinding Extension support.	amsr_doipbinding_null_extension
-DAMSR_ENABLE_SOVD	Enable supported SOVD functionalities.	OFF
-DAMSR_DIAG_DAEMON_REPORT_RUNNING_AFTER_INIT	Enable running state report to Execution Manager (ara::em) after initialization.	OFF

Table 3-12 CMake Options

based on template version 1.00.00

3.7 Installation

The binary of Diagnostics Daemon is typically installed into `/opt/amsr_vector_fs_diagnosticmanager/bin`. The generated configuration files are typically installed at `/opt/amsr_vector_fs_diagnosticmanager/etc` The persistency data is stored in `/opt/amsr_vector_fs_diagnosticmanager/etc/nv_data`.

3.8 Execution

The Diagnostics Daemon is represented by the executable `amsr_vector_fs_diagnosticmanager`. Diagnostics Daemon does not support command line parameters. It can be started by the execution manager.

3.8.1 Directory Structure of the Working Directory

Figure 3-1 shows the folder structure expected by the Diagnostics Daemon.

3.8.2 Start-Up behavior

On first/initial start-up of the Diagnostics Daemon, the non-volatile files (please see 3.5 for further details) are not present. The Diagnostics Daemon will create them, but also log a message about the missing files.

Caution
The Diagnostics Daemon does not provide a migration strategy for non-volatile files between deliveries.

3.8.3 Measuring execution runtime

Diagnostics Daemon supports watch dog and stop watch utility to measure the runtime of added tasks into `EventLoop` section 4.1. This measurement includes the tasks internally added by Diagnostics Daemon like connection monitoring task to find available diag services between Diagnostics Daemon and customer applications etc. The measurements are also done on externally added task to `EventLoop` by customer applications.

3.8.3.1 Watch Dog

Watch Dog is disabled by default and can be enabled only from source code, no configuration parameter supported. When enabled watch dog will terminates the process when any task in `EventLoop` exceeds execution time of 100 milliseconds.

Caution
Do not enable the Watch Dog. It is only provided for debugging capabilities and should be disabled in production code.

3.8.3.2 Stop Watch

Stop Watch is enabled by default and logs warning message when any tasks in `EventLoop` exceeds execution time of 100 milliseconds.

```
amsr-vector-fs-diagnosticmanager
├─ bin
│  └─ amsr-vector-fs-diagnosticmanager (executable)
├─ etc
│  ├─ dm_server_config_software_cluster.json
│  ├─ com_application.json
│  ├─ diagnosticmanager.json
│  ├─ diagnosticmanager_doip.json (if DoIP library is used)
│  ├─ exec_config.json (if executionmanager is used)
│  ├─ logging_config.json
│  └─ per_file_storage_config.json
└─ nv_data
   ├─ dm_response_after_reset (if Response After Reset is used)
   ├─ dm_security_access_faam (if brute-force-attack prevention is used)
   ├─ doip_response_after_reset_00 (if Response After Reset is used)
   ├─ security_event_type_0 (if Security Event Log is used)
   ├─ ...
   ├─ security_event_type_k (if Security Event Log is used)
   ├─ PrimaryMemory0 (if file handling mode set to Multifile)
   ├─ ...
   ├─ PrimaryMemoryn (if file handling mode set to Multifile)
   ├─ <UserMemoryShortName>n + 1 (if file handling mode set to Multifile)
   ├─ ...
   ├─ <UserMemoryShortName>m (if file handling mode set to Multifile)
   ├─ PrimaryMemory.singlefile (if file handling mode set to Singlefile)
   └─ <UserMemoryShortName>.singlefile (if file handling mode set to Singlefile)
```

Figure 3-1 Working Directory Structure of the Diagnostics Daemon

based on template version 1.00.00

3.9 Application IDs

For logging via the LOG component, Adaptive Applications provided by DM use an Application ID. The Application ID can be used to relate log messages to the respective Adaptive Application (refer to the TechnicalReference_LogAndTrace.pdf located in the `Doc/TechnicalReferences` folder of your AMSR SIP). Table 3-13 specifies the used Application IDs of the Adaptive Applications.

Application Name	Application ID
Diagnostics Daemon	`VDMd`

Table 3-13 Used Application ID of the DM

Please ensure to use other Application IDs for your Adaptive Applications.

3.10 Context IDs

The Context ID is an identifier that is used to logically group logging information within the scope of an Application process (refer to the TechnicalReference_LogAndTrace.pdf located in the `Doc/TechnicalReferences` folder of your AMSR SIP), e.g. to simplify debugging.

For DM and its sub-components, the following Context IDs are used:

The following Logging Context IDs are used by the components and libraries provided by DM.

Context ID	Description
`vdal`	Diagnostic Application Library
`vdba`	DiagnosticRpcBcpBinding (Application Side)
`vdbd`	DiagnosticRpcBcpBinding (Daemon Side)
`vdfm`	Diagnostic Manager Daemon Fault Memory
`vdip`	DoIpBinding
`vdmd`	Diagnostic Manager Daemon
`vdpd`	PduHdrTpBinding
`vdra`	DiagnosticRpc (Application Side)
`vdrc`	DiagnosticRpc (Common Parts)
`vdrd`	DiagnosticRpc (Daemon Side)
`vdso`	Diagnostic Manager SOVD
`vdsp`	Diagnostic Manager Daemon Service Processing
`vdut`	DiagnosticUtility
`vDRS`	Diagnostic memory resource usage in context of Steady Mode (see 2.10)

Table 3-14 List of Context IDs used by the diagnostic manager

based on template version 1.00.00

4 Diagnostic Library API Description

Caution

The API of the AMSR product is built by public classes, their methods, free functions and types. Classes, their accessible methods, free functions and other types that are intended to be used by customers are annotated with the documentation tag `\vpublic`. These items build the product public API. Customer software shall just use the public API of AMSR. The private part shall not be used.

based on template version 1.00.00

4.1 amsr::diag_utility::EventLoop

Description	
Provide functionnality for task execution synchronization	
Include	**Generalize**
amsr/diag_utility/event_loop.h	
MemberClass	
Handle	Handle for periodic tasks
Buzzer	Buzzer for events
MemberFunctions	
RegisterPeriodicTask	Register a periodic task
GetRegisterEvent	Register an event
RaiseEvent	Raise an event with a specific name
Unregister	Unregister an event with a specific name
SetDeferTask	Defer a task

Table 4-1 amsr::diag_utility::EventLoop

4.1.1 amsr::diag_utility::EventLoop::RegisterPeriodicTask

Prototype

```
amsr::diag_utility::EventLoop::Handle
RegisterPeriodicTask(std::chrono::milliseconds period,
  amsr::diag_utility::EventLoop::Task task,
  amsr::diag_utility::EventLoop::UnregisterStrategy strategy) noexcept
```

Description	
Register a periodic task	
Parameter	
period	The period in millisseconds
task	The task to execute
strategy	The unregistration strategy
Returns	
`amsr::diag_utility::EventLoop::Handle`	Handle allowing unregistration of the task
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints
N/A

Table 4-2 amsr::diag_utility::EventLoop::RegisterPeriodicTask

4.1.2 amsr::diag_utility::EventLoop::RegisterEvent

4.1.2.1 Register an event using specific name

Prototype				

`ara::core::Result<void>`
`RegisterEvent(ara::core::StringView name,`
`  amsr::diag_utility::EventLoop::Task task,`
`  amsr::diag_utility::EventLoop::UnregisterStrategy strategy) noexcept`				

Description				
Register an event using specific name				
Parameter				
name	The event name			
task	The task to execute			
strategy	The unregistration strategy			
Returns				
`ara::core::Result<void>`	Result containing void if registration succeed, error otherwise			
Errors				
DiagUtilityErrc::kGenericError	A task is already registered with the given name			
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-3 amsr::diag_utility::EventLoop::RegisterEventName

4.1.2.2 Register an event

Prototype	

`amsr::diag_utility::EventLoop::Buzzer`
`RegisterEvent(`
`  amsr::diag_utility::EventLoop::Task task,`
`  amsr::diag_utility::EventLoop::UnregisterStrategy strategy) noexcept`	

Description	
Register an event task	
Parameter	
task	The task to execute
strategy	The unregistration strategy
Returns	
`amsr::diag_utility::` `  EventLoop::Buzzer`	Buzzer allowing unregistration and raise of the event
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-4 amsr::diag_utility::EventLoop::RegisterEvent

4.1.3 amsr::diag_utility::EventLoop::RaiseEvent

Prototype	
`void RaiseEvent(` `  ara::core::StringView name) noexcept`	
Description	
Raise an event using specific name	
Parameter	
name	The event name
Returns	
void	
Errors	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-5 amsr::diag_utility::EventLoop::RaiseEvent

4.1.4 amsr::diag_utility::EventLoop::Unregister

Prototype	
`void Unregister(` `  ara::core::StringView name) noexcept`	
Description	
Unregister an event using specific name	
Parameter	
name	The event name
Returns	
void	
Errors	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-6 amsr::diag_utility::EventLoop::Unregister

4.1.5 amsr::diag_utility::EventLoop::DeferTask

Prototype	
`void DeferTask(` `  amsr::diag_utility::EventLoop::Task task) noexcept`	
Description	
Defer a task	
Parameter	
task	The task to Defer
Returns	
void	
Errors	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-7 amsr::diag_utility::EventLoop::DeferTask

4.1.6 amsr::diag_utility::EventLoop::Handle

Description	
Allow unregistration of a periodic task	
Include	**Generalize**
amsr/diag_utility/event_loop.h	
Constructors	
Handle	Move constructs instance of class
Destructors	
~Handle	Desctruct instance of class and unregister the task if not yet done
Operators	

operator=	Move assigns instance of class
MemberFunctions	
Unergister	Unregister the periodic task

Table 4-8 amsr::diag_utility::EventLoop::Handle

4.1.6.1 amsr::diag_utility::EventLoop::Handle::Handle

Prototype				
`Handle(Handle&& handle) noexcept`				
Description				
Move constructs instance of class				
Parameter				
handle	The instance to move			
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-9 amsr::diag_utility::EventLoop::Handle::Handle

4.1.6.2 amsr::diag_utility::EventLoop::Handle::~Handle

Prototype				
`~Handle() noexcept`				
Description				
Destructs instance of class				
Parameter				
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-10 amsr::diag_utility::EventLoop::Handle::~Handle

4.1.6.3 amsr::diag_utility::EventLoop::Handle::operator=

Prototype				
`Handle & operator=(Handle&& handle) noexcept`				
Description				
Move assigns instance of class				
Parameter				
handle	The instance to move			
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-11 amsr::diag_utility::EventLoop::Handle::operator=

4.1.6.4 amsr::diag_utility::EventLoop::Handle::Unregister

Prototype				
`void Unregister() noexcept`				
Description				
Unregister the periodic task				
Parameter				
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-12 amsr::diag_utility::EventLoop::Handle::Unregister

4.1.7 amsr::diag_utility::EventLoop::Buzzer

Description	
Allow unregistration and raise of an Event	
Include	**Generalize**
amsr/diag_utility/event_loop.h	
Constructors	
Buzzer	Move constructs instance of class
Destructors	
~Buzzer	Desctruct instance of class and unregister the event if not yet done
Operators	
operator=	Move assigns instance of class
MemberFunctions	
Unergister	Unregister the event
RaiseEvent	Raise the event

Table 4-13 amsr::diag_utility::EventLoop::Buzzer

4.1.7.1 amsr::diag_utility::EventLoop::Buzzer::Buzzer

Prototype				
`Buzzer(Buzzer&& buzzer) noexcept`				
Description				
Move constructs instance of class				
Parameter				
buzzer	The instance to move			
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-14 amsr::diag_utility::EventLoop::Buzzer::Buzzer

4.1.7.2 amsr::diag_utility::EventLoop::Buzzer::~Buzzer

Prototype				
`~Buzzer() noexcept`				
Description				
Destructs instance of class				
Parameter				
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-15 amsr::diag_utility::EventLoop::Buzzer::~Buzzer

4.1.7.3 amsr::diag_utility::EventLoop::Buzzer::operator=

Prototype				
`Buzzer & operator=(Buzzer&& buzzer) noexcept`				
Description				
Move assigns instance of class				
Parameter				
buzzer	The instance to move			
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-16 amsr::diag_utility::EventLoop::Buzzer::operator=

4.1.7.4 amsr::diag_utility::EventLoop::Buzzer::Unregister

Prototype

`void Unregister() noexcept`				
Description				
Unregister the event				
Parameter				
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-17 amsr::diag_utility::EventLoop::Buzzer::Unregister

4.1.7.5 amsr::diag_utility::EventLoop::Buzzer::RaiseEvent

Prototype				
`void RaiseEvent() noexcept`				
Description				
Raise the event				
Parameter				
Returns				
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-18 amsr::diag_utility::EventLoop::Buzzer::RaiseEvent

based on template version 1.00.00

4.2 amsr::diag_utility::RootSwc

Description	
Provides a central RootSwc entity to manage process specifics	
Include	**Generalize**
amsr/diag_utility/root_swc.h	
MemberFunctions	
`static amsr::generic::SingletonAccess <RootSwc> GetAccess() noexcept`	Gets an access to the RootSwc singleton This SingletonAccess shall be kept as long as RootSwc members are accessed
`EventLoop & GetEventLoop() noexcept`	Get a reference to the EventLoop

Table 4-19 amsr::diag_utility::RootSwc

4.2.1 amsr::diag_utility::RootSwc::GetAccess

Prototype				
`static amsr::generic::SingletonAccess<RootSwc> GetAccess() noexcept`				
Description				
Get an access to the RootSwc singleton				
Parameter				
Returns				
amsr::generic::SingletonAccess <RootSwc>	Singleton access to the RootSwc			
Errors				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-20 amsr::diag_utility::RootSwc::GetAccess

4.2.2 amsr::diag_utility::RootSwc::GetEventLoop

Prototype
`amsr::diag_utility::EventLoop& GetEventLoop() noexcept`
Description
Get reference of the EventLoop
Parameter

Returns	
amsr::diag_utility::EventLoop&	Reference to the EventLoop
Errors	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-21 amsr::diag_utility::RootSwc::GetEventLoop

4.3 amsr::diag::DiagUdsNrc

Prototype	
`amsr::diag::DiagUdsNrc`	
Description	
Specifies the UDS specific NRC types	
Type	
std::uint8_t	
Enumerations	
kGeneralReject	0x10
kServiceNotSupported	0x11
kSubFunctionNotSupported	0x12
kIncorrectMessageLengthOrInvalidFormat	0x13
kResponseTooLong	0x14
kBusyRepeatRequest	0x21
kConditionsNotCorrect	0x22
kRequestSequenceError	0x24
kNoResponseFromSubnetComponent	0x25
kFailurePreventsExecutionOfRequestedAction	0x26
kRequestOutOfRange	0x31
kSecurityAccessDenied	0x33
kAuthenticationRequired	0x34
kInvalidKey	0x35
kExceedNumberOfAttempts	0x36
kRequiredTimeDelayNotExpired	0x37
kCertificateVerificationFailedInvalidTimePeriod	0x50
kCertificateVerificationFailedInvalidSignature	0x51
kCertificateVerificationFailedInvalidChainOfTrust	0x52
kCertificateVerificationFailedInvalidType	0x53
kCertificateVerificationFailedInvalidFormat	0x54
kCertificateVerificationFailedInvalidContent	0x55
kCertificateVerificationFailedInvalidScope	0x56
kCertificateVerificationFailedInvalidCertificateRevoked	0x57
kOwnershipVerificationFailed	0x58
kChallengeCalculationFailed	0x59
kSettingAccessRightsFailed	0x5A
kSessionKeyCreationDerivationFailed	0x5B
kConfigurationDataUsageFailed	0x5C
kDeAuthenticationFailed	0x5D

kUploadDownloadNotAccepted	0x70
kTransferDataSuspended	0x71
kGeneralProgrammingFailure	0x72
kWrongBlockSequenceCounter	0x73
kRequestCorrectlyReceived_responsePending	0x78
kSubFunctionNotSupportedInActiveSession	0x7E
kServiceNotSupportedInActiveSession	0x7F
kRpmTooHigh	0x81
kRpmTooLow	0x82
kEngineIsRunning	0x83
kEngineIsNotRunning	0x84
kEngineRunTimeTooLow	0x85
kTemperatureTooHigh	0x86
kTemperatureTooLow	0x87
kVehicleSpeedTooHigh	0x88
kVehicleSpeedTooLow	0x89
kThrottlePedalTooHigh	0x8A
kThrottlePedalTooLow	0x8B
kTransmissionRangeNotInNeutral	0x8C
kTransmissionRangeNotInGear	0x8D
kBrakeSwitchNotClosed	0x8F
kShifterLeverNotInPark	0x90
kTorqueConverterClutchLocked	0x91
kVoltageTooHigh	0x92
kVoltageTooLow	0x93
kResourceTemporarilyNotAvailable	0x94

Table 4-22 amsr::diag::DiagUdsNrc

based on template version 1.00.00

4.4 amsr::diag::NegativeResponseReason

Prototype	
`amsr::diag::NegativeResponseReason`	
Description	
Specifies the negative response reason types	
Type	
std::uint8_t	
Enumerations	
kAuthentication	0x00
kSession	0x01
kSecurity	0x02
kOther	0x03

Table 4-23 amsr::diag::NegativeResponseReason

based on template version 1.00.00

4.5 amsr::diag::r21_11::DiagErrorDomain

Description	
Specific error domain for all relevant diagnostic application errors	
Include	**Generalize**
amsr/diag/r21_11/diag_error_domain.h	generalized by ErrorDomain
Detailed Description	
This class inherits the base error domain, which is the controlling entity for ErrorCode's error code values, and defines the mapping of such error code values to textual representations.	
Constructors	
DiagErrorDomain	Constructs instance of class

Table 4-24 amsr::diag::r21_11::DiagErrorDomain

4.5.1 amsr::diag::r21_11::DiagErrorDomain::DiagErrorDomain

Prototype				
`amsr::diag::r21_11::DiagErrorDomain::DiagErrorDomain () noexcept`				
Description				
Constructs instance of class				
Parameter				
N/A	N/A			
Detailed Description				
Initializes the diagnostic application error domain ID with DiagErrorDomain::kId				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-25 amsr::diag::r21_11::DiagErrorDomain::DiagErrorDomain

4.6 amsr::diag::r21_11::DiagOfferErrorDomain

Description	
Specific error domain for all relevant diagnostic application errors	
Include	**Generalize**
amsr/diag/r21_11/diag_error_domain.h	generalized by ErrorDomain
Detailed Description	
This class inherits the base error domain, which is the controlling entity for ErrorCode's error code values, and defines the mapping of such error code values to textual representations.	
Constructors	
DiagOfferErrorDomain	Constructs instance of class

Table 4-26 amsr::diag::r21_11::DiagOfferErrorDomain

4.6.1 amsr::diag::r21_11::DiagOfferErrorDomain::DiagOfferErrorDomain

Prototype				
`amsr::diag::r21_11::DiagOfferErrorDomain::DiagOfferErrorDomain () noexcept`				
Description				
Constructs instance of class				
Parameter				
N/A	N/A			
Detailed Description				
Initializes the diagnostic application error domain ID with DiagOfferErrorDomain::kId				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-27 amsr::diag::r21_11::DiagOfferErrorDomain::DiagOfferErrorDomain

4.7 amsr::diag::r21_11::DiagReportingErrorDomain

Description	
Specific error domain for all relevant diagnostic application errors	
Include	**Generalize**
amsr/diag/r21_11/diag_error_domain.h	generalized by ErrorDomain
Detailed Description	
This class inherits the base error domain, which is the controlling entity for ErrorCode's error code values, and defines the mapping of such error code values to textual representations.	
Constructors	
DiagReportingErrorDomain	Constructs instance of class

Table 4-28 amsr::diag::r21_11::DiagReportingErrorDomain

4.7.1 amsr::diag::r21_11::DiagReportingErrorDomain::DiagReportingErrorDomain

Prototype				
`amsr::diag::r21_11::DiagReportingErrorDomain::DiagReportingErrorDomain () noexcept`				
Description				
Constructs instance of class				
Parameter				
N/A	N/A			
Detailed Description				
Initializes the diagnostic application error domain ID with DiagReportingErrorDomain::kId				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-29 amsr::diag::r21_11::DiagReportingErrorDomain::DiagReportingErrorDomain

4.8 amsr::diag::r21_11::Monitor

Description	
Diagnostic Monitor interface	
Include	**Generalize**
amsr/diag/r21_11/monitor.h	
Constructors	
Monitor	Move constructs instance of class
Monitor	Constructs a monitor with counter-based debouncing
Monitor	Constructs a monitor with time-based debouncing
Monitor	Constructs a monitor with monitor-internal debouncing
Operators	
operator=	Move assigns instance of class
Types	
InitMonitorReason	Represents the status information reported to AAs why the monitor may be re-initialized
CounterBased	Represents the parameters for counter-based debouncing
TimeBased	Represents the parameters for time-based debouncing
MonitorAction	Represents the possible monitor actions for version R21-11
MemberFunctions	
IsValidInstanceSpecifier	Checks if the given InstanceSpecifier is existing in the configuration
Offer	Enables the DM to forward request messages to this handler
ReportMonitorAction	Contains either the last (un-)qualified test result of the diagnostic monitor or commands to control the debouncing or to force a prestorage
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-30 amsr::diag::r21_11::Monitor

4.8.1 amsr::diag::r21_11::Monitor::Monitor

Prototype

```
amsr::diag::r21_11::Monitor::Monitor (
   amsr::diag::r21_11::Monitor &&
) noexcept
```

Description

Move constructs instance of class

Parameter

Errors

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-31 amsr::diag::r21_11::Monitor::Monitor

4.8.2 amsr::diag::r21_11::Monitor::Monitor

Prototype

```
amsr::diag::r21_11::Monitor::Monitor (
   const ara::core::InstanceSpecifier & specifier,
   std::function<void (InitMonitorReason)> init_monitor_cbk,
   amsr::diag::r21_11::Monitor::CounterBased debouncing
)
```

Description

Constructs a monitor with counter-based debouncing

Parameter

specifier	InstanceSpecifier to an PortPrototype of an DiagnosticMonitorInterface
init_monitor_cbk	Possibility to register an InitMonitor callback
debouncing	CounterBased debouncing option is added to the monitor

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE

Constraints

A port with that specifier is configured and no other port is right now instantiated with it

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the destruction of this Monitor has completed.

The failed threshold must be greater than zero and passed threshold must be lesser than zero

Table 4-32 amsr::diag::r21_11::Monitor::Monitor

4.8.3 amsr::diag::r21_11::Monitor::Monitor

Prototype

```
amsr::diag::r21_11::Monitor::Monitor (
   const ara::core::InstanceSpecifier & specifier,
   std::function<void (InitMonitorReason)> init_monitor_cbk,
   amsr::diag::r21_11::Monitor::TimeBased debouncing
)
```

Description	
Constructs a monitor with time-based debouncing	
Parameter	
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticMonitorInterface
init_monitor_cbk	Possibility to register an InitMonitor callback
debouncing	TimeBased debouncing option is added to the monitor
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this Monitor has completed. The failed timeout and passed timeout must be greater than zero.				

Table 4-33 amsr::diag::r21_11::Monitor::Monitor

4.8.4 amsr::diag::r21_11::Monitor::Monitor

Prototype

```
amsr::diag::r21_11::Monitor::Monitor (
   const ara::core::InstanceSpecifier & specifier,
   std::function<void (InitMonitorReason)> init_monitor_cbk,
   std::function<std::int8_t (void)> get_fdc_cbk
)
```

Description	
Constructs a monitor with monitor-internal debouncing	
Parameter	
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticMonitorInterface
init_monitor_cbk	Possibility to register an InitMonitor callback
get_fdc_cbk	Getter function for the fault detection counter
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it The passed in functions must not throw any exceptions. The passed in functions must be valid to be called until the destruction of this Monitor has completed.				

Table 4-34 amsr::diag::r21_11::Monitor::Monitor

4.8.5 amsr::diag::r21_11::Monitor::operator=

Prototype	

`amsr::diag::r21_11::Monitor &`
`amsr::diag::r21_11::Monitor::operator= (`
`  amsr::diag::r21_11::Monitor &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
amsr::diag::r21_11::Monitor &	Reference to self
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-35 amsr::diag::r21_11::Monitor::operator=

4.8.6 amsr::diag::r21_11::Monitor::InitMonitorReason

Prototype
`amsr::diag::r21_11::Monitor::InitMonitorReason`
Description
Represents the status information reported to AAs why the monitor may be re-initialized
Type
std::uint8_t

Parameter	
kClear	0x00
kRestart	0x01
kReenabled	0x02
kDisabled	0x03

Table 4-36 amsr::diag::r21_11::Monitor::InitMonitorReason

4.8.7 amsr::diag::r21_11::Monitor::CounterBased

Prototype		
`struct amsr::diag::r21_11::Monitor::CounterBased`		
Description		
Represents the parameters for counter-based debouncing		
Members		
std::int16_t	failed_threshold	Threshold until qualified failed
std::int16_t	passed_threshold	Threshold until qualified passed
std::uint16_t	failed_stepsize	Stepsize per pre-failed report. Valid values [0, 32767]
std::uint16_t	passed_stepsize	Stepsize per pre-passed report. Valid values [0, 32767]
std::int16_t	failed_jump_value	Failed to jump value
std::int16_t	passed_jump_value	Passed to jump value
bool	use_jump_to_failed	Is jump supported
bool	use_jump_to_passed	Is jump supported

Table 4-37 amsr::diag::r21_11::Monitor::CounterBased

4.8.8 amsr::diag::r21_11::Monitor::TimeBased

Prototype		
`struct amsr::diag::r21_11::Monitor::TimeBased`		
Description		
Represents the parameters for time-based debouncing		
Members		
std::uint32_t	failed_ms	time until failed in (ms)
std::uint32_t	passed_ms	time until passed in (ms)

Table 4-38 amsr::diag::r21_11::Monitor::TimeBased

4.8.9 amsr::diag::r21_11::Monitor::MonitorAction

Prototype
`amsr::diag::r21_11::Monitor::MonitorAction`

Description	
Represents the possible monitor actions for version R21-11	
Type	
std::uint16_t	
Enumerations	
kPassed	0x00
kFailed	0x01
kPrepassed	0x02
kPrefailed	0x03
kFdcThresholdReached	0x04
kResetTestFailed	0x05
kFreezeDebouncing	0x06
kResetDebouncing	0x07
kPrestore	0x08
kClearPrestore	0x09

Table 4-39 amsr::diag::r21_11::Monitor::MonitorAction

4.8.10 amsr::diag::r21_11::Monitor::IsValidInstanceSpecifier

Prototype

```
bool
amsr::diag::r21_11::Monitor::IsValidInstanceSpecifier (
   const ara::core::InstanceSpecifier & instance_specifier
) noexcept
```

Description

Checks if the given InstanceSpecifier is existing in the configuration

Parameter

instance_specifier	The to be checked InstanceSpecifier

Detailed Description

This function does currently not check if the InstanceSpecifier is matching the configured interface port in the configuration.

Returns

bool	true, if the InstanceSpecifier exists, false otherwise

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints

N/A
Steady
TRUE

Table 4-40 amsr::diag::r21_11::Monitor::IsValidInstanceSpecifier

4.8.11 amsr::diag::r21_11::Monitor::Offer

Prototype	
`ara::core::Result<void>` `amsr::diag::r21_11::Monitor::Offer () noexcept`	
Description	
Enables the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Result of type void containing an error if the Monitor could not be offered
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered This service was already offered.	
amsr::diag::DiagLibErrc::kGenericError The service request could not be offered	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-41 amsr::diag::r21_11::Monitor::Offer

4.8.12 amsr::diag::r21_11::Monitor::ReportMonitorAction

Prototype	

`void`
`amsr::diag::r21_11::Monitor::ReportMonitorAction (`
`  amsr::diag::r21_11::Monitor::MonitorAction monitor_action`
`)`	

Description	
Contains either the last (un-)qualified test result of the diagnostic monitor or commands to control the debouncing or to force a prestorage	
Parameter	
monitor_action	Represents the status information being relevant for error monitoring.
Returns	

void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-42 amsr::diag::r21_11::Monitor::ReportMonitorAction

4.8.13 amsr::diag::r21_11::Monitor::StopOffer

Prototype				
`void amsr::diag::r21_11::Monitor::StopOffer () noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-43 amsr::diag::r21_11::Monitor::StopOffer

4.9 amsr::diag::r21_11::OperationCycle

Description	
Provides a means to read current operation cycle and set a new operation cycles	
Include	**Generalize**
amsr/diag/r21_11/operation_cycle.h	
Constructors	
OperationCycle	Move constructs instance of class
OperationCycle	Constructor for DiagnosticOperationCycleInterface
Operators	
operator=	Move assigns instance of class
MemberFunctions	
RestartOperationCycle	Restarts the OperationCycle
SetNotifier	Registers a notifier function which is called if the operation cycle is changed

Table 4-44 amsr::diag::r21_11::OperationCycle

4.9.1 amsr::diag::r21_11::OperationCycle::OperationCycle

Prototype

```
amsr::diag::r21_11::OperationCycle::OperationCycle (
   amsr::diag::r21_11::OperationCycle &&
) noexcept
```

Description				
Move constructs instance of class				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 4-45 amsr::diag::r21_11::OperationCycle::OperationCycle

4.9.2 amsr::diag::r21_11::OperationCycle::OperationCycle

Prototype

```
amsr::diag::r21_11::OperationCycle::OperationCycle (
   const ara::core::InstanceSpecifier & specifier
)
```

Description

Constructor for DiagnosticOperationCycleInterface				
Parameter				
specifier		InstanceSpecifier to an PortPrototype of an DiagnosticOperationCycleInterface		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-46 amsr::diag::r21_11::OperationCycle::OperationCycle

4.9.3 amsr::diag::r21_11::OperationCycle::~OperationCycle

Prototype				
`amsr::diag::r21_11::OperationCycle::~OperationCycle () noexcept`				
Description				
Destructor of DiagnosticOperationCycleInterface				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-47 amsr::diag::r21_11::OperationCycle::~OperationCycle

4.9.4 amsr::diag::r21_11::OperationCycle::operator=

Prototype	

`amsr::diag::r21_11::OperationCycle &`
`amsr::diag::r21_11::OperationCycle::operator= (`
`  amsr::diag::r21_11::OperationCycle &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
amsr::diag::r21_11::OperationCycle &	Reference to self

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-48 amsr::diag::r21_11::OperationCycle::operator=

4.9.5 amsr::diag::r21_11::OperationCycle::RestartOperationCycle

Prototype	
`ara::core::Result<void>` `amsr::diag::r21_11::OperationCycle::RestartOperationCycle ()`	
Description	
Restarts the OperationCycle	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Result of type void
Errors	
ara::diag::DiagErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-49 amsr::diag::r21_11::OperationCycle::RestartOperationCycle

4.9.6 amsr::diag::r21_11::OperationCycle::SetNotifier

Prototype

`ara::core::Result<void>`
`amsr::diag::r21_11::OperationCycle::SetNotifier (`
`  std::function<void (void)> notifier`
`)`

Description
Registers a notifier function which is called if the operation cycle is changed
Parameter

notifier	The notifier function - AUTOSAR: DIRECTION NOT DEFINED
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	void Result.
Errors	
ara::diag::DiagErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints
The passed in function must not throw any exceptions.
The passed in function must be valid to be called until the destruction of this OperationCycle has completed or until it is replaced by a new notifier function.
Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Steady
TRUE

Table 4-50 amsr::diag::r21_11::OperationCycle::SetNotifier

based on template version 1.00.00

4.10 amsr::diag::ReleaseHandler

Description	
ReleaseHandler contains a shared state (context) if the processing should be canceled	
Include	**Generalize**
amsr/diag/release_handler.h	
Detailed Description	
ReleaseHandler is used to for the application to release the span of the file download	
Constructors	
ReleaseHandler	Move constructs instance of class
Operators	
operator=	Move assigns instance of class
MemberFunctions	
MayRelease	Reports whether the shared resource is no longer in use
SetNotifier	Registering a notifier function which is called if the shared resource can be freed

Table 4-51 amsr::diag::ReleaseHandler

4.10.1 amsr::diag::ReleaseHandler::ReleaseHandler

Prototype

```
amsr::diag::ReleaseHandler::ReleaseHandler (
   amsr::diag::ReleaseHandler && other
) noexcept
```

Description	
Move constructs instance of class	
Parameter	
other	The object to move contruct from
Errors	
N/A	

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-52 amsr::diag::ReleaseHandler::ReleaseHandler

4.10.2 amsr::diag::ReleaseHandler::operator=

Prototype

```
amsr::diag::ReleaseHandler &
amsr::diag::ReleaseHandler::operator= (
   amsr::diag::ReleaseHandler && other
) & noexcept
```

Description				
Move assigns instance of class				
Parameter				
other	The object to move assign from			
Returns				
amsr::diag::ReleaseHandler &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-53 amsr::diag::ReleaseHandler::operator=

4.10.3 amsr::diag::ReleaseHandler::MayRelease

Prototype				
`bool amsr::diag::ReleaseHandler::MayRelease () const noexcept`				
Description				
Reports whether the shared resource is no longer in use				
Parameter				
N/A	N/A			
Returns				
bool	True if the shared resource is no longer in use, false otherwise			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-54 amsr::diag::ReleaseHandler::MayRelease

4.10.4 amsr::diag::ReleaseHandler::SetNotifier

Prototype				
`void amsr::diag::ReleaseHandler::SetNotifier ( std::function<void (void)> notifier ) noexcept`				
Description				
Registering a notifier function which is called if the shared resource can be freed				
Parameter				
notifier	Notification function that is called upon releasing the shared resource			
Detailed Description				
A previously set notifier gets overwritten. If the shared resource is already released, the given notifier is invoked directly.				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until it is eventually called or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Table 4-55 amsr::diag::ReleaseHandler::SetNotifier

based on template version 1.00.00

4.11 amsr::diag::SecurityEvent

Description	
Class to implement the security event reporting	
Include	**Generalize**
amsr/diag/security_event.h	
Constructors	
SecurityEvent	Move constructs instance of class
SecurityEvent	Constructor for a SecurityEvent instance
Operators	
operator=	Move assigns instance of class
MemberFunctions	
Report	Reports the security event with necessary logging context

Table 4-56 amsr::diag::SecurityEvent

4.11.1 amsr::diag::SecurityEvent::SecurityEvent

Prototype

```
amsr::diag::SecurityEvent::SecurityEvent (
   amsr::diag::SecurityEvent && other
) noexcept
```

Description

Move constructs instance of class

Parameter	
other	The object to move contruct from

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-57 amsr::diag::SecurityEvent::SecurityEvent

4.11.2 amsr::diag::SecurityEvent::SecurityEvent

Prototype

```
amsr::diag::SecurityEvent::SecurityEvent (
   const ara::core::InstanceSpecifier & specifier
) noexcept
```

Description

Constructor for a SecurityEvent instance

Parameter				
specifier		InstanceSpecifier to a PortPrototype of a SecurityEvent		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-58 amsr::diag::SecurityEvent::SecurityEvent

4.11.3 amsr::diag::SecurityEvent::~SecurityEvent

Prototype				
`amsr::diag::SecurityEvent::~SecurityEvent () noexcept`				
Description				
Destructor of class SecurityEvent				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-59 amsr::diag::SecurityEvent::~SecurityEvent

4.11.4 amsr::diag::SecurityEvent::operator=

Prototype	

`amsr::diag::SecurityEvent &`
`amsr::diag::SecurityEvent::operator= (`
`  amsr::diag::SecurityEvent && other`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
other	The object to move assign from
Returns	
amsr::diag::SecurityEvent &	Reference to self
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-60 amsr::diag::SecurityEvent::operator=

4.11.5 amsr::diag::SecurityEvent::Report

Prototype	
`void amsr::diag::SecurityEvent::Report ( ara::core::Span<const std::uint8_t> context ) noexcept`	
Description	
Reports the security event with necessary logging context	
Parameter	
context	The logging context
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-61 amsr::diag::SecurityEvent::Report

4.12 amsr::diag::SecurityEventLog

Description	
Class to implement the security event reporting	
Include	**Generalize**
amsr/diag/security_event_log.h	
Constructors	
SecurityEventLog	Move constructs instance of class
SecurityEventLog	Constructor for a SecurityEventLog instance
Operators	
operator=	Move assigns instance of class
Types	
Event	Log of security event
EventType	Log of security event type
MemberFunctions	
AddRawEntryToAll	Applies a pattern to all event type buffers
GetEventLog	Get a snapshot of currently reported events
GetEventType	Get the event type
Report	Reports a security event with the given logging context for the given event ID and event type ID
SetNotifierNumberOfBufferEntries Changed	Register a notifier function which is called if the number of buffer entries has changed
SetNotifierSecurityEvent	Register a notifier function which is called if a security event has been registered

Table 4-62 amsr::diag::SecurityEventLog

4.12.1 amsr::diag::SecurityEventLog::SecurityEventLog

Prototype

```
amsr::diag::SecurityEventLog::SecurityEventLog (
  amsr::diag::SecurityEventLog && other
) noexcept
```

Description

Move constructs instance of class

Parameter

other	The object to move contruct from

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-63 amsr::diag::SecurityEventLog::SecurityEventLog

4.12.2 amsr::diag::SecurityEventLog::SecurityEventLog

Prototype				
`amsr::diag::SecurityEventLog::SecurityEventLog ( const ara::core::InstanceSpecifier & specifier ) noexcept`				
Description				
Constructor for a SecurityEventLog instance				
Parameter				
specifier		InstanceSpecifier to a PortPrototype of a SecurityEventLog		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-64 amsr::diag::SecurityEventLog::SecurityEventLog

4.12.3 amsr::diag::SecurityEventLog::~SecurityEventLog

Prototype				
`amsr::diag::SecurityEventLog::~SecurityEventLog () noexcept`				
Description				
Destructor of class SecurityEventLog				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-65 amsr::diag::SecurityEventLog::~SecurityEventLog

4.12.4 amsr::diag::SecurityEventLog::operator=

Prototype

```
amsr::diag::SecurityEventLog &
amsr::diag::SecurityEventLog::operator= (
   amsr::diag::SecurityEventLog && other
) & noexcept
```

Description				
Move assigns instance of class				
Parameter				
other	The object to move assign from			
Returns				
amsr::diag::SecurityEventLog &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-66 amsr::diag::SecurityEventLog::operator=

4.12.5 amsr::diag::SecurityEventLog::Event

Description	
Declaration of Event	
Include	**Generalize**
amsr/diag/state_types/security_event_log_type.h	
Constructors	
SecurityEventLogEvent	Move constructs instance of class
SecurityEventLogEvent	Copy constructs instance of class
Operators	
operator=	Move assigns instance of class
operator=	Copy assigns instance of class
MemberFunctions	
GetCounter	Get counter
GetId	Get the Event id

Table 4-67 amsr::diag::SecurityEventLog::Event

4.12.5.1 amsr::diag::SecurityEventLog::Event::Event

Prototype

```
amsr::diag::SecurityEventLog::Event::Event (
   amsr::diag::state_types::SecurityEventLogEvent &&
) noexcept
```

Description
Move constructs instance of class

Parameter	

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-68 amsr::diag::SecurityEventLog::Event::Event

4.12.5.2 amsr::diag::SecurityEventLog::Event::Event

Prototype

```
amsr::diag::SecurityEventLog::Event::Event (
   const class amsr::diag::state_types::SecurityEventLogEvent &
) noexcept
```

Description
Copy constructs instance of class

Parameter	

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-69 amsr::diag::SecurityEventLog::Event::Event

4.12.5.3 amsr::diag::SecurityEventLog::Event::~Event

Prototype

```
amsr::diag::SecurityEventLog::Event::~Event () noexcept
```

Description
Destructor of class Event

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-70 amsr::diag::SecurityEventLog::Event::~Event

4.12.5.4 amsr::diag::SecurityEventLog::Event::operator=

Prototype	

`amsr::diag::state_types::SecurityEventLogEvent &`
`amsr::diag::SecurityEventLog::Event::operator= (`
`  amsr::diag::state_types::SecurityEventLogEvent &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
amsr::diag::state_types::Security EventLogEvent &	Reference to self
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-71 amsr::diag::SecurityEventLog::Event::operator=

4.12.5.5 amsr::diag::SecurityEventLog::Event::operator=

Prototype

`amsr::diag::state_types::SecurityEventLogEvent &`
`amsr::diag::SecurityEventLog::Event::operator= (`
`  const class amsr::diag::state_types::SecurityEventLogEvent &`
`) noexcept`

Description
Copy assigns instance of class

Parameter	
Returns	
amsr::diag::state_types::Security EventLogEvent &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-72 amsr::diag::SecurityEventLog::Event::operator=

4.12.5.6 amsr::diag::SecurityEventLog::Event::GetCounter

Prototype	
`std::uint32_t amsr::diag::SecurityEventLog::Event::GetCounter () const noexcept`	
Description	
Get counter	
Parameter	
N/A	N/A
Returns	
std::uint32_t	The Event counter
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-73 amsr::diag::SecurityEventLog::Event::GetCounter

4.12.5.7 amsr::diag::SecurityEventLog::Event::GetId

Prototype
`std::uint32_t amsr::diag::SecurityEventLog::Event::GetId () const noexcept`
Description

Get the Event id				
Parameter				
N/A		N/A		
Returns				
std::uint32_t		The Event id		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-74 amsr::diag::SecurityEventLog::Event::GetId

4.12.6 amsr::diag::SecurityEventLog::EventType

Description	
Declaration of EventType	
Include	Generalize
amsr/diag/state_types/security_event_log_type.h	
Constructors	
SecurityEventLogEventType	Move constructs instance of class
Operators	
operator=	Move assigns instance of class
MemberFunctions	
GetBuffer	Get buffer
GetEvent	Get Event
GetEvents	Get the Events
GetId	Get the Event type id

Table 4-75 amsr::diag::SecurityEventLog::EventType

4.12.6.1 amsr::diag::SecurityEventLog::EventType::EventType

Prototype

```
amsr::diag::SecurityEventLog::EventType::EventType (
   amsr::diag::state_types::SecurityEventLogEventType &&
) noexcept
```

Description

Move constructs instance of class

Parameter

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-76 amsr::diag::SecurityEventLog::EventType::EventType

4.12.6.2 amsr::diag::SecurityEventLog::EventType::~EventType

Prototype				
`amsr::diag::SecurityEventLog::EventType::~EventType () noexcept`				
Description				
Destructor of class EventType				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-77 amsr::diag::SecurityEventLog::EventType::~EventType

4.12.6.3 amsr::diag::SecurityEventLog::EventType::operator=

Prototype	

`amsr::diag::state_types::SecurityEventLogEventType &`
`amsr::diag::SecurityEventLog::EventType::operator= (`
`  amsr::diag::state_types::SecurityEventLogEventType &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
amsr::diag::state_types::Security EventLogEventType &	Reference to self
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-78 amsr::diag::SecurityEventLog::EventType::operator=

4.12.6.4 amsr::diag::SecurityEventLog::EventType::GetBuffer

Prototype	
`ara::core::Span<const std::uint8_t>` `amsr::diag::SecurityEventLog::EventType::GetBuffer () const noexcept`	
Description	
Get buffer	
Parameter	
N/A	N/A
Returns	
ara::core::Span<const std::uint8_t>	The buffer
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-79 amsr::diag::SecurityEventLog::EventType::GetBuffer

4.12.6.5 amsr::diag::SecurityEventLog::EventType::GetEvent

Prototype	

`ara::core::Result<const SecurityEventLogEvent &>`
`amsr::diag::SecurityEventLog::EventType::GetEvent (`
`  std::uint32_t event_id`
`) const noexcept`	

Description	
Get Event	
Parameter	
event_id	The event id
Returns	
ara::core::Result<const SecurityEvent LogEvent &>	Result containing reference to the Event

Errors				
DiagLibErrc::kUnknownKey The Event id is unknown				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-80 amsr::diag::SecurityEventLog::EventType::GetEvent

4.12.6.6 amsr::diag::SecurityEventLog::EventType::GetEvents

Prototype	
`ara::core::Span<const SecurityEventLogEvent>` `amsr::diag::SecurityEventLog::EventType::GetEvents () const noexcept`	
Description	
Get the Events	
Parameter	
N/A	N/A
Returns	
ara::core::Span<const SecurityEvent LogEvent>	The Events
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-81 amsr::diag::SecurityEventLog::EventType::GetEvents

4.12.6.7 amsr::diag::SecurityEventLog::EventType::GetId

Prototype	
`std::uint32_t` `amsr::diag::SecurityEventLog::EventType::GetId () const noexcept`	
Description	
Get the Event type id	
Parameter	
N/A	N/A
Returns	

std::uint32_t		The Event id		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-82 amsr::diag::SecurityEventLog::EventType::GetId

4.12.7 amsr::diag::SecurityEventLog::AddRawEntryToAll

Prototype	
`void amsr::diag::SecurityEventLog::AddRawEntryToAll ( std::uint8_t buffer_pattern, ara::core::Span<const std::uint32_t> excluded_event_types ) noexcept`	
Description	
Applies a pattern to all event type buffers	
Parameter	
buffer_pattern	The byte pattern to fill each event type buffer-line with
excluded_event_types	An optional list of event types that will not add the pattern
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-83 amsr::diag::SecurityEventLog::AddRawEntryToAll

4.12.8 amsr::diag::SecurityEventLog::GetEventLog

Prototype
`ara::core::Result<ara::core::Vector<EventType> > amsr::diag::SecurityEventLog::GetEventLog () const noexcept`
Description
Get a snapshot of currently reported events

Parameter	
N/A	N/A
Returns	
ara::core::Result<ara::core::Vector< EventType> >	The EventTypes
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-84 amsr::diag::SecurityEventLog::GetEventLog

4.12.9 amsr::diag::SecurityEventLog::GetEventType

Prototype

```
ara::core::Result<EventType>
amsr::diag::SecurityEventLog::GetEventType (
   std::uint32_t id
) noexcept
```

Description	
Get the event type	
Parameter	
id	The event type id
Detailed Description	
This method is currently not implemented and always return an error	
Returns	
ara::core::Result<EventType>	The EventType
Errors	
Any/Custom Function is not implemented yet	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-85 amsr::diag::SecurityEventLog::GetEventType

4.12.10 amsr::diag::SecurityEventLog::Report

Prototype

```
void
amsr::diag::SecurityEventLog::Report (
   std::uint32_t event_type_id,
   std::uint32_t event_id,
   ara::core::Span<const std::uint8_t> context
) noexcept
```

Description	
Reports a security event with the given logging context for the given event ID and event type ID	
Parameter	
event_type_id	The ID of the security event type
event_id	The ID of the security event
context	The context to be logged
Returns	
void	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-86 amsr::diag::SecurityEventLog::Report

4.12.11 amsr::diag::SecurityEventLog::SetNotifierNumberOfBufferEntriesChanged

```
ara::core::Result<void>
amsr::diag::SecurityEventLog::SetNotifierNumberOfBufferEntriesChanged (
   std::function<void (std::uint32_t, std::uint32_t, std::uint32_t)> notifier
) noexcept
```

Description	
Register a notifier function which is called if the number of buffer entries has changed	
Parameter	
notifier	The notifier to be called
Notifier Parameter	
Type	Description
std::uint32_t	ID of the associated SecurityEventType
std::uint32_t	The number of reported SecurityEvents for the associated SecurityEventType
std::uint32_t	The maximum number of entries that can be logged for the associated SecurityEventType

Detailed Description				
If successful, a previously set notifier gets overwritten.				
Returns				
ara::core::Result<void>	-			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this SecurityEventLog has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetNotifierNumberOfBufferEntriesChanged method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Table 4-87 amsr::diag::SecurityEventLog::SetNotifierNumberOfBufferEntriesChanged

4.12.12 amsr::diag::SecurityEventLog::SetNotifierSecurityEvent

Prototype	

`ara::core::Result<void>`
`amsr::diag::SecurityEventLog::SetNotifierSecurityEvent (`
`  std::function<void (std::uint32_t)> notifier`
`) noexcept`	

Description	
Register a notifier function which is called if a security event has been reported	
Parameter	
notifier	The notifier to be called
Notifier Parameter	
Type	Description
std::uint32_t	ID of the reported SecurityEvent
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	-
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this SecurityEventLog has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetNotifierSecurityEvent method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Table 4-88 amsr::diag::SecurityEventLog::SetNotifierSecurityEvent

based on template version 1.00.00

4.13 amsr::diag::ServiceProcessingNotification

Description	
DiagnosticServiceProcessingInterface	
Include	**Generalize**
amsr/diag/service_processing_notification.h	
Constructors	
ServiceProcessingNotification	Move constructs an instance of ServiceProcessingNotification
ServiceProcessingNotification	Constructor of ServiceProcessingNotification (inherited)
Operators	
operator=	Move assigns an instance of ServiceProcessingNotification
Types	
ConfirmationStatusType	Represents the status of the service processing
NegativeResponseDetails	Represents the negative response details of the service request
MemberFunctions	
Offer	This Offer will enable the DM to forward request messages to this handler
ProcessingEnd	Called for any request message after processing is finished
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-89 amsr::diag::ServiceProcessingNotification

4.13.1 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification

Prototype

```
amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification (
  amsr::diag::ServiceProcessingNotification && other
) noexcept
```

Description

Move constructs an instance of ServiceProcessingNotification

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-90 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification

4.13.2 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification

Prototype				
`amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor of ServiceProcessingNotification (inherited)				
Parameter				
specifier		InstanceSpecifier to an PortPrototype of an DiagnosticServiceProcessingNotificationInterface		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-91 amsr::diag::ServiceProcessingNotification::ServiceProcessingNotification

4.13.3 amsr::diag::ServiceProcessingNotification::~ServiceProcessingNotification

Prototype				
`amsr::diag::ServiceProcessingNotification:: ~ServiceProcessingNotification () noexcept`				
Description				
Destructor of ServiceProcessingNotification				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-92 amsr::diag::ServiceProcessingNotification::~ServiceProcessingNotification

4.13.4 amsr::diag::ServiceProcessingNotification::operator=

Prototype

`amsr::diag::ServiceProcessingNotification & amsr::diag::ServiceProcessingNotification::operator= ( amsr::diag::ServiceProcessingNotification && other ) & noexcept`				
Description				
Move assigns an instance of ServiceProcessingNotification				
Parameter				
other	The other object			
Returns				
amsr::diag::ServiceProcessing Notification &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-93 amsr::diag::ServiceProcessingNotification::operator=

4.13.5 amsr::diag::ServiceProcessingNotification::ConfirmationStatusType

Prototype	
`amsr::diag::ServiceProcessingNotification::ConfirmationStatusType`	
Description	
Represents the status of the service processing	
Type	
std::uint8_t	
Enumerations	
ResPosOk	0x00
kResPosNotOk	0x01
kResNegOk	0x02
kResNegNotOk	0x03
kResPosSuppressed	0x04
kResNegSuppressed	0x05
kCanceled	0x06
kNoProcessingNoResponse	0x07

Table 4-94 amsr::diag::ServiceProcessingNotification::ConfirmationStatusType

4.13.6 amsr::diag::ServiceProcessingNotification::NegativeResponseDetails

Description	
Type for negative response message details	
Include	**Generalize**
amsr/diag/state_types/negative_response_details.h	
MemberFunctions	
GetNegativeResponseReason	Get the negative response reason of the process
GetRawNrc	Get the raw NRC of the process
GetUdsNrc	Get the UDS specific NRC if applicable

Table 4-95 amsr::diag::NegativeResponseDetails

4.13.7 amsr::diag::NegativeResponseDetails::GetNegativeResponseReason

Prototype				
`enum amsr::diag::NegativeResponseReasonType amsr::diag::NegativeResponseDetails::GetNegativeResponseReason () const noexcept`				
Description				
Get the negative response reason of the process				
Parameter				
N/A	N/A			
Returns				
enum amsr::diag::NegativeResponse ReasonType	Received reason			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-96 amsr::diag::NegativeResponseDetails::GetNegativeResponseReason

4.13.8 amsr::diag::NegativeResponseDetails::GetRawNrc

Prototype
`std::uint8_t amsr::diag::NegativeResponseDetails::GetRawNrc () const noexcept`
Description
Get the raw NRC of the process

Parameter				
N/A		N/A		
Returns				
std::uint8_t		Received NRC		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-97 amsr::diag::NegativeResponseDetails::GetRawNrc

4.13.9 amsr::diag::NegativeResponseDetails::GetUdsNrc

Prototype	
`ara::core::Optional<UdsNrcType>` `amsr::diag::NegativeResponseDetails::GetUdsNrc () const noexcept`	
Description	
Get the UDS specific NRC if applicable	
Parameter	
N/A	N/A
Returns	
ara::core::Optional<UdsNrcType>	No value if conversion to UDS specific NRC was not applicable, otherwise the UDS specific NRC
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-98 amsr::diag::NegativeResponseDetails::GetUdsNrc

4.13.10 amsr::diag::ServiceProcessingNotification::Offer

Prototype				
`ara::core::Result<void>` `amsr::diag::ServiceProcessingNotification::Offer () noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-99 amsr::diag::ServiceProcessingNotification::Offer

4.13.11 amsr::diag::ServiceProcessingNotification::ProcessingEnd

Prototype	

`void`
`amsr::diag::ServiceProcessingNotification::ProcessingEnd (`
`  ara::core::Span<const std::uint8_t> request_service_header,`
`  amsr::diag::ServiceProcessingNotification::ConfirmationStatusType status,`
`  ara::core::Optional<NegativeResponseDetails> negative_response_details,`
`  const ara::diag::MetaInfo & meta_info`
`)`	

Description	
Called for any request message after processing is finished	
Parameter	
request_service_header	The first eight bytes of the UDS service request message.
status	Status/Outcome of the service processing.
negative_response_details	Details of the negative response.
meta_info	MetaInfo of the request.
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-100 amsr::diag::ServiceProcessingNotification::ProcessingEnd

4.13.12 amsr::diag::ServiceProcessingNotification::StopOffer

Prototype				
`void amsr::diag::ServiceProcessingNotification::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-101 amsr::diag::ServiceProcessingNotification::StopOffer

4.14 amsr::diag::EventStatusByte

Description	
Current event status bit-encoded	
Include	**Generalize**
amsr/diag/state_types/event_status_byte.h	
MemberFunctions	
IsFailedAndTested	Returns true if the associated event is tested and failed
IsNotSet	Returns true if the given bit is not set
IsNotSet	Returns true if all the given bits are not set
IsPassedAndTested	Returns true if the associated event is tested and passed
IsSet	Returns true if the given bit is set
IsSet	Returns true if the all the given bits as parameters are set

Table 4-102 amsr::diag::EventStatusByte

4.14.1 amsr::diag::EventStatusByte::IsFailedAndTested

Prototype				
`bool amsr::diag::EventStatusByte::IsFailedAndTested () const noexcept`				
Description				
Returns true if the associated event is tested and failed				
Parameter				
N/A	N/A			
Returns				
bool	True if failed and tested, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-103 amsr::diag::EventStatusByte::IsFailedAndTested

4.14.2 amsr::diag::EventStatusByte::IsNotSet

Prototype
`bool amsr::diag::EventStatusByte::IsNotSet ( enum amsr::diag::EventStatusBit bit ) const noexcept`

Description				
Returns true if the given bit is not set				
Parameter				
bit		The bit to be tested		
Returns				
bool		True if the bit is not set, false otherwise.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-104 amsr::diag::EventStatusByte::IsNotSet

4.14.3 amsr::diag::EventStatusByte::IsNotSet

Prototype				

`bool`
`amsr::diag::EventStatusByte::IsNotSet (`
`  enum amsr::diag::EventStatusBit bit,`
`  EventStatusBits...  bits`
`) const noexcept`				

Description				
Returns true if all the given bits are not set				
Template Parameter				
EventStatusBits		Types of the remaining bits. Must all be EventStatusBit.		
Parameter				
bit		The first bit.		
bits		List of all other bits		
Returns				
bool		True if all the bits are not set, false otherwise.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
See parameter description				

Table 4-105 amsr::diag::EventStatusByte::IsNotSet

4.14.4 amsr::diag::EventStatusByte::IsPassedAndTested

Prototype				
`bool amsr::diag::EventStatusByte::IsPassedAndTested () const noexcept`				
Description				
Returns true if the associated event is tested and passed				
Parameter				
N/A	N/A			
Returns				
bool	True if passed and tested, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-106 amsr::diag::EventStatusByte::IsPassedAndTested

4.14.5 amsr::diag::EventStatusByte::IsSet

Prototype				
`bool amsr::diag::EventStatusByte::IsSet ( enum amsr::diag::EventStatusBit bit ) const noexcept`				
Description				
Returns true if the given bit is set				
Parameter				
bit	The bit to be tested			
Returns				
bool	True if the bit is set, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				

N/A

Table 4-107 amsr::diag::EventStatusByte::IsSet

4.14.6 amsr::diag::EventStatusByte::IsSet

Prototype				
`bool amsr::diag::EventStatusByte::IsSet ( enum amsr::diag::EventStatusBit bit, EventStatusBits... bits ) const noexcept`				
Description				
Returns true if the all the given bits as parameters are set				
Template Parameter				
EventStatusBits	Types of the remaining bits. Must all be EventStatusBit.			
Parameter				
bit	The first bit.			
bits	List of all other bits			
Returns				
bool	True if all the bits are set, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
See parameter description				

Table 4-108 amsr::diag::EventStatusByte::IsSet

based on template version 1.00.00

4.15 amsr::diag::UdsDTCStatusByteType

Description	
Type for UdsDTCStatusByteType status	
Include	**Generalize**
amsr/diag/state_types/uds_status_byte_type.h	
Operators	
operator==	Equality operator
MemberFunctions	
IsFailedAndTested	Returns true if the associated dtc is tested and failed
IsNotSet	Returns true if the bit given as parameter is not set
IsNotSet	Returns true if the all the given bits are not set
IsPassedAndTested	Returns true if the associated dtc is tested and passed
IsSet	Returns true if the bit given is set
IsSet	Returns true if all the given bits are set

Table 4-109 amsr::diag::UdsDTCStatusByteType

4.15.1 amsr::diag::UdsDTCStatusByteType::operator==

Prototype

```
bool
amsr::diag::UdsDTCStatusByteType::operator== (
   const class amsr::diag::UdsDTCStatusByteType rhs
) const noexcept
```

Description	
Equality operator	
Parameter	
rhs	UdsDTCStatusByteType to compare to this status
Returns	
bool	true of the status bytes match
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE

Constraints
N/A

Table 4-110 amsr::diag::UdsDTCStatusByteType::operator==

4.15.2 amsr::diag::UdsDTCStatusByteType::IsFailedAndTested

Prototype

`bool amsr::diag::UdsDTCStatusByteType::IsFailedAndTested () const noexcept`				
Description				
Returns true if the associated dtc is tested and failed				
Parameter				
N/A		N/A		
Returns				
bool		True if failed and tested, false otherwise.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-111 amsr::diag::UdsDTCStatusByteType::IsFailedAndTested

4.15.3 amsr::diag::UdsDTCStatusByteType::IsNotSet

Prototype				
`bool amsr::diag::UdsDTCStatusByteType::IsNotSet ( enum amsr::diag::UdsStatusBit bit ) const noexcept`				
Description				
Returns true if the bit given as parameter is not set				
Parameter				
bit		The bit to be tested		
Returns				
bool		True if the bit is not set, false otherwise.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-112 amsr::diag::UdsDTCStatusByteType::IsNotSet

4.15.4 amsr::diag::UdsDTCStatusByteType::IsNotSet

Prototype				
`bool amsr::diag::UdsDTCStatusByteType::IsNotSet ( enum amsr::diag::UdsStatusBit bit, UdsStatusBits... bits ) const noexcept`				
Description				
Returns true if the all the given bits are not set				
Template Parameter				
UdsStatusBits	Types of the remaining bits. Must all be UdsStatusBit.			
Parameter				
bit	The first bit.			
bits	List of all other bits			
Returns				
bool	True if all the bits are not set, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
See parameter description				

Table 4-113 amsr::diag::UdsDTCStatusByteType::IsNotSet

4.15.5 amsr::diag::UdsDTCStatusByteType::IsPassedAndTested

Prototype				
`bool amsr::diag::UdsDTCStatusByteType::IsPassedAndTested () const noexcept`				
Description				
Returns true if the associated dtc is tested and passed				
Parameter				
N/A	N/A			
Returns				
bool	True if passed and tested, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE

Constraints
N/A

Table 4-114 amsr::diag::UdsDTCStatusByteType::IsPassedAndTested

4.15.6 amsr::diag::UdsDTCStatusByteType::IsSet

Prototype				
`bool amsr::diag::UdsDTCStatusByteType::IsSet ( enum amsr::diag::UdsStatusBit bit ) const noexcept`				
Description				
Returns true if the bit given is set				
Parameter				
bit	The bit to be tested			
Returns				
bool	True if the bit is set, false otherwise.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-115 amsr::diag::UdsDTCStatusByteType::IsSet

4.15.7 amsr::diag::UdsDTCStatusByteType::IsSet

Prototype	
`bool amsr::diag::UdsDTCStatusByteType::IsSet ( enum amsr::diag::UdsStatusBit bit, UdsStatusBits... bits ) const noexcept`	
Description	
Returns true if all the given bits are set	
Template Parameter	
UdsStatusBits	Types of the remaining bits. Must all be UdsStatusBit.
Parameter	
bit	The first bit.
bits	List of all other bits

Returns				
bool		True if all the bits are set, false otherwise.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
See parameter description				

Table 4-116 amsr::diag::UdsDTCStatusByteType::IsSet

based on template version 1.00.00

4.16 amsr::diag::DataTransfer

4.16.1 amsr::diag::DataTransferReadByPullHandler

Description	
Handles data transfers initiated by RequestReadFile/-Directory with data pulled by AraDiag from the implementation	
Include	**Generalize**
amsr/diag/data_transfer.h	
Constructors	
DataTransferReadByPullHandler	Constructs an instance of this class
DataTransferReadByPullHandler	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class
MemberFunctions	
ExitRead	Exits an ongoing data transfer session
Read	Reads data chunk from the application

Table 4-117 amsr::diag::DataTransferReadByPullHandler

4.16.2 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler

Prototype				
`amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler () noexcept`				
Description				
Constructs an instance of this class				
Parameter				
N/A		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-118 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler

4.16.3 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler

Prototype
`amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler ( amsr::diag::DataTransferReadByPullHandler && other ) noexcept`

Description				
Move constructs an instance of this class				
Parameter				
other	The other object			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-119 amsr::diag::DataTransferReadByPullHandler::DataTransferReadByPullHandler

4.16.4 amsr::diag::DataTransferReadByPullHandler::~DataTransferReadByPullHandler

Prototype				
`amsr::diag::DataTransferReadByPullHandler::~DataTransferReadByPullHandler () noexcept`				
Description				
Destructs an instance of this class				
Detailed Description				
An instance of this class must not be destroyed before ExitRead() is called				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-120 amsr::diag::DataTransferReadByPullHandler::~DataTransferReadByPullHandler

4.16.5 amsr::diag::DataTransferReadByPullHandler::operator=

Prototype
`amsr::diag::DataTransferReadByPullHandler & amsr::diag::DataTransferReadByPullHandler::operator= ( amsr::diag::DataTransferReadByPullHandler && other ) & noexcept`
Description
Move assigns an instance of this class
Parameter

other	The other object
Returns	
amsr::diag::DataTransferReadByPullHandler &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-121 amsr::diag::DataTransferReadByPullHandler::operator=

4.16.6 amsr::diag::DataTransferReadByPullHandler::ExitRead

Prototype

```
ara::core::Future<ara::core::Vector<std::uint8_t> >
amsr::diag::DataTransferReadByPullHandler::ExitRead (
  enum amsr::diag::DataTransferExitType exit_type,
  ara::core::Span<std::uint8_t> transfer_request_parameter_record,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Exits an ongoing data transfer session

Parameter

exit_type	Specifies the exit reason
transfer_request_parameter_record	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<ara::core::Vector<std::uint8_t> >	returns a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.

Errors				
Any Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-122 amsr::diag::DataTransferReadByPullHandler::ExitRead

4.16.7 amsr::diag::DataTransferReadByPullHandler::Read

Prototype

```
ara::core::Future<std::uint32_t>
amsr::diag::DataTransferReadByPullHandler::Read (
  ara::core::Span<std::uint8_t> response_data,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Reads data chunk from the application

Parameter

response_data	The view over a pre-allocated by AraDiag memory to write in this data chunk.
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Detailed Description

The first or subsequent file/directory info content chunk to be sent back to the UDS client will be composed by the application and copied into the provided by AraDiag Span with up to the requested Span size.

Returns

ara::core::Future<std::uint32_t>	A Future, which either gets readied to actual amount of data provided in the Span (for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in response_data will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.

Errors				
Any Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-123 amsr::diag::DataTransferReadByPullHandler::Read

4.16.8 amsr::diag::DataTransferReadByPushHandler

Description	
Handles data transfers initiated by RequestReadFile/-Directory with data pushed by the implementation	
Include	Generalize
amsr/diag/data_transfer.h	
Constructors	
DataTransferReadByPushHandler	Constructs an instance of this class
DataTransferReadByPushHandler	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class
MemberFunctions	
ExitRead	Exits an ongoing data transfer session
Read	Reads data chunk from the application

Table 4-124 amsr::diag::DataTransferReadByPushHandler

4.16.9 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler

Prototype	
`amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler () noexcept`	
Description	
Constructs an instance of this class	
Parameter	
N/A	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-125 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler

4.16.10 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler

Prototype	
`amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler ( amsr::diag::DataTransferReadByPushHandler && other ) noexcept`	
Description	
Move constructs an instance of this class	
Parameter	
other	The other object
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-126 amsr::diag::DataTransferReadByPushHandler::DataTransferReadByPushHandler

4.16.11 amsr::diag::DataTransferReadByPushHandler::~DataTransferReadByPushHandler

Prototype
`amsr::diag::DataTransferReadByPushHandler::~DataTransferReadByPushHandler () noexcept`
Description
Destructs an instance of this class
Detailed Description
An instance of this class must not be destroyed before ExitRead() is called
Errors
N/A
Particularities and Limitations
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 4-127 amsr::diag::DataTransferReadByPushHandler::~DataTransferReadByPushHandler

4.16.12 amsr::diag::DataTransferReadByPushHandler::operator=

Prototype	

`amsr::diag::DataTransferReadByPushHandler &`
`amsr::diag::DataTransferReadByPushHandler::operator= (`
`  amsr::diag::DataTransferReadByPushHandler && other`
`) & noexcept`	

Description	
Move assigns an instance of this class	
Parameter	
other	The other object
Returns	
amsr::diag::DataTransferReadBy PushHandler &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 4-128 amsr::diag::DataTransferReadByPushHandler::operator=

4.16.13 amsr::diag::DataTransferReadByPushHandler::ExitRead

Prototype	

`ara::core::Future<ara::core::Vector<std::uint8_t> >`
`amsr::diag::DataTransferReadByPushHandler::ExitRead (`
`  enum amsr::diag::DataTransferExitType exit_type,`
`  ara::core::Span<std::uint8_t> transfer_request_parameter_record,`
`  const ara::diag::MetaInfo & meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`) noexcept`	

Description	
Exits an ongoing data transfer session	
Parameter	
exit_type	Specifies the exit reason

transfer_request_parameter_record	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<ara::core::Vector<std::uint8_t> >	returns a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-129 amsr::diag::DataTransferReadByPushHandler::ExitRead

4.16.14 amsr::diag::DataTransferReadByPushHandler::Read

Prototype

```
ara::core::Future<ara::core::Vector<std::uint8_t> >
amsr::diag::DataTransferReadByPushHandler::Read (
  std::size_t recommended_number_bytes_to_return,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Reads data chunk from the application

Parameter

recommended_number_bytes_to_return	The recommended number of bytes to send back to DM in order to get optimum data throughput
meta_info	Contains additional meta information

cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Detailed Description	
The first or subsequent file/directory info content chunk to be sent back to the UDS client will be composed by the application and its ownership is passed as a result to AraDiag.	
Returns	
ara::core::Future<ara::core::Vector<std::uint8_t> >	A Future, which either gets readied to ReadPushOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in ReadPushOutput.responseData will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-130 amsr::diag::DataTransferReadByPushHandler::Read

based on template version 1.00.00

4.16.15 amsr::diag::DataTransferReadSession

Description	
Encapsulates all data transfer reading variants	
Include	**Generalize**
amsr/diag/data_transfer.h	
Constructors	
DataTransferReadSession	Construct with a DataTransferReadByPullHandler
DataTransferReadSession	Construct with a DataTransferReadByPushHandler
DataTransferReadSession	Construct with a DataTransferReadSharedDataHandler
DataTransferReadSession	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class

Table 4-131 amsr::diag::DataTransferReadSession

4.16.16 amsr::diag::DataTransferReadSession::DataTransferReadSession

Prototype

```
amsr::diag::DataTransferReadSession::DataTransferReadSession (
  ReadByPullHandler && read_handler
) noexcept
```

Description	
Construct with a DataTransferReadByPullHandler	
Template Parameter	
ReadByPullHandler	Application specific ReadHandler type inheriting from DataTransferReadByPullHandler
Parameter	
read_handler	Reader handler which is either moved into the constructor or passed as reference - pre: if passed as a reference it must be ensured that the referenced object is valid until ExitRead() is called
Errors	
N/A	

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

See parameter description

Table 4-132 amsr::diag::DataTransferReadSession::DataTransferReadSession

4.16.17 amsr::diag::DataTransferReadSession::DataTransferReadSession

Prototype				
`amsr::diag::DataTransferReadSession::DataTransferReadSession ( ReadByPushHandler && read_handler ) noexcept`				
Description				
Construct with a DataTransferReadByPushHandler				
Template Parameter				
ReadByPushHandler	Application specific ReadHandler type inheriting from DataTransferReadByPushHandler			
Parameter				
read_handler	Reader handler which is either moved into the constructor or passed as reference - pre: if passed as a reference it must be ensured that the referenced object is valid until ExitRead() is called			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
See parameter description				

Table 4-133 amsr::diag::DataTransferReadSession::DataTransferReadSession

4.16.18 amsr::diag::DataTransferReadSession::DataTransferReadSession

Prototype	
`amsr::diag::DataTransferReadSession::DataTransferReadSession ( ReadSharedDataHandler && read_handler ) noexcept`	
Description	
Construct with a DataTransferReadSharedDataHandler	
Template Parameter	
ReadSharedDataHandler	Application specific ReadHandler type inheriting from DataTransferReadSharedDataHandler
Parameter	
read_handler	Reader handler which is either moved into the constructor or passed as reference - pre: if passed as a reference it must be ensured that the referenced object is valid until ExitRead() is called

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
See parameter description				

Table 4-134 amsr::diag::DataTransferReadSession::DataTransferReadSession

4.16.19 amsr::diag::DataTransferReadSession::DataTransferReadSession

Prototype	
`amsr::diag::DataTransferReadSession::DataTransferReadSession ( amsr::diag::DataTransferReadSession && other ) noexcept`	
Description	
Move constructs an instance of this class	
Parameter	
other	The other object
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-135 amsr::diag::DataTransferReadSession::DataTransferReadSession

4.16.20 amsr::diag::DataTransferReadSession::~DataTransferReadSession

Prototype				
`amsr::diag::DataTransferReadSession::~DataTransferReadSession () noexcept`				
Description				
Destructs an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-136 amsr::diag::DataTransferReadSession::~DataTransferReadSession

4.16.21 amsr::diag::DataTransferReadSession::operator=

Prototype				

`amsr::diag::DataTransferReadSession &`
`amsr::diag::DataTransferReadSession::operator= (`
`  amsr::diag::DataTransferReadSession && other`
`) & noexcept`				

Description				
Move assigns an instance of this class				
Parameter				
other	The other object			
Returns				
amsr::diag::DataTransferRead Session &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-137 amsr::diag::DataTransferReadSession::operator=

4.16.22 amsr::diag::DataTransferReadSharedDataHandler

Description	
Handles data transfers initiated by RequestReadFile/-Directory with shared data from the implementation	
Include	**Generalize**
amsr/diag/data_transfer.h	
Constructors	
DataTransferReadSharedData Handler	Constructs an instance of this class
DataTransferReadSharedData Handler	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class
MemberFunctions	
ExitRead	Exits an ongoing data transfer session

Read	Provides the whole data to be read out from the application in form of a shared data view

Table 4-138 amsr::diag::DataTransferReadSharedDataHandler

4.16.23 amsr::diag::DataTransferReadSharedDataHandler:: DataTransferReadSharedDataHandler

Prototype				
`amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler () noexcept`				
Description				
Constructs an instance of this class				
Parameter				
N/A		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-139 amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler

4.16.24 amsr::diag::DataTransferReadSharedDataHandler:: DataTransferReadSharedDataHandler

Prototype				
`amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler ( amsr::diag::DataTransferReadSharedDataHandler && other ) noexcept`				
Description				
Move constructs an instance of this class				
Parameter				
other		The other object		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-140 amsr::diag::DataTransferReadSharedDataHandler::DataTransferReadSharedDataHandler

4.16.25 amsr::diag::DataTransferReadSharedDataHandler::~DataTransferReadSharedDataHandler

Prototype				
`amsr::diag::DataTransferReadSharedDataHandler::~DataTransferReadSharedDataHandler () noexcept`				
Description				
Destructs an instance of this class				
Detailed Description				
An instance of this class must not be destroyed before ExitRead() is called				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-141 amsr::diag::DataTransferReadSharedDataHandler::~DataTransferReadSharedDataHandler

4.16.26 amsr::diag::DataTransferReadSharedDataHandler::operator=

Prototype				
`amsr::diag::DataTransferReadSharedDataHandler & amsr::diag::DataTransferReadSharedDataHandler::operator= ( amsr::diag::DataTransferReadSharedDataHandler && other ) & noexcept`				
Description				
Move assigns an instance of this class				
Parameter				
other	The other object			
Returns				
amsr::diag::DataTransferReadSharedDataHandler &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-142 amsr::diag::DataTransferReadSharedDataHandler::operator=

4.16.27 amsr::diag::DataTransferReadSharedDataHandler::ExitRead

Prototype

```
ara::core::Future<ara::core::Vector<std::uint8_t> >
amsr::diag::DataTransferReadSharedDataHandler::ExitRead (
  enum amsr::diag::DataTransferExitType exit_type,
  ara::core::Span<std::uint8_t> transfer_request_parameter_record,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Exits an ongoing data transfer session

Parameter

exit_type	Specifies the exit reason
transfer_request_parameter_record	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<ara::core::Vector<std::uint8_t> >	returns a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE

Constraints

N/A

Table 4-143 amsr::diag::DataTransferReadSharedDataHandler::ExitRead

4.16.28 amsr::diag::DataTransferReadSharedDataHandler::Read

Prototype	

`ara::core::Future<ara::core::Span<std::uint8_t> >`
`amsr::diag::DataTransferReadSharedDataHandler::Read (`
`  amsr::diag::ReleaseHandler release_handler,`
`  const ara::diag::MetaInfo & meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`) noexcept`	

Description	
Provides the whole data to be read out from the application in form of a shared data view	
Parameter	
release_handler	The release shared resource handler to be stored and used by the application until a new one is passed for the same meta_info (UDS client)
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Detailed Description	
The whole file/directory info content to be sent back to the UDS client will be allocated by the application and its shared ownership (view over the memory location) is passed as a result to AraDiag. The allocated memory must be kept until the ReleaseHandler's notifier is invoked or the status polled, signalling that the memory addressed by the Span now can safely be released.	
Returns	
ara::core::Future<ara::core::Span<std::uint8_t> >	A Future, which either gets readied to ReadSharedOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in ReleaseSharedHandler.responseData will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE

Constraints

N/A

Table 4-144 amsr::diag::DataTransferReadSharedDataHandler::Read

based on template version 1.00.00

4.16.29 amsr::diag::DataTransferWriteHandler

Description	
Handles data transfers initiated by AddFile, ResumeFile or ReplaceFile ModeOfOperation handling per requested RequestWriteFile	
Include	**Generalize**
amsr/diag/data_transfer.h	
Constructors	
DataTransferWriteHandler	Constructs an instance of this class
DataTransferWriteHandler	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class
MemberFunctions	
ExitWrite	Exits an ongoing data transfer session
Write	Appends a data chunk into application memory

Table 4-145 amsr::diag::DataTransferWriteHandler

4.16.30 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler

Prototype				
`amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler ()`				
Description				
Constructs an instance of this class				
Parameter				
N/A	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-146 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler

4.16.31 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler

Prototype

`amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler (`
`  amsr::diag::DataTransferWriteHandler && other`
`) noexcept`

Description
Move constructs an instance of this class

Parameter	
other	The other object
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-147 amsr::diag::DataTransferWriteHandler::DataTransferWriteHandler

4.16.32 amsr::diag::DataTransferWriteHandler::~DataTransferWriteHandler

Prototype

```
amsr::diag::DataTransferWriteHandler::~DataTransferWriteHandler () noexcept
```

Description

Destructs an instance of this class

Detailed Description

An instance of this class must not be destroyed before ExitWrite() is called

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-148 amsr::diag::DataTransferWriteHandler::~DataTransferWriteHandler

4.16.33 amsr::diag::DataTransferWriteHandler::operator=

Prototype

```
amsr::diag::DataTransferWriteHandler &
amsr::diag::DataTransferWriteHandler::operator= (
   amsr::diag::DataTransferWriteHandler && other
) & noexcept
```

Description

Move assigns an instance of this class

Parameter	
other	The other object
Returns	

amsr::diag::DataTransferWrite Handler &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-149 amsr::diag::DataTransferWriteHandler::operator=

4.16.34 amsr::diag::DataTransferWriteHandler::ExitWrite

Prototype

```
ara::core::Future<ara::core::Vector<std::uint8_t> >
amsr::diag::DataTransferWriteHandler::ExitWrite (
  enum amsr::diag::DataTransferExitType exit_type,
  ara::core::Span<std::uint8_t> transfer_request_parameter_record,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Exits an ongoing data transfer session

Parameter

exit_type	Specifies the exit reason
transfer_request_parameter_record	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<ara::core:: Vector<std::uint8_t> >	returns a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-150 amsr::diag::DataTransferWriteHandler::ExitWrite

4.16.35 amsr::diag::DataTransferWriteHandler::Write

Prototype

```
ara::core::Future<void>
amsr::diag::DataTransferWriteHandler::Write (
  ara::core::Span<const std::uint8_t> request_data,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Appends a data chunk into application memory

Parameter

request_data	The first or subsequent file content chunk received from the UDS client to be written into the file.
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<void>	A Future, which either is of type void or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message).

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				

N/A

Table 4-151 amsr::diag::DataTransferWriteHandler::Write

based on template version 1.00.00

4.16.36 amsr::diag::DataTransferWriteSession

Description	
Encapsulates all data transfer writing variants	
Include	**Generalize**
amsr/diag/data_transfer.h	
Constructors	
DataTransferWriteSession	Vendor specific implementation
DataTransferWriteSession	Vendor specific implementation
DataTransferWriteSession	Move constructs an instance of this class
Operators	
operator=	Move assigns an instance of this class

Table 4-152 amsr::diag::DataTransferWriteSession

4.16.37 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

Prototype

```
amsr::diag::DataTransferWriteSession::DataTransferWriteSession (
  WriteHandler & write_handler
) noexcept
```

Description	
Construct with a reference to a DataTransferWriteHandler	
Template Parameter	
WriteHandler	Application specific WriteHandler type which inherits from DataTransferWriteHandler
Parameter	
write_handler	The write handler that will receive the transferred data - pre: it must be ensured that the referenced object is valid until ExitWrite() is called
Errors	
N/A	

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-153 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

4.16.38 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

Prototype

```
amsr::diag::DataTransferWriteSession::DataTransferWriteSession (
   WriteHandler && write_handler
) noexcept
```

Description				
Construct with a DataTransferWriteHandler (takes ownership of the write handler)				
Template Parameter				
WriteHandler	Application specific WriteHandler type which inherits from DataTransferWriteHandler			
Parameter				
write_handler	The write handler that will receive the transferred data			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-154 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

4.16.39 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

Prototype				
`amsr::diag::DataTransferWriteSession::DataTransferWriteSession ( amsr::diag::DataTransferWriteSession && other ) noexcept`				
Description				
Move constructs an instance of this class				
Parameter				
other	The other object			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-155 amsr::diag::DataTransferWriteSession::DataTransferWriteSession

4.16.40 amsr::diag::DataTransferWriteSession::~DataTransferWriteSession

Prototype				
`amsr::diag::DataTransferWriteSession::~DataTransferWriteSession () noexcept`				
Description				
Destructs an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-156 amsr::diag::DataTransferWriteSession::~DataTransferWriteSession

4.16.41 amsr::diag::DataTransferWriteSession::operator=

Prototype	

`amsr::diag::DataTransferWriteSession &`
`amsr::diag::DataTransferWriteSession::operator= (`
`  amsr::diag::DataTransferWriteSession && other`
`) & noexcept`	

Description	
Move assigns an instance of this class	
Parameter	
other	The other object
Returns	
amsr::diag::DataTransferWrite Session &	Reference to self
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-157 amsr::diag::DataTransferWriteSession::operator=

4.17 amsr::diag::FileTransferService

Description	
File transfer service interface	
Include	**Generalize**
amsr/diag/file_transfer.h	
Constructors	
FileTransferService	Move constructs an instance of FileTransferService
FileTransferService	Constructor for FileTransferService (inherited)
Operators	
operator=	Move assigns an instance of FileTransferService
MemberFunctions	
DeleteFile	Called for RequestFileTransfer with ModeOfOperation DeleteFile
Offer	This Offer will enable the DM to forward request messages to this handler
RequestReadDirectory	Called for RequestFileTransfer with ModeOfOperation ReadDir
RequestReadFile	Called for RequestFileTransfer with ModeOfOperation ReadFile
RequestResumeWriteFile	Called for RequestFileTransfer with ModeOfOperation ResumeFile
RequestWriteFile	Called for RequestFileTransfer with ModeOfOperation WriteFile
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-158 amsr::diag::FileTransferService

4.17.1 amsr::diag::FileTransferService::FileTransferService

Prototype

```
amsr::diag::FileTransferService::FileTransferService (
  amsr::diag::FileTransferService && other
) noexcept
```

Description

Move constructs an instance of FileTransferService

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
The port passed as parameter is not offered

Table 4-159 amsr::diag::FileTransferService::FileTransferService

4.17.2 amsr::diag::FileTransferService::FileTransferService

Prototype				
`amsr::diag::FileTransferService::FileTransferService ( const ara::core::InstanceSpecifier & instance_specifier, ara::diag::ReentrancyType reentrancy_type ) noexcept`				
Description				
Constructor for FileTransferService (inherited)				
Parameter				
instance_specifier	InstanceSpecifier to an PortPrototype of an DownloadServiceInterface			
reentrancy_type	Specifies if interface is callable fully- or non-reentrant			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-160 amsr::diag::FileTransferService::FileTransferService

4.17.3 amsr::diag::FileTransferService::~FileTransferService

Prototype				
`amsr::diag::FileTransferService::~FileTransferService () noexcept`				
Description				
Destructor of class FileTransferService				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-161 amsr::diag::FileTransferService::~FileTransferService

4.17.4 amsr::diag::FileTransferService::operator=

Prototype				

`amsr::diag::FileTransferService &`
`amsr::diag::FileTransferService::operator= (`
`  amsr::diag::FileTransferService && other`
`) & noexcept`				

Description				
Move assigns an instance of FileTransferService				
Parameter				
other	The other object			
Returns				
amsr::diag::FileTransferService &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-162 amsr::diag::FileTransferService::operator=

4.17.5 amsr::diag::FileTransferService::DeleteFile

Prototype	

`ara::core::Future<void>`
`amsr::diag::FileTransferService::DeleteFile (`
`  ara::core::String file_name,`
`  const ara::diag::MetaInfo & meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`) noexcept`	

Description	
Called for RequestFileTransfer with ModeOfOperation DeleteFile	
Parameter	
file_name	Path including name of the file to delete
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Detailed Description	
This method is the complete operation of deleting a file	
Returns	

ara::core::Future<void>	A future with either a void data result or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-163 amsr::diag::FileTransferService::DeleteFile

4.17.6 amsr::diag::FileTransferService::Offer

Prototype	
`ara::core::Result<void>` `amsr::diag::FileTransferService::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
DiagLibErrc::kConfigurationMismatch if handle is an invalid handle	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-164 amsr::diag::FileTransferService::Offer

4.17.7 amsr::diag::FileTransferService::RequestReadDirectory

Prototype

```
ara::core::Future<std::tuple<DataTransferReadSession, std::uint64_t> >
amsr::diag::FileTransferService::RequestReadDirectory (
   ara::core::String directory_name,
   const ara::diag::MetaInfo & meta_info,
   ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description
Called for RequestFileTransfer with ModeOfOperation ReadDir

Parameter	
directory_name	Path including name of the directory to read
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<std::tuple<Data TransferReadSession, std::uint64_t> >	A future with either the access strategy to be used during reading and the directory information size to be reported to the UDS client for a positive response message or an UDS NRC value (for a negative response message)

Errors
Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE

Constraints
N/A

Table 4-165 amsr::diag::FileTransferService::RequestReadDirectory

4.17.8 amsr::diag::FileTransferService::RequestReadFile

Prototype

```
ara::core::Future<std::tuple<DataTransferReadSession, FileSizes> >
amsr::diag::FileTransferService::RequestReadFile (
   ara::core::String file_name,
   std::uint8_t data_format_identifier,
   const ara::diag::MetaInfo & meta_info,
   ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Called for RequestFileTransfer with ModeOfOperation ReadFile	
Parameter	
file_name	Path including name of the file to read
data_format_identifier	UDS dataFormat Identifier
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<std::tuple<Data TransferReadSession, FileSizes> >	A future with either the access strategy to be used during reading and the file sizes to be reported to the UDS client for a positive response message or an UDS NRC value (for a negative response message). The compressed size parameter: If no compression is used, the value shall be zero.
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-166 amsr::diag::FileTransferService::RequestReadFile

4.17.9 amsr::diag::FileTransferService::RequestResumeWriteFile

Prototype

```
ara::core::Future<std::tuple<amsr::diag::DataTransferWriteSession, std::uint64_t>
>
amsr::diag::FileTransferService::RequestResumeWriteFile (
  ara::core::String file_name,
  std::uint8_t data_format_identifier,
  struct amsr::diag::FileTransferService::FileSizes file_sizes,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Called for RequestFileTransfer with ModeOfOperation ResumeFile

Parameter

file_name	Path including name of the file to write
data_format_identifier	UDS dataFormat Identifier
file_sizes	The compressed/uncompressed file sizes
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<std::tuple<amsr::diag::DataTransferWriteSession, std::uint64_t>>	A future with either an instance of the file writing session and the byte position the client shall start resuming from or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-167 amsr::diag::FileTransferService::RequestResumeWriteFile

4.17.10 amsr::diag::FileTransferService::RequestWriteFile

Prototype

```
ara::core::Future<DataTransferWriteSession>
amsr::diag::FileTransferService::RequestWriteFile (
  ara::core::String file_name,
  std::uint8_t data_format_identifier,
  struct amsr::diag::FileTransferService::FileSizes file_sizes,
  enum amsr::diag::FileTransferService::WriteFileMode mode,
  const ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
) noexcept
```

Description

Called for RequestFileTransfer with ModeOfOperation WriteFile

Parameter

file_name	Path including name of the file to write
data_format_identifier	UDS dataFormat Identifier
file_sizes	The compressed/uncompressed file sizes

mode	The file replacement mode
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<DataTransferWriteSession>	A future with either an instance of the file writing session or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	FALSE
Constraints				
N/A				

Table 4-168 amsr::diag::FileTransferService::RequestWriteFile

4.17.11 amsr::diag::FileTransferService::StopOffer

Prototype	
`void amsr::diag::FileTransferService::StopOffer () const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Detailed Description	
Any DataTransferReadSession associated with this port, respectively its data transfer read handler, will be exited (for abort reason)	
Parameter	
N/A	N/A
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-169 amsr::diag::FileTransferService::StopOffer

based on template version 1.00.00

4.18 amsr::diag::DownloadService

Description	
Download service interface	
Include	**Generalize**
amsr/diag/download.h	
Constructors	
DownloadService	Move constructs an instance of DownloadService
DownloadService	Constructor for DownloadService (inherited)
Operators	
operator=	Move assigns an instance of DownloadService
MemberFunctions	
DownloadData	Called for TransferData following a previous RequestDownload
Offer	This Offer will enable the DM to forward request messages to this handler
RequestDownload	Called for RequestDownload
RequestDownloadExit	Called for RequestTransferExit
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-170 amsr::diag::DownloadService

4.18.1 amsr::diag::DownloadService::DownloadService

Prototype

```
amsr::diag::DownloadService::DownloadService (
   amsr::diag::DownloadService && other
) noexcept
```

Description

Move constructs an instance of DownloadService

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-171 amsr::diag::DownloadService::DownloadService

4.18.2 amsr::diag::DownloadService::DownloadService

Prototype				
`amsr::diag::DownloadService::DownloadService ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for DownloadService (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DownloadServiceInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-172 amsr::diag::DownloadService::DownloadService

4.18.3 amsr::diag::DownloadService::~DownloadService

Prototype				
`amsr::diag::DownloadService::~DownloadService () noexcept`				
Description				
Destructor of class DownloadService				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-173 amsr::diag::DownloadService::~DownloadService

4.18.4 amsr::diag::DownloadService::operator=

Prototype
`amsr::diag::DownloadService &` `amsr::diag::DownloadService::operator= ( amsr::diag::DownloadService && other ) & noexcept`
Description

Move assigns an instance of DownloadService				
Parameter				
other		The other object		
Returns				
amsr::diag::DownloadService &		Reference to self		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-174 amsr::diag::DownloadService::operator=

4.18.5 amsr::diag::DownloadService::DownloadData

Prototype

```
ara::core::Future<OperationOutput>
amsr::diag::DownloadService::DownloadData (
  ara::core::Span<const std::uint8_t> transfer_request_parameter_record,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for TransferData following a previous RequestDownload

Parameter

transfer_request_parameter_record	data to be transferred (copied/downloaded to the ECU/server).
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<OperationOutput>	a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.

Errors

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-175 amsr::diag::DownloadService::DownloadData

4.18.6 amsr::diag::DownloadService::Offer

Prototype	
`ara::core::Result<void>` `amsr::diag::DownloadService::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-176 amsr::diag::DownloadService::Offer

4.18.7 amsr::diag::DownloadService::RequestDownload

Prototype

```
ara::core::Future<void>
amsr::diag::DownloadService::RequestDownload (
  std::uint8_t data_format_identifier,
  std::uint8_t address_and_length_format_identifier,
  ara::core::Span<const std::uint8_t> memory_address_and_size,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description	
Called for RequestDownload	
Parameter	
data_format_identifier	UDS dataFormat Identifier
address_and_length_format_identifier	UDS addressAndLengthFormatIdentifier
memory_address_and_size	memoryAddress and memorySize part of the request
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<void>	a Future, which either gets readied to void (for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message)
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-177 amsr::diag::DownloadService::RequestDownload

4.18.8 amsr::diag::DownloadService::RequestDownloadExit

Prototype

```
ara::core::Future<OperationOutput>
amsr::diag::DownloadService::RequestDownloadExit (
  ara::core::Span<const std::uint8_t> transferRequestParameterRecord,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for RequestTransferExit

Parameter

transferRequestParameterRecord	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<OperationOutput>	a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message) Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-178 amsr::diag::DownloadService::RequestDownloadExit

4.18.9 amsr::diag::DownloadService::StopOffer

Prototype	
`void amsr::diag::DownloadService::StopOffer () const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Parameter	
N/A	N/A
Returns	
void	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-179 amsr::diag::DownloadService::StopOffer

4.19 amsr::diag::SovdAuthorization

Description	
Class to implement the SOVD Authorization interfaces in the application	
Include	**Generalize**
amsr/diag/sovd_authorization.h	
Constructors	
SovdAuthorization	Constructs an instance of SovdAuthorization (inherited)
SovdAuthorization	Move constructs an instance of SovdAuthorization
Destructor	
~SovdAuthorization	Destructs an instance of SovdAuthorization
Operators	
operator=	Move assigns an instance of SovdAuthorization
MemberFunctions	
Offer	This Offer will enable the SOVD lib to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from SOVD lib After calling, no further calls will be provided to the specific handlers since they will get destructed.
ValidateAuthorization	This function validates the authorization token sent by the client

Table 4-180 amsr::diag::SovdAuthorization

4.19.1 amsr::diag::SovdAuthorization::~SovdAuthorization

Prototype				
`amsr::diag::SovdAuthorization::~SovdAuthorization() noexcept`				
Description				
Destructs an instance of SovdAuthorization				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
All SovdTokenAuthorizationHandles must be destroyed				

Table 4-181 amsr::diag::SovdAuthorization::~SovdAuthorization

4.19.2 amsr::diag::SovdAuthorization::Offer

Prototype

`auto amsr::diag::SovdAuthorization::Offer() noexcept->ara::core::Result<void>`				
Description				
This Offer will enable the SOVD lib to forward request messages to this handler				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-182 amsr::diag::SovdAuthorization::Offer

4.19.3 amsr::diag::SovdAuthorization::SovdAuthorization

Prototype				

`amsr::diag::SovdAuthorization::SovdAuthorization(`
`  amsr::diag::SovdAuthorization&& other`
`) noexcept`				

Description				
Move constructs an instance of SovdAuthorization				
Parameter				
other		The other object		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-183 amsr::diag::SovdAuthorization::SovdAuthorization

4.19.4 amsr::diag::SovdAuthorization::SovdAuthorization

Prototype

```
amsr::diag::SovdAuthorization::SovdAuthorization(
   const ara::core::InstanceSpecifier& instance_specifier,
   ara::diag::ReentrancyType reentrancy_type
) noexcept
```

Description	
Constructs an instance of SovdAuthorization (inherited)	
Parameter	
instance_specifier	An InstanceSpecifier linking this instance with the PortPrototype in the manifest
reentrancy_type	The supported reentrancy for this port (currently this parameter is ignored and ReentrancyType::kNot is used)
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-184 amsr::diag::SovdAuthorization::SovdAuthorization

4.19.5 amsr::diag::SovdAuthorization::StopOffer

Prototype	
`void amsr::diag::SovdAuthorization::StopOffer() const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from SOVD lib After calling, no further calls will be provided to the specific handlers since they will get destructed.	
Parameter	
N/A	N/A
Returns	
void	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-185 amsr::diag::SovdAuthorization::StopOffer

4.19.6 amsr::diag::SovdAuthorization::ValidateAuthorization

Prototype				

`auto amsr::diag::SovdAuthorization::ValidateAuthorization (`
`  ara::core::StringView token,`
`  amsr::diag::SovdTokenAuthorization& sovd_token_authorization,`
`  const ara::diag::MetaInfo& meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`) noexcept->ara::core::Future<void>`				

Description				
This function validates the authorization token sent by the client				
Parameter				
token	The authorization token sent by the SOVD client in the HTTP Authorization header.			
sovd_token_authorization	The SOVD tokens authorization to validate the token. Note: the reference to the statements is only valid until this request is completed, that is the promise is set (error/value).			
meta_info	The meta information associated to this request			
cancellation_handler	Set if the current service operation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.			
Returns				
ara::core::Future<void>	A future, which either is of type void or containing an appropriate ErrorCode			
Errors				
Error while handling the request				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-186 amsr::diag::SovdAuthorization::ValidateAuthorization

4.19.7 amsr::diag::SovdAuthorization::operator=

Prototype

`auto amsr::diag::SovdAuthorization::operator= (`
`  amsr::diag::SovdAuthorization&& other`
`) & noexcept`
`->amsr::diag::SovdAuthorization&`

Description
Move assigns an instance of SovdAuthorization
Parameter

other		The other object		
Returns				
amsr::diag::SovdAuthorization&		Reference to self		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-187 amsr::diag::SovdAuthorization::operator=

based on template version 1.00.00

4.20 amsr::diag::SovdTokenAuthorization

Description	
The SOVD token authorization	
Include	**Generalize**
amsr/diag/sovd_token_authorization.h	
MemberFunctions	
Authorize	Requests the token authorization

Table 4-188 amsr::diag::SovdTokenAuthorization

4.20.1 amsr::diag::SovdTokenAuthorization::Authorize

Prototype

```
auto amsr::diag::SovdTokenAuthorization::Authorize (
   ara::core::String client_id,
   ara::core::Vector<ara::core::String> user_roles,
   std::chrono::system_clock::time_point expiry_time_point) noexcept
->ara::core::Result<amsr::diag::SovdTokenAuthorizationHandle>
```

Description

Requests the token authorization

Parameter	
client_id	The client ID
user_roles	The user roles to be applied for the authorized client
expiry_time_point	The time point when the client's authorization is invalid. Must be aligned to time since epoch. Must be in range]0, std::chrono::nanoseconds::max()[.
Returns	
ara::core::Result<amsr::diag::Sovd TokenAuthorizationHandle>	The authorization token handle if operation succeeds, otherwise error code that shall not be evaluated

Errors

Do not evaluate the error code just that the result contains an error

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-189 amsr::diag::SovdTokenAuthorization::Authorize

4.21 amsr::diag::SovdTokenAuthorizationHandle

Description	
The handle to an authorized SOVD token	
Include	**Generalize**
amsr/diag/sovd_token_authorization_handle.h	
Constructors	
SovdTokenAuthorizationHandle	Move constructs an instance of this class
Destructor	
~SovdTokenAuthorizationHandle	Destructs an instance of SovdAuthorization
Operators	
operator=	Move assigns an instance of this class
MemberFunctions	
Revoke	Revokes the token which is associated to this handle
SetNotifier	Registers a (single) notifier to be invoked on token deletion (i.e. token is revoked or timed out) If a notifier is set already, this method will replace the previous one. If the notifier is invoked, the associated token is invalid.

Table 4-190 amsr::diag::SovdTokenAuthorizationHandle

4.21.1 amsr::diag::SovdTokenAuthorizationHandle::~SovdTokenAuthorizationHandle

Prototype				
`amsr::diag::SovdTokenAuthorizationHandle::~SovdTokenAuthorizationHandle() noexcept`				
Description				
Destructs an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
Must be destroyed before SovdAuthorization				

Table 4-191 amsr::diag::SovdTokenAuthorizationHandle::~SovdTokenAuthorizationHandle

4.21.2 amsr::diag::SovdTokenAuthorizationHandle::Revoke

Prototype
`auto amsr::diag::SovdTokenAuthorizationHandle::Revoke() noexcept->ara::core::Result<void>`
Description
Revokes the token which is associated to this handle

Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	void on successful revoked authorization, otherwise error code that shall not be evaluated
Errors	
Failed to forward the authorization token revoke to the diagnostic manager	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-192 amsr::diag::SovdTokenAuthorizationHandle::Revoke

4.21.3 amsr::diag::SovdTokenAuthorizationHandle::SetNotifier

Prototype

```
auto amsr::diag::SovdTokenAuthorizationHandle::SetNotifier (
   std::function<void()> notifier) noexcept
->ara::core::Result<void>
```

Description

Registers a (single) notifier to be invoked on token deletion (i.e. token is revoked or timed out) If a notifier is set already, this method will replace the previous one. If the notifier is invoked, the associated token is invalid.

Parameter	
notifier	The notifier to be called on token deletion
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function or it was invoked (note: single notifier).

Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Table 4-193 amsr::diag::SovdTokenAuthorizationHandle::SetNotifier

4.21.4 amsr::diag::SovdTokenAuthorizationHandle::SovdTokenAuthorizationHandle

Prototype				

`amsr::diag::SovdTokenAuthorizationHandle::SovdTokenAuthorizationHandle(`
`  amsr::diag::SovdTokenAuthorizationHandle&&`
`) noexcept`				

Description				
Move constructs an instance of this class				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-194 amsr::diag::SovdTokenAuthorizationHandle::SovdTokenAuthorizationHandle

4.21.5 amsr::diag::SovdTokenAuthorizationHandle::operator=

Prototype				

`auto amsr::diag::SovdTokenAuthorizationHandle::operator= (`
`  amsr::diag::SovdTokenAuthorizationHandle&&`
`) & noexcept->amsr::diag::SovdTokenAuthorizationHandle&`				

Description				
Move assigns an instance of this class				
Parameter				
Returns				
amsr::diag::SovdTokenAuthorization Handle&	Reference to this SovdTokenAuthorizationHandle			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 4-195 amsr::diag::SovdTokenAuthorizationHandle::operator=

based on template version 1.00.00

4.22 amsr::diag::SovdLock

Description	
SovdLock	
Include	**Generalize**
amsr/diag/sovd_lock.h	
Constructors	
SovdLock	Constructor of SovdLock port
SovdLock	Move constructs an instance of SovdLock
Operators	
operator=	Move assigns an instance of SovdLock
Destructor	
~SovdLock	Destructs an instance of SovdLock
MemberFunctions	
GetAllLocks	Gets a list of all locks held by sovd clients
GetExpirationTime	Gets the expiration time for a specific lock
SetSovdLockStateHandler	Register a handler function which is called if a lock state is changed

Table 4-196 amsr::diag::SovdLock

4.22.1 amsr::diag::SovdLock::~SovdLock

Prototype				
`amsr::diag::SovdLock::~SovdLock() noexcept`				
Description				
Destructor of SovdLock				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-197 amsr::diag::SovdLock::~SovdLock

4.22.2 amsr::diag::SovdLock::GetAllLocks

Prototype
`auto amsr::diag::SovdLock::GetAllLocks() noexcept -> ara::core::Result<ara::core::Vector<amsr::diag::state_types::LockDescription>>`
Description
Gets a list of all locks held by sovd clients

Parameter	
N/A	N/A
Returns	
ara::core::Result<ara::core::Vector<amsr::diag::state_types::LockDescription»	Result containing a vector of LockDescription if successful, error otherwise
Errors	
Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				

Table 4-198 amsr::diag::SovdLock::GetAllLocks

4.22.3 amsr::diag::SovdLock::GetExpirationTime

Prototype

```
auto amsr::diag::SovdLock::GetExpirationTime(
  ara::core::String lock_id
) noexcept -> ara::core::Result<std::chrono::system_clock::time_point>
```

Description	
Gets the expiration time for a specific lock	
Parameter	
lock_id	The lock id to request the expiration time for
Returns	
ara::core::Result <std::chrono::system_clock::time_point>	Result containing the expiration timepoint of the respective lock if successful, error otherwise
Errors	
Do not evaluate the error code just that the result contains an error, potentially the lock_id was unknown	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				

Table 4-199 amsr::diag::SovdLock::GetExpirationTime

4.22.4 amsr::diag::SovdLock::SetSovdLockStateHandler

Prototype

```
void amsr::diag::SovdLock::SetSovdLockStateHandler(
  std::function<void(state_types::LockDescription, state_types::StateType)>
handler) noexcept
```

Description				
Register a handler function which is called if a lock state is changed If successful, a previously set handler gets overwritten.				
Parameter				
handler	The handler function			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this lock has completed or until it is replaced by a new handler function. Whenever a previously set handler is in execution, the SetLockStateHandler method is blocking until the execution of that handler is completed. Consider this for the implementation of the handlers (deadlock hazard).				

Table 4-200 amsr::diag::SovdLock::SetSovdLockStateHandler

4.22.5 amsr::diag::SovdLock::SovdLock

Prototype				
`void amsr::diag::SovdLock::SovdLock(amsr::diag::SovdLock&& other) noexcept`				
Description				
Move constructs an instance of SovdLock				
Parameter				
other	The other object			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-201 amsr::diag::SovdLock::SovdLock

4.22.6 amsr::diag::SovdLock::SovdLock

Prototype				
`void amsr::diag::SovdLock::SovdLock(const ara::core::InstanceSpecifier& specifier)`				
Description				
Constructor of sovd lock port				
Parameter				
specifier	InstanceSpecifier to a PortPrototype of an SovdLock			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, for different this pointer	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-202 amsr::diag::SovdLock::SovdLock

4.22.7 amsr::diag::SovdLock::operator=

Prototype				

`auto amsr::diag::SovdLock::operator=(`
`  amsr::diag::SovdLock&& other`
`) & noexcept -> amsr::diag::SovdLock&`				

Description				
Move assigns an instance of SovdLock				
Parameter				
other	The other object			
Returns				
amsr::diag::SovdLock&	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-203 amsr::diag::SovdLock::operator=

4.23 ara::diag::Authentication

Description	
Authentication interface	
Include	**Generalize**
ara/diag/authentication.h	
Constructors	
Authentication	Move constructs an instance of Authentication
Authentication	Constructor for Authentication
Operators	
operator=	Move assigns an instance of Authentication
MemberFunctions	
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM
TransmitCertificate	Accepts the certificate received from the tester, verifies it, and takes necessary actions in the application
VerifyCertificateBidirectional	Verifies certificate received from the tester and creates a challenge, Ephemeral Public Key and Proof Of Ownership as well as the server certificate that will be used by the tester to verify the Proof Of Ownership
VerifyCertificateUnidirectional	Verifies certificate received from the tester and creates a challenge and Ephemeral Public Key
VerifyOwnership	Verifies Proof Of Ownership against the server challenge

Table 4-204 ara::diag::Authentication

4.23.1 ara::diag::Authentication::Authentication

Prototype

```
ara::diag::Authentication::Authentication (
  ara::diag::Authentication && other
) noexcept
```

Description

Move constructs an instance of Authentication

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
The port passed as parameter is not offered

Table 4-205 ara::diag::Authentication::Authentication

4.23.2 ara::diag::Authentication::Authentication

Prototype	
`ara::diag::Authentication::Authentication ( const ara::core::InstanceSpecifier & specifier, enum ara::diag::ReentrancyType reentrancy_type )`	
Description	
Constructor for Authentication	
Parameter	
specifier	InstanceSpecifier to an PortPrototype of an AuthenticationInterface
reentrancy_type	Parameter is currently ignored and instead the parameter "numberOfParallelRequestExecutions" specifies if interface must be callable from multiple threads (must be threadsafe if numberOfParallelRequestExecutions > 1)
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints
A port with that specifier is configured and no other port is right now instantiated with it

Table 4-206 ara::diag::Authentication::Authentication

4.23.3 ara::diag::Authentication::~Authentication

Prototype
`ara::diag::Authentication::~Authentication () noexcept`
Description
Destructor of class Authentication
Errors
N/A
Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-207 ara::diag::Authentication::~Authentication

4.23.4 ara::diag::Authentication::operator=

Prototype				

`ara::diag::Authentication &`
`ara::diag::Authentication::operator= (`
`  ara::diag::Authentication && other`
`) & noexcept`				

Description				
Move assigns an instance of Authentication				
Parameter				
other	The other object			
Returns				
ara::diag::Authentication &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-208 ara::diag::Authentication::operator=

4.23.5 ara::diag::Authentication::Offer

Prototype				
`ara::core::Result<void>` `ara::diag::Authentication::Offer () noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<void>	Positive result if service was offered, error if offer failed			
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 4-209 ara::diag::Authentication::Offer

4.23.6 ara::diag::Authentication::StopOffer

Prototype				
`void` `ara::diag::Authentication::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A	N/A			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-210 ara::diag::Authentication::StopOffer

4.23.7 ara::diag::Authentication::TransmitCertificate

Prototype

`ara::core::Future<void>`
`ara::diag::Authentication::TransmitCertificate (`
`  ara::core::Span<std::uint8_t> certificate_data,`
`  const ara::diag::MetaInfo & meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`

Description
Accepts the certificate received from the tester, verifies it, and takes necessary actions in the application

Parameter	
certificate_data	Certificate to be verified, transmitted by the tester
meta_info	MetaInfo of the request
cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<void>	void in case of no error
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-211 ara::diag::Authentication::TransmitCertificate

4.23.8 ara::diag::Authentication::VerifyCertificateBidirectional

Prototype

```
ara::core::Future<std::tuple<ara::core::Span<std::uint8_t>, ara::core::
Span<std::uint8_t>, ara::core::Span<std::uint8_t>, ara::core::Span<std::uint8_t> >
>
ara::diag::Authentication::VerifyCertificateBidirectional (
   std::uint8_t communication_configuration,
   ara::core::Span<std::uint8_t> client_certificate,
   ara::core::Span<std::uint8_t> client_challenge,
   const ara::diag::MetaInfo & meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description

Verifies certificate received from the tester and creates a challenge, Ephemeral Public Key and Proof Of Ownership as well as the server certificate that will be used by the tester to verify the Proof Of Ownership

Parameter

communication_configuration	Information about how to proceed with security in further diagnostic communication after the Authentication
client_certificate	Certificate that is received from the tester during Bidirectional Authentication
client_challenge	Challenge received fom the tester
meta_info	MetaInfo of the request
cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core:: Future<std::tuple<ara::core:: Span<std::uint8_t>, ara::core:: Span<std::uint8_t>, ara::core:: Span<std::uint8_t>, ara::core:: Span<std::uint8_t> > >	Tuple of: 1. challenge created by the application; 2. Certificate of the Server; 3. Proof Of Ownership; 4. Ephemeral Public Key
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-212 ara::diag::Authentication::VerifyCertificateBidirectional

4.23.9 ara::diag::Authentication::VerifyCertificateUnidirectional

Prototype

```
ara::core::Future<std::tuple<ara::core::Span<std::uint8_t>, ara::core::
Span<std::uint8_t> > >
ara::diag::Authentication::VerifyCertificateUnidirectional (
  std::uint8_t communication_configuration,
  ara::core::Span<std::uint8_t> client_certificate,
  ara::core::Span<std::uint8_t> client_challenge,
  const class ara::diag::MetaInfo & meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Verifies certificate received from the tester and creates a challenge and Ephemeral Public Key

Parameter

communication_configuration	Information about how to proceed with security in further diagnostic communication after the Authentication
client_certificate	certificate that is received from the tester during Unidirectional Authentication
client_challenge	Challenge received fom the tester during Unidirectional Authentication
meta_info	MetaInfo of the request
cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<std::tuple<ara::core::Span<std::uint8_t>, ara::core::Span<std::uint8_t> > >	Tuple of challenge created by the application and Ephemeral Public Key of Server

Errors
Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints
N/A

Table 4-213 ara::diag::Authentication::VerifyCertificateUnidirectional

4.23.10 ara::diag::Authentication::VerifyOwnership

Prototype

```
ara::core::Future<ara::core::Span<std::uint8_t> >
ara::diag::Authentication::VerifyOwnership (
   ara::core::Span<std::uint8_t> client_pown,
   ara::core::Span<std::uint8_t> client_ephemeral_public_key,
   const ara::diag::MetaInfo & meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description

Verifies Proof Of Ownership against the server challenge

Parameter

client_pown	The Proof Of Ownership provided by the Tester to the previously exchanged Server Challenge
client_ephemeral_public_key	The Ephemeral public key generated by the client for Diffie-Hellman key agreement.
meta_info	MetaInfo of the request
cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Detailed Description

Accepts the received Proof Of Ownership and verifies it with the Public Key of the certificate received in the VerifyCertificateUnidirectional / VerifyCertificateBidirectional against the server challenge created in the last call to VerifyCertificateUnidirectional / VerifyCertificateBidirectional

Returns	
ara::core::Future<ara::core::Span<std::uint8_t> >	Session Key Info
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-214 ara::diag::Authentication::VerifyOwnership

based on template version 1.00.00

4.24 ara::diag::CancellationHandler

Description	
CancellationHandler contains a shared state if the processing should be canceled	
Include	**Generalize**
ara/diag/cancellation_handler.h	
Detailed Description	
This object shall not be stored or accessed beyond the lifetime of the operation at which this object was received.	
Constructors	
CancellationHandler	Move constructs instance of class
Operators	
operator=	Move assigns instance of class
MemberFunctions	
IsCanceled	Returns true in if the diagnostic service execution is canceled in DM
SetNotifier	Registering a notifier function which is called if the diagnostic service execution is canceled in DM

Table 4-215 ara::diag::CancellationHandler

4.24.1 ara::diag::CancellationHandler::CancellationHandler

Prototype				
`ara::diag::CancellationHandler::CancellationHandler ( ara::diag::CancellationHandler && ) noexcept`				
Description				
Move constructs instance of class				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	**Call Context**	**Reentrant**	**ThreadSafe**	**Can Throw**
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-216 ara::diag::CancellationHandler::CancellationHandler

4.24.2 ara::diag::CancellationHandler::operator=

Prototype				

`ara::diag::CancellationHandler &`
`ara::diag::CancellationHandler::operator= (`
`  ara::diag::CancellationHandler &&`
`) & noexcept`				

Description				
Move assigns instance of class				
Parameter				
Returns				
ara::diag::CancellationHandler &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-217 ara::diag::CancellationHandler::operator=

4.24.3 ara::diag::CancellationHandler::IsCanceled

Prototype				
`bool` `ara::diag::CancellationHandler::IsCanceled () const`				
Description				
Returns true in if the diagnostic service execution is canceled in DM				
Parameter				
N/A	N/A			
Returns				
bool	True if service execution is canceled, false otherwise			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				

N/A
Steady
TRUE

Table 4-218 ara::diag::CancellationHandler::IsCanceled

4.24.4 ara::diag::CancellationHandler::SetNotifier

Prototype	
`void ara::diag::CancellationHandler::SetNotifier ( std::function<void (void)> notifier )`	
Description	
Registering a notifier function which is called if the diagnostic service execution is canceled in DM	
Parameter	
notifier	Notification function that is called upon cancellation
Detailed Description	
A previously set notifier gets overwritten. The notifier will be invoked immediately if processing is already cancelled.	
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE

Constraints
The passed in function must not throw any exceptions.
The passed in function must be valid to be called until the destruction of this CancellationHandler has completed or until it is replaced by a new notifier function.
Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).
Steady
TRUE

Table 4-219 ara::diag::CancellationHandler::SetNotifier

based on template version 1.00.00

4.25 ara::diag::ClientAuthenticationHandle

Description	
The diagnostic client specific authentication management interface	
Include	**Generalize**
ara/diag/client_authentication_handle.h	
Constructors	
ClientAuthenticationHandle	The move constructor
Operators	
operator=	The move assignment
MemberFunctions	
Append	Extends the dynamic access list of services
Refresh	Restarts the timeout
Revoke	De-authenticates the client, resp. revokes overridden defaults
Set	Replaces an existing the dynamic access list of services

Table 4-220 ara::diag::ClientAuthenticationHandle

4.25.1 ara::diag::ClientAuthenticationHandle::ClientAuthenticationHandle

Prototype

```
ara::diag::ClientAuthenticationHandle::ClientAuthenticationHandle (
  ara::diag::ClientAuthenticationHandle &&
) noexcept
```

Description

The move constructor

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-221 ara::diag::ClientAuthenticationHandle::ClientAuthenticationHandle

4.25.2 ara::diag::ClientAuthenticationHandle::~ClientAuthenticationHandle

Prototype

```
ara::diag::ClientAuthenticationHandle::~ClientAuthenticationHandle () noexcept
```

Description

Destructs the client specific authentication management				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
Must be destroyed before ExternalAuthentication Must be destroyed before ClientAuthentication				

Table 4-222 ara::diag::ClientAuthenticationHandle::~ClientAuthenticationHandle

4.25.3 ara::diag::ClientAuthenticationHandle::operator=

Prototype	

`ara::diag::ClientAuthenticationHandle &`
`ara::diag::ClientAuthenticationHandle::operator= (`
`  ara::diag::ClientAuthenticationHandle &&`
`) & noexcept`	

Description	
The move assignment	
Parameter	
Returns	
ara::diag::ClientAuthentication Handle &	Reference to this ClientAuthenticationHandle
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-223 ara::diag::ClientAuthenticationHandle::operator=

4.25.4 ara::diag::ClientAuthenticationHandle::Append

Prototype

`ara::core::Result<void>`
`ara::diag::ClientAuthenticationHandle::Append (`
`  ara::diag::DiagnosticServiceDynamicAccessList dynamic_access_list`
`) noexcept`

Description				
Extends the dynamic access list of services				
Parameter				
dynamic_access_list	The dynamic access list of additional services to enable append			
Returns				
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-224 ara::diag::ClientAuthenticationHandle::Append

4.25.5 ara::diag::ClientAuthenticationHandle::Refresh

Prototype				
`ara::core::Result<void>` `ara::diag::ClientAuthenticationHandle::Refresh () noexcept`				
Description				
Restarts the timeout				
Parameter				
N/A	N/A			
Detailed Description				
Both authentication or override timeouts are affected, depending on what issued this handle				
Returns				
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-225 ara::diag::ClientAuthenticationHandle::Refresh

4.25.6 ara::diag::ClientAuthenticationHandle::Revoke

Prototype				
`ara::core::Result<void>` `ara::diag::ClientAuthenticationHandle::Revoke () noexcept`				
Description				
De-authenticates the client, resp. revokes overridden defaults				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-226 ara::diag::ClientAuthenticationHandle::Revoke

4.25.7 ara::diag::ClientAuthenticationHandle::Set

Prototype				

`ara::core::Result<void>`
`ara::diag::ClientAuthenticationHandle::Set (`
`  ara::diag::DiagnosticServiceDynamicAccessList dynamic_access_list`
`) noexcept`				

Description				
Replaces an existing the dynamic access list of services				
Parameter				
dynamic_access_list	The dynamic access list of additional services to enable			
Returns				
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-227 ara::diag::ClientAuthenticationHandle::Set

based on template version 1.00.00

4.26 ara::diag::ClientAuthentication

Description	
The diagnostic client related authentication instance	
Include	**Generalize**
ara/diag/client_authentication.h	
Constructors	
ClientAuthentication	Move constructor
Operators	
operator=	Move Assignment
Types	
DiagnosticAuthState	The authentication state of the client
DiagnosticAuthRole	The authentication role of the client
MemberFunctions	
Authenticate	Authenticates the client for specific role(s)
GetState	Reports the current authentication state of the diagnostic client
OverrideDefaultRoles	Temporarily overrides the default roles without authenticating the client
SetNotifier	Registers a single notifier on authentication state change for the diagnostic client

Table 4-228 ara::diag::ClientAuthentication

4.26.1 ara::diag::ClientAuthentication::ClientAuthentication

Prototype

```
ara::diag::ClientAuthentication::ClientAuthentication (
  ara::diag::ClientAuthentication &&
) noexcept
```

Description

Move constructor

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-229 ara::diag::ClientAuthentication::ClientAuthentication

4.26.2 ara::diag::ClientAuthentication::~ClientAuthentication

Prototype				
`ara::diag::ClientAuthentication::~ClientAuthentication () noexcept`				
Description				
Destructs the client related authentication				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
Must be destroyed before ExternalAuthentication				

Table 4-230 ara::diag::ClientAuthentication::~ClientAuthentication

4.26.3 ara::diag::ClientAuthentication::operator=

Prototype	

`ara::diag::ClientAuthentication &`
`ara::diag::ClientAuthentication::operator= (`
`  ara::diag::ClientAuthentication &&`
`) & noexcept`	

Description	
Move Assignment	
Parameter	
Returns	
ara::diag::ClientAuthentication &	Reference to this ClientAuthentication
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-231 ara::diag::ClientAuthentication::operator=

4.26.4 ara::diag::ClientAuthentication::DiagnosticAuthState

Prototype
`ara::diag::ClientAuthentication::DiagnosticAuthState`
Description

The authentication state of the client	
Type	
std::uint8_t	
Enumerations	
kDeAuthenticated	0x00
kAuthenticated	0x01

Table 4-232 ara::diag::ClientAuthentication::DiagnosticAuthState

4.26.5 ara::diag::ClientAuthentication::DiagnosticAuthRole

Prototype
`ara::diag::ClientAuthentication::DiagnosticAuthRole`
Description
The authentication role of the client
Type
ara::core::String

Table 4-233 ara::diag::ClientAuthentication::DiagnosticAuthRole

4.26.6 ara::diag::ClientAuthentication::Authenticate

Prototype				

`ara::core::Result<ClientAuthenticationHandle>`
`ara::diag::ClientAuthentication::Authenticate (`
`  ara::core::Vector<DiagnosticAuthRole> user_roles`
`)`				

Description				
Authenticates the client for specific role(s)				
Parameter				
user_roles	The user roles to set on the diagnostic client			
Returns				
ara::core::Result<Client AuthenticationHandle>	Result contains "client authentication handle" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-234 ara::diag::ClientAuthentication::Authenticate

4.26.7 ara::diag::ClientAuthentication::GetState

Prototype				
`ara::core::Result<DiagnosticAuthState>` `ara::diag::ClientAuthentication::GetState () const noexcept`				
Description				
Reports the current authentication state of the diagnostic client				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<DiagnosticAuth State>	Result contains "current authentication state" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-235 ara::diag::ClientAuthentication::GetState

4.26.8 ara::diag::ClientAuthentication::OverrideDefaultRoles

Prototype	

`ara::core::Result<ClientAuthenticationHandle>`
`ara::diag::ClientAuthentication::OverrideDefaultRoles (`
`  ara::core::Vector<DiagnosticAuthRole> default_roles,`
`  std::chrono::milliseconds timeout`
`)`	

Description	
Temporarily overrides the default roles without authenticating the client	
Parameter	
default_roles	The default roles to set on the diagnostic client
timeout	The timeout the override is active
Detailed Description	
To extend the timeout the Refresh() method. To prematurely revoke the override, call the Revoke() method.	
Returns	
ara::core::Result<Client AuthenticationHandle>	Result contains "client authentication handle" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-236 ara::diag::ClientAuthentication::OverrideDefaultRoles

4.26.9 ara::diag::ClientAuthentication::SetNotifier

Prototype	

`ara::core::Result<void>`
`ara::diag::ClientAuthentication::SetNotifier (`
`  std::function<void (DiagnosticAuthState)> notifier`
`) noexcept`	

Description	
Registers a single notifier on authentication state change for the diagnostic client	
Parameter	
notifier	The notifier to call on state transition
Detailed Description	
If another ClientAuthentication object representing the same client authentication has already set the notifier, calling this method will replace the previous one	
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Table 4-237 ara::diag::ClientAuthentication::SetNotifier

4.27 ara::diag::Condition

Description	
DiagnosticConditionInterface	
Include	**Generalize**
ara/diag/condition.h	
Constructors	
Condition	Move constructs instance of class
Condition	Class for an Condition
Operators	
operator=	Move assigns instance of class
Types	
ConditionType	Type for Condition status
MemberFunctions	
GetCondition	Get current condition
SetCondition	Set condition
SetNotifier	Register a notifier function which is called if the condition is changed

Table 4-238 ara::diag::Condition

4.27.1 ara::diag::Condition::Condition

Prototype

```
ara::diag::Condition::Condition (
   ara::diag::Condition &&
) noexcept
```

Description

Move constructs instance of class

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-239 ara::diag::Condition::Condition

4.27.2 ara::diag::Condition::Condition

Prototype

```
ara::diag::Condition::Condition (
   const ara::core::InstanceSpecifier & specifier
)
```

Description
Class for an Condition

Parameter	
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticConditionInterface

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-240 ara::diag::Condition::Condition

4.27.3 ara::diag::Condition::~Condition

Prototype
`ara::diag::Condition::~Condition () noexcept`

Description
Destructor of class Condition

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-241 ara::diag::Condition::~Condition

4.27.4 ara::diag::Condition::operator=

Prototype

```
ara::diag::Condition &
ara::diag::Condition::operator= (
   ara::diag::Condition &&
) & noexcept
```

Description
Move assigns instance of class

Parameter	
Returns	
ara::diag::Condition &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-242 ara::diag::Condition::operator=

4.27.5 ara::diag::Condition::ConditionType

Prototype	
`ara::diag::Condition::ConditionType`	
Description	
Type for Condition status	
Type	
std::uint8_t	
Enumerations	
kConditionFalse	0x00
kConditionTrue	0x01

Table 4-243 ara::diag::Condition::ConditionType

4.27.6 ara::diag::Condition::GetCondition

Prototype	
`ara::core::Result<ConditionType>` `ara::diag::Condition::GetCondition ()`	
Description	
Get current condition	
Parameter	
N/A	N/A
Returns	
ara::core::Result<ConditionType>	Result contains the "current condition" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-244 ara::diag::Condition::GetCondition

4.27.7 ara::diag::Condition::SetCondition

Prototype	

`ara::core::Result<void>`
`ara::diag::Condition::SetCondition (`
`  ara::diag::Condition::ConditionType condition`
`)`	

Description	
Set condition	
Parameter	
condition	The current condition
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-245 ara::diag::Condition::SetCondition

4.27.8 ara::diag::Condition::SetNotifier

Prototype

`ara::core::Result<void>`
`ara::diag::Condition::SetNotifier (`
`  std::function<void (ConditionType)> notifier`
`)`

Description	
Register a notifier function which is called if the condition is changed	
Parameter	
notifier	The notifier function
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code
Errors	
ara::diag::DiagReportingErrc::kGenericError	The instance specifier has not been registered

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this Condition has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				
Steady				
TRUE				

Table 4-246 ara::diag::Condition::SetNotifier

based on template version 1.00.00

4.28 ara::diag::Conversation

Description	
Conversation interface	
Include	**Generalize**
ara/diag/conversation.h	
Types	
ActivityStatusType	Type for current activity status
ConversationIdentifierType	Properties allowing an identification of the conversation
MemberFunctions	
GetActivityStatus	Represents the status of an active conversation
GetAllConversations	Get all possible conversations
GetConversation	Get one conversation based on given MetaInfo
GetConversationIdentifier	Getter for the current identification properties of the active conversation
GetCurrentActiveConversations	Get all currently active conversations
GetDiagnosticSecurityLevel	Represents the current active diagnostic security level of an active conversation
GetDiagnosticSession	Represents the current active diagnostic session of an active conversation
ResetToDefaultSession	Method to reset the current session to default session
SetActivityNotifier	Register a notifier function which is called if the activity is changed
SetDiagnosticSessionNotifier	Register a notifier function which is called if the Session is changed
SetSecurityLevelNotifier	Register a notifier function which is called if the SecurityLevel is changed

Table 4-247 ara::diag::Conversation

4.28.1 ara::diag::Conversation::ActivityStatusType

Prototype	
`ara::diag::Conversation::ActivityStatusType`	
Description	
Type for current activity status	
Type	
std::uint16_t	
Enumerations	
kActive	0x00
kInactive	0x01

Table 4-248 ara::diag::Conversation::ActivityStatusType

4.28.2 ara::diag::Conversation::ConversationIdentifierType

Prototype		
`struct ara::diag::Conversation::ConversationIdentifierType`		
Description		
Properties allowing an identification of the conversation		
Members		
ara::core::StringView	diagnosticProtocolKind	The diagnostic protocol kind
std::uint16_t	diagnosticServerAddress	The diagnostic server address
std::uint16_t	testerAddress	The tester address

Table 4-249 ara::diag::Conversation::ConversationIdentifierType

4.28.3 ara::diag::Conversation::GetActivityStatus

Prototype				
`ara::core::Result<ActivityStatusType>` `ara::diag::Conversation::GetActivityStatus ()`				
Description				
Represents the status of an active conversation				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<ActivityStatus Type>	Result contains "current activity status" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-250 ara::diag::Conversation::GetActivityStatus

4.28.4 ara::diag::Conversation::GetAllConversations

Prototype
`ara::core::Vector<std::reference_wrapper<Conversation> >` `ara::diag::Conversation::GetAllConversations ()`
Description
Get all possible conversations

Parameter	
N/A	N/A
Returns	
ara::core:: Vector<std::reference_wrapper< Conversation> >	a vector of all possible Conversation objects
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-251 ara::diag::Conversation::GetAllConversations

4.28.5 ara::diag::Conversation::GetConversation

Prototype

`ara::core::Result<Conversation &>`
`ara::diag::Conversation::GetConversation (`
`  const ara::diag::MetaInfo & meta_info`
`)`

Description
Get one conversation based on given MetaInfo

Parameter	
meta_info	The meta information
Returns	
ara::core::Result<Conversation &>	Result contains "conversation object" if operation succeeds, otherwise error code
Errors	
ara::diag::DiagReportingErrc::kInvalidArgument The client authentication object was not found	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-252 ara::diag::Conversation::GetConversation

4.28.6 ara::diag::Conversation::GetConversationIdentifier

Prototype				
`ara::core::Result<ConversationIdentifierType>` `ara::diag::Conversation::GetConversationIdentifier ()`				
Description				
Getter for the current identification properties of the active conversation				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<Conversation IdentifierType>	Result contains "current conversation identifier" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-253 ara::diag::Conversation::GetConversationIdentifier

4.28.7 ara::diag::Conversation::GetCurrentActiveConversations

Prototype	
`ara::core::Vector<std::reference_wrapper<Conversation> >` `ara::diag::Conversation::GetCurrentActiveConversations ()`	
Description	
Get all currently active conversations	
Parameter	
N/A	N/A
Returns	
ara::core::Vector<std::reference_wrapper<Conversation> >	a vector of all currently active (GetActivityStatus() == kActive) Conversation objects
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-254 ara::diag::Conversation::GetCurrentActiveConversations

4.28.8 ara::diag::Conversation::GetDiagnosticSecurityLevel

Prototype				
`ara::core::Result<ara::core::StringView>` `ara::diag::Conversation::GetDiagnosticSecurityLevel ()`				
Description				
Represents the current active diagnostic security level of an active conversation				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<ara::core::String View>		Result contains "current security level" if operation succeeds, otherwise error code that shall not be evaluated		
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-255 ara::diag::Conversation::GetDiagnosticSecurityLevel

4.28.9 ara::diag::Conversation::GetDiagnosticSession

Prototype
`ara::core::Result<ara::core::StringView>` `ara::diag::Conversation::GetDiagnosticSession ()`
Description
Represents the current active diagnostic session of an active conversation
Parameter

N/A		N/A		
Returns				
ara::core::Result<ara::core::String View>		Result contains "current session" if operation succeeds, otherwise error code that shall not be evaluated		
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-256 ara::diag::Conversation::GetDiagnosticSession

4.28.10 ara::diag::Conversation::ResetToDefaultSession

Prototype				
`void ara::diag::Conversation::ResetToDefaultSession ()`				
Description				
Method to reset the current session to default session				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-257 ara::diag::Conversation::ResetToDefaultSession

4.28.11 ara::diag::Conversation::SetActivityNotifier

Prototype

```
ara::core::Result<void>
ara::diag::Conversation::SetActivityNotifier (
   std::function<void (ActivityStatusType)> activity_status_notifier
)
```

Description
Register a notifier function which is called if the activity is changed

Parameter	
activity_status_notifier	The notifier function

Detailed Description
A previously set notifier gets overwritten.

Returns	
ara::core::Result<void>	Always a result containing "void"

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints
The passed in function must not throw any exceptions.
The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function.
Whenever a previously set notifier is in execution, the SetActivityNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Steady
TRUE

Table 4-258 ara::diag::Conversation::SetActivityNotifier

4.28.12 ara::diag::Conversation::SetDiagnosticSessionNotifier

Prototype

```
ara::core::Result<void>
ara::diag::Conversation::SetDiagnosticSessionNotifier (
   std::function<void (ara::core::StringView)> session_notifier
)
```

Description
Register a notifier function which is called if the Session is changed

Parameter	
session_notifier	The notifier function

Detailed Description
A previously set notifier gets overwritten.

Returns	
ara::core::Result<void>	Always a result containing "void"
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function.

Whenever a previously set notifier is in execution, the SetDiagnosticSessionNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Steady

TRUE

Table 4-259 ara::diag::Conversation::SetDiagnosticSessionNotifier

4.28.13 ara::diag::Conversation::SetSecurityLevelNotifier

Prototype

```
ara::core::Result<void>
ara::diag::Conversation::SetSecurityLevelNotifier (
   std::function<void (ara::core::StringView)> security_level_notifier
)
```

Description

Register a notifier function which is called if the SecurityLevel is changed

Parameter	
security_level_notifier	The notifier function

Detailed Description

A previously set notifier gets overwritten.

Returns	
ara::core::Result<void>	Always a result containing "void"
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetSecurityLevelNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).
Steady
TRUE

Table 4-260 ara::diag::Conversation::SetSecurityLevelNotifier

based on template version 1.00.00

4.29 ara::diag::DiagUdsNrcErrorDomain

Description	
Specific error domain for all relevant UDS NRC errors	
Include	**Generalize**
ara/diag/diag_uds_nrc_error_domain.h	generalized by ErrorDomain
Detailed Description	
This class inherits the base error domain, which is the controlling entity for ErrorCode's error code values, and defines the mapping of such error code values to textual representations.	
Constructors	
DiagUdsNrcErrorDomain	Constructs instance of class

Table 4-261 ara::diag::DiagUdsNrcErrorDomain

4.29.1 ara::diag::DiagUdsNrcErrorDomain::DiagUdsNrcErrorDomain

Prototype				
`ara::diag::DiagUdsNrcErrorDomain::DiagUdsNrcErrorDomain () noexcept`				
Description				
Constructs instance of class				
Parameter				
N/A	N/A			
Detailed Description				
Initializes the diagnostic daemon error domain ID with DiagUdsNrcErrorDomain::kId				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-262 ara::diag::DiagUdsNrcErrorDomain::DiagUdsNrcErrorDomain

4.30 ara::diag::DiagUdsNrcErrc

Prototype
`ara::diag::DiagUdsNrcErrc`
Description
Specific error codes for the UDS NRC ara::diag error codes
Detailed Description
This enum does not contain values for the manufacturer specific NRC values 0xF0-0xFE. To report such NRCs the amsr::diag::AsUdsNrc method can be used.
Type
ara::core::ErrorDomain::CodeType
Enumerations

kBrakeSwitchNotClosed	0x8F
kBusyRepeatRequest	0x21
kConditionsNotCorrect	0x22
kEngineIsNotRunning	0x84
kEngineIsRunning	0x83
kEngineRunTimeTooLow	0x85
kExceedNumberOfAttempts	0x36
kSecureDataTransmissionRequired	0x38
kSecureDataTransmissionNotAllowed	0x39
kSecureDataVerificationFailed	0x3A
kFailurePreventsExecutionOfRequestedAction	0x26
kGeneralProgrammingFailure	0x72
kGeneralReject	0x10
kIncorrectMessageLengthOrInvalidFormat	0x13
kInvalidKey	0x35
kNoProcessingNoResponse	0xFF
kNoResponseFromSubnetComponent	0x25
kRequestOutOfRange	0x31
kRequestSequenceError	0x24
kRequiredTimeDelayNotExpired	0x37
kResponseTooLong	0x14
kRpmTooHigh	0x81
kRpmTooLow	0x82
kSecurityAccessDenied	0x33
kServiceNotSupported	0x11
kServiceNotSupportedInActiveSession	0x7F
kShifterLeverNotInPark	0x90

kSubFunctionNotSupportedInActiveSession	0x7E
kSubfunctionNotSupported	0x12
kTemperatureTooHigh	0x86
kTemperatureTooLow	0x87
kThrottlePedalTooHigh	0x8A
kThrottlePedalTooLow	0x8B
kTorqueConverterClutchLocked	0x91
kTransferDataSuspended	0x71
kTransmissionRangeNotInGear	0x8D
kTransmissionRangeNotInNeutral	0x8C
kUploadDownloadNotAccepted	0x70
kVehicleSpeedTooHigh	0x88
kVehicleSpeedTooLow	0x89
kVoltageTooHigh	0x92
kVoltageTooLow	0x93
kWrongBlockSequenceCounter	0x73
kCertificateVerificationFailedInvalidTimePeriod	0x50
kCertificateVerificationFailedInvalidSignature	0x51
kCertificateVerificationFailedInvalidChainOfTrust	0x52
kCertificateVerificationFailedInvalidType	0x53
kCertificateVerificationFailedInvalidFormat	0x54
kCertificateVerificationFailedInvalidContent	0x55
kCertificateVerificationFailedInvalidScope	0x56
kCertificateVerificationFailedInvalidCertificateRevoked	0x57
kOwnershipVerificationFailed	0x58
kChallengeCalculationFailed	0x59
kSettingAccessRightsFailed	0x5A
kSessionKeyCreationDerivationFailed	0x5B
kConfigurationDataUsageFailed	0x5C
kDeAuthenticationFailed	0x5D
kResourceTemporarilyNotAvailable	0x94

Table 4-263 ara::diag::DiagUdsNrcErrc

4.31 ara::diag::DiagnosticServiceDynamicAccessList

Description	
The diagnostic service dynamic access list to be used for additional clients access rights	
Include	**Generalize**
ara/diag/diagnostic_service_dynamic_access_list.h	
Constructors	
DiagnosticServiceDynamicAccessList	Constructs an instance of DiagnosticServiceDynamicAccessList
DiagnosticServiceDynamicAccessList	Move constructs an instance of DiagnosticServiceDynamicAccessList
DiagnosticServiceDynamicAccessList	Copy constructs an instance of DiagnosticServiceDynamicAccessList
Operators	
operator=	Move assigns an instance of DiagnosticServiceDynamicAccessList
operator=	Copy assigns an instance of DiagnosticServiceDynamicAccessList
MemberFunctions	
MakeServiceBuilder	Constructs a diagnostic service pattern builder for a specific ServiceIdentifier
MakeServiceBuilder	Constructs a diagnostic service pattern builder starting with a specific sequence of values
Reserve	Allocates memory for all service heads to fit into the dynamic access list

Table 4-264 ara::diag::DiagnosticServiceDynamicAccessList

4.31.1 ara::diag::DiagnosticServiceDynamicAccessList:: DiagnosticServiceDynamicAccessList

Prototype				
`ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList ( ) noexcept`				
Description				
Constructs an instance of DiagnosticServiceDynamicAccessList				
Parameter				
N/A		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	**Call Context**	**Reentrant**	**ThreadSafe**	**Can Throw**
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-265 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList

4.31.2 ara::diag::DiagnosticServiceDynamicAccessList:: DiagnosticServiceDynamicAccessList

Prototype				
`ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList ( ara::diag::DiagnosticServiceDynamicAccessList && other ) noexcept`				
Description				
Move constructs an instance of DiagnosticServiceDynamicAccessList				
Parameter				
other	Object to move-construct from			
Detailed Description				
A builder instance must always get destroyed before the dynamic access list that created it is moved				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-266 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList

4.31.3 ara::diag::DiagnosticServiceDynamicAccessList:: DiagnosticServiceDynamicAccessList

Prototype				
`ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList ( const class ara::diag::DiagnosticServiceDynamicAccessList & other ) noexcept`				
Description				
Copy constructs an instance of DiagnosticServiceDynamicAccessList				
Parameter				
other	Object to copy-construct from			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-267 ara::diag::DiagnosticServiceDynamicAccessList::DiagnosticServiceDynamicAccessList

4.31.4 ara::diag::DiagnosticServiceDynamicAccessList::~DiagnosticServiceDynamicAccessList

Prototype				
`ara::diag::DiagnosticServiceDynamicAccessList::~DiagnosticServiceDynamicAccessList () noexcept`				
Description				
Destructs an instance of DiagnosticServiceDynamicAccessList				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
All the builder instances created by the call of MakeServiceBuilder functions must be destroyed before calling this destructor.				

Table 4-268 ara::diag::DiagnosticServiceDynamicAccessList::~DiagnosticServiceDynamicAccessList

4.31.5 ara::diag::DiagnosticServiceDynamicAccessList::operator=

Prototype	
`ara::diag::DiagnosticServiceDynamicAccessList & ara::diag::DiagnosticServiceDynamicAccessList::operator= ( ara::diag::DiagnosticServiceDynamicAccessList && other ) & noexcept`	
Description	
Move assigns an instance of DiagnosticServiceDynamicAccessList	
Parameter	
other	Object to move-assign from
Detailed Description	
A builder instance must always get destroyed before the dynamic access list that created it is moved	
Returns	
ara::diag::DiagnosticServiceDynamicAccessList &	Reference to self
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-269 ara::diag::DiagnosticServiceDynamicAccessList::operator=

4.31.6 ara::diag::DiagnosticServiceDynamicAccessList::operator=

Prototype	
`ara::diag::DiagnosticServiceDynamicAccessList & ara::diag::DiagnosticServiceDynamicAccessList::operator= ( const class ara::diag::DiagnosticServiceDynamicAccessList & other ) & noexcept`	
Description	
Copy assigns an instance of DiagnosticServiceDynamicAccessList	
Parameter	
other	Object to copy-assign from
Returns	
ara::diag::DiagnosticServiceDynamicAccessList &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-270 ara::diag::DiagnosticServiceDynamicAccessList::operator=

4.31.7 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder

Prototype	
`ara::diag::DynamicAccessListDiagServiceBuilder ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder ( ara::diag::class DynamicAccessListDiagServiceBuilder::Byte sid ) noexcept`	
Description	
Constructs a diagnostic service pattern builder for a specific ServiceIdentifier	
Parameter	
sid	The diagnostic service identifier

Detailed Description				
A builder instance must always get destroyed before the dynamic access list that created it is destroyed				
Returns				
ara::diag::DynamicAccessListDiagServiceBuilder	An instance of a diagnostic service pattern builder			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The previously created DynamicAccessListDiagServiceBuilder by this instance must be first finished by calling ara::diag::DynamicAccessListDiagServiceBuilder::Build() and then destroyed.				

Table 4-271 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder

4.31.8 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder

Prototype				
`ara::diag::DynamicAccessListDiagServiceBuilder ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder ( ara::diag::class DynamicAccessListDiagServiceBuilder::ByteString service_head ) noexcept`				
Description				
Constructs a diagnostic service pattern builder starting with a specific sequence of values				
Parameter				
service_head	The diagnostic service header to start matching with			
Detailed Description				
A builder instance must always get destroyed before the dynamic access list that created it is destroyed				
Returns				
ara::diag::DynamicAccessListDiagServiceBuilder	An instance of a diagnostic service pattern builder			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				

The content must not be empty, i.e. at least SID must be there. The previously created DynamicAccessListDiagServiceBuilder by this instance must be first finished by calling ara::diag::DynamicAccessListDiagServiceBuilder::Build() and then destroyed.

Table 4-272 ara::diag::DiagnosticServiceDynamicAccessList::MakeServiceBuilder

4.31.9 ara::diag::DiagnosticServiceDynamicAccessList::Reserve

Prototype				

`void`
`ara::diag::DiagnosticServiceDynamicAccessList::Reserve (`
`  std::size_t number_of_service_heads,`
`  std::size_t max_service_head_size`
`) noexcept`				

Description				
Allocates memory for all service heads to fit into the dynamic access list				
Parameter				
number_of_service_heads	The number of diagnostic service patterns			
max_service_head_size	The expected maximum number of diagnostic service bytes in a single pattern			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-273 ara::diag::DiagnosticServiceDynamicAccessList::Reserve

4.32 ara::diag::DoIPActivationLine

Description	
DoIPActivationLine interface	
Include	**Generalize**
ara/diag/doip_activation_line.h	
Constructors	
DoIPActivationLine	Move constructs an instance of DoIPActivationLine
DoIPActivationLine	Constructor of class DoIPActivationLine (inherited)
Operators	
operator=	Move assigns an instance of DoIPActivationLine
MemberFunctions	
GetActivationLineState	Called to get the current activation line state
GetNetworkInterfaceId	Called to get the network interface id for which the activation line is responsible
Offer	This Offer will enable the DM to listen to activation line state changes
StopOffer	This StopOffer will disable the provision of activation line state to DM
UpdateActivationLineState	Called to update the current activation line state

Table 4-274 ara::diag::DoIPActivationLine

4.32.1 ara::diag::DoIPActivationLine::DoIPActivationLine

Prototype

```
ara::diag::DoIPActivationLine::DoIPActivationLine (
  ara::diag::DoIPActivationLine && other
) noexcept
```

Description

Move constructs an instance of DoIPActivationLine

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-275 ara::diag::DoIPActivationLine::DoIPActivationLine

4.32.2 ara::diag::DoIPActivationLine::DoIPActivationLine

Prototype				
`ara::diag::DoIPActivationLine::DoIPActivationLine ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor of class DoIPActivationLine (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DoIPActivationLineInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-276 ara::diag::DoIPActivationLine::DoIPActivationLine

4.32.3 ara::diag::DoIPActivationLine::~DoIPActivationLine

Prototype				
`ara::diag::DoIPActivationLine::~DoIPActivationLine () noexcept`				
Description				
Destructor of class DoIPActivationLine				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-277 ara::diag::DoIPActivationLine::~DoIPActivationLine

4.32.4 ara::diag::DoIPActivationLine::operator=

Prototype
`ara::diag::DoIPActivationLine & ara::diag::DoIPActivationLine::operator= ( ara::diag::DoIPActivationLine && other ) & noexcept`
Description

Move assigns an instance of DoIPActivationLine				
Parameter				
other		The other object		
Returns				
ara::diag::DoIPActivationLine &		Reference to self		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-278 ara::diag::DoIPActivationLine::operator=

4.32.5 ara::diag::DoIPActivationLine::GetActivationLineState

Prototype				
`ara::core::Future<bool>` `ara::diag::DoIPActivationLine::GetActivationLineState ()`				
Description				
Called to get the current activation line state				
Parameter				
N/A		N/A		
Returns				
ara::core::Future<bool>		Future containing the current activation line state		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 4-279 ara::diag::DoIPActivationLine::GetActivationLineState

4.32.6 ara::diag::DoIPActivationLine::GetNetworkInterfaceId

Prototype
`ara::core::Future<std::uint8_t>` `ara::diag::DoIPActivationLine::GetNetworkInterfaceId ()`
Description

Called to get the network interface id for which the activation line is responsible				
Parameter				
N/A		N/A		
Returns				
ara::core::Future<std::uint8_t>		Future containing the network interface id		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-280 ara::diag::DoIPActivationLine::GetNetworkInterfaceId

4.32.7 ara::diag::DoIPActivationLine::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::DoIPActivationLine::Offer () noexcept`	
Description	
This Offer will enable the DM to listen to activation line state changes	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-281 ara::diag::DoIPActivationLine::Offer

4.32.8 ara::diag::DoIPActivationLine::StopOffer

Prototype

`void ara::diag::DoIPActivationLine::StopOffer () const noexcept`				
Description				
This StopOffer will disable the provision of activation line state to DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-282 ara::diag::DoIPActivationLine::StopOffer

4.32.9 ara::diag::DoIPActivationLine::UpdateActivationLineState

Prototype				
`void ara::diag::DoIPActivationLine::UpdateActivationLineState ( bool activation_line_state ) noexcept`				
Description				
Called to update the current activation line state				
Parameter				
activation_line_state		The current activation line state to set		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-283 ara::diag::DoIPActivationLine::UpdateActivationLineState

based on template version 1.00.00

4.33 ara::diag::DoIPGroupIdentification

Description	
DoIPGroupIdentificationInterface	
Include	**Generalize**
ara/diag/doip_group_identification.h	
Constructors	
DoIPGroupIdentification	Move constructs an instance of DoIPGroupIdentification
DoIPGroupIdentification	Constructor of DoIPGroupIdentification (inherited)
Operators	
operator=	Move assigns an instance of DoIPGroupIdentification
Types	
GidStatus	Response data of positive response message
MemberFunctions	
GetGidStatus	Called to get the current GID state for the DoIP protocol
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-284 ara::diag::DoIPGroupIdentification

4.33.1 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification

Prototype

```
ara::diag::DoIPGroupIdentification::DoIPGroupIdentification (
  ara::diag::DoIPGroupIdentification && other
) noexcept
```

Description

Move constructs an instance of DoIPGroupIdentification

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-285 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification

4.33.2 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification

Prototype				
`ara::diag::DoIPGroupIdentification::DoIPGroupIdentification ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor of DoIPGroupIdentification (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticDoIPGroupIdentificationInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-286 ara::diag::DoIPGroupIdentification::DoIPGroupIdentification

4.33.3 ara::diag::DoIPGroupIdentification::~DoIPGroupIdentification

Prototype				
`ara::diag::DoIPGroupIdentification::~DoIPGroupIdentification () noexcept`				
Description				
Destructor of DoIPGroupIdentification				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-287 ara::diag::DoIPGroupIdentification::~DoIPGroupIdentification

4.33.4 ara::diag::DoIPGroupIdentification::operator=

Prototype
`ara::diag::DoIPGroupIdentification & ara::diag::DoIPGroupIdentification::operator= ( ara::diag::DoIPGroupIdentification && other ) & noexcept`
Description

Move assigns an instance of DoIPGroupIdentification				
Parameter				
other	The other object			
Returns				
ara::diag::DoIPGroupIdentification &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-288 ara::diag::DoIPGroupIdentification::operator=

4.33.5 ara::diag::DoIPGroupIdentification::GidStatus

Prototype		
`struct ara::diag::DoIPGroupIdentification::GidStatus`		
Description		
Response data of positive response message		
Members		
ara::core::Array <std::uint8_t, 6>	group_identification	The identification group
std::uint8_t	further_action_required	A further action is required
std::uint8_t	sync_status	The synchronisation status

Table 4-289 ara::diag::DoIPGroupIdentification::GidStatus

4.33.6 ara::diag::DoIPGroupIdentification::GetGidStatus

Prototype	
`ara::core::Future<GidStatus>` `ara::diag::DoIPGroupIdentification::GetGidStatus ()`	
Description	
Called to get the current GID state for the DoIP protocol	
Parameter	
N/A	N/A
Detailed Description	
Providing the GID in DoIP protocol messages If the DM needs to know the GID and the status of the GID to be able to react or answer on any DoIP message, it shall obtain it by calling the method ara::diag::DoIPGroupIdentification::GetGidStatus()	

Returns	
ara::core::Future<GidStatus>	Future containing the group identification and state
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-290 ara::diag::DoIPGroupIdentification::GetGidStatus

4.33.7 ara::diag::DoIPGroupIdentification::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::DoIPGroupIdentification::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-291 ara::diag::DoIPGroupIdentification::Offer

4.33.8 ara::diag::DoIPGroupIdentification::StopOffer

Prototype
`void` `ara::diag::DoIPGroupIdentification::StopOffer () const noexcept`
Description
This StopOffer will disable the forwarding of request messages from DM

Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-292 ara::diag::DoIPGroupIdentification::StopOffer

based on template version 1.00.00

4.34 ara::diag::DoIPPowerMode

Description	
DiagnosticDoIPPowerModeInterface	
Include	**Generalize**
ara/diag/doip_power_mode.h	
Constructors	
DoIPPowerMode	Move constructs an instance of DoIPPowerMode
DoIPPowerMode	Constructor for DoIPPowerMode (inherited)
Operators	
operator=	Move assigns an instance of DoIPPowerMode
Types	
PowerModeType	PowerMode as defined in ISO13400-2
MemberFunctions	
GetDoIPPowerMode	Called to get the current Power Mode for the DoIP protocol
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-293 ara::diag::DoIPPowerMode

4.34.1 ara::diag::DoIPPowerMode::DoIPPowerMode

Prototype

```
ara::diag::DoIPPowerMode::DoIPPowerMode (
  ara::diag::DoIPPowerMode && other
) noexcept
```

Description

Move constructs an instance of DoIPPowerMode

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-294 ara::diag::DoIPPowerMode::DoIPPowerMode

4.34.2 ara::diag::DoIPPowerMode::DoIPPowerMode

Prototype				
`ara::diag::DoIPPowerMode::DoIPPowerMode ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for DoIPPowerMode (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticDoIPPowerModeInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-295 ara::diag::DoIPPowerMode::DoIPPowerMode

4.34.3 ara::diag::DoIPPowerMode::~DoIPPowerMode

Prototype				
`ara::diag::DoIPPowerMode::~DoIPPowerMode () noexcept`				
Description				
Destructor of DoIPPowerMode				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-296 ara::diag::DoIPPowerMode::~DoIPPowerMode

4.34.4 ara::diag::DoIPPowerMode::operator=

Prototype
`ara::diag::DoIPPowerMode & ara::diag::DoIPPowerMode::operator= ( ara::diag::DoIPPowerMode && other ) & noexcept`
Description

Move assigns an instance of DoIPPowerMode				
Parameter				
other	The other object			
Returns				
ara::diag::DoIPPowerMode &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-297 ara::diag::DoIPPowerMode::operator=

4.34.5 ara::diag::DoIPPowerMode::PowerModeType

Prototype	
`ara::diag::DoIPPowerMode::PowerModeType`	
Description	
PowerMode as defined in ISO13400-2	
Type	
std::uint8_t	
Enumerations	
kNotReady	0x00
kReady	0x01
kNotSupported	0x02

Table 4-298 ara::diag::DoIPPowerMode::PowerModeType

4.34.6 ara::diag::DoIPPowerMode::GetDoIPPowerMode

Prototype	
`ara::core::Future<PowerModeType>` `ara::diag::DoIPPowerMode::GetDoIPPowerMode ()`	
Description	
Called to get the current Power Mode for the DoIP protocol	
Parameter	
N/A	N/A
Returns	
ara::core::Future<PowerModeType>	Future containing the current diagnostic power mode
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-299 ara::diag::DoIPPowerMode::GetDoIPPowerMode

4.34.7 ara::diag::DoIPPowerMode::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::DoIPPowerMode::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-300 ara::diag::DoIPPowerMode::Offer

4.34.8 ara::diag::DoIPPowerMode::StopOffer

Prototype	
`void` `ara::diag::DoIPPowerMode::StopOffer () const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Parameter	
N/A	N/A
Returns	
void	N/A
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-301 ara::diag::DoIPPowerMode::StopOffer

based on template version 1.00.00

4.35 ara::diag::DownloadService

Description	
Download service interface	
Include	**Generalize**
ara/diag/download.h	
Constructors	
DownloadService	Move constructs an instance of DownloadService
DownloadService	Constructor for DownloadService (inherited)
Operators	
operator=	Move assigns an instance of DownloadService
Types	
OperationOutput	Response data of positive response message
MemberFunctions	
DownloadData	Called for TransferData following a previous RequestDownload
Offer	This Offer will enable the DM to forward request messages to this handler
RequestDownload	Called for RequestDownload
RequestDownloadExit	Called for RequestTransferExit
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-302 ara::diag::DownloadService

4.35.1 ara::diag::DownloadService::DownloadService

Prototype

```
ara::diag::DownloadService::DownloadService (
  ara::diag::DownloadService && other
) noexcept
```

Description				
Move constructs an instance of DownloadService				
Parameter				
other	The other object			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-303 ara::diag::DownloadService::DownloadService

4.35.2 ara::diag::DownloadService::DownloadService

Prototype				
`ara::diag::DownloadService::DownloadService ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for DownloadService (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DownloadServiceInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-304 ara::diag::DownloadService::DownloadService

4.35.3 ara::diag::DownloadService::~DownloadService

Prototype				
`ara::diag::DownloadService::~DownloadService () noexcept`				
Description				
Destructor of class DownloadService				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-305 ara::diag::DownloadService::~DownloadService

4.35.4 ara::diag::DownloadService::operator=

Prototype
`ara::diag::DownloadService & ara::diag::DownloadService::operator= ( ara::diag::DownloadService && other ) & noexcept`
Description

Move assigns an instance of DownloadService				
Parameter				
other	The other object			
Returns				
ara::diag::DownloadService &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-306 ara::diag::DownloadService::operator=

4.35.5 ara::diag::DownloadService::OperationOutput

Prototype		
`struct ara::diag::DownloadService::OperationOutput`		
Description		
Response data of positive response message		
Members		
ara::core::Vector<std::uint8_t>	response_data	Content of positive response message (without SID)

Table 4-307 ara::diag::DownloadService::OperationOutput

4.35.6 ara::diag::DownloadService::DownloadData

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::DownloadService::DownloadData (
  ara::core::Span<std::uint8_t> transfer_request_parameter_record,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description	
Called for TransferData following a previous RequestDownload	
Parameter	
transfer_request_parameter_record	data to be transferred (copied/downloaded to the ECU/server).
meta_info	contains additional meta information

cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<OperationOutput>	a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-308 ara::diag::DownloadService::DownloadData

4.35.7 ara::diag::DownloadService::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::DownloadService::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-309 ara::diag::DownloadService::Offer

4.35.8 ara::diag::DownloadService::RequestDownload

Prototype

`ara::core::Future<void>`
`ara::diag::DownloadService::RequestDownload (`
`  std::uint8_t data_format_identifier,`
`  std::uint8_t address_and_length_format_identifier,`
`  ara::core::Span<std::uint8_t> memory_address_and_size,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`

Description
Called for RequestDownload
Parameter

data_format_identifier	UDS dataFormat Identifier
address_and_length_format_identifier	UDS addressAndLengthFormatIdentifier
memory_address_and_size	memoryAddress and memorySize part of the request
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<void>	a Future, which either gets readied to void (for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message)

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-310 ara::diag::DownloadService::RequestDownload

4.35.9 ara::diag::DownloadService::RequestDownloadExit

Prototype

`ara::core::Future<OperationOutput>`
`ara::diag::DownloadService::RequestDownloadExit (`
`  ara::core::Span<std::uint8_t> transferRequestParameterRecord,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`

Description
Called for RequestTransferExit
Parameter

transferRequestParameterRecord	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<OperationOutput>	a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message) Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints

N/A

Table 4-311 ara::diag::DownloadService::RequestDownloadExit

4.35.10 ara::diag::DownloadService::StopOffer

Prototype
`void` `ara::diag::DownloadService::StopOffer () const noexcept`

Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-312 ara::diag::DownloadService::StopOffer

based on template version 1.00.00

4.36 ara::diag::DTCInformation

Description	
Class to implement operations on DTC information per configured DiagnosticMemory Destination	
Include	**Generalize**
ara/diag/dtc_information.h	
Constructors	
DTCInformation	Move constructs instance of class
DTCInformation	Constructor for a DTCInformation instance which allows for DTC related operation per DiagnosticMemoryDestination
Operators	
operator=	Move assigns instance of class
Types	
ControlDtcStatusType	Type for ControlDTCStatus status as requested by UDS service 0x85 ControlDTCSetting
UdsStatusBit	Type alias for UDS status bits according to ISO 14229-1
UdsDTCStatusByteType	Type for UdsDTCStatusByteType status
SnapshotDataIdentiferType	Type for SnapshotDataIdentifierType status
SnapshotDataRecordType	Type for SnapshotDataRecordType status
SnapshotRecordUpdatedType	Type for SnapshotRecordUpdatedType status
MemberFunctions	
Clear	Method for Clearing a DTC or a group of DTCs
EnableControlDtc	Enforce restoring ControlDTCStatus setting to enabled in case the monitor has some conditions or states demands to do so
GetControlDTCStatus	Contains the current status of the ControlDTCStatus
GetCurrentStatus	Retrieves the current UDS DTC status byte of the given DTC identifier
GetNumberOfStoredEntries	Contains the number of currently stored fault memory entries
SetControlDtcStatusNotifier	Registers a notifier function which is called if the control DTC setting is changed
SetDTCStatusChangedNotifier	Register a notifier function which is called if a DTC is changed
SetNumberOfStoredEntriesNotifier	Register a notifier function which is called if the number of currently stored fault memory entries changed
SetSnapshotRecordUpdatedNotifier	Register a notifier function which is called if the SnapshotRecord is changed

Table 4-313 ara::diag::DTCInformation

4.36.1 ara::diag::DTCInformation::DTCInformation

Prototype				
`ara::diag::DTCInformation::DTCInformation ( ara::diag::DTCInformation && ) noexcept`				
Description				
Move constructs instance of class				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-314 ara::diag::DTCInformation::DTCInformation

4.36.2 ara::diag::DTCInformation::DTCInformation

Prototype				
`ara::diag::DTCInformation::DTCInformation ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for a DTCInformation instance which allows for DTC related operation per DiagnosticMemoryDestination				
Parameter				
specifier		InstanceSpecifier to an PortPrototype of an DiagnosticDTCInformationInterface		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-315 ara::diag::DTCInformation::DTCInformation

4.36.3 ara::diag::DTCInformation::~DTCInformation

Prototype

ara::diag::DTCInformation::~DTCInformation () noexcept				
Description				
Destructor of class DTCInformation				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-316 ara::diag::DTCInformation::~DTCInformation

4.36.4 ara::diag::DTCInformation::operator=

Prototype	

`ara::diag::DTCInformation &`
`ara::diag::DTCInformation::operator= (`
`  ara::diag::DTCInformation &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
ara::diag::DTCInformation &	Reference to self
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 4-317 ara::diag::DTCInformation::operator=

4.36.5 ara::diag::DTCInformation::ControlDtcStatusType

Prototype
`ara::diag::DTCInformation::ControlDtcStatusType`
Description
Type for ControlDTCStatus status as requested by UDS service 0x85 ControlDTCSetting

Type	
std::uint8_t	
Enumerations	
kDTCSettingOn	0x00
kDTCSettingOff	0x01

Table 4-318 ara::diag::DTCInformation::ControlDtcStatusType

4.36.6 ara::diag::DTCInformation::UdsStatusBit

Prototype	
`ara::diag::DTCInformation::UdsStatusBit`	
Description	
Type alias for UDS status bits according to ISO 14229-1	
Type	
std::uint8_t	
Enumerations	
kTestFailed	0x01
kTestFailedThisOperationCycle	0x02
kPendingDTC	0x04
kConfirmedDTC	0x08
kTestNotCompletedSinceLastClear	0x10
kTestFailedSinceLastClear	0x20
kTestNotCompletedThisOperationCycle	0x40
kWarningIndicatorRequested	0x80

Table 4-319 ara::diag::DTCInformation::UdsStatusBit

4.36.7 ara::diag::DTCInformation::SnapshotDataIdentiferType

Prototype		
`struct ara::diag::DTCInformation::SnapshotDataIdentiferType`		
Description		
Type for SnapshotDataIdentiferType status		
Members		
std::uint16_t	dataIdentifer	The data identifier
ara::core::Vector <std::uint8_t>	data	The vector containing the snapshot data

Table 4-320 ara::diag::DTCInformation::SnapshotDataIdentiferType

4.36.8 ara::diag::DTCInformation::SnapshotDataRecordType

Prototype

struct ara::diag::DTCInformation::SnapshotDataRecordType		
Description		
Type for SnapshotDataRecordType		
Members		
std::uint8_t	snapshotRecordNumber	The snapshot record number
ara::core::Vector <Snapshot-DataIdentifer-Type>	snapshotDataIdentifiers	List of data identifiers snapshots

Table 4-321 ara::diag::DTCInformation::SnapshotDataRecordType

4.36.9 ara::diag::DTCInformation::SnapshotRecordUpdatedType

Prototype		
struct ara::diag::DTCInformation::SnapshotRecordUpdatedType		
Description		
Type for SnapshotDataRecordType		
Members		
std::uint32_t	DTC	The DTC which is changed
ara::core::Vector <Snapshot-DataRecord-Type>	snapshotDataRecord	List of snapshot records

Table 4-322 ara::diag::DTCInformation::SnapshotRecordUpdatedType

4.36.10 ara::diag::DTCInformation::Clear

Prototype	
ara::core::Result<void> ara::diag::DTCInformation::Clear (std::uint32_t dtc_group)	
Description	
Method for Clearing a DTC or a group of DTCs	
Parameter	
dtc_group	group to be cleared.
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-323 ara::diag::DTCInformation::Clear

4.36.11 ara::diag::DTCInformation::EnableControlDtc

Prototype	
`ara::core::Result<void>` `ara::diag::DTCInformation::EnableControlDtc ()`	
Description	
Enforce restoring ControlDTCStatus setting to enabled in case the monitor has some conditions or states demands to do so	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-324 ara::diag::DTCInformation::EnableControlDtc

4.36.12 ara::diag::DTCInformation::GetControlDTCStatus

Prototype	
`ara::core::Result<ControlDtcStatusType>` `ara::diag::DTCInformation::GetControlDTCStatus ()`	
Description	
Contains the current status of the ControlDTCStatus	
Parameter	
N/A	N/A

Returns	
ara::core::Result<ControlDtcStatus Type>	Result contains "current ControlDTCStatus" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-325 ara::diag::DTCInformation::GetControlDTCStatus

4.36.13 ara::diag::DTCInformation::GetCurrentStatus

Prototype

```
ara::core::Result<UdsDTCStatusByteType>
ara::diag::DTCInformation::GetCurrentStatus (
   std::uint32_t dtc_number
)
```

Description	
Retrieves the current UDS DTC status byte of the given DTC identifier	
Parameter	
dtc_number	The diagnostic trouble code
Returns	
ara::core::Result<UdsDTCStatus ByteType>	Result contains "current status of ControlDtcStatus" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-326 ara::diag::DTCInformation::GetCurrentStatus

4.36.14 ara::diag::DTCInformation::GetNumberOfStoredEntries

Prototype				
`ara::core::Result<std::uint32_t>` `ara::diag::DTCInformation::GetNumberOfStoredEntries ()`				
Description				
Contains the number of currently stored fault memory entries				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<std::uint32_t>	Result contains the "number of currently stored fault memory entries" if operation succeeds, otherwise error code that shall not be evaluated			
Errors				
Any/Custom Do not evaluate the error code just that the result contains an error				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-327 ara::diag::DTCInformation::GetNumberOfStoredEntries

4.36.15 ara::diag::DTCInformation::SetControlDtcStatusNotifier

Prototype	

`ara::core::Result<void>`
`ara::diag::DTCInformation::SetControlDtcStatusNotifier (`
`  std::function<void (ControlDtcStatusType)> notifier`
`)`	

Description	
Registers a notifier function which is called if the control DTC setting is changed	
Parameter	
notifier	The notifier function
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	void Result
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this DTCInformation has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetControlDtcStatusNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				
Steady				
TRUE				

Table 4-328 ara::diag::DTCInformation::SetControlDtcStatusNotifier

4.36.16 ara::diag::DTCInformation::SetDTCStatusChangedNotifier

Prototype

```
ara::core::Result<void>
ara::diag::DTCInformation::SetDTCStatusChangedNotifier (
  std::function<void (std::uint32_t, UdsDTCStatusByteType, UdsDTCStatusByteType)>
notifier
)
```

Description

Register a notifier function which is called if a DTC is changed

Parameter

notifier	The notifier function

Detailed Description

If successful, a previously set notifier gets overwritten.

Returns

ara::core::Result<void>	void Result.

Errors

ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this DTCInformation has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetDTCStatusChangedNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Steady
TRUE

Table 4-329 ara::diag::DTCInformation::SetDTCStatusChangedNotifier

4.36.17 ara::diag::DTCInformation::SetNumberOfStoredEntriesNotifier

Prototype

```
ara::core::Result<void>
ara::diag::DTCInformation::SetNumberOfStoredEntriesNotifier (
  std::function<void (std::uint32_t)> notifier
)
```

Description

Register a notifier function which is called if the number of currently stored fault memory entries changed

Parameter

notifier	The notifier function

Detailed Description

If successful, a previously set notifier gets overwritten.

Returns

ara::core::Result<void>	void Result.

Errors

ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the destruction of this DTCInformation has completed or until it is replaced by a new notifier function.

Whenever a previously set notifier is in execution, the SetNumberOfStoredEntriesNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Steady

TRUE

Table 4-330 ara::diag::DTCInformation::SetNumberOfStoredEntriesNotifier

4.36.18 ara::diag::DTCInformation::SetSnapshotRecordUpdatedNotifier

Prototype

```
ara::core::Result<void>
ara::diag::DTCInformation::SetSnapshotRecordUpdatedNotifier (
   std::function<void (SnapshotRecordUpdatedType)> notifier
)
```

Description	
Register a notifier function which is called if the SnapshotRecord is changed	
Parameter	
notifier	The notifier function
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	void Result.
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints
The passed in function must not throw any exceptions.
The passed in function must be valid to be called until the destruction of this DTCInformation has completed or until it is replaced by a new notifier function.
Whenever a previously set notifier is in execution, the SetSnapshotRecordUpdatedNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Steady
TRUE

Table 4-331 ara::diag::DTCInformation::SetSnapshotRecordUpdatedNotifier

based on template version 1.00.00

4.37 ara::diag::DynamicAccessListDiagServiceBuilder

Description	
Single diagnostic service pattern builder to match in the dynamic access list	
Include	**Generalize**
ara/diag/dynamic_access_list_diag_service_builder.h	
Types	
ByteString	Type alias of a sequence of diagnostic service pattern elements in the dynamic access list
Byte	Type alias of a single diagnostic service pattern element in the dynamic access list
MemberFunctions	
Add	Adds a single byte value to the current pattern
Add	Adds a sequence of values to the current pattern
Add	Adds a closed range of a single byte values to the current pattern
Any	Specifies the number of bytes to consider as irrelevant for the pattern match
Build	Finalizes building the dynamic access list
EndsWith	Specifies a single byte value to match in the end of the diagnostic service protocol information
EndsWith	Specifies a closed range of a single byte values to match in the end of the diagnostic service

Table 4-332 ara::diag::DynamicAccessListDiagServiceBuilder

4.37.1 ara::diag::DynamicAccessListDiagServiceBuilder::~DynamicAccessListDiagServiceBuilder

Prototype				
`ara::diag::DynamicAccessListDiagServiceBuilder::~DynamicAccessListDiagServiceBuilder () noexcept`				
Description				
Destructor				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE for different instances	FALSE
Constraints				
N/A				

Table 4-333 ara::diag::DynamicAccessListDiagServiceBuilder::~DynamicAccessListDiagServiceBuilder

4.37.2 ara::diag::DynamicAccessListDiagServiceBuilder::ByteString

Prototype
`ara::diag::DynamicAccessListDiagServiceBuilder::ByteString`
Description
Type alias of a sequence of diagnostic service pattern elements in the dynamic access list
Type
ara::core::Span<Byte>

Table 4-334 ara::diag::DynamicAccessListDiagServiceBuilder::ByteString

4.37.3 ara::diag::DynamicAccessListDiagServiceBuilder::Byte

Prototype
`ara::diag::DynamicAccessListDiagServiceBuilder::Byte`
Description
Type alias of a single diagnostic service pattern element in the dynamic access list
Type
std::uint8_t

Table 4-335 ara::diag::DynamicAccessListDiagServiceBuilder::Byte

4.37.4 ara::diag::DynamicAccessListDiagServiceBuilder::Add

Prototype

```
ara::diag::DynamicAccessListDiagServiceBuilder &
ara::diag::DynamicAccessListDiagServiceBuilder::Add (
   ara::diag::DynamicAccessListDiagServiceBuilder::Byte value
) noexcept
```

Description

Adds a single byte value to the current pattern

Parameter

value	The value to exactly match

Returns

ara::diag::DynamicAccessListDiagServiceBuilder &	The instance of the same object to allow fluent API usage

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE for different instances	FALSE

Constraints

N/A

Table 4-336 ara::diag::DynamicAccessListDiagServiceBuilder::Add

4.37.5 ara::diag::DynamicAccessListDiagServiceBuilder::Add

Prototype				

`ara::diag::DynamicAccessListDiagServiceBuilder &`
`ara::diag::DynamicAccessListDiagServiceBuilder::Add (`
`  ara::diag::DynamicAccessListDiagServiceBuilder::ByteString values`
`) noexcept`				

Description				
Adds a sequence of values to the current pattern				
Parameter				
values	The view over list of sequence of values to exactly match			
Returns				
ara::diag::DynamicAccessListDiag ServiceBuilder &	The instance of the same object to allow fluent API usage			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE for different instances	FALSE
Constraints				
The input ByteString must not be empty				

Table 4-337 ara::diag::DynamicAccessListDiagServiceBuilder::Add

4.37.6 ara::diag::DynamicAccessListDiagServiceBuilder::Add

Prototype	

`ara::diag::DynamicAccessListDiagServiceBuilder &`
`ara::diag::DynamicAccessListDiagServiceBuilder::Add (`
`  ara::diag::DynamicAccessListDiagServiceBuilder::ByteRange range`
`) noexcept`	

Description	
Adds a closed range of a single byte values to the current pattern	
Parameter	
range	The range of values to match
Detailed Description	
Not supported yet	
Returns	
ara::diag::DynamicAccessListDiag ServiceBuilder &	The instance of the same object to allow fluent API usage

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE for different instances	FALSE
Constraints				
Unavailable functionality. Must not be called				

Table 4-338 ara::diag::DynamicAccessListDiagServiceBuilder::Add

4.37.7 ara::diag::DynamicAccessListDiagServiceBuilder::Any

Prototype	

`ara::diag::DynamicAccessListDiagServiceBuilder &`
`ara::diag::DynamicAccessListDiagServiceBuilder::Any (`
`  std::size_t number_of_bytes_to_ignore`
`) noexcept`	

Description	
Specifies the number of bytes to consider as irrelevant for the pattern match	
Parameter	
number_of_bytes_to_ignore	The number of bytes to ignore
Detailed Description	
Not supported yet	
Returns	
ara::diag::DynamicAccessListDiag ServiceBuilder &	The instance of the same object to allow fluent API usage

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
Unavailable functionality. Must not be called				

Table 4-339 ara::diag::DynamicAccessListDiagServiceBuilder::Any

4.37.8 ara::diag::DynamicAccessListDiagServiceBuilder::Build

Prototype
`void` `ara::diag::DynamicAccessListDiagServiceBuilder::Build () noexcept`
Description

Finalizes building the dynamic access list				
Parameter				
N/A		N/A		
Detailed Description				
Must be called before destroying the ServiceBuilder object.				
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE for different instances	FALSE
Constraints				
N/A				

Table 4-340 ara::diag::DynamicAccessListDiagServiceBuilder::Build

4.37.9 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith

Prototype				
`void ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith ( ara::diag::DynamicAccessListDiagServiceBuilder::Byte value ) noexcept`				
Description				
Specifies a single byte value to match in the end of the diagnostic service protocol information				
Parameter				
value		The value to exactly match		
Detailed Description				
Not supported yet				
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
Unavailable functionality. Must not be called				

Table 4-341 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith

4.37.10 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith

Prototype				
`void ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith ( ara::diag::DynamicAccessListDiagServiceBuilder::ByteRange range ) noexcept`				
Description				
Specifies a closed range of a single byte values to match in the end of the diagnostic service				
Parameter				
range	The value to exactly match			
Detailed Description				
Not supported yet				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
Unavailable functionality. Must not be called				

Table 4-342 ara::diag::DynamicAccessListDiagServiceBuilder::EndsWith

based on template version 1.00.00

4.38 ara::diag::Event

Description	
Event interface	
Include	**Generalize**
ara/diag/event.h	
Constructors	
Event	Move constructs instance of class
Event	Class for an Event
Operators	
operator=	Move assigns instance of class
Types	
EventStatusByte	Current event status bit-encoded
DTCFormatType	Represents the type of the DTC format according to ISO 14229-1
EventStatusBit	Type alias for event status bit
DebouncingState	Debounce status of event
MemberFunctions	
GetDTCNumber	Returns the DTC related to the event
GetDebouncingStatus	Get the current debouncing status
GetEventStatus	Contains the current diagnostic event status
GetFaultDetectionCounter	Returns the current value of fault detection counter of this event
GetLatchedWIRStatus	Contains the current warning indicator status
GetTestComplete	Get the test completed flag (corresponds to FDC = -128 or FDC = 127)
SetEventStatusChangedNotifier	Register a notifier function which is called if a diagnostic event is changed
SetLatchedWIRStatus	Set the warning indicator status

Table 4-343 ara::diag::Event

4.38.1 ara::diag::Event::Event

Prototype

`ara::diag::Event::Event (`
`  ara::diag::Event &&`
`) noexcept`

Description
Move constructs instance of class
Parameter
Errors

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-344 ara::diag::Event::Event

4.38.2 ara::diag::Event::Event

Prototype	
`ara::diag::Event::Event ( const ara::core::InstanceSpecifier & specifier )`	
Description	
Class for an Event	
Parameter	
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticEventInterface
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-345 ara::diag::Event::Event

4.38.3 ara::diag::Event::~Event

Prototype				
`ara::diag::Event::~Event () noexcept`				
Description				
Destructor of class Event				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-346 ara::diag::Event::~Event

4.38.4 ara::diag::Event::operator=

Prototype	

`ara::diag::Event &`
`ara::diag::Event::operator= (`
`  ara::diag::Event &&`
`) & noexcept`	

Description	
Move assigns instance of class	
Parameter	
Returns	
ara::diag::Event &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-347 ara::diag::Event::operator=

4.38.5 ara::diag::Event::GetDTCNumber

Prototype	

`ara::core::Result<std::uint32_t>`
`ara::diag::Event::GetDTCNumber (`
`  enum ara::diag::Event::DTCFormatType dtc_format`
`)`	

Description	
Returns the DTC related to the event	
Parameter	
dtc_format	Define DTC format for the return value.
Returns	
ara::core::Result<std::uint32_t>	result containing DTC number if DTCFormatType is kDTCFormatUDS else kInvalidArgument for other format types
Errors	

ara::diag::DiagReportingErrc::kInvalidArgument The given DTC format differs from DTC format UDS				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-348 ara::diag::Event::GetDTCNumber

4.38.6 ara::diag::Event::DTCFormatType

Prototype		
`ara::diag::Event::DTCFormatType`		
Description		
Represents the type of the DTC format according to ISO 14229-1		
Type		
std::uint8_t		
Enumerations		
kDTCFormatOBD	0	
kDTCFormatUDS	1	
kDTCFormatJ1939	2	

Table 4-349 ara::diag::Event::DTCFormatType

4.38.7 ara::diag::Event::EventStatusBit

Prototype	
`ara::diag::Event::EventStatusBit`	
Description	
Type alias for event status bit	
Type	
std::uint16_t	
Enumerations	
kTestFailed	0x01
kTestFailedThisOperationCycle	0x02
kTestNotCompletedThisOperationCycle	0x40

Table 4-350 ara::diag::Event::EventStatusBit

4.38.8 ara::diag::Event::DebouncingState

Prototype	
`ara::diag::Event::DebouncingState`	
Description	
Represents the type of the DTC format according to ISO 14229-1	
Type	
std::uint8_t	
Enumerations	
kNeutral	0x00
kTemporarilyDefective	0x01
kFinallyDefective	0x02
kTemporarilyHealed	0x04
kFinallyHealed	0x08

Table 4-351 ara::diag::Event::DebouncingState

4.38.9 ara::diag::Event::GetDebouncingStatus

Prototype	
`ara::core::Result<DebouncingState>` `ara::diag::Event::GetDebouncingStatus ()`	
Description	
Get the current debouncing status	
Parameter	
N/A	N/A
Detailed Description	
Function not yet implemented. Always returning generic error.	
Returns	
ara::core::Result<DebouncingState>	Result contains the "current debouncing status" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
ara::diag::DiagReportingErrc::kGenericError Always	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE

Constraints
N/A

Table 4-352 ara::diag::Event::GetDebouncingStatus

4.38.10 ara::diag::Event::GetEventStatus

Prototype	
`ara::core::Result<EventStatusByte> ara::diag::Event::GetEventStatus ()`	
Description	
Contains the current diagnostic event status	
Parameter	
N/A	N/A
Returns	
ara::core::Result<EventStatusByte>	Result contains the "current diagnostic event status" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-353 ara::diag::Event::GetEventStatus

4.38.11 ara::diag::Event::GetFaultDetectionCounter

Prototype	
`ara::core::Result<std::int8_t> ara::diag::Event::GetFaultDetectionCounter ()`	
Description	
Returns the current value of fault detection counter of this event	
Fault detection counter of an event debouncing towards failed shall be between 1 and 127	
Fault detection counter of an event debouncing towards passed shall be between -1 and -128	
Parameter	
N/A	N/A
Returns	
ara::core::Result<std::int8_t>	Result contains the "current value of fault detection counter" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
ara::diag::DiagReportingErrc::kGenericError Always	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
Function must not be invoked directly from an AraDiag thread. It must be ensured that the invoking thread is not blocking any AraDiag thread.				

Table 4-354 ara::diag::Event::GetFaultDetectionCounter

4.38.12 ara::diag::Event::GetLatchedWIRStatus

Prototype	
`ara::core::Result<bool>` `ara::diag::Event::GetLatchedWIRStatus ()`	
Description	
Contains the current warning indicator status	
Parameter	
N/A	N/A
Detailed Description	
Function not yet implemented. Always returning generic error.	
Returns	
ara::core::Result<bool>	Result contains the "current warning indicator status" if operation succeeds, otherwise error code
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-355 ara::diag::Event::GetLatchedWIRStatus

4.38.13 ara::diag::Event::GetTestComplete

Prototype	
`ara::core::Result<bool>` `ara::diag::Event::GetTestComplete ()`	
Description	
Get the test completed flag (corresponds to FDC = -128 or FDC = 127)	
Parameter	
N/A	N/A
Returns	

ara::core::Result<bool>		Result containing a boolean indicating if test is complete		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-356 ara::diag::Event::GetTestComplete

4.38.14 ara::diag::Event::SetEventStatusChangedNotifier

Prototype	

`ara::core::Result<void>`
`ara::diag::Event::SetEventStatusChangedNotifier (`
`  std::function<void (EventStatusByte)> notifier`
`)`	

Description	
Register a notifier function which is called if a diagnostic event is changed	
Parameter	
notifier	The notifier function
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	void Result.
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this Event has completed or until it is replaced by a new notifier function. Whenever a previously set notifier is in execution, the SetEventStatusChangedNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				
Steady				

TRUE

Table 4-357 ara::diag::Event::SetEventStatusChangedNotifier

4.38.15 ara::diag::Event::SetLatchedWIRStatus

Prototype	

`ara::core::Result<void>`
`ara::diag::Event::SetLatchedWIRStatus (`
`  bool wir_status`
`)`	

Description	
Set the warning indicator status	
Parameter	
wir_status	Current warning indicator status
Detailed Description	
Function not yet implemented. Always returning generic error.	
Returns	
ara::core::Result<void>	Result contains "void" if operation succeeds, otherwise error code
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-358 ara::diag::Event::SetLatchedWIRStatus

based on template version 1.00.00

4.39 ara::diag::ExternalAuthentication

Description	
Provides externally managed diagnostic client authentication	
Include	**Generalize**
ara/diag/external_authentication.h	
Constructors	
ExternalAuthentication	Move constructor
ExternalAuthentication	Constructs the port for authentication of diagnostic clients
Operators	
operator=	Move Assignment
Types	
Address	Alias for tester address
MemberFunctions	
Get	Provides a diagnostic client authenticate state object
Get	Provides a diagnostic client authenticate state object
GetAll	Provides all diagnostic client authenticate state objects

Table 4-359 ara::diag::ExternalAuthentication

4.39.1 ara::diag::ExternalAuthentication::ExternalAuthentication

Prototype

```
ara::diag::ExternalAuthentication::ExternalAuthentication (
  ara::diag::ExternalAuthentication &&
) noexcept
```

Description

Move constructor

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-360 ara::diag::ExternalAuthentication::ExternalAuthentication

4.39.2 ara::diag::ExternalAuthentication::ExternalAuthentication

Prototype

```
ara::diag::ExternalAuthentication::ExternalAuthentication (
   const ara::core::InstanceSpecifier & specifier,
   enum ara::diag::ReentrancyType reentrancy_type
) noexcept
```

Description	
Constructs the port for authentication of diagnostic clients	
Parameter	
specifier	The instance specifier
reentrancy_type	Parameter is currently ignored and instead the parameter "numberOfParallelRequestExecutions" specifies if interface must be callable from multiple threads (must be threadsafe if numberOfParallelRequestExecutions > 1)
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-361 ara::diag::ExternalAuthentication::ExternalAuthentication

4.39.3 ara::diag::ExternalAuthentication::~ExternalAuthentication

Prototype				
`ara::diag::ExternalAuthentication::~ExternalAuthentication () noexcept`				
Description				
Destructor				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-362 ara::diag::ExternalAuthentication::~ExternalAuthentication

4.39.4 ara::diag::ExternalAuthentication::operator=

Prototype

```
ara::diag::ExternalAuthentication &
ara::diag::ExternalAuthentication::operator= (
   ara::diag::ExternalAuthentication &&
) & noexcept
```

Description	
Move Assignment	
Parameter	
Returns	
ara::diag::ExternalAuthentication &	Reference to this ExternalAuthentication
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-363 ara::diag::ExternalAuthentication::operator=

4.39.5 ara::diag::ExternalAuthentication::Address

Prototype
`ara::diag::ExternalAuthentication::Address`
Description
Alias for tester address
Type
std::uint16_t

Table 4-364 ara::diag::ExternalAuthentication::Address

4.39.6 ara::diag::ExternalAuthentication::Get

```
ara::core::Result<ClientAuthentication>
ara::diag::ExternalAuthentication::Get (
   ara::diag::ExternalAuthentication::Address source_address
) noexcept
```

Description	
Provides a diagnostic client authenticate state object	
Parameter	
source_address	The source address of the client to authenticate
Returns	

ara::core::Result<Client Authentication>	The associated diagnostic client authentication object or error
Errors	
ara::diag::DiagReportingErrc::kInvalidArgument	The client authentication object was not found

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-365 ara::diag::ExternalAuthentication::Get

4.39.7 ara::diag::ExternalAuthentication::Get

Prototype

```
ara::core::Result<ClientAuthentication>
ara::diag::ExternalAuthentication::Get (
  ara::diag::MetaInfo & meta_info
) noexcept
```

Description

Provides a diagnostic client authenticate state object

Parameter	
meta_info	The meta information of a diagnostic service port
Returns	
ara::core::Result<Client Authentication>	The diagnostic client associated authentication object or error
Errors	
ara::diag::DiagReportingErrc::kGenericError	The source address string could not be converted
ara::diag::DiagReportingErrc::kInvalidArgument	The client authentication object was not found

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-366 ara::diag::ExternalAuthentication::Get

4.39.8 ara::diag::ExternalAuthentication::GetAll

Prototype

```
ara::core::Vector<ClientAuthentication>
ara::diag::ExternalAuthentication::GetAll () noexcept
```

Description

Provides all diagnostic client authenticate state objects				
Parameter				
N/A		N/A		
Returns				
ara::core::Vector<Client Authentication>		The list of all diagnostic client associated authentication objects or empty list if none available.		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-367 ara::diag::ExternalAuthentication::GetAll

4.40 ara::diag::GenericDataIdentifier

Description	
Generic DataIdentifier interface	
Include	**Generalize**
ara/diag/generic_data_identifier.h	
Constructors	
GenericDataIdentifier	Move constructs an instance of GenericDataIdentifier
GenericDataIdentifier	Constructor for GenericDataIdentifier (inherited)
Operators	
operator=	Move assigns an instance of GenericDataIdentifier
Types	
OperationOutput	Response data of positive response message
MemberFunctions	
Offer	This Offer will enable the DM to forward request messages to this handler
Read	Called for ReadDataByIdentifier request for this DiagnosticDataIdentifier
StopOffer	This StopOffer will disable the forwarding of request messages from DM
Write	Called for WriteDataByIdentifier request for this DiagnosticDataIdentifier

Table 4-368 ara::diag::GenericDataIdentifier

4.40.1 ara::diag::GenericDataIdentifier::GenericDataIdentifier

Prototype

```
ara::diag::GenericDataIdentifier::GenericDataIdentifier (
  ara::diag::GenericDataIdentifier && other
) noexcept
```

Description				
Move constructs an instance of GenericDataIdentifier				
Parameter				
other	The other object			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-369 ara::diag::GenericDataIdentifier::GenericDataIdentifier

4.40.2 ara::diag::GenericDataIdentifier::GenericDataIdentifier

Prototype				
`ara::diag::GenericDataIdentifier::GenericDataIdentifier ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for GenericDataIdentifier (inherited)				
Parameter				
specifier		InstanceSpecifier to an PortPrototype of an DiagnosticDataIdentifierGenericInterface		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-370 ara::diag::GenericDataIdentifier::GenericDataIdentifier

4.40.3 ara::diag::GenericDataIdentifier::~GenericDataIdentifier

Prototype				
`ara::diag::GenericDataIdentifier::~GenericDataIdentifier () noexcept`				
Description				
Destructor				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-371 ara::diag::GenericDataIdentifier::~GenericDataIdentifier

4.40.4 ara::diag::GenericDataIdentifier::operator=

Prototype
`ara::diag::GenericDataIdentifier & ara::diag::GenericDataIdentifier::operator= ( ara::diag::GenericDataIdentifier && other ) & noexcept`
Description

Move assigns an instance of GenericDataIdentifier				
Parameter				
other		The other object		
Returns				
ara::diag::GenericDataIdentifier &		Reference to self		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-372 ara::diag::GenericDataIdentifier::operator=

4.40.5 ara::diag::GenericDataIdentifier::OperationOutput

Prototype		
`struct ara::diag::GenericDataIdentifier::OperationOutput`		
Description		
Response data of positive response message		
Members		
ara::core::Vector<std::uint8_t>	response_data	Content of positive response message (without DataIdentifier)

Table 4-373 ara::diag::GenericDataIdentifier::OperationOutput

4.40.6 ara::diag::GenericDataIdentifier::Offer

Prototype				
`ara::core::Result<void>` `ara::diag::GenericDataIdentifier::Offer () noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-374 ara::diag::GenericDataIdentifier::Offer

4.40.7 ara::diag::GenericDataIdentifier::Read

Prototype	

`ara::core::Future<OperationOutput>`
`ara::diag::GenericDataIdentifier::Read (`
`  std::uint16_t data_identifier,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`	

Description	
Called for ReadDataByIdentifier request for this DiagnosticDataIdentifier	
Parameter	
data_identifier	the corresponding DataIdentifier
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<OperationOutput>	A future with either OperationOutput (for a positive response message) or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-375 ara::diag::GenericDataIdentifier::Read

4.40.8 ara::diag::GenericDataIdentifier::StopOffer

Prototype				
`void ara::diag::GenericDataIdentifier::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-376 ara::diag::GenericDataIdentifier::StopOffer

4.40.9 ara::diag::GenericDataIdentifier::Write

Prototype	

`ara::core::Future<void>`
`ara::diag::GenericDataIdentifier::Write (`
`  std::uint16_t data_identifier,`
`  ara::core::Span<std::uint8_t> request_data,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`	

Description	
Called for WriteDataByIdentifier request for this DiagnosticDataIdentifier	
Parameter	
data_identifier	the corresponding DataIdentifier
request_data	Content of request message (without DataIdentifier)
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<void>	A future with either void (for a positive response message) or an UDS NRC value (for a negative response message)
Errors	

Any Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-377 ara::diag::GenericDataIdentifier::Write

based on template version 1.00.00

4.41 ara::diag::GenericRoutine

Description	
Generic Routine interface	
Include	**Generalize**
ara/diag/generic_routine.h	
Constructors	
GenericRoutine	Move constructs an instance of GenericRoutine
GenericRoutine	Constructor for GenericRoutine (inherited)
Operators	
operator=	Move assigns an instance of GenericRoutine
Types	
OperationOutput	Response data of positive response message
MemberFunctions	
Offer	This Offer will enable the DM to forward request messages to this handler
RequestResults	Called for RoutineControl with SubFunction RequestResults request for this DiagnosticRoutineIdentifier
Start	Called for RoutineControl with SubFunction Start request for this DiagnosticRoutineIdentifier
Stop	Called for RoutineControl with SubFunction Stop request for this DiagnosticRoutineIdentifier
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-378 ara::diag::GenericRoutine

4.41.1 ara::diag::GenericRoutine::GenericRoutine

Prototype

```
ara::diag::GenericRoutine::GenericRoutine (
  ara::diag::GenericRoutine && other
) noexcept
```

Description

Move constructs an instance of GenericRoutine

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
The port passed as parameter is not offered

Table 4-379 ara::diag::GenericRoutine::GenericRoutine

4.41.2 ara::diag::GenericRoutine::GenericRoutine

Prototype				
`ara::diag::GenericRoutine::GenericRoutine ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for GenericRoutine (inherited)				
Parameter				
specifier		InstanceSpecifier to an PortPrototype of an DiagnosticRoutineGenericInterface		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-380 ara::diag::GenericRoutine::GenericRoutine

4.41.3 ara::diag::GenericRoutine::~GenericRoutine

Prototype				
`ara::diag::GenericRoutine::~GenericRoutine () noexcept`				
Description				
Destructor of class GenericRoutine				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-381 ara::diag::GenericRoutine::~GenericRoutine

4.41.4 ara::diag::GenericRoutine::operator=

Prototype				

`ara::diag::GenericRoutine &`
`ara::diag::GenericRoutine::operator= (`
`  ara::diag::GenericRoutine && other`
`) & noexcept`				

Description				
Move assigns an instance of GenericRoutine				
Parameter				
other	The other object			
Returns				
ara::diag::GenericRoutine &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-382 ara::diag::GenericRoutine::operator=

4.41.5 ara::diag::GenericRoutine::OperationOutput

Prototype		
`struct ara::diag::GenericRoutine::OperationOutput`		
Description		
Response data of positive response message		
Members		
ara::core::Vector<std::uint8_t>	response_data	Content of positive response message (without RoutineIdentifier)

Table 4-383 ara::diag::GenericRoutine::OperationOutput

4.41.6 ara::diag::GenericRoutine::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::GenericRoutine::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	

ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-384 ara::diag::GenericRoutine::Offer

4.41.7 ara::diag::GenericRoutine::RequestResults

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::GenericRoutine::RequestResults (
   std::uint16_t routine_id,
   ara::core::Span<std::uint8_t> request_data,
   ara::diag::MetaInfo meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for RoutineControl with SubFunction RequestResults request for this DiagnosticRoutineIdentifier

Parameter

routine_id	the corresponding RoutineIdentifier
request_data	Content of request message (without RoutineIdentifier)
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<OperationOutput>	A future with either OperationOutput (for a positive response message) or an UDS NRC value (for a negative response message)

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-385 ara::diag::GenericRoutine::RequestResults

4.41.8 ara::diag::GenericRoutine::Start

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::GenericRoutine::Start (
  std::uint16_t routine_id,
  ara::core::Span<std::uint8_t> request_data,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for RoutineControl with SubFunction Start request for this DiagnosticRoutineIdentifier

Parameter

routine_id	the corresponding RoutineIdentifier
request_data	Content of request message (without RoutineIdentifier)
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<OperationOutput>	A future with either OperationOutput (for a positive response message) or an UDS NRC value (for a negative response message)

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints
N/A
Steady
TRUE

Table 4-386 ara::diag::GenericRoutine::Start

4.41.9 ara::diag::GenericRoutine::Stop

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::GenericRoutine::Stop (
  std::uint16_t routine_id,
  ara::core::Span<std::uint8_t> request_data,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for RoutineControl with SubFunction Stop request for this DiagnosticRoutineIdentifier

Parameter

routine_id	the corresponding RoutineIdentifier
request_data	Content of request message (without RoutineIdentifier)
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<OperationOutput>	A future with either OperationOutput (for a positive response message) or an UDS NRC value (for a negative response message)

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints
N/A
Steady
TRUE

Table 4-387 ara::diag::GenericRoutine::Stop

4.41.10 ara::diag::GenericRoutine::StopOffer

Prototype				
`void ara::diag::GenericRoutine::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-388 ara::diag::GenericRoutine::StopOffer

based on template version 1.00.00

4.42 ara::diag::GenericUDSService

Description	
Generic UDS service interface	
Include	**Generalize**
ara/diag/generic_uds_service.h	
Constructors	
GenericUDSService	Move constructs an instance of GenericUDSService
GenericUDSService	Constructor of GenericUDSService (inherited)
Operators	
operator=	Move assigns an instance of GenericUDSService
Types	
OperationOutput	Response data of positive response message
MemberFunctions	
HandleMessage	Called for any request message
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-389 ara::diag::GenericUDSService

4.42.1 ara::diag::GenericUDSService::GenericUDSService

Prototype

```
ara::diag::GenericUDSService::GenericUDSService (
  ara::diag::GenericUDSService && other
) noexcept
```

Description

Move constructs an instance of GenericUDSService

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-390 ara::diag::GenericUDSService::GenericUDSService

4.42.2 ara::diag::GenericUDSService::GenericUDSService

Prototype				

`ara::diag::GenericUDSService::GenericUDSService (`
`  const ara::core::InstanceSpecifier & specifier`
`)`				

Description				
Constructor of GenericUDSService (inherited)				
Parameter				
specifier	An InstanceSpecifier linking this instance with the PortPrototype in the manifest			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-391 ara::diag::GenericUDSService::GenericUDSService

4.42.3 ara::diag::GenericUDSService::~GenericUDSService

Prototype				
`ara::diag::GenericUDSService::~GenericUDSService () noexcept`				
Description				
Destructor of GenericUDSService				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-392 ara::diag::GenericUDSService::~GenericUDSService

4.42.4 ara::diag::GenericUDSService::operator=

Prototype

`ara::diag::GenericUDSService &`
`ara::diag::GenericUDSService::operator= (`
`  ara::diag::GenericUDSService && other`
`) & noexcept`

Description

Move assigns an instance of GenericUDSService				
Parameter				
other	The other object			
Returns				
ara::diag::GenericUDSService &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-393 ara::diag::GenericUDSService::operator=

4.42.5 ara::diag::GenericUDSService::OperationOutput

Prototype		
`struct ara::diag::GenericUDSService::OperationOutput`		
Description		
Response data of positive response message		
Members		
ara::core::Vector<std::uint8_t>	response_data	Content of positive response message (without SID)

Table 4-394 ara::diag::GenericUDSService::OperationOutput

4.42.6 ara::diag::GenericUDSService::HandleMessage

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::GenericUDSService::HandleMessage (
  std::uint8_t sid,
  ara::core::Span<std::uint8_t> request_data,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for any request message

Parameter	
sid	Diagnostic Request Service Identifier.
request_data	Diagnostic request data (starting after SID).
meta_info	MetaInfo of the request.

cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<OperationOutput>	A future with either a OperationOutput (Diagnostic response data (starting after SID)) or an error
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-395 ara::diag::GenericUDSService::HandleMessage

4.42.7 ara::diag::GenericUDSService::Offer

Prototype

```
ara::core::Result<void>
ara::diag::GenericUDSService::Offer () noexcept
```

Description

This Offer will enable the DM to forward request messages to this handler

Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-396 ara::diag::GenericUDSService::Offer

4.42.8 ara::diag::GenericUDSService::StopOffer

Prototype				
`void ara::diag::GenericUDSService::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A	N/A			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-397 ara::diag::GenericUDSService::StopOffer

based on template version 1.00.00

4.43 ara::diag::Indicator

Description	
DiagnosticIndicatorInterface provides functionality for handling indicators	
Include	**Generalize**
ara/diag/indicator.h	
Constructors	
Indicator	Move constructs instance of class
Indicator	Constructor for DiagnosticIndicatorInterface
Operators	
operator=	Move assigns instance of class
Types	
IndicatorType	Represents the state of an indicator
MemberFunctions	
GetIndicator	Get current Indicator
SetNotifier	Register a notifier function which is called if the indicator is updated

Table 4-398 ara::diag::Indicator

4.43.1 ara::diag::Indicator::Indicator

Prototype

```
ara::diag::Indicator::Indicator (
  ara::diag::Indicator &&
) noexcept
```

Description

Move constructs instance of class

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 4-399 ara::diag::Indicator::Indicator

4.43.2 ara::diag::Indicator::Indicator

Prototype

```
ara::diag::Indicator::Indicator (
   const ara::core::InstanceSpecifier & specifier
)
```

Description
Constructor for DiagnosticIndicatorInterface

Parameter	
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticIndicatorInterface

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-400 ara::diag::Indicator::Indicator

4.43.3 ara::diag::Indicator::~Indicator

Prototype
`ara::diag::Indicator::~Indicator () noexcept`

Description
Destructor of DiagnosticIndicatorInterface

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-401 ara::diag::Indicator::~Indicator

4.43.4 ara::diag::Indicator::operator=

Prototype

```
ara::diag::Indicator &
ara::diag::Indicator::operator= (
   ara::diag::Indicator &&
) & noexcept
```

Description
Move assigns instance of class

Parameter	
Returns	
ara::diag::Indicator &	Reference to self
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-402 ara::diag::Indicator::operator=

4.43.5 ara::diag::Indicator::IndicatorType

Prototype	
`ara::diag::Indicator::IndicatorType`	
Description	
Represents the state of an indicator	
Type	
std::uint8_t	
Enumerations	
kOff	0x00
kContinuous	0x01
kBlinking	0x02
kBlinkingAndContinuous	0x03
kSlowFlash	0x04
kFastFlash	0x05
kOnDemand	0x06
kShort	0x07

Table 4-403 ara::diag::Indicator::IndicatorType

4.43.6 ara::diag::Indicator::GetIndicator

Prototype
`ara::core::Result<IndicatorType>` `ara::diag::Indicator::GetIndicator ()`
Description
Get current Indicator
Parameter

N/A	N/A
Returns	
ara::core::Result<IndicatorType>	Result contains the "current indicator" if operation succeeds, otherwise error code that shall not be evaluated
Errors	
Any/Custom Do not evaluate the error code just that the result contains an error	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-404 ara::diag::Indicator::GetIndicator

4.43.7 ara::diag::Indicator::SetNotifier

Prototype

```
ara::core::Result<void>
ara::diag::Indicator::SetNotifier (
   std::function<void (IndicatorType)> notifier
)
```

Description	
Register a notifier function which is called if the indicator is updated	
Parameter	
notifier	notifier function
Detailed Description	
If successful, a previously set notifier gets overwritten.	
Returns	
ara::core::Result<void>	void Result
Errors	
ara::diag::DiagReportingErrc::kGenericError The instance specifier has not been registered	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the destruction of this Indicator has completed or until it is replaced by a new notifier function.				

Whenever a previously set notifier is in execution, the SetNotifier method is blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).	
Steady	
TRUE	

Table 4-405 ara::diag::Indicator::SetNotifier

based on template version 1.00.00

4.44 ara::diag::MetaInfo

Description	
Type definition of a MetaInfo class	
Include	**Generalize**
ara/diag/meta_info.h	
Constructors	
MetaInfo	Move constructor
Operators	
operator=	Move assignment
MemberFunctions	
GetValue	Returns the stored value for the given key

Table 4-406 ara::diag::MetaInfo

4.44.1 ara::diag::MetaInfo::MetaInfo

Prototype

```
ara::diag::MetaInfo::MetaInfo (
  ara::diag::MetaInfo &&
) noexcept
```

Description

Move constructor

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Steady

TRUE

Table 4-407 ara::diag::MetaInfo::MetaInfo

4.44.2 ara::diag::MetaInfo::~MetaInfo

Prototype

```
ara::diag::MetaInfo::~MetaInfo () noexcept
```

Description

Destructs the meta info

Errors

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-408 ara::diag::MetaInfo::~MetaInfo

4.44.3 ara::diag::MetaInfo::operator=

Prototype	

`ara::diag::MetaInfo &`
`ara::diag::MetaInfo::operator= (`
`  ara::diag::MetaInfo &&`
`) & noexcept`	

Description	
Move assignment	
Parameter	
Returns	
ara::diag::MetaInfo &	Reference to the meta info
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-409 ara::diag::MetaInfo::operator=

4.44.4 ara::diag::MetaInfo::GetValue

Prototype

`ara::core::Result<ara::core::StringView>`
`ara::diag::MetaInfo::GetValue (`
`  const ara::core::String & key`
`) const noexcept`

Description
Returns the stored value for the given key

Parameter	
key	The requested key
Returns	
ara::core::Result<ara::core::String View>	The stored value, or an error result if the key does not exist
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				
Steady				
TRUE				

Table 4-410 ara::diag::MetaInfo::GetValue

based on template version 1.00.00

4.45 ara::diag::ReentrancyType

Prototype	
`ara::diag::ReentrancyType`	
Description	
Specifies the reentrancy types	
Type	
std::uint8_t	
Enumerations	
kFully	0x00
kNot	0x01

Table 4-411 ara::diag::ReentrancyType

based on template version 1.00.00

4.46 ara::diag::SecurityAccess

Description	
DiagnosticSecurityAccessInterface	
Include	**Generalize**
ara/diag/security_access.h	
Constructors	
SecurityAccess	Move constructs an instance of SecurityAccess
SecurityAccess	Constructor of SecurityAccess (inherited)
Operators	
operator=	Move assigns an instance of SecurityAccess
Types	
KeyCompareResultType	Represents the status of the key compare
MemberFunctions	
CompareKey	This method is called, when a diagnostic request has been finished, to notify about the outcome
GetSeed	Called for any request message
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-412 ara::diag::SecurityAccess

4.46.1 ara::diag::SecurityAccess::SecurityAccess

Prototype

```
ara::diag::SecurityAccess::SecurityAccess (
  ara::diag::SecurityAccess && other
) noexcept
```

Description

Move constructs an instance of SecurityAccess

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-413 ara::diag::SecurityAccess::SecurityAccess

4.46.2 ara::diag::SecurityAccess::SecurityAccess

Prototype				
`ara::diag::SecurityAccess::SecurityAccess ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor of SecurityAccess (inherited)				
Parameter				
specifier	InstanceSpecifier to a PortPrototype of a DiagnosticSecurityAccessInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-414 ara::diag::SecurityAccess::SecurityAccess

4.46.3 ara::diag::SecurityAccess::~SecurityAccess

Prototype				
`ara::diag::SecurityAccess::~SecurityAccess () noexcept`				
Description				
Destructor of SecurityAccess				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-415 ara::diag::SecurityAccess::~SecurityAccess

4.46.4 ara::diag::SecurityAccess::operator=

Prototype
`ara::diag::SecurityAccess & ara::diag::SecurityAccess::operator= ( ara::diag::SecurityAccess && other ) & noexcept`
Description

Move assigns an instance of SecurityAccess				
Parameter				
other	The other object			
Returns				
ara::diag::SecurityAccess &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-416 ara::diag::SecurityAccess::operator=

4.46.5 ara::diag::SecurityAccess::KeyCompareResultType

Prototype	
`ara::diag::SecurityAccess::KeyCompareResultType`	
Description	
Type alias for event status bit	
Type	
std::uint8_t	
Enumerations	
kKeyValid	0x00
kKeyInvalid	0x01

Table 4-417 ara::diag::SecurityAccess::KeyCompareResultType

4.46.6 ara::diag::SecurityAccess::CompareKey

Prototype

```
ara::core::Future<KeyCompareResultType>
ara::diag::SecurityAccess::CompareKey (
   ara::core::Span<std::uint8_t> key,
   ara::diag::MetaInfo meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description	
This method is called, when a diagnostic request has been finished, to notify about the outcome	
Parameter	
key	The key to be validated
meta_info	MetaInfo of the request.

cancellation_handler	Set if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<KeyCompare ResultType>	A future with either status of the key compare or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-418 ara::diag::SecurityAccess::CompareKey

4.46.7 ara::diag::SecurityAccess::GetSeed

Prototype

```
ara::core::Future<ara::core::Vector<std::uint8_t> >
ara::diag::SecurityAccess::GetSeed (
  ara::core::Span<std::uint8_t> security_access_data_record,
  ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for any request message

Parameter

security_access_data_record	Diagnostic request data.
meta_info	MetaInfo of the request.
cancellation_handler	The call back in case the request is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<ara::core::Vector<std::uint8_t> >	A future with either provided seed or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-419 ara::diag::SecurityAccess::GetSeed

4.46.8 ara::diag::SecurityAccess::Offer

Prototype	
`ara::core::Result<void>` `ara::diag::SecurityAccess::Offer () noexcept`	
Description	
This Offer will enable the DM to forward request messages to this handler	
Parameter	
N/A	N/A
Returns	
ara::core::Result<void>	Positive result if service was offered, error if offer failed
Errors	
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-420 ara::diag::SecurityAccess::Offer

4.46.9 ara::diag::SecurityAccess::StopOffer

Prototype	
`void` `ara::diag::SecurityAccess::StopOffer () const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Parameter	
N/A	N/A
Returns	
void	N/A

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-421 ara::diag::SecurityAccess::StopOffer

based on template version 1.00.00

4.47 ara::diag::ServiceValidation

Description	
DiagnosticServiceValidationInterface	
Include	**Generalize**
ara/diag/service_validation.h	
Constructors	
ServiceValidation	Move constructs an instance of ServiceValidation
ServiceValidation	Constructor of ServiceValidation (inherited)
Operators	
operator=	Move assigns an instance of ServiceValidation
Types	
ConfirmationStatusType	Represents the status of the service processing
MemberFunctions	
Confirmation	This method is called, when a diagnostic request has been finished, to notify about the outcome
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM
Validate	Called for any request message

Table 4-422 ara::diag::ServiceValidation

4.47.1 ara::diag::ServiceValidation::ServiceValidation

Prototype

```
ara::diag::ServiceValidation::ServiceValidation (
  ara::diag::ServiceValidation && other
) noexcept
```

Description

Move constructs an instance of ServiceValidation

Parameter

other	The other object

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

The port passed as parameter is not offered

Table 4-423 ara::diag::ServiceValidation::ServiceValidation

4.47.2 ara::diag::ServiceValidation::ServiceValidation

Prototype				
`ara::diag::ServiceValidation::ServiceValidation ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor of ServiceValidation (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DiagnosticServiceValidationInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-424 ara::diag::ServiceValidation::ServiceValidation

4.47.3 ara::diag::ServiceValidation::~ServiceValidation

Prototype				
`ara::diag::ServiceValidation::~ServiceValidation () noexcept`				
Description				
Destructor of ServiceValidation				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-425 ara::diag::ServiceValidation::~ServiceValidation

4.47.4 ara::diag::ServiceValidation::operator=

Prototype
`ara::diag::ServiceValidation & ara::diag::ServiceValidation::operator= ( ara::diag::ServiceValidation && other ) & noexcept`
Description

Move assigns an instance of ServiceValidation				
Parameter				
other	The other object			
Returns				
ara::diag::ServiceValidation &	Reference to self			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-426 ara::diag::ServiceValidation::operator=

4.47.5 ara::diag::ServiceValidation::ConfirmationStatusType

Prototype	
`ara::diag::ServiceValidation::ConfirmationStatusType`	
Description	
Represents the status of the service processing	
Type	
std::uint8_t	
Enumerations	
ResPosOk	0x00
kResPosNotOk	0x01
kResNegOk	0x02
kResNegNotOk	0x03
kResPosSuppressed	0x04
kResNegSuppressed	0x05
kCanceled	0x06
kNoProcessingNoResponse	0x07

Table 4-427 ara::diag::ServiceValidation::ConfirmationStatusType

4.47.6 ara::diag::ServiceValidation::Confirmation

Prototype

```
ara::core::Future<void>
ara::diag::ServiceValidation::Confirmation (
   ara::diag::ServiceValidation::ConfirmationStatusType status,
   ara::diag::MetaInfo meta_info
)
```

Description				
This method is called, when a diagnostic request has been finished, to notify about the outcome				
Parameter				
status	status/outcome of the service processing.			
meta_info	MetaInfo of the request.			
Returns				
ara::core::Future<void>	A future of type void or an UDS NRC value (for a negative response message)			
Errors				
Any Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-428 ara::diag::ServiceValidation::Confirmation

4.47.7 ara::diag::ServiceValidation::Offer

Prototype				
`ara::core::Result<void>` `ara::diag::ServiceValidation::Offer () noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A	N/A			
Returns				
ara::core::Result<void>	Positive result if service was offered, error if offer failed			
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 4-429 ara::diag::ServiceValidation::Offer

4.47.8 ara::diag::ServiceValidation::StopOffer

Prototype				
`void ara::diag::ServiceValidation::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A	N/A			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-430 ara::diag::ServiceValidation::StopOffer

4.47.9 ara::diag::ServiceValidation::Validate

Prototype	

`ara::core::Future<void>`
`ara::diag::ServiceValidation::Validate (`
`  ara::core::Span<std::uint8_t> request_data,`
`  ara::diag::MetaInfo meta_info`
`)`	

Description	
Called for any request message	
Parameter	
request_data	Diagnostic request data (including SID).
meta_info	MetaInfo of the request.
Returns	
ara::core::Future<void>	A future of type void or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-431 ara::diag::ServiceValidation::Validate

based on template version 1.00.00

4.48 ara::diag::UploadService

Description	
Upload service interface	
Include	**Generalize**
ara/diag/upload.h	
Constructors	
UploadService	Move constructs an instance of UploadService
UploadService	Constructor for UploadService (inherited)
Operators	
operator=	Move assigns an instance of UploadService
Types	
OperationOutput	Response data of positive response message
MemberFunctions	
Offer	This Offer will enable the DM to forward request messages to this handler
RequestUpload	Called for RequestUpload
RequestUploadExit	Called for RequestTransferExit
StopOffer	This StopOffer will disable the forwarding of request messages from DM
UploadData	Called for TransferData following a previous RequestUpload

Table 4-432 ara::diag::UploadService

4.48.1 ara::diag::UploadService::UploadService

Prototype				
`ara::diag::UploadService::UploadService ( ara::diag::UploadService && other ) noexcept`				
Description				
Move constructs an instance of UploadService				
Parameter				
other	The other object			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-433 ara::diag::UploadService::UploadService

4.48.2 ara::diag::UploadService::UploadService

Prototype				
`ara::diag::UploadService::UploadService ( const ara::core::InstanceSpecifier & specifier )`				
Description				
Constructor for UploadService (inherited)				
Parameter				
specifier	InstanceSpecifier to an PortPrototype of an DownloadServiceInterface			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-434 ara::diag::UploadService::UploadService

4.48.3 ara::diag::UploadService::~UploadService

Prototype				
`ara::diag::UploadService::~UploadService () noexcept`				
Description				
Destructor of class UploadService				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-435 ara::diag::UploadService::~UploadService

4.48.4 ara::diag::UploadService::operator=

Prototype
`ara::diag::UploadService & ara::diag::UploadService::operator= ( ara::diag::UploadService && other ) & noexcept`
Description

Move assigns an instance of UploadService				
Parameter				
other		The other object		
Returns				
ara::diag::UploadService &		Reference to self		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
The port passed as parameter is not offered				

Table 4-436 ara::diag::UploadService::operator=

4.48.5 ara::diag::UploadService::OperationOutput

Prototype		
`struct ara::diag::UploadService::OperationOutput`		
Description		
Response data of positive response message		
Members		
ara::core::Vector<std::uint8_t>	response_data	Content of positive response message (without SID)

Table 4-437 ara::diag::UploadService::OperationOutput

4.48.6 ara::diag::UploadService::Offer

Prototype				
`ara::core::Result<void>` `ara::diag::UploadService::Offer () noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered if service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-438 ara::diag::UploadService::Offer

4.48.7 ara::diag::UploadService::RequestUpload

Prototype	

`ara::core::Future<void>`
`ara::diag::UploadService::RequestUpload (`
`  std::uint8_t data_format_identifier,`
`  std::uint8_t address_and_length_format_identifier,`
`  ara::core::Span<std::uint8_t> memory_address_and_size,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`	

Description	
Called for RequestUpload	
Parameter	
data_format_identifier	UDS dataFormat Identifier
address_and_length_format_identifier	UDS addressAndLengthFormatIdentifier
memory_address_and_size	memoryAddress and memorySize part of the request
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<void>	A future with either void (for a positive response message) or an UDS NRC value (for a negative response message)
Errors	
Any Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-439 ara::diag::UploadService::RequestUpload

4.48.8 ara::diag::UploadService::RequestUploadExit

Prototype

```
ara::core::Future<OperationOutput>
ara::diag::UploadService::RequestUploadExit (
   ara::core::Span<std::uint8_t> transferRequestParameterRecord,
   ara::diag::MetaInfo meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called for RequestTransferExit

Parameter

transferRequestParameterRecord	This parameter record contains parameter(s), which are required by the server to support the transfer of data. Format and length of this parameter(s) are vehicle manufacturer specific.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<OperationOutput>	returns a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message) Data in OperationOutput.response_data will be placed after SID as transferResponseParameterRecord in the positive response.

Errors

Any Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints

N/A

Table 4-440 ara::diag::UploadService::RequestUploadExit

4.48.9 ara::diag::UploadService::StopOffer

Prototype

`void` `ara::diag::UploadService::StopOffer () const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-441 ara::diag::UploadService::StopOffer

4.48.10 ara::diag::UploadService::UploadData

Prototype	

`ara::core::Future<OperationOutput>`
`ara::diag::UploadService::UploadData (`
`  std::size_t num_bytes_to_return,`
`  ara::diag::MetaInfo meta_info,`
`  ara::diag::CancellationHandler cancellation_handler`
`)`	

Description	
Called for TransferData following a previous RequestUpload	
Parameter	
num_bytes_to_return	number of bytes DM accepts (due to its internal buffer) for this chunk.
meta_info	contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	

ara::core::Future<OperationOutput>	a Future, which either gets readied to OperationOutput (transferResponseParameterRecord for a positive response message) or readied with ErrorCode from DiagUdsNrcErrc (for a negative response message). Data in OperationOutput.response_data will be placed after blockSequenceCounter as transferResponseParameterRecord in the positive response.

Errors				
Any Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				

Table 4-442 ara::diag::UploadService::UploadData

based on template version 1.00.00

4.49 Free Functions

4.49.1 amsr::diag::AddIsReadyNotifier

Prototype	
`void amsr::diag::AddIsReadyNotifier ( amsr::diag::LibraryIsReadyNotifier notifier ) noexcept`	
Include	
amsr/diag/lib_ara_diag.h	
Description	
Registers an additional notifier function for ara::diag library readiness for interaction	
Parameter	
notifier	The function to be called once the daemon is ready to interact
Detailed Description	
This callout does not guarantee that the daemon is still available, but notifies for the first feasible point in time to try interacting with it	
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function (using the SetIsReadyNotifier function).

Whenever a previously set notifier is in execution, AddIsReadyNotifier and SetIsReadyNotifier are blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Table 4-443 amsr::diag::AddIsReadyNotifier

4.49.2 amsr::diag::AddIsReadyNotifier

Prototype
`void amsr::diag::AddIsReadyNotifier ( amsr::diag::LibraryStateNotifier notifier ) noexcept`
Include

amsr/diag/lib_ara_diag.h				
Description				
Registers an additional notifier function for ara::diag library readiness for interaction				
Parameter				
notifier	The function to be called once the daemon is ready to interact or disconnected			
Detailed Description				
This callout notifies that either the ara::diag library is connected to the daemon and ready for use or that it is disconnected from the daemon. The reason of notification is conveyed in the callback function's parameter.				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
The passed in function must not throw any exceptions. The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function (using the SetIsReadyNotifier function). Whenever a previously set notifier is in execution, AddIsReadyNotifier and SetIsReadyNotifier are blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).				

Table 4-444 amsr::diag::AddIsReadyNotifier

4.49.3 amsr::diag::IsReady

Prototype	
`bool` `amsr::diag::IsReady () noexcept`	
Include	
amsr/diag/lib_ara_diag.h	
Description	
Reports whether the DM daemon is connected with the ara::diag library	
Parameter	
N/A	N/A
Returns	
bool	true - if the library is ready to interact, false - otherwise

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 4-445 amsr::diag::IsReady

4.49.4 amsr::diag::SetIsReadyNotifier

Prototype

```
void
amsr::diag::SetIsReadyNotifier (
   amsr::diag::LibraryIsReadyNotifier notifier
) noexcept
```

Include

amsr/diag/lib_ara_diag.h

Description

Registers a single notifier function for ara::diag library readiness for interaction

Parameter

notifier	The function to be called once the daemon is ready to interact

Detailed Description

> This callout does not guarantee that the daemon is still available, but notifies for the first feasible point in time to try interacting with it
> The notifiers previously added using AddIsReadyNotifier or SetIsReadyNotifier will be replaced.

Returns

void	N/A

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function.

Whenever a previously set notifier is in execution, AddIsReadyNotifier and SetIsReadyNotifier are blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Table 4-446 amsr::diag::SetIsReadyNotifier

4.49.5 amsr::diag::SetIsReadyNotifier

Prototype

```
void
amsr::diag::SetIsReadyNotifier (
   amsr::diag::LibraryStateNotifier notifier
) noexcept
```

Include

amsr/diag/lib_ara_diag.h

Description

Registers a single notifier function for ara::diag library readiness for interaction

Parameter

notifier	The function to be called once the daemon is ready to interact or disconnected

Detailed Description

- This callout notifies that either the ara::diag library is connected to the daemon and ready for use or that it is disconnected from the daemon. The reason of notification is conveyed in the callback function's parameter.
- The notifiers previously added using AddIsReadyNotifier or SetIsReadyNotifier will be replaced.

Returns

void	N/A

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the deinitalization of the diagnostic library has completed or until it is replaced by a new notifier function.

Whenever a previously set notifier is in execution, AddIsReadyNotifier and SetIsReadyNotifier are blocking until the execution of that notifier is completed. Consider this for the implementation of the notifiers (deadlock hazard).

Table 4-447 amsr::diag::SetIsReadyNotifier

4.49.6 amsr::diag::AsUdsDtcStatusByte

Prototype

```
std::uint8_t
amsr::diag::AsUdsDtcStatusByte (
   amsr::diag::EventStatusByte event_status_byte
) noexcept
```

Include	
amsr/diag/state_types/event_status_converter.h	
Description	
Converts the given EventStatusByte to a UDS DTC status byte	
Parameter	
event_status_byte	The EventStatusByte to convert
Returns	
std::uint8_t	The status as a UDS DTC status byte
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 4-448 amsr::diag::AsUdsDtcStatusByte

4.49.7 amsr::diag::conversation::OverrideS3Timeout

Prototype

```
ara::core::Result<void>
amsr::diag::conversation::OverrideS3Timeout (
   std::chrono::milliseconds timeout,
   ara::diag::MetaInfo const& meta_info
) noexcept
```

Include	
amsr/diag/conversation/s3_timeout_config.h	
Description	
Set the s3 timer timeout to the time specified	
Parameter	
timeout	The duration of the timeout to be set
meta_info	The meta information
Returns	
ara::core::Result<void>	Result contains the currently configured s3 timeout if operation succeeds, otherwise error code
Errors	

The timeout entered is invalid or the meta-info does not contain an instance handle				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-449 amsr::diag::conversation::OverrideS3Timeout

4.49.8 amsr::diag::AsUdsNrc

Prototype

```
constexpr ara::core::ErrorCode
amsr::diag::AsUdsNrc (
  std::uint8_t uds_nrc,
  ara::diag::DiagUdsNrcErrorDomain::SupportDataType data = {}
) noexcept
```

Include

amsr/diag/diag_uds_nrc_error_domain.h

Description

Creates an error code for the DiagUdsNrcErrorDomain from a raw uint8_t value

Parameter

uds_nrc	The raw UDS-NRC value
data	Optional vendor defined support data.

Returns

ara::core::ErrorCode	The error with the UDS NRC as code

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE

Constraints

N/A

Table 4-450

4.50 Typed DataElement

Description	
Typed DataElement interface	
Include	**Generalize**
ara/diag/<DataElementPortInterfaceName>.h	
Constructors	
<DataElementPortInterfaceName>	Constructor for the Typed DataElement
MemberClasses	
Output type	Data elements' value(s)
MemberFunctions	
Read	Called for reading a DataElement
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-451 <DataElementPortInterfaceName>

4.50.1 <DataElementPortInterfaceName>::Output type

Prototype		
`struct namespace1_of_portinterface::namespace2_data_element_of_portinterface::<DataElementPortInterfaceName>::Output`		
Description		
Data elements' value(s)		
Members		
For each data type there is one member. The data types supported can be found in 6.3.2, but please be aware that DataElement cannot return any Vector type. The DataElementName is configured as `ARGUMENT-DATA-PROTOTYPE` in the model.		
<DataType>	<DataElementName>	data element value

Table 4-452 DataElement::Output

4.50.2 <DataElementPortInterfaceName> constructor

Prototype	
`<DataElementPortInterfaceName>::<DataElementPortInterfaceName> ( ara::core::InstanceSpecifier const& instance_specifier)`	
Description	
Constructor of typed DataElement interface	
Parameter	
instance_specifier	InstanceSpecifier to a PortPrototype
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-453 <DataElementPortInterfaceName>::<DataElementPortInterfaceName>

4.50.3 <DataElementPortInterfaceName> destructor

Prototype				
`<DataElementPortInterfaceName>::~<DataElementPortInterfaceName> ( ) noexcept`				
Description				
Destructor of typed DataElement interface				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-454 <DataElementPortInterfaceName>::~<DataElementPortInterfaceName>

4.50.4 <DataElementPortInterfaceName>::Offer

Prototype				
`ara::core::Result<void> <DataElementPortInterfaceName>::Offer ( ) noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Returns				
ara::core::Result<void>	Positive result if service was offered, error if offer failed			
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered	If service has been offered already.			
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-455 <DataElementPortInterfaceName>::Offer

4.50.5 <DataElementPortInterfaceName>::Read

Prototype				
`ara::core::Future<Output> <DataElementPortInterfaceName>::Read ( ara::diag::MetaInfo meta_info, ara::diag::CancellationHandler cancellation_handler)`				
Description				
Called whenever the data associated with this port is needed.				
Parameter				
meta_info	Contains additional meta information.			
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.			
Returns				
ara::core::Future<Output>	A Result with either Output (for a positive response message) or an UDS NRC value (for a negative response message)			
Errors				
Application responded with a UDS NRC value from DiagUdsNrcErrc				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-456 <DataElementPortInterfaceName>::Read

4.50.6 <DataElementPortInterfaceName>::StopOffer

Prototype	
`void <DataElementPortInterfaceName>::StopOffer ( ) const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Returns	
void	N/A
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-457 <DataElementPortInterfaceName>::StopOffer

based on template version 1.00.00

4.51 Typed DataIdentifier

Description	
Typed DataIdentifier interface	
Include	
ara/diag/<DataIdentifierPortInterfaceName>.h	
Constructors	
<DataIdentifierPortInterfaceName>	Constructor for the Typed DataIdentifier
MemberClasses	
Output type	Response data
MemberFunctions	
Read	Called whenever the data associated with this port is needed
Write	Called whenever the data associated with this port needs to be written
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-458 <DataIdentifierPortInterfaceName>

4.51.1 <DataIdentifierPortInterfaceName>::Output type

Prototype		
`struct namespace1_of_portinterface::namespace2_of_portinterface::<DataIdentifierPortInterfaceName>::Output`		
Description		
Response data		
Members		
For each data type there is one member. The data types supported can be found in 6.3.2. The DataElementName is configured as `ARGUMENT-DATA-PROTOTYPE` in the model.		
<DataType>	<DataElementName>	data element value

Table 4-459 <DataIdentifierPortInterfaceName>::Output

4.51.2 <DataIdentifierPortInterfaceName> constructor

Prototype
`explicit namespace1_of_portinterface::namespace2_of_portinterface::<DataIdentifierPortInterfaceName>::<DataIdentifierPortInterfaceName>(ara::core::InstanceSpecifier const& instance_specifier)`
Description
Constructor of typed DataIdentifier interface

Parameter	
instance_specifier	InstanceSpecifier to a PortPrototype
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-460 <DataIdentifierPortInterfaceName> constructor

4.51.3 <DataIdentifierPortInterfaceName> destructor

Prototype
`<DataIdentifierPortInterfaceName>::~<DataIdentifierPortInterfaceName> ( )`
Description
Destructor of typed DataIdentifier interface
Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-461 <DataIdentifierPortInterfaceName>::~<DataIdentifierPortInterfaceName>

4.51.4 <DataIdentifierPortInterfaceName>::Read

Prototype

```
ara::core::Future<Output>
<DataIdentifierPortInterfaceName>::Read (
   ara::diag::MetaInfo meta_info,
   ara::diag::CancellationHandler cancellation_handler
)
```

Description

Called when the data identifier content needs to be acquired.

Parameter	
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future<Output>	A Result with either Output (for a positive response message) or an UDS NRC value (for a negative response message)
Errors	
Application responded with a UDS NRC value from DiagUdsNrcErrc	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-462 <DataIdentifierPortInterfaceName>::Read

4.51.5 <DataIdentifierPortInterfaceName>::Write

Prototype

```
virtual ara::core::Future<void> <DataIdentifierPortInterfaceName>::Write (
  namespace1_of_type_of_argument_data_prototype::Type1OfArgumentDataPrototype
  shortname1_of_argument_data_prototype,
  namespace2_of_type_of_argument_data_prototype::Type2OfArgumentDataPrototype
  shortname2_of_ArgumentDataPrototype, ..., ara::diag::MetaInfo meta_info,
  ara::diag::CancellationHandler cancellation_handler)
```

Description

Called when the data identifier content needs to be written.

Parameter

meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns

ara::core::Future<void>	An empty Result (for a positive response message) or an UDS NRC value (for a negative response message)

Errors

Application responded with a UDS NRC value from DiagUdsNrcErrc

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-463 <DataIdentifierPortInterfaceName>::Write

4.51.6 <DataIdentifierPortInterfaceName>::Offer

Prototype				
`ara::core::Result<void> <DataIdentifierPortInterfaceName>::Offer ( ) noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Parameter				
N/A		N/A		
Returns				
ara::core::Result<void>		Positive result if service was offered, error if offer failed		
Errors				
If service has been offered already				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-464 <DataIdentifierPortInterfaceName>::Offer

4.51.7 <DataIdentifierPortInterfaceName>::StopOffer

Prototype	
`void <DataIdentifierPortInterfaceName>::StopOffer ( ) const noexcept`	
Description	
This StopOffer will disable the forwarding of request messages from DM	
Returns	
void	N/A
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-465 <DataIdentifierPortInterfaceName>::StopOffer

4.52 Typed Routine

Description	
Typed Routine interface	
Include	**Generalize**
ara/diag/<RoutinePortInterfaceName>.h	
Constructors	
<RoutinePortInterfaceName>	Constructor for the Typed Routine
MemberClasses	
StartOutput type	Data elements' value(s) when routine is started
StopOutput type	Data elements' value(s) when routine is stopped
RequestResultsOutput type	Data elements' value(s) when result requested
MemberFunctions	
Start	Called for RoutineControl with SubFunction 'Start' request for this DiagnosticRoutineIdentifier
Stop	Called for RoutineControl with SubFunction 'Stop' request for this DiagnosticRoutineIdentifier
RequestResults	Called for RoutineControl with SubFunction 'RequestResults' request for this DiagnosticRoutineIdentifier
Offer	This Offer will enable the DM to forward request messages to this handler
StopOffer	This StopOffer will disable the forwarding of request messages from DM

Table 4-466 <RoutinePortInterfaceName>::<RoutinePortInterfaceName>

4.52.1 <RoutinePortInterfaceName>::StartOutput type

Prototype		
`namespace1_of_portinterface::namespace2_of_portinterface::<RoutinePortInterfaceName>::StartOutput`		
Description		
Data elements' value(s) returned when routine started.		
Members		
For each data type there is one member. The data types supported can be found in 6.3.2. The DataElementName is configured as `ARGUMENT-DATA-PROTOTYPE` in the model.		
<DataType>	<DataElementName>	data element value

Table 4-467 <RoutinePortInterfaceName>::StartOutput

4.52.2 <RoutinePortInterfaceName>::StopOutput type

Prototype

`namespace1_of_portinterface::namespace2_of_portinterface::<RoutinePortInterfaceName>::StopOutput`		
Description		
Data elements' value(s) returned when routine stops.		
Members		
For each data type there is one member. The data types supported can be found in 6.3.2. The DataElementName is configured as `ARGUMENT-DATA-PROTOTYPE` in the model.		
<DataType>	<DataElementName>	data element value

Table 4-468 <RoutinePortInterfaceName>::StopOutput

4.52.3 <RoutinePortInterfaceName>::RequestResultsOutput type

Prototype		
`struct namespace1_of_portinterface::namespace2_of_portinterface::<RoutinePortInterfaceName>::RequestResultsOutput`		
Description		
Data elements' value(s) returned when result requested.		
Members		
For each data type there is one member. The data types supported can be found in 6.3.2. The DataElementName is configured as `ARGUMENT-DATA-PROTOTYPE` in the model.		
<DataType>	<DataElementName>	data element value

Table 4-469 <RoutinePortInterfaceName>::RequestResultsOutput

4.52.4 <RoutinePortInterfaceName> constructor

Prototype				
`<RoutinePortInterfaceName>::<RoutinePortInterfaceName> ( ara::core::InstanceSpecifier const& instance_specifier)`				
Description				
Constructor of typed Routine interface.				
Parameter				
instance_specifier	InstanceSpecifier to a PortPrototype of the typed Routine			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
A port with that specifier is configured and no other port is right now instantiated with it				

Table 4-470 <RoutinePortInterfaceName>::<RoutinePortInterfaceName>

4.52.5 <RoutinePortInterfaceName> destructor

Prototype				
`<RoutinePortInterfaceName>::~<RoutinePortInterfaceName> ( )`				
Description				
Destructor of typed Routine interface				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 4-471 <RoutinePortInterfaceName>::~<RoutinePortInterfaceName>

4.52.6 <RoutinePortInterfaceName>::Start

Prototype

```
ara::core::Future<StartOutput>
namespace1_of_portinterface::namespace2_of_portinterface::
<RoutinePortInterfaceName>::Start(
   namespace1_of_type_of_argument_data_prototype::Type1OfArgumentDataPrototype
   shortname1_of_argument_data_prototype,
   namespace2_of_type_of_argument_data_prototype::Type2OfArgumentDataPrototype
   shortname2_of_ArgumentDataPrototype, ...  MetaInfo meta_info,
   CancellationHandler cancellation_handler)
```

Description

Called for RoutineControl with SubFunction 'Start' request for this DiagnosticRoutineIdentifier.

Parameter	
meta_info	Contains additional meta information.
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<StartOutput>	A Result with either StartOutput (for a positive response message) or an UDS NRC value (for a negative response message).
Errors	
UDS NRC value (for a negative response message)	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-472 <RoutinePortInterfaceName>::Start

4.52.7 <RoutinePortInterfaceName>::Stop

Prototype	
`ara::core::Future<StopOutput> namespace1_of_portinterface::namespace2_of_portinterface:: <RoutinePortInterfaceName>::Stop(namespace1_of_type_of_argument_data_prototype::Type1OfArgumentDataPrototype shortname1_of_argument_data_prototype, namespace2_of_type_of_argument_data_prototype::Type2OfArgumentDataPrototype shortname2_of_argument_data_prototype, ... MetaInfo meta_info, CancellationHandler cancellation_handler)`	
Description	
Called for RoutineControl with SubFunction 'Stop' request for this DiagnosticRoutineIdentifier.	
Parameter	
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.
Returns	
ara::core::Future<StopOutput>	A Result with either StopOutput (for a positive response message) or an UDS NRC value (for a negative response message)
Errors	
UDS NRC value (for a negative response message)	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE

Constraints
N/A
Steady
TRUE

Table 4-473 <RoutinePortInterfaceName>::Stop

4.52.8 <RoutinePortInterfaceName>::RequestResults

Prototype

`ara::core::Future<RequestResultsOutput>`
`namespace1_of_portinterface::namespace2_of_portinterface::`
`<RoutinePortInterfaceName>::RequestResults(`
`  namespace1_of_type_of_argument_data_prototype::Type1OfArgumentDataPrototype`
`  shortname1_of_argument_data_prototype,`
`  namespace2_of_type_of_argument_data_prototype::Type2OfArgumentDataPrototype`
`  shortname2_of_argument_data_prototype, ...  MetaInfo meta_info,`
`  CancellationHandler cancellation_handler)`

Description
Data element value returned when result is requested

Parameter	
meta_info	Contains additional meta information
cancellation_handler	Informs if the current conversation is cancelled. This object shall not be stored or accessed beyond the lifetime of this operation.

Returns	
ara::core::Future <RequestResultsOutput>	A Result with either RequestResultsOutput (for a positive response message) or an UDS NRC value (for a negative response message)

Errors
UDS NRC value (for a negative response message)

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE, if model parameter "numberOfParallelRequestExecutions" > 1	TRUE
Constraints				
N/A				
Steady				
TRUE				

Table 4-474 <RoutinePortInterfaceName>::RequestResults

4.52.9 <RoutinePortInterfaceName>::Offer

Prototype				
`ara::core::Result<void> <RoutinePortInterfaceName>::Offer ( ) noexcept`				
Description				
This Offer will enable the DM to forward request messages to this handler				
Returns				
ara::core::Result<void>	Positive result if service was offered, error if offer failed			
Errors				
ara::diag::DiagOfferErrc::kAlreadyOffered	If service has been offered already			
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-475 <RoutinePortInterfaceName>::Offer

4.52.10 <RoutinePortInterfaceName>::StopOffer

Prototype				
`void <RoutinePortInterfaceName>::StopOffer ( ) const noexcept`				
Description				
This StopOffer will disable the forwarding of request messages from DM.				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 4-476 <RoutinePortInterfaceName>::StopOffer

5 UDS Transport Protocol API Description

Caution

The API of the AMSR product is build by public classes, their methods, free functions and types. Classes, their accessible methods, free functions and other types that are intended to be used by customers are annotated with the documentation tag `\vpublic`. These items build the product public API. Customer software shall just use the public API of AMSR. The private part shall not be used.

based on template version 1.00.00

5.1 amsr::uds_transport::SecuredDataTransmissionHandler

Description	
Abstract Class, which a specific SecuredDataTransmission (plugin) shall subclass	
Include	**Generalize**
amsr/uds_transport/secured_data_transmission_handler.h	
Constructors	
SecuredDataTransmissionHandler	Constructor of SecuredDataTransmissionHandler
SecuredDataTransmissionHandler	Move constructs instance
SecuredDataTransmissionHandler	Copy constructs instance
MemberFunctions	
GetServiceValidator	Returns a service validation object
HandleRequest	Processes an incoming SDT request message
HandleResponse	Processes an outgoing SDT response message
Initialize	Initializes handler and makes it ready for use via the other class methods
Shutdown	Method to indicate that this SecuredDataTransmissionHandler shall terminate

Table 5-1 amsr::uds_transport::SecuredDataTransmissionHandler

5.1.1 amsr::uds_transport::SecuredDataTransmissionHandler:: SecuredDataTransmissionHandler

Prototype				
`amsr::uds_transport::SecuredDataTransmissionHandler:: SecuredDataTransmissionHandler () noexcept`				
Description				
Constructor of SecuredDataTransmissionHandler				
Parameter				
N/A	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-2 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler

5.1.2 amsr::uds_transport::SecuredDataTransmissionHandler:: SecuredDataTransmissionHandler

Prototype

```
amsr::uds_transport::SecuredDataTransmissionHandler::
SecuredDataTransmissionHandler (
   amsr::uds_transport::SecuredDataTransmissionHandler &&
) noexcept
```

Description
Move constructs instance

Parameter	

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-3 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler

5.1.3 amsr::uds_transport::SecuredDataTransmissionHandler:: SecuredDataTransmissionHandler

Prototype

```
amsr::uds_transport::SecuredDataTransmissionHandler::
SecuredDataTransmissionHandler (
   const class amsr::uds_transport::SecuredDataTransmissionHandler &
) noexcept
```

Description
Copy constructs instance

Parameter	

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-4 amsr::uds_transport::SecuredDataTransmissionHandler::SecuredDataTransmissionHandler

5.1.4 amsr::uds_transport::SecuredDataTransmissionHandler:: ~SecuredDataTransmissionHandler

Prototype

```
amsr::uds_transport::SecuredDataTransmissionHandler::
~SecuredDataTransmissionHandler () noexcept
```

Description				
Destructor of SecuredDataTransmissionHandler				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-5 amsr::uds_transport::SecuredDataTransmissionHandler::~SecuredDataTransmissionHandler

5.1.5 amsr::uds_transport::SecuredDataTransmissionHandler::GetServiceValidator

Prototype

```
amsr::uds_transport::ServiceValidation *
amsr::uds_transport::SecuredDataTransmissionHandler::GetServiceValidator (
  amsr::uds_transport::class SecuredDataTransmissionHandler::ServiceValidation
Type service_validation_type
) noexcept
```

Description

Returns a service validation object

Parameter

service_validation_type	Type of the service validation object to be returned

Detailed Description

This API shall return a service validation object, if it is desired to receive service validation/confirmation calls of the specified kind to this SDT handler. Otherwise a nullptr shall be returned. This API will be called once for each kind of service validation (manufacturer/supplier). The returned service validation object must be valid until the SdtPlugin is shutdown.

Returns

amsr::uds_transport::Service Validation *	Either a pointer to a service validation object or nullptr. If a pointer to a service validation object is returned, the pointer must be valid (and the service validation object must be able to be called) until the Shutdown method of this handler is invoked.

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 5-6 amsr::uds_transport::SecuredDataTransmissionHandler::GetServiceValidator

5.1.6 amsr::uds_transport::SecuredDataTransmissionHandler::HandleRequest

Prototype	

`ara::core::Result<ara::core::Span<const std::uint8_t>, UdsNegativeResponseCode>`
`amsr::uds_transport::SecuredDataTransmissionHandler::HandleRequest (`
`  amsr::uds_transport::SecuredDataTransmissionRequestMessage request_message,`
`  ara::core::Span<std::uint8_t> output_buffer`
`) noexcept`	

Description	
Processes an incoming SDT request message	
Parameter	
request_message	The structured representation of SDT request message. This object shall not live once this method call returns
output_buffer	Pre-allocated memory section to write the extracted/verified internal UDS message into.
Detailed Description	
This API inspects the request_message parameters, validates the enclosed payload and on success, must return either a span over the internal message part of the "request_message" (i.e. when just verifying a plain text signed message), or a sub-span of the "output_buffer" that will contain a dedicated copy of the enclosed request message.	
Returns	
ara::core::Result<ara::core::Span<const std::uint8_t>, UdsNegativeResponseCode>	In case of successfully processed SDT request - A span holding the enclosed request message. The span must be either a sub-span from the "request_message" or "output_buffer", to allow maximum efficiency of memory operations within DiagnosticManager. In any case: The returned span size must not exceed the "output_buffer" size! In case of error during processing, returns the appropriate UDS NRC as error
Errors	
amsr::uds_transport::UdsNegativeResponseCode Any UDS NRC	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE for different UDS client's source addresses	TRUE for different UDS client's source addresses	FALSE
Constraints				
N/A				

Table 5-7 amsr::uds_transport::SecuredDataTransmissionHandler::HandleRequest

5.1.7 amsr::uds_transport::SecuredDataTransmissionHandler::HandleResponse

Prototype	

`ara::core::Result<ara::diag::uds_transport::UdsMessagePtr, UdsNegativeResponseCode>`
`amsr::uds_transport::SecuredDataTransmissionHandler::HandleResponse (`
`  ara::core::Span<const std::uint8_t> plain_response_buffer,`
`  amsr::uds_transport::SecuredDataTransmissionResponseMessage response_message`
`) noexcept`	

Description	
Processes an outgoing SDT response message	
Parameter	
plain_response_buffer	The structured representation of SDT request message
response_message	The structured representation of SDT response message. Initial parameter values are derived from the corresponding SDT request message.
Detailed Description	
This API composes from the plain UDS response message a SDT response by setting the corresponding SDT parameters on the response_message and providing the enclosed response message in the appropriated form.	
Returns	
ara::core::Result<ara::diag::uds_transport::UdsMessagePtr, UdsNegativeResponseCode>	In case of successfully processed SDT response - the modified "response_message", holding the enclosed response message is returned. In case of error during processing, returns the appropriate UDS NRC as error
Errors	
amsr::uds_transport::UdsNegativeResponseCode Any UDS NRC	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE for different UDS client's source addresses	TRUE for different UDS client's source addresses	FALSE
Constraints				
N/A				

Table 5-8 amsr::uds_transport::SecuredDataTransmissionHandler::HandleResponse

5.1.8 amsr::uds_transport::SecuredDataTransmissionHandler::Initialize

Prototype
`enum amsr::uds_transport::SecuredDataTransmissionHandler::InitializationResult` `amsr::uds_transport::SecuredDataTransmissionHandler::Initialize () noexcept`
Description
Initializes handler and makes it ready for use via the other class methods
Parameter

N/A		N/A		
Returns				
enum amsr::uds_transport::Secured DataTransmissionHandler:: InitializationResult		kInitializeOk if initialization was successful, else kInitializeFailed		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-9 amsr::uds_transport::SecuredDataTransmissionHandler::Initialize

5.1.9 amsr::uds_transport::SecuredDataTransmissionHandler::Shutdown

Prototype				
`void amsr::uds_transport::SecuredDataTransmissionHandler::Shutdown () noexcept`				
Description				
Method to indicate that this SecuredDataTransmissionHandler shall terminate				
Parameter				
N/A		N/A		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-10 amsr::uds_transport::SecuredDataTransmissionHandler::Shutdown

5.2 amsr::uds_transport::AdministrativeParameter

Description	
Typed representation of the UDS SDT administrative parameter	
Include	**Generalize**
amsr/uds_transport/secured_data_transmission_message.h	
Constructors	
AdministrativeParameter	Default constructs a new instance of the object
AdministrativeParameter	Move constructs a new instance of this class
AdministrativeParameter	Copy constructs a new instance of this class
AdministrativeParameter	Constructs a new instance of the object with specific raw value
Operators	
operator=	Copy assigns an instance of this class to another
operator=	Move assigns an instance of this class to another
MemberFunctions	
GetValue	Returns the raw value of the parameter
IsMessageEncrypted	Returns the MessageEncrypted flag value
IsMessageSigned	Returns the MessageSigned flag value
IsPreEstablishedKeyUsed	Returns the PreEstablishedKeyUsed flag value
IsSignatureOnResponseRequested	Returns the SignatureOnResponseRequested flag value
SetMessageEncrypted	Replaces the MessageEncrypted flag value with a new one
SetMessageSigned	Replaces the MessageSigned flag value with a new one
SetPreEstablishedKeyUsed	Replaces the PreEstablishedKeyUsed flag value with a new one
SetSignatureOnResponse Requested	Replaces the SignatureOnResponseRequested flag value with a new one
SetValue	Replaces the raw value with a new one

Table 5-11 amsr::uds_transport::AdministrativeParameter

5.2.1 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

Prototype	
`amsr::uds_transport::AdministrativeParameter::AdministrativeParameter () noexcept`	
Description	
Default constructs a new instance of the object	
Parameter	
N/A	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-12 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

5.2.2 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

Prototype				
`amsr::uds_transport::AdministrativeParameter::AdministrativeParameter ( amsr::uds_transport::AdministrativeParameter && other ) noexcept`				
Description				
Move constructs a new instance of this class				
Parameter				
other	The origin to move from			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-13 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

5.2.3 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

Prototype				
`amsr::uds_transport::AdministrativeParameter::AdministrativeParameter ( const class amsr::uds_transport::AdministrativeParameter & other ) noexcept`				
Description				
Copy constructs a new instance of this class				
Parameter				
other	The origin to copy			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 5-14 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

5.2.4 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

Prototype
`amsr::uds_transport::AdministrativeParameter::AdministrativeParameter ( std::uint16_t value ) noexcept`
Description
Constructs a new instance of the object with specific raw value
Parameter

value	The raw value

Errors
N/A
Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 5-15 amsr::uds_transport::AdministrativeParameter::AdministrativeParameter

5.2.5 amsr::uds_transport::AdministrativeParameter::~AdministrativeParameter

Prototype
`amsr::uds_transport::AdministrativeParameter::~AdministrativeParameter () noexcept`
Description
Destructs an instance of this class
Errors
N/A
Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 5-16 amsr::uds_transport::AdministrativeParameter::~AdministrativeParameter

5.2.6 amsr::uds_transport::AdministrativeParameter::operator=

Prototype

```
amsr::uds_transport::AdministrativeParameter &
amsr::uds_transport::AdministrativeParameter::operator= (
   amsr::uds_transport::AdministrativeParameter && other
) & noexcept
```

Description	
Copy assigns an instance of this class to another	
Parameter	
other	The origin to copy
Returns	
amsr::uds_transport::Administrative Parameter &	This instance of the class
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-17 amsr::uds_transport::AdministrativeParameter::operator=

5.2.7 amsr::uds_transport::AdministrativeParameter::operator=

Prototype

```
amsr::uds_transport::AdministrativeParameter &
amsr::uds_transport::AdministrativeParameter::operator= (
   const class amsr::uds_transport::AdministrativeParameter & other
) & noexcept
```

Description	
Move assigns an instance of this class to another	
Parameter	
other	The origin to copy
Returns	
amsr::uds_transport::Administrative Parameter &	This instance of the class
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 5-18 amsr::uds_transport::AdministrativeParameter::operator=

5.2.8 amsr::uds_transport::AdministrativeParameter::GetValue

Prototype				
`std::uint16_t amsr::uds_transport::AdministrativeParameter::GetValue () const noexcept`				
Description				
Returns the raw value of the parameter				
Parameter				
N/A		N/A		
Returns				
std::uint16_t		The raw value		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-19 amsr::uds_transport::AdministrativeParameter::GetValue

5.2.9 amsr::uds_transport::AdministrativeParameter::IsMessageEncrypted

Prototype				
`bool amsr::uds_transport::AdministrativeParameter::IsMessageEncrypted () const noexcept`				
Description				
Returns the MessageEncrypted flag value				
Parameter				
N/A		N/A		
Returns				
bool		True if flag is set, otherwise - false		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				

N/A

Table 5-20 amsr::uds_transport::AdministrativeParameter::IsMessageEncrypted

5.2.10 amsr::uds_transport::AdministrativeParameter::IsMessageSigned

Prototype				
`bool amsr::uds_transport::AdministrativeParameter::IsMessageSigned () const noexcept`				
Description				
Returns the MessageSigned flag value				
Parameter				
N/A		N/A		
Returns				
bool		True if flag is set, otherwise - false		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-21 amsr::uds_transport::AdministrativeParameter::IsMessageSigned

5.2.11 amsr::uds_transport::AdministrativeParameter::IsRequestMessage

Prototype				
`bool amsr::uds_transport::AdministrativeParameter::IsRequestMessage () const noexcept`				
Description				
Returns the RequestMessage flag value				
Parameter				
N/A		N/A		
Returns				
bool		True if flag is set, otherwise - false		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				

N/A

Table 5-22 amsr::uds_transport::AdministrativeParameter::IsRequestMessage

5.2.12 amsr::uds_transport::AdministrativeParameter::IsPreEstablishedKeyUsed

Prototype				
`bool amsr::uds_transport::AdministrativeParameter::IsPreEstablishedKeyUsed ( ) const noexcept`				
Description				
Returns the PreEstablishedKeyUsed flag value				
Parameter				
N/A	N/A			
Returns				
bool	True if flag is set, otherwise - false			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-23 amsr::uds_transport::AdministrativeParameter::IsPreEstablishedKeyUsed

5.2.13 amsr::uds_transport::AdministrativeParameter::IsSignatureOnResponseRequested

Prototype				
`bool amsr::uds_transport::AdministrativeParameter::IsSignatureOnResponseRequested ( ) const noexcept`				
Description				
Returns the SignatureOnResponseRequested flag value				
Parameter				
N/A	N/A			
Returns				
bool	True if flag is set, otherwise - false			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE

Constraints
N/A

Table 5-24 amsr::uds_transport::AdministrativeParameter::IsSignatureOnResponseRequested

5.2.14 amsr::uds_transport::AdministrativeParameter::SetMessageEncrypted

Prototype				
`void amsr::uds_transport::AdministrativeParameter::SetMessageEncrypted ( bool value ) noexcept`				
Description				
Replaces the MessageEncrypted flag value with a new one				
Parameter				
value	The new flag state			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-25 amsr::uds_transport::AdministrativeParameter::SetMessageEncrypted

5.2.15 amsr::uds_transport::AdministrativeParameter::SetMessageSigned

Prototype	
`void amsr::uds_transport::AdministrativeParameter::SetMessageSigned ( bool value ) noexcept`	
Description	
Replaces the MessageSigned flag value with a new one	
Parameter	
value	The new flag state
Returns	
void	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-26 amsr::uds_transport::AdministrativeParameter::SetMessageSigned

5.2.16 amsr::uds_transport::AdministrativeParameter::SetRequestMessage

Prototype	
`void amsr::uds_transport::AdministrativeParameter::SetRequestMessage ( bool value ) noexcept`	
Description	
Replaces the RequestMessage flag value with a new one	
Parameter	
value	The new flag state
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-27 amsr::uds_transport::AdministrativeParameter::SetRequestMessage

5.2.17 amsr::uds_transport::AdministrativeParameter::SetPreEstablishedKeyUsed

Prototype	
`void amsr::uds_transport::AdministrativeParameter::SetPreEstablishedKeyUsed ( bool value ) noexcept`	
Description	
Replaces the PreEstablishedKeyUsed flag value with a new one	
Parameter	
value	The new flag state
Returns	

void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-28 amsr::uds_transport::AdministrativeParameter::SetPreEstablishedKeyUsed

5.2.18 amsr::uds_transport::AdministrativeParameter::SetSignatureOnResponseRequested

Prototype	
`void amsr::uds_transport::AdministrativeParameter::SetSignatureOnResponseRequested ( bool value ) noexcept`	
Description	
Replaces the SignatureOnResponseRequested flag value with a new one	
Parameter	
value	The new flag state
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-29 amsr::uds_transport::AdministrativeParameter::SetSignatureOnResponseRequested

5.2.19 amsr::uds_transport::AdministrativeParameter::SetValue

Prototype
`void amsr::uds_transport::AdministrativeParameter::SetValue ( std::uint16_t value ) noexcept`
Description
Replaces the raw value with a new one

Parameter				
value		The new raw value		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-30 amsr::uds_transport::AdministrativeParameter::SetValue

based on template version 1.00.00

5.3 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage

Description	
Typed representation of the enclosed UDS SDT message content (AntiReplayCounter, InternalMessage, Signature)	
Include	**Generalize**
amsr/uds_transport/secured_data_transmission_message.h	
Constructors	
SecuredDataTransmissionEnclosedMessage	Move constructs a new instance of this class
SecuredDataTransmissionEnclosedMessage	Copy constructs a new instance of this class
Operators	
operator=	Move assigns an instance of this class to another
operator=	Copy assigns an instance of this class to another
MemberFunctions	
Create	Attempts to constructs a new instance of this class
GetAntiReplayCounter	Returns the AntiReplayCounter parameter value
GetInternalMessage	Returns a writeable span over the pre-allocated memory for the internal UDS message content
GetInternalMessage	Returns a read-only span over the pre-allocated memory for the internal UDS message content
GetPayload	Returns a writeable span over the pre-allocated memory for the enclosed message parameters (AntiReplayCounter, InternalMessage, Signature)
GetPayload	Returns a span over the pre-allocated memory for the enclosed message parameters (AntiReplayCounter, InternalMessage, Signature)
GetSignature	Returns a writeable span over the pre-allocated memory for the internal UDS SDT message signature content
GetSignature	Returns a read-only span over the pre-allocated memory for the internal UDS SDT message signature content
SetAntiReplayCounter	Replaces the AntiReplayCounter parameter value with a new one

Table 5-31 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage

5.3.1 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: SecuredDataTransmissionEnclosedMessage

Prototype

```
amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::
SecuredDataTransmissionEnclosedMessage (
   amsr::uds_transport::SecuredDataTransmissionEnclosedMessage && other
) noexcept
```

Description				
Move constructs a new instance of this class				
Parameter				
other		The origin to move from		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-32 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SecuredDataTransmissionEnclosedMessage

5.3.2 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: SecuredDataTransmissionEnclosedMessage

Prototype				
`amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: SecuredDataTransmissionEnclosedMessage ( const class amsr::uds_transport::SecuredDataTransmissionEnclosedMessage & other ) noexcept`				
Description				
Copy constructs a new instance of this class				
Parameter				
other		The origin to copy		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-33 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SecuredDataTransmissionEnclosedMessage

5.3.3 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: ~SecuredDataTransmissionEnclosedMessage

Prototype
`amsr::uds_transport::SecuredDataTransmissionEnclosedMessage:: ~SecuredDataTransmissionEnclosedMessage () noexcept`
Description

Destructs an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-34 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::~SecuredDataTransmissionEnclosedMessage

5.3.4 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator=

Prototype	
`amsr::uds_transport::SecuredDataTransmissionEnclosedMessage & amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= ( amsr::uds_transport::SecuredDataTransmissionEnclosedMessage && other ) & noexcept`	
Description	
Move assigns an instance of this class to another	
Parameter	
other	The origin to move from
Returns	
amsr::uds_transport::SecuredDataTransmissionEnclosedMessage &	This instance of the class
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-35 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator=

5.3.5 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator=

Prototype
`amsr::uds_transport::SecuredDataTransmissionEnclosedMessage & amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator= ( const class amsr::uds_transport::SecuredDataTransmissionEnclosedMessage & other ) & noexcept`
Description

Copy assigns an instance of this class to another				
Parameter				
other	The origin to copy			
Returns				
amsr::uds_transport::SecuredDataTransmissionEnclosedMessage &	This instance of the class			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-36 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::operator=

5.3.6 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::Create

Prototype

```
ara::core::Result<SecuredDataTransmissionEnclosedMessage>
amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::Create (
  ara::core::Span<std::uint8_t> message,
  std::size_t message_size,
  std::size_t signature_size
) noexcept
```

Description

Attempts to constructs a new instance of this class

Parameter	
message	The pre-allocated buffer over the enclosed UDS SDT message including AntiReplayCounter, InternalMessage, Signature
message_size	The enclosed UDS message size. Valid range [1, message.size()]
signature_size	The enclosed UDS SDT message signature size. Valid range [0, (message.size()-message_size - 2)]
Returns	
ara::core::Result<SecuredDataTransmissionEnclosedMessage>	If valid object can be constructed, returns the object. In case of precondition violations, returns an error

Errors				
amsr::uds_transport::UdsTransportErrc::kGenericError An error occurred				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
Refer to individual parameter preconditions				

Table 5-37 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::Create

5.3.7 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetAntiReplayCounter

Prototype				
`std::uint16_t amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetAntiReplayCounter () const noexcept`				
Description				
Returns the AntiReplayCounter parameter value				
Parameter				
N/A		N/A		
Returns				
std::uint16_t		The parameter value		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-38 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetAntiReplayCounter

5.3.8 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage

Prototype	
`ara::core::Span<std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage () noexcept`	
Description	
Returns a writeable span over the pre-allocated memory for the internal UDS message content	
Parameter	
N/A	N/A
Returns	
ara::core::Span<std::uint8_t>	A writeable span over the UDS message content
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-39 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage

5.3.9 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage

Prototype	
`ara::core::Span<const std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage ( ) const noexcept`	
Description	
Returns a read-only span over the pre-allocated memory for the internal UDS message content	
Parameter	
N/A	N/A
Returns	
ara::core::Span<const std::uint8_t>	A read-only span over the UDS message content
Errors	
N/A	
Particularities and Limitations	
Synchronous / Call Context / Reentrant / ThreadSafe / Can Throw	
TRUE / ANY / TRUE / FALSE / FALSE	
Constraints	
N/A	

Table 5-40 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetInternalMessage

5.3.10 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload

Prototype	
`ara::core::Span<std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload () noexcept`	
Description	
Returns a writeable span over the pre-allocated memory for the enclosed message parameters (AntiReplayCounter, InternalMessage, Signature)	
Parameter	
N/A	N/A
Returns	
ara::core::Span<std::uint8_t>	A writeable span over the enclosed message content (AntiReplayCounter, InternalMessage, Signature)

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-41 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload

5.3.11 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload

Prototype	
`ara::core::Span<const std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload ( ) const noexcept`	
Description	
Returns a span over the pre-allocated memory for the enclosed message parameters (AntiReplayCounter, InternalMessage, Signature)	
Parameter	
N/A	N/A
Returns	
ara::core::Span<const std::uint8_t>	A read-only span over the enclosed message content (AntiReplayCounter, InternalMessage, Signature)
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-42 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetPayload

5.3.12 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature

Prototype
`ara::core::Span<std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature ( ) noexcept`
Description
Returns a writeable span over the pre-allocated memory for the internal UDS SDT message signature content

Parameter	
N/A	N/A
Detailed Description	
If no signature is desired, the span will have zero size	
Returns	
ara::core::Span<std::uint8_t>	A writeable span over the signature content
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-43 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature

5.3.13 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature

Prototype	
`ara::core::Span<const std::uint8_t> amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature ( ) const noexcept`	
Description	
Returns a read-only span over the pre-allocated memory for the internal UDS SDT message signature content	
Parameter	
N/A	N/A
Detailed Description	
If no signature is desired, the span will have zero size	
Returns	
ara::core::Span<const std::uint8_t>	A read-only span over the signature content
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-44 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::GetSignature

5.3.14 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SetAntiReplayCounter

Prototype				
`void amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SetAntiReplayCounter ( std::uint16_t value ) noexcept`				
Description				
Replaces the AntiReplayCounter parameter value with a new one				
Parameter				
value	The new anti-replay counter value			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-45 amsr::uds_transport::SecuredDataTransmissionEnclosedMessage::SetAntiReplayCounter

based on template version 1.00.00

5.4 amsr::uds_transport::SecuredDataTransmissionRequestMessage

Description	
A typed representation of the UDS SDT request message	
Include	**Generalize**
amsr/uds_transport/secured_data_transmission_message.h	
Constructors	
SecuredDataTransmissionRequestMessage	Move constructs a new instance
Operators	
operator=	Move assigns an instance of this class to another
MemberFunctions	
AddMetaInfoValue	Adds a new key-value in the MetaInfo
Create	Attempts to compose a structured SDT request object
GetAdministrativeParameters	Returns the administrative parameters
GetClientAddress	Returns the UDS client's source address
GetMessage	Returns the secured message content: AntiReplayCounter, InternalMessage, Signature
GetMetaInfoValue	Searches in the MetaInfo map for a specific key
GetPayload	Returns the whole SDT UDS request payload
GetSignEncryptCalculation	Returns the signature/encryption calculation identifier

Table 5-46 amsr::uds_transport::SecuredDataTransmissionRequestMessage

5.4.1 amsr::uds_transport::SecuredDataTransmissionRequestMessage::SecuredDataTransmissionRequestMessage

Prototype

```
amsr::uds_transport::SecuredDataTransmissionRequestMessage::
SecuredDataTransmissionRequestMessage (
  amsr::uds_transport::SecuredDataTransmissionRequestMessage && other
) noexcept
```

Description

Move constructs a new instance

Parameter

other	The origin to move from

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

Table 5-47 amsr::uds_transport::SecuredDataTransmissionRequestMessage::SecuredDataTransmissionRequestMessage

5.4.2 amsr::uds_transport::SecuredDataTransmissionRequestMessage::~SecuredDataTransmissionRequestMessage

Prototype				
`amsr::uds_transport::SecuredDataTransmissionRequestMessage::~SecuredDataTransmissionRequestMessage () noexcept`				
Description				
Destroys an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-48 amsr::uds_transport::SecuredDataTransmissionRequestMessage::~SecuredDataTransmissionRequestMessage

5.4.3 amsr::uds_transport::SecuredDataTransmissionRequestMessage::operator=

Prototype				
`amsr::uds_transport::SecuredDataTransmissionRequestMessage & amsr::uds_transport::SecuredDataTransmissionRequestMessage::operator= ( amsr::uds_transport::SecuredDataTransmissionRequestMessage && other ) & noexcept`				
Description				
Move assigns an instance of this class to another				
Parameter				
other	The origin to move from			
Returns				
amsr::uds_transport::SecuredDataTransmissionRequestMessage &	This instance of the class			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-49 amsr::uds_transport::SecuredDataTransmissionRequestMessage::operator=

5.4.4 amsr::uds_transport::SecuredDataTransmissionRequestMessage::AddMetaInfoValue

Prototype

```
void
amsr::uds_transport::SecuredDataTransmissionRequestMessage::AddMetaInfoValue (
   ara::core::StringView key,
   ara::core::StringView value
) noexcept
```

Description	
Adds a new key-value in the MetaInfo	
Parameter	
key	The key to store
value	The associated value
Detailed Description	
If the key already exists, the new value will be ignored	
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-50 amsr::uds_transport::SecuredDataTransmissionRequestMessage::AddMetaInfoValue

5.4.5 amsr::uds_transport::SecuredDataTransmissionRequestMessage::Create

Prototype

```
ara::core::Result<SecuredDataTransmissionRequestMessage, UdsNegativeResponseCode>
amsr::uds_transport::SecuredDataTransmissionRequestMessage::Create (
   ara::diag::uds_transport::UdsMessage & message
) noexcept
```

Description	
Attempts to compose a structured SDT request object	
Parameter	
message	A Uds message to bring in a structure
Returns	
ara::core::Result<SecuredDataTransmissionRequestMessage, UdsNegativeResponseCode>	Returns a valid object instance on successfully interpret UDS message. In case of error -an appropriate UDS NRC is returned.
Errors	
amsr::uds_transport::UdsNegativeResponseCode Any UDS NRC	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE, for different message parameter instances	TRUE, for different message parameter instances	FALSE
Constraints				
N/A				

Table 5-51 amsr::uds_transport::SecuredDataTransmissionRequestMessage::Create

5.4.6 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetAdministrativeParameters

Prototype	
`amsr::uds_transport::AdministrativeParameter amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetAdministrativeParameters () const noexcept`	
Description	
Returns the administrative parameters	
Parameter	
N/A	N/A
Returns	
amsr::uds_transport::Administrative Parameter	The corresponding SDT UDS request parameter value
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-52 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetAdministrativeParameters

5.4.7 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetClientAddress

Prototype	
`ara::diag::uds_transport::class UdsMessage::Address amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetClientAddress ( ) const noexcept`	
Description	
Returns the UDS client's source address	
Parameter	
N/A	N/A

Returns	
ara::diag::uds_transport::class Uds Message::Address	The UDS client source address
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-53 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetClientAddress

5.4.8 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMessage

Prototype

```
const class amsr::uds_transport::SecuredDataTransmissionEnclosedMessage &
amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMessage (
) const noexcept
```

Description

Returns the secured message content: AntiReplayCounter, InternalMessage, Signature

Parameter	
N/A	N/A
Returns	
const class amsr::uds_transport:: SecuredDataTransmissionEnclosed Message &	The secured message content
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-54 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMessage

5.4.9 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMetaInfoValue

Prototype

```
ara::core::Optional<ara::core::StringView>
amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMetaInfoValue (
  ara::core::StringView key
) const noexcept
```

Description				
Searches in the MetaInfo map for a specific key				
Parameter				
key	The key to look for			
Returns				
ara::core::Optional<ara::core::String View>	If the key exists - the corresponding value. Otherwise - an empty optional.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-55 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetMetaInfoValue

5.4.10 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetPayload

Prototype				
`ara::core::Span<const std::uint8_t> amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetPayload ( ) const noexcept`				
Description				
Returns the whole SDT UDS request payload				
Parameter				
N/A	N/A			
Detailed Description				
This is the raw request byte stream starting behind the SID 0x84				
Returns				
ara::core::Span<const std::uint8_t>	A read-only view over the SDT UDS request message payload (without SID).			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-56 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetPayload

5.4.11 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetSignEncryptCalculation

Prototype				
`std::uint8_t amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetSignEncryptCalculation () const noexcept`				
Description				
Returns the signature/encryption calculation identifier				
Parameter				
N/A	N/A			
Returns				
std::uint8_t	The corresponding SDT UDS request parameter value			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-57 amsr::uds_transport::SecuredDataTransmissionRequestMessage::GetSignEncryptCalculation

5.5 amsr::uds_transport::SecuredDataTransmissionResponseMessage

Description	
A typed representation of the UDS SDT response message	
Include	**Generalize**
amsr/uds_transport/secured_data_transmission_message.h	
Constructors	
SecuredDataTransmissionResponseMessage	Move constructs a new instance
MemberFunctions	
AddMetaInfoValue	Adds a new key-value in the MetaInfo
AllocateMessage	Returns a span for SDT UDS response payload
AllocateMessage	Returns a typed view over the pre-allocated enclosed SDT message parameters (AntiReplayCounter, InternalMessage, Signature)
Create	Attempts to compose a structured SDT response object
Finalize	Returns the formed SDT UDS Message
GetAdministrativeParameters	Returns the administrative parameters
GetClientAddress	Returns the UDS client's source address
GetMetaInfoValue	Searches in the MetaInfo map for a specific key
GetSignEncryptCalculation	Returns the signature/encryption calculation identifier
SetAdministrativeParameters	Replaces the the administrative parameters with new one
SetSignEncryptCalculation	Replaces the the signature/encryption calculation identifier with new one

Table 5-58 amsr::uds_transport::SecuredDataTransmissionResponseMessage

5.5.1 amsr::uds_transport::SecuredDataTransmissionResponseMessage:: SecuredDataTransmissionResponseMessage

Prototype

```
amsr::uds_transport::SecuredDataTransmissionResponseMessage::
SecuredDataTransmissionResponseMessage (
  amsr::uds_transport::SecuredDataTransmissionResponseMessage && other
) noexcept
```

Description

Move constructs a new instance

Parameter

other	The origin to move from

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-59 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SecuredDataTransmissionResponseMessage

5.5.2 amsr::uds_transport::SecuredDataTransmissionResponseMessage::~SecuredDataTransmissionResponseMessage

Prototype				
`amsr::uds_transport::SecuredDataTransmissionResponseMessage::~SecuredDataTransmissionResponseMessage () noexcept`				
Description				
Destroys an instance of this class				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-60 amsr::uds_transport::SecuredDataTransmissionResponseMessage::~SecuredDataTransmissionResponseMessage

5.5.3 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AddMetaInfoValue

Prototype	
`void amsr::uds_transport::SecuredDataTransmissionResponseMessage::AddMetaInfoValue ( ara::core::StringView key, ara::core::StringView value ) noexcept`	
Description	
Adds a new key-value in the MetaInfo	
Parameter	
key	The key to store
value	The associated value
Detailed Description	
If the key already exists, the new value will be ignored	
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-61 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AddMetaInfoValue

5.5.4 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage

Prototype

```
ara::core::Result<ara::core::Span<std::uint8_t> >
amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage (
   std::size_t size
) noexcept
```

Description

Returns a span for SDT UDS response payload

Parameter

size	The SDT response payload size without excluding the SID byte

Detailed Description

This is the raw response byte stream starting behind the response SID 0xC4. The call of this API is an alternative to its typed variant. Any call of this API does not change the content, but the size of the final message.

Returns

ara::core::Result<ara::core::Span<std::uint8_t> >	A writeable view over the SDT UDS request message payload (without SID). In case of enough room - the typed representation of the pre-allocated SDT message buffer

Errors

ara::diag::uds_transport::kInvalidParameter If the requested size cannot be satisfied

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-62 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage

5.5.5 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage

Prototype

```
ara::core::Result<SecuredDataTransmissionEnclosedMessage>
amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage (
   std::size_t message_size,
   std::uint16_t signature_size
) noexcept
```

Description

Returns a typed view over the pre-allocated enclosed SDT message parameters (AntiReplayCounter, InternalMessage, Signature)	
Parameter	
message_size	The required amount of bytes for the enclosed UDS response message. Valid range [1, message.size()]
signature_size	The required amount of bytes for signature storage. Valid range [0, max(message.size()-message_size, 65535)]
Detailed Description	
These parameters must be provided by the user of this API Any subsequent call of this method will not invalidate the already written data, but the caller must assure non-concurrent access to the memory provided with the result, since all the results represent an overlapping memory view	
Returns	
ara::core::Result<SecuredData TransmissionEnclosedMessage>	In case of enough room - the typed representation of the pre-allocated SDT message buffer
Errors	
ara::diag::uds_transport::UdsTransportErrc::kInvalidParameter If the requested sizes cannot be satisfied	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
Refer to individual parameter preconditions

Table 5-63 amsr::uds_transport::SecuredDataTransmissionResponseMessage::AllocateMessage

5.5.6 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Create

Prototype

```
ara::core::Result<SecuredDataTransmissionResponseMessage>
amsr::uds_transport::SecuredDataTransmissionResponseMessage::Create (
  ara::diag::uds_transport::UdsMessagePtr message
) noexcept
```

Description	
Attempts to compose a structured SDT response object	
Parameter	
message	A Uds message to bring in a structure
Returns	
ara::core::Result<SecuredData TransmissionResponseMessage>	In case of minimum required size available, returns a valid object instance. In case no object can be instantiated, returns an error
Errors	
ara::diag::uds_transport::UdsTransportErrc::kInvalidParameter UDS message is too short	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE for different UDS messages	TRUE for different UDS messages	FALSE
Constraints				
N/A				

Table 5-64 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Create

5.5.7 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Finalize

Prototype	
`ara::core::Result<ara::diag::uds_transport::UdsMessagePtr>` `amsr::uds_transport::SecuredDataTransmissionResponseMessage::Finalize () noexcept`	
Description	
Returns the formed SDT UDS Message	
Parameter	
N/A	N/A
Detailed Description	
If the message is not completely set up, an error will be returned. The SDT UDS message will be disposed	
Returns	
ara::core::Result<ara::diag::uds_transport::UdsMessagePtr>	In case of success - the SDT UDS message ready to be sent over the transport layer. In case of failure - error
Errors	
ara::diag::uds_transport::UdsTransportErrc::kGenericError If the finalization failed, or message already finalized	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
Can be called only once				

Table 5-65 amsr::uds_transport::SecuredDataTransmissionResponseMessage::Finalize

5.5.8 amsr::uds_transport::SecuredDataTransmissionResponseMessage ::GetAdministrativeParameters

Prototype

`amsr::uds_transport::AdministrativeParameter`
`amsr::uds_transport::SecuredDataTransmissionResponseMessage::`
`GetAdministrativeParameters () const noexcept`

Description

Returns the administrative parameters				
Parameter				
N/A		N/A		
Detailed Description				
The initial value is always the one from the corresponding SDT request message				
Returns				
amsr::uds_transport::Administrative Parameter		The corresponding SDT UDS request parameter value		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-66 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetAdministrativeParameters

5.5.9 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetClientAddress

Prototype				
`ara::diag::uds_transport::class UdsMessage::Address amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetClientAddress ( ) const noexcept`				
Description				
Returns the UDS client's source address				
Parameter				
N/A		N/A		
Returns				
ara::diag::uds_transport::class Uds Message::Address		The UDS client source address		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-67 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetClientAddress

5.5.10 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetMetaInfoValue

Prototype				

`ara::core::Optional<ara::core::StringView>`
`amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetMetaInfoValue (`
`  ara::core::StringView key`
`) const noexcept`				

Description				
Searches in the MetaInfo map for a specific key				
Parameter				
key	The key to look for			
Returns				
ara::core::Optional<ara::core::String View>	If the key exists - the corresponding value. Otherwise - an empty optional.			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-68 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetMetaInfoValue

5.5.11 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetSignEncryptCalculation

Prototype				

`std::uint8_t`
`amsr::uds_transport::SecuredDataTransmissionResponseMessage::`
`GetSignEncryptCalculation () const noexcept`				

Description				
Returns the signature/encryption calculation identifier				
Parameter				
N/A	N/A			
Detailed Description				
The initial value is always the one from the corresponding SDT request message				
Returns				
std::uint8_t	The corresponding SDT UDS request parameter value			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw

TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-69 amsr::uds_transport::SecuredDataTransmissionResponseMessage::GetSignEncryptCalculation

5.5.12 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetAdministrativeParameters

Prototype				
`void amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetAdministrativeParameters ( amsr::uds_transport::AdministrativeParameter value ) noexcept`				
Description				
Replaces the administrative parameters with new one				
Parameter				
value	The new parameters			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-70 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetAdministrativeParameters

5.5.13 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetSignEncryptCalculation

Prototype	
`void amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetSignEncryptCalculation ( std::uint8_t value ) noexcept`	
Description	
Replaces the signature/encryption calculation identifier with new one	
Parameter	
value	The new value

Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-71 amsr::uds_transport::SecuredDataTransmissionResponseMessage::SetSignEncryptCalculation

based on template version 1.00.00

5.6 ara::diag::uds_transport::UdsTransportProtocolHandler

Description	
Abstract Class, which a specific UDS Transport Protocol (plugin) shall subclass	
Include	**Generalize**
ara/diag/uds_transport/protocol_handler.h	
Constructors	
UdsTransportProtocolHandler	Default move constructs instance
UdsTransportProtocolHandler	Constructor of UdsTransportProtocolHandler
UdsTransportProtocolHandler	Default copy constructs instance
MemberFunctions	
GetHandlerID	Return the UdsTransportProtocolPluginID, which was given to the implementation during construction (ctor call)
GetPeriodicHandler	Returns the corresponding periodic TP handler
Initialize	Initializes handler
NotifyReestablishment	Tells the UdsTransportProtocolHandler, that it shall notify the DM core via UdsTransportProtocolMgr::ChannelReestablished()) if the given channel has been re-established after next UdsTransportProtocolHandler::Start()
Start	Start processing the implemented Uds Transport Protocol
Stop	Method to indicate that this UdsTransportProtocolHandler should terminate
StopConnection	Stop the connection with UDS clients and lock TP Handler for further requests
Transmit	Transmit a Uds message via the underlying Uds Transport Protocol channel

Table 5-72 ara::diag::uds_transport::UdsTransportProtocolHandler

5.6.1 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

Prototype

```
ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler
(
  ara::diag::uds_transport::UdsTransportProtocolHandler &&
) noexcept
```

Description

Default move constructs instance

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-73 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

5.6.2 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

Prototype

```
ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler
(
  const ara::diag::uds_transport::UdsTransportProtocolHandlerID handler_id,
  ara::diag::uds_transport::UdsTransportProtocolMgr & transport_protocol_mgr
)
```

Description

Constructor of UdsTransportProtocolHandler

Parameter

handler_id	The handler ID used by DM to identify this handler. This is just a number/identification given by the DM core when instantiating a UdsTransportProtocolHandler instance to be able to distinguish it from other handler-plugins or built-in UdsTransportProtocolHandler implementations.
transport_protocol_mgr	Reference to UdsTransportProtocolMgr owned by this DM, with which UdsTransportProtocolHandler instance shall interact.

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-74 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

5.6.3 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

Prototype

```
ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler
(
  const class ara::diag::uds_transport::UdsTransportProtocolHandler &
) noexcept
```

Description

Default copy constructs instance

Parameter

Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-75 ara::diag::uds_transport::UdsTransportProtocolHandler::UdsTransportProtocolHandler

5.6.4 ara::diag::uds_transport::UdsTransportProtocolHandler::~UdsTransportProtocolHandler

Prototype
`ara::diag::uds_transport::UdsTransportProtocolHandler::~UdsTransportProtocolHandler ()`
Description
Destructor of UdsTransportProtocolHandler
Errors
N/A
Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-76 ara::diag::uds_transport::UdsTransportProtocolHandler::~UdsTransportProtocolHandler

5.6.5 ara::diag::uds_transport::UdsTransportProtocolHandler::GetHandlerID

Prototype	
`ara::diag::uds_transport::UdsTransportProtocolHandlerID ara::diag::uds_transport::UdsTransportProtocolHandler::GetHandlerID () const`	
Description	
Return the UdsTransportProtocolPluginID, which was given to the implementation during construction (ctor call)	
Parameter	
N/A	N/A
Returns	
ara::diag::uds_transport::UdsTransportProtocolHandlerID	The handler ID used by DM to identify this handler
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-77 ara::diag::uds_transport::UdsTransportProtocolHandler::GetHandlerID

5.6.6 ara::diag::uds_transport::UdsTransportProtocolHandler::GetPeriodicHandler

Prototype	
`ara::core::Result<UdsTransportProtocolPeriodicHandler &>` `ara::diag::uds_transport::UdsTransportProtocolHandler::GetPeriodicHandler ()`	
Description	
Returns the corresponding periodic TP handler	
Parameter	
N/A	N/A
Detailed Description	
The life-time of the returned object must be equal with the life-time of the UdsTransportProtocolHandler object that holds it. Returns an error result for all calls in the default implementation.	
Returns	
ara::core::Result<UdsTransport ProtocolPeriodicHandler &>	UdsTransportProtocolPeriodicHandler reference if periodic transmissions are supported on this transport protocol, an error if not supported
Errors	
UdsTransportErrc::kPeriodicHandlerNotAvailable Periodic transmissions are not supported on this TP	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-78 ara::diag::uds_transport::UdsTransportProtocolHandler::GetPeriodicHandler

5.6.7 ara::diag::uds_transport::UdsTransportProtocolHandler::Initialize

Prototype
`enum ara::diag::uds_transport::UdsTransportProtocolHandler::InitializationResult` `ara::diag::uds_transport::UdsTransportProtocolHandler::Initialize ()`
Description
Initializes handler
Parameter

N/A	N/A
Detailed Description	
Must be called before Start(). The idea is to have "initialization" of handler-plugin separated from its ctor.	
Returns	
enum ara::diag::uds_transport::UdsTransportProtocolHandler::InitializationResult	kInitializeOk if initialization was successful, else kInitializeFailed
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-79 ara::diag::uds_transport::UdsTransportProtocolHandler::Initialize

5.6.8 ara::diag::uds_transport::UdsTransportProtocolHandler::NotifyReestablishment

Prototype

```
bool
ara::diag::uds_transport::UdsTransportProtocolHandler::NotifyReestablishment (
  ara::diag::uds_transport::ChannelID channel_id
)
```

Description

Tells the UdsTransportProtocolHandler, that it shall notify the DM core via UdsTransportProtocolMgr::ChannelReestablished()) if the given channel has been re-established after next UdsTransportProtocolHandler::Start()

Parameter

channel_id	channelID, whose re-establishment shall be notified to UdsTransportProtocolMgr

Detailed Description

The main purpose of this method is to allow DM to provide an ECU-Reset (0x11 service), with configuration option "Pos. response AFTER reset". In this scenario the request for 0x11 will be received on a certain channel with identifying tuple <p_x, c_y> (GlobalChannelIdentifier). Then the ECU-Reset takes place and after ECU-Restart all UdsProtocolHandlers/plugins get restarted via call to UdsTransportProtocolHandler::Start(). Now there are two expectations, when this method has been called before and returned "true":

- if the same remote client connects to the UdsProtocolHandler, it shall get a channel identification with the same identifying tuple <p_x, c_y> as last time.
- it shall call UdsTransportProtocolMgr::ChannelReestablished(GlobalChannelIdentifier <p_x, c_y>)

Returns	
bool	true if notification request is accepted and can be fulfilled, false otherwise.
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-80 ara::diag::uds_transport::UdsTransportProtocolHandler::NotifyReestablishment

5.6.9 ara::diag::uds_transport::UdsTransportProtocolHandler::Start

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolHandler::Start ()
```

Description

Start processing the implemented Uds Transport Protocol

Parameter	
N/A	N/A
Detailed Description	

The implementation shall call its superclass Start() method as there might be some stack specific implementation. Implementation shall be asynchronous as DM might start many/different UdsTransportProtocolHandler in parallel and strong serialization of all those starts just unnecessarily slows down DM startup.

Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
Initialize() was successfully called before.				

Table 5-81 ara::diag::uds_transport::UdsTransportProtocolHandler::Start

5.6.10 ara::diag::uds_transport::UdsTransportProtocolHandler::Stop

Prototype

`void ara::diag::uds_transport::UdsTransportProtocolHandler::Stop ()`				
Description				
Method to indicate that this UdsTransportProtocolHandler should terminate				
Parameter				
N/A		N/A		
Detailed Description				
If UdsTransportProtocolHandler has stopped, it shall call UdsTransportProtocolMgr::HandlerStopped(UdsTransportProtocolHandlerID)				
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-82 ara::diag::uds_transport::UdsTransportProtocolHandler::Stop

5.6.11 ara::diag::uds_transport::UdsTransportProtocolHandler::StopConnection

Prototype	
`void ara::diag::uds_transport::UdsTransportProtocolHandler::StopConnection ()`	
Description	
Stop the connection with UDS clients and lock TP Handler for further requests	
Parameter	
connection_id	The connection id to close (source/tester address and channel id). In case of empty optional, all connections shall be closed
target_address	The connection to this target address (diagnostic server) shall be closed and no further request to this target address shall be accepted. In case of empty optional, no further request shall be accepted from the TP handler
meta_info	The meta information associated to the request requesting the connections to be stopped
connection_end_type	The connection end type
Detailed Description	

The optional parameters shall be considered as minimum requirement to close the connections. If a parameter is non-empty but the TP Handler cannot execute the Stop command for only the specified parameter, the TP Handler may ignore the parameter and execute the StopConnection method as if the optional were empty.				
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
Initialize() was successfully called before.				

Table 5-83 ara::diag::uds_transport::UdsTransportProtocolHandler::StopConnection

5.6.12 ara::diag::uds_transport::UdsTransportProtocolHandler::Transmit

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolHandler::Transmit (
  ara::diag::uds_transport::UdsMessageConstPtr message,
  ara::diag::uds_transport::ChannelID channel_id
)
```

Description

Transmit a Uds message via the underlying Uds Transport Protocol channel

Parameter

message	The message to be transmitted as a UdsMessagePtr (unique_ptr style). UdsTransportProtocolHandler has to give back this UdsMessagePtr via UdsTransportProtocolMgr::TransmitConfirmation() to signal, that it is done with this message.
channel_id	Identification of channel on which to transmit.

Detailed Description

This transmit API covers T_Data.req of ISO 14229-2 Figure 2.

Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE

Constraints
N/A

Table 5-84 ara::diag::uds_transport::UdsTransportProtocolHandler::Transmit

based on template version 1.00.00

5.7 ara::diag::uds_transport::UdsTransportProtocolMgr

Description	
Interface for UDS transport protocol manager	
Include	**Generalize**
ara/diag/uds_transport/protocol_mgr.h	
Constructors	
UdsTransportProtocolMgr	Default constructs instance
UdsTransportProtocolMgr	Default move constructs instance
UdsTransportProtocolMgr	Default copy constructs instance
Operators	
operator=	Default move assigns instance
operator=	Default copy assigns instance
MemberFunctions	
ChannelReestablished	Notification call from the given transport channel, that it has been reestablished since the last (Re)Start from the UdsTransportProtocolHandler to which this channel belongs
HandleMessage	Hands over a valid received Uds message (currently this is only a request type) from transport layer to session layer
HandlerStopped	Notification from handler, that it has stopped now (e.g. closed down network connections, freed resources, etc...)
IndicateMessage	Indicates a message start
IndicateMessage	Indicates a message start
NotifyMessageFailure	Indicates, that the message indicated via IndicateMessage() has failure and will not lead to a final HandleMessage() call
PeriodicTransmitConfirmation	Confirmation of sent messages periodic messages
TransmitConfirmation	Notification about the outcome of a transmit request called by core DM at the handler via Uds TransportProtocolHandler::Transmit

Table 5-85 ara::diag::uds_transport::UdsTransportProtocolMgr

5.7.1 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

Prototype	
`ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr ( ) noexcept`	
Description	
Default constructs instance	
Parameter	
N/A	N/A
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-86 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

5.7.2 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

Prototype				
`ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr ( ara::diag::uds_transport::UdsTransportProtocolMgr && ) noexcept`				
Description				
Default move constructs instance				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-87 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

5.7.3 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

Prototype				
`ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr ( const class ara::diag::uds_transport::UdsTransportProtocolMgr & ) noexcept`				
Description				
Default copy constructs instance				
Parameter				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-88 ara::diag::uds_transport::UdsTransportProtocolMgr::UdsTransportProtocolMgr

5.7.4 ara::diag::uds_transport::UdsTransportProtocolMgr::~UdsTransportProtocolMgr

Prototype				
`ara::diag::uds_transport::UdsTransportProtocolMgr::~UdsTransportProtocolMgr ()`				
Description				
Destructor				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-89 ara::diag::uds_transport::UdsTransportProtocolMgr::~UdsTransportProtocolMgr

5.7.5 ara::diag::uds_transport::UdsTransportProtocolMgr::operator=

Prototype	

`ara::diag::uds_transport::UdsTransportProtocolMgr &`
`ara::diag::uds_transport::UdsTransportProtocolMgr::operator= (`
`  ara::diag::uds_transport::UdsTransportProtocolMgr &&`
`) & noexcept`	

Description	
Default move assigns instance	
Parameter	
Returns	
ara::diag::uds_transport::Uds TransportProtocolMgr &	Reference to assignee
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-90 ara::diag::uds_transport::UdsTransportProtocolMgr::operator=

5.7.6 ara::diag::uds_transport::UdsTransportProtocolMgr::operator=

Prototype				
`ara::diag::uds_transport::UdsTransportProtocolMgr & ara::diag::uds_transport::UdsTransportProtocolMgr::operator= ( const class ara::diag::uds_transport::UdsTransportProtocolMgr & ) & noexcept`				
Description				
Default copy assigns instance				
Parameter				
Returns				
ara::diag::uds_transport::Uds TransportProtocolMgr &	Reference to assignee			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-91 ara::diag::uds_transport::UdsTransportProtocolMgr::operator=

5.7.7 ara::diag::uds_transport::UdsTransportProtocolMgr::ChannelReestablished

Prototype	
`void ara::diag::uds_transport::UdsTransportProtocolMgr::ChannelReestablished ( ara::diag::uds_transport::UdsTransportProtocolMgr::GlobalChannelIdentifier global_channel_id )`	
Description	
Notification call from the given transport channel, that it has been reestablished since the last (Re)Start from the UdsTransportProtocolHandler to which this channel belongs	
Parameter	
global_channel_id	Transport protocol channel, which is available again.
Detailed Description	
To activate this notification a previous call to UdsTransportProtocolHandler::NotifyReestablishment() has to be done. See further documentation at UdsTransportProtocolHandler::NotifyReestablishment()	
Returns	
void	N/A
Errors	

N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
UdsTransportProtocolHandler::NotifyReestablishment() was successfully called before.				

Table 5-92 ara::diag::uds_transport::UdsTransportProtocolMgr::ChannelReestablished

5.7.8 ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage (
   ara::diag::uds_transport::UdsMessagePtr message
)
```

Description

Hands over a valid received Uds message (currently this is only a request type) from transport layer to session layer

Parameter

message	The Uds message ptr (unique_ptr semantics) with the request. Ownership of the UdsMessage is given back to the generic DM core here.

Detailed Description

It corresponds to T_Data.ind of Figure 2 from ISO 14229-2. The behavior is asynchronously. I.e. the UdsMessage is handed over to Session Layer and it is expected, that it "instantly" returns, which means, that real processing of the message shall be done asynchronously!

Returns

void	N/A

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE

Constraints

N/A

Table 5-93 ara::diag::uds_transport::UdsTransportProtocolMgr::HandleMessage

5.7.9 ara::diag::uds_transport::UdsTransportProtocolMgr::HandlerStopped

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolMgr::HandlerStopped (
   ara::diag::uds_transport::UdsTransportProtocolHandlerID handler_id
)
```

Description
Notification from handler, that it has stopped now (e.g. closed down network connections, freed resources, etc...)

Parameter	
handler_id	indication, which plugin stopped.

Detailed Description
This callback is expected as a reaction from handler to a call to UdsTransportProtocolHandler::Stop.

Returns	
void	N/A

Errors
N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE

Constraints
All UdsMessages must have been destructed or passed back to the ara::diag::uds_transport::UdsTransportProtocolMgr.

Table 5-94 ara::diag::uds_transport::UdsTransportProtocolMgr::HandlerStopped

5.7.10 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage

Prototype

```
std::pair<IndicationResult, UdsMessagePtr>
ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage (
   UdsMessage::Address source_addr,
   UdsMessage::Address target_addr,
   UdsMessage::TargetAddressType type,
   ara::diag::uds_transport::UdsTransportProtocolMgr::GlobalChannelIdentifier
global_channel_id,
   std::size_t size,
   ara::diag::uds_transport::Priority priority,
   ara::diag::uds_transport::ProtocolKind protocol_kind
)
```

Description
Indicates a message start

Parameter	
source_addr	UDS source address of message

target_addr	UDS target address of message
type	Indication whether its is phys/func request
global_channel_id	Transport protocol channel on which message start happened
size	Size in bytes of the UdsMessage starting from SID.
priority	The priority
protocol_kind	The protocol kind
Detailed Description	
This is an interface, which is just served/called by UdsTransportProtocolHandlers, which return true from UdsTransportProtocolHandlers::isStartOfMessageIndicationSupported().	
Returns	
std::pair<IndicationResult, UdsMessagePtr>	A pair of IndicationResult to be filled with Indication result and a pointer to UdsMessage owned/created by DM core and returned to the handler to get filled.
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-95 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage

5.7.11 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage

Prototype

```
std::pair<IndicationResult, UdsMessagePtr>
ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage (
  UdsMessage::Address source_addr,
  UdsMessage::Address target_addr,
  UdsMessage::TargetAddressType type,
  ara::diag::uds_transport::UdsTransportProtocolMgr::GlobalChannelIdentifier
global_channel_id,
  std::size_t size,
  ara::diag::uds_transport::Priority priority,
  ara::diag::uds_transport::ProtocolKind protocol_kind,
  ara::core::Span<const std::uint8_t> payload_info
)
```

Description

Indicates a message start

Parameter	
source_addr	UDS source address of message

target_addr	UDS target address of message
type	Indication whether its is phys/func request
global_channel_id	Transport protocol channel on which message start happened
size	Size in bytes of the UdsMessage starting from SID.
priority	The priority
protocol_kind	The protocol kind
payload_info	View onto the first received payload bytes, if any.
Detailed Description	
This is an interface, which is just served/called by UdsTransportProtocolHandlers, which return true from UdsTransportProtocolHandlers::isStartOfMessageIndicationSupported().	
Returns	
std::pair<IndicationResult, UdsMessagePtr>	A pair of IndicationResult to be filled with Indication result and a pointer to UdsMessage owned/created by DM core and returned to the handler to get filled.
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-96 ara::diag::uds_transport::UdsTransportProtocolMgr::IndicateMessage

5.7.12 ara::diag::uds_transport::UdsTransportProtocolMgr::NotifyMessageFailure

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolMgr::NotifyMessageFailure (
  ara::diag::uds_transport::UdsMessagePtr message
)
```

Description

Indicates, that the message indicated via IndicateMessage() has failure and will not lead to a final HandleMessage() call

Parameter	
message	The pointer to UdsMessage handed back over to the session layer.
Returns	
void	N/A
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-97 ara::diag::uds_transport::UdsTransportProtocolMgr::NotifyMessageFailure

5.7.13 ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation (
  ara::core::Vector<UdsMessageConstPtr> messages,
  std::size_t numberOfSentMessages
)
```

Description

Confirmation of sent messages periodic messages

Parameter	
messages	The same ordered list of messages previously passed to UdsTransportProtocolPeriodicHandler::PeriodicTransmit
numberOfSentMessages	The number of successfully sent messages from the "messages" list

Returns	
void	N/A

Errors

N/A

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-98 ara::diag::uds_transport::UdsTransportProtocolMgr::PeriodicTransmitConfirmation

5.7.14 ara::diag::uds_transport::UdsTransportProtocolMgr::TransmitConfirmation

Prototype

```
void
ara::diag::uds_transport::UdsTransportProtocolMgr::TransmitConfirmation (
  ara::diag::uds_transport::UdsMessageConstPtr message,
  enum ara::diag::uds_transport::UdsTransportProtocolMgr::TransmissionResult
result
)
```

Description				
Notification about the outcome of a transmit request called by core DM at the handler via Uds TransportProtocolHandler::Transmit				
Parameter				
message	For which message (created in IndicateMessage()) this is the confirmation.			
result	Result of transmission. In case UDS message could be transmitted on network layer: kTransmitOk), kTransmitFailed else.			
Detailed Description				
This transmit API covers T_Data.con of ISO 14229-2 Figure 2.				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-99 ara::diag::uds_transport::UdsTransportProtocolMgr::TransmitConfirmation

based on template version 1.00.00

5.8 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler

Description	
Interface for UDS transport protocol periodic handler	
Include	**Generalize**
ara/diag/uds_transport/protocol_periodic_handler.h	
Detailed Description	
UdsTransportProtocolPeriodicHandler class to support 0x2A service from ISO.	
Constructors	
UdsTransportProtocolPeriodic Handler	Default constructs instance
UdsTransportProtocolPeriodic Handler	Default move constructs instance
UdsTransportProtocolPeriodic Handler	Default copy constructs instance
Operators	
operator=	Default move assigns instance
operator=	Default copy assigns instance
MemberFunctions	
GetMaxPayloadLength	Reports the maximum payload length supported for a single periodic transmission on the channel
GetNumberOfPeriodicMessages	Reports the TP implementation and connection specific number of periodic messages
PeriodicTransmit	Sends all the messages in the list in the given order. If one message transmission fails, the send process is stopped and the PeriodicTransmitConfirmation is invoked with the number of sent messages and the same Vector with UdsMessages

Table 5-100 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler

5.8.1 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler:: UdsTransportProtocolPeriodicHandler

Prototype	
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler:: UdsTransportProtocolPeriodicHandler () noexcept`	
Description	
Default constructs instance	
Parameter	
N/A	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-101 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler

5.8.2 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler:: UdsTransportProtocolPeriodicHandler

Prototype

```
ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
UdsTransportProtocolPeriodicHandler (
  ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler &&
) noexcept
```

Description

Default move constructs instance

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

Table 5-102 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler

5.8.3 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler:: UdsTransportProtocolPeriodicHandler

Prototype

```
ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::
UdsTransportProtocolPeriodicHandler (
  const class ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler &
) noexcept
```

Description

Default copy constructs instance

Parameter

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

Table 5-103 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::UdsTransportProtocolPeriodicHandler

5.8.4 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::~UdsTransportProtocolPeriodicHandler

Prototype
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::~UdsTransportProtocolPeriodicHandler ()`
Description
Destructor
Errors
N/A
Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints
N/A

Table 5-104 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::~UdsTransportProtocolPeriodicHandler

5.8.5 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator=

Prototype
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler & ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= ( ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler && ) & noexcept`
Description
Default move assigns instance
Parameter

Returns	
ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler &	Reference to assignee
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints
N/A

Table 5-105 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator=

5.8.6 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator=

Prototype				
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler & ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator= ( const class ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler & ) & noexcept`				
Description				
Default copy assigns instance				
Parameter				
Returns				
ara::diag::uds_transport::Uds TransportProtocolPeriodicHandler &	Reference to assignee			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-106 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::operator=

5.8.7 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetMaxPayloadLength

Prototype	
`std::size_t ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetMaxPayloadLength ( ara::diag::uds_transport::ChannelID channel_id ) const`	
Description	
Reports the maximum payload length supported for a single periodic transmission on the channel	
Parameter	
channel_id	The concrete connection for which the maximum payload length is reported
Returns	
std::size_t	The supported payload length
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-107 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetMaxPayloadLength

5.8.8 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetNumberOfPeriodicMessages

Prototype

`std::size_t`
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::`
`GetNumberOfPeriodicMessages (`
`  ara::diag::uds_transport::ChannelID channel_id`
`) const`

Description
Reports the TP implementation and connection specific number of periodic messages

Parameter	
channel_id	The concrete connection to send over
Returns	
std::size_t	The number of periodic messages
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-108 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::GetNumberOfPeriodicMessages

5.8.9 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::PeriodicTransmit

Prototype

`void`
`ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::PeriodicTransmit (`
`  ara::diag::uds_transport::ChannelID channel_id,`
`  ara::core::Vector<UdsMessageConstPtr> messages`
`)`

Description

Sends all the messages in the list in the given order. If one message transmission fails, the send process is stopped and the PeriodicTransmitConfirmation is invoked with the number of sent messages and the same Vector with UdsMessages				
Parameter				
channel_id		The concrete connection to send over		
messages		Ordered list of messages to send at once		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-109 ara::diag::uds_transport::UdsTransportProtocolPeriodicHandler::PeriodicTransmit

based on template version 1.00.00

5.9 ara::diag::uds_transport::UdsMessage

Description	
Represents an UDS message exchanged between DM generic core (UdsTransport ProtocolMgr) and a specific implementation of UdsTransportProtocolHandler on diagnostic request reception path or diagnostic response transmission path	
Include	**Generalize**
ara/diag/uds_transport/uds_message.h	
Detailed Description	
UdsMessage provides the storage for UDS requests/responses. Instances of UdsMessage (with optimized resource allocation) are only created by DM generic core. UdsTransport ProtocolHandler read/write on it.	
Constructors	
UdsMessage	Move constructing the UDS message
UdsMessage	Copy constructing the UDS message
Operators	
operator=	Move assigning the UDS message
operator=	Copy assigning the UDS message
MemberFunctions	
AddMetaInfo	Add new metaInfo to this message.
AddMetaInfo	Add new metaInfo as shared object to this message.
Capacity	Reports the original pre-allocated buffer capacity
ChangePayloadView	Changes the Payload() view over the pre-allocated buffer
GetGlobalChannelIdentifier	Return the global channel identifier
GetPayload	Return the underlying buffer for write access
GetPayload	Get the UDS message data starting with the SID (A_Data as per ISO)
GetSa	Get the source address of the uds message
GetTa	Get the target address of the uds message
GetTaType	Get the target address type (phys/func) of the uds message

Table 5-110 ara::diag::uds_transport::UdsMessage

5.9.1 ara::diag::uds_transport::UdsMessage::UdsMessage

Prototype

```
ara::diag::uds_transport::UdsMessage::UdsMessage (
   ara::diag::uds_transport::UdsMessage && other
) noexcept
```

Description

Move constructing the UDS message

Parameter

other		Object to move-construct from		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-111 ara::diag::uds_transport::UdsMessage::UdsMessage

5.9.2 ara::diag::uds_transport::UdsMessage::UdsMessage

Prototype				
`ara::diag::uds_transport::UdsMessage::UdsMessage ( const class ara::diag::uds_transport::UdsMessage & other ) noexcept`				
Description				
Copy constructing the UDS message				
Parameter				
other		Object to copy-construct from		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-112 ara::diag::uds_transport::UdsMessage::UdsMessage

5.9.3 ara::diag::uds_transport::UdsMessage::~UdsMessage

Prototype				
`ara::diag::uds_transport::UdsMessage::~UdsMessage ()`				
Description				
Destructing the uds message				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE

Constraints

Table 5-113 ara::diag::uds_transport::UdsMessage::~UdsMessage

5.9.4 ara::diag::uds_transport::UdsMessage::operator=

Prototype				

`ara::diag::uds_transport::UdsMessage &`
`ara::diag::uds_transport::UdsMessage::operator= (`
`  ara::diag::uds_transport::UdsMessage && other`
`) & noexcept`				

Description				
Move assigning the UDS message				
Parameter				
other	Object to move-assign from.			
Returns				
ara::diag::uds_transport::Uds Message &	Moved instance			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	FALSE	FALSE
Constraints				
N/A				

Table 5-114 ara::diag::uds_transport::UdsMessage::operator=

5.9.5 ara::diag::uds_transport::UdsMessage::operator=

Prototype	

`ara::diag::uds_transport::UdsMessage &`
`ara::diag::uds_transport::UdsMessage::operator= (`
`  const class ara::diag::uds_transport::UdsMessage & other`
`) & noexcept`	

Description	
Copy assigning the UDS message	
Parameter	
other	Object to copy-assign from.
Returns	
ara::diag::uds_transport::Uds Message &	Copied instance
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-115 ara::diag::uds_transport::UdsMessage::operator=

5.9.6 ara::diag::uds_transport::UdsMessage::AddMetaInfo

Prototype	
`void ara::diag::uds_transport::UdsMessage::AddMetaInfo ( ara::diag::uds_transport::UdsMessage::MetaInfoMap meta_info )`	
Description	
Add new metaInfo to this message Typically called by the transport plugin to add channel specific meta-info. (see SWS - there are already predefined meta-info keys for DoIP....) If the meta info map storage is full already, the call is ignored.	
Parameter	
meta_info	meta information relevant for UdsMessage
Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-116 ara::diag::uds_transport::UdsMessage::AddMetaInfo

5.9.7 ara::diag::uds_transport::UdsMessage::AddMetaInfo

Prototype
`void ara::diag::uds_transport::UdsMessage::AddMetaInfo ( std::shared_ptr<const MetaInfoMap> meta_info )`
Description
Add new metaInfo as shared object to this message Typically called by the transport plugin to add channel specific meta-info. (see SWS - there are already predefined meta-info keys for DoIP....) If the meta info map storage is full already, the call is ignored.

Parameter				
meta_info		meta information relevant for UdsMessage		
Returns				
void		N/A		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-117 ara::diag::uds_transport::UdsMessage::AddMetaInfo

5.9.8 ara::diag::uds_transport::UdsMessage::Capacity

Prototype				
`std::size_t` `ara::diag::uds_transport::UdsMessage::Capacity () const noexcept`				
Description				
Reports the original pre-allocated buffer capacity				
Parameter				
N/A		N/A		
Returns				
std::size_t		The original buffer capacity		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-118 ara::diag::uds_transport::UdsMessage::Capacity

5.9.9 ara::diag::uds_transport::UdsMessage::ChangePayloadView

Prototype

`void`
`ara::diag::uds_transport::UdsMessage::ChangePayloadView (`
`  std::size_t new_begin,`
`  std::size_t new_size`
`) noexcept`

Description				
Changes the Payload() view over the pre-allocated buffer				
Parameter				
new_begin	The new start offset of the payload. Valid range: [0..(Capacity()-1)]			
new_size	The new size of the payload. Valid range: [0..Capacity()-new_begin]			
Detailed Description				
The new size affects the GetPayload() result only				
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-119 ara::diag::uds_transport::UdsMessage::ChangePayloadView

5.9.10 ara::diag::uds_transport::UdsMessage::GetGlobalChannelIdentifier

Prototype				
`std::tuple<UdsTransportProtocolHandlerID, ChannelID>` `ara::diag::uds_transport::UdsMessage::GetGlobalChannelIdentifier () const`				
Description				
Return the global channel identifier				
Parameter				
N/A	N/A			
Returns				
std::tuple<UdsTransportProtocol HandlerID, ChannelID>	Tuple (HandlerID, ChannelID): - HandlerID: The ID of the protocol handler - ChannelID: The ID of the communication channel			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	TRUE
Constraints				
N/A				

Table 5-120 ara::diag::uds_transport::UdsMessage::GetGlobalChannelIdentifier

5.9.11 ara::diag::uds_transport::UdsMessage::GetPayload

Prototype	
`ara::diag::uds_transport::ByteVector &` `ara::diag::uds_transport::UdsMessage::GetPayload ()`	
Description	
Return the underlying buffer for write access	
Parameter	
N/A	N/A
Detailed Description	
Needed by Plugin implementation to fill the created UdsMessage with data. I.e. plugin impl. will typically create UdsMessage via createUdsMessage() and then call this method on it and write into returned uds_transport::ByteVector. Marked as "unsafe" with regard to threadsafety as implementation is allowed to do resource allocation of buffer in the context of this call.	
Returns	
ara::diag::uds_transport::ByteVector &	Payload of the UDSMessage starting from SID.
Errors	
N/A	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-121 ara::diag::uds_transport::UdsMessage::GetPayload

5.9.12 ara::diag::uds_transport::UdsMessage::GetPayload

Prototype	
`const ara::diag::uds_transport::ByteVector &` `ara::diag::uds_transport::UdsMessage::GetPayload () const`	
Description	
Get the UDS message data starting with the SID (A_Data as per ISO)	
Parameter	
N/A	N/A
Detailed Description	
Marked as "unsafe" with regard to threadsafety as implementation is allowed to do resource allocation of buffer in the context of this call.	
Returns	

const ara::diag::uds_transport::Byte Vector &	The entire payload (A_Data)
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	TRUE
Constraints				
N/A				

Table 5-122 ara::diag::uds_transport::UdsMessage::GetPayload

5.9.13 ara::diag::uds_transport::UdsMessage::GetSa

Prototype	
`ara::diag::uds_transport::UdsMessage::Address ara::diag::uds_transport::UdsMessage::GetSa () const noexcept`	
Description	
Get the source address of the uds message	
Parameter	
N/A	N/A
Returns	
ara::diag::uds_transport::Uds Message::Address	The source address of the uds message.
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-123 ara::diag::uds_transport::UdsMessage::GetSa

5.9.14 ara::diag::uds_transport::UdsMessage::GetTa

Prototype	
`ara::diag::uds_transport::UdsMessage::Address ara::diag::uds_transport::UdsMessage::GetTa () const noexcept`	
Description	
Get the target address of the uds message	
Parameter	
N/A	N/A

Returns	
ara::diag::uds_transport::Uds Message::Address	The target address of the uds message.
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-124 ara::diag::uds_transport::UdsMessage::GetTa

5.9.15 ara::diag::uds_transport::UdsMessage::GetTaType

Prototype	
`enum ara::diag::uds_transport::UdsMessage::TargetAddressType ara::diag::uds_transport::UdsMessage::GetTaType () const noexcept`	
Description	
Get the target address type (phys/func) of the uds message	
Parameter	
N/A	N/A
Returns	
enum ara::diag::uds_transport::Uds Message::TargetAddressType	The target address type of the uds message.
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	TRUE	TRUE	FALSE
Constraints				
N/A				

Table 5-125 ara::diag::uds_transport::UdsMessage::GetTaType

5.10 amsr::uds_transport::CancellationHandler

Description	
CancellationHandler contains a shared state if the processing should be canceled	
Include	**Generalize**
amsr/uds_transport/cancellation_handler.h	
Detailed Description	
This object shall not be stored or accessed beyond the lifetime of the operation at which this object was received.	
Constructors	
CancellationHandler	Move constructs instance of CancellationHandler
Operators	
operator=	Move assigns instance of CancellationHandler
MemberFunctions	
IsCanceled	Returns the cancellation state of this handler
SetNotifier	Registering a notifier function which is called if the associated operation is canceled

Table 5-126 amsr::uds_transport::CancellationHandler

5.10.1 amsr::uds_transport::CancellationHandler::CancellationHandler

Prototype

```
amsr::uds_transport::CancellationHandler::CancellationHandler (
   amsr::uds_transport::CancellationHandler && other
) noexcept
```

Description				
Move constructs instance of CancellationHandler				
Parameter				
other		The object used for move-constructor		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-127 amsr::uds_transport::CancellationHandler::CancellationHandler

5.10.2 amsr::uds_transport::CancellationHandler::~CancellationHandler

Prototype

```
amsr::uds_transport::CancellationHandler::~CancellationHandler () noexcept
```

Description				
Destructs an instance of CancellationHandler				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-128 amsr::uds_transport::CancellationHandler::~CancellationHandler

5.10.3 amsr::uds_transport::CancellationHandler::operator=

Prototype	

`amsr::uds_transport::CancellationHandler &`
`amsr::uds_transport::CancellationHandler::operator= (`
`  amsr::uds_transport::CancellationHandler && other`
`) & noexcept`	

Description	
Move assigns instance of CancellationHandler	
Parameter	
other	The object used for move-assignment
Returns	
amsr::uds_transport::Cancellation Handler &	Reference to self
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 5-129 amsr::uds_transport::CancellationHandler::operator=

5.10.4 amsr::uds_transport::CancellationHandler::IsCanceled

Prototype
`bool` `amsr::uds_transport::CancellationHandler::IsCanceled () const noexcept`
Description
Returns the cancellation state of this handler

Parameter	
N/A	N/A
Returns	
bool	True if the associated operation is cancelled, false otherwise
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints
N/A

Table 5-130 amsr::uds_transport::CancellationHandler::IsCanceled

5.10.5 amsr::uds_transport::CancellationHandler::SetNotifier

Prototype

```
void
amsr::uds_transport::CancellationHandler::SetNotifier (
   amsr::uds_transport::CancellationHandler::Notifier notifier
) noexcept
```

Description

Registering a notifier function which is called if the associated operation is canceled

Parameter	
notifier	Notification function that is called upon cancellation

Detailed Description

A previously set notifier gets overwritten. The notifier will be invoked immediately if processing is already cancelled (the notifier is then executed in the thread that calls SetNotifier).

Returns	
void	N/A
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE

Constraints

The passed in function must not throw any exceptions.

The passed in function must be valid to be called until the destruction of this CancellationHandler has completed or until it is replaced by a new notifier function.

The passed in function must not set any other notifier to this CancellationHandler.

Table 5-131 amsr::uds_transport::CancellationHandler::SetNotifier

based on template version 1.00.00

5.11 amsr::uds_transport::MetaInfo

Description	
Type definition of a MetaInfo class	
Include	**Generalize**
amsr/uds_transport/meta_info.h	
Constructors	
MetaInfo	Move constructor
Operators	
operator=	Move assignment
MemberFunctions	
GetValue	Returns the stored value for the given key

Table 5-132 amsr::uds_transport::MetaInfo

5.11.1 amsr::uds_transport::MetaInfo::MetaInfo

Prototype

```
amsr::uds_transport::MetaInfo::MetaInfo (
  amsr::uds_transport::MetaInfo && other
) noexcept
```

Description

Move constructor

Parameter

other	The object used for move-constructor

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 5-133 amsr::uds_transport::MetaInfo::MetaInfo

5.11.2 amsr::uds_transport::MetaInfo::~MetaInfo

Prototype

```
amsr::uds_transport::MetaInfo::~MetaInfo () noexcept
```

Description

Destructs the meta info

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-134 amsr::uds_transport::MetaInfo::~MetaInfo

5.11.3 amsr::uds_transport::MetaInfo::operator=

Prototype				

`amsr::uds_transport::MetaInfo &`
`amsr::uds_transport::MetaInfo::operator= (`
`  amsr::uds_transport::MetaInfo && other`
`) & noexcept`				

Description				
Move assignment				
Parameter				
other	The object used for move-assignment			
Returns				
amsr::uds_transport::MetaInfo &	Reference to the meta info			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-135 amsr::uds_transport::MetaInfo::operator=

5.11.4 amsr::uds_transport::MetaInfo::GetValue

Prototype	

`ara::core::Result<ara::core::StringView>`
`amsr::uds_transport::MetaInfo::GetValue (`
`  const ara::core::String & key`
`) const noexcept`	

Description	
Returns the stored value for the given key	
Parameter	
key	The requested key
Returns	

ara::core::Result<ara::core::StringView>	The stored value, or an error if the key does not exist			
Errors				
ara::diag::uds_transport::UdsTransportErrc::kInvalidParameter No entry exists for the given key				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-136 amsr::uds_transport::MetaInfo::GetValue

based on template version 1.00.00

5.12 amsr::uds_transport::ServiceValidation

Description	
The interface of in-library service validation (to be implemented by a SDT plugin)	
Include	**Generalize**
amsr/uds_transport/service_validation.h	
Constructors	
ServiceValidation	Constructor of ServiceValidation
ServiceValidation	Move constructs an instance of ServiceValidation
ServiceValidation	Copy constructs an instance of ServiceValidation
MemberFunctions	
Confirmation	Handles a service confirmation
Validate	Handles a service validation

Table 5-137 amsr::uds_transport::ServiceValidation

5.12.1 amsr::uds_transport::ServiceValidation::ServiceValidation

Prototype				
`amsr::uds_transport::ServiceValidation::ServiceValidation () noexcept`				
Description				
Constructor of ServiceValidation				
Parameter				
N/A	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-138 amsr::uds_transport::ServiceValidation::ServiceValidation

5.12.2 amsr::uds_transport::ServiceValidation::ServiceValidation

Prototype	

`amsr::uds_transport::ServiceValidation::ServiceValidation (`
`  amsr::uds_transport::ServiceValidation && other`
`) noexcept`	

Description	
Move constructs an instance of ServiceValidation	
Parameter	
other	Object to move-construct from

Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-139 amsr::uds_transport::ServiceValidation::ServiceValidation

5.12.3 amsr::uds_transport::ServiceValidation::ServiceValidation

Prototype	
`amsr::uds_transport::ServiceValidation::ServiceValidation ( const class amsr::uds_transport::ServiceValidation & other ) noexcept`	
Description	
Copy constructs an instance of ServiceValidation	
Parameter	
other	Object to copy-construct from
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 5-140 amsr::uds_transport::ServiceValidation::ServiceValidation

5.12.4 amsr::uds_transport::ServiceValidation::~ServiceValidation

Prototype				
`amsr::uds_transport::ServiceValidation::~ServiceValidation () noexcept`				
Description				
Destructor of ServiceValidation				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				

N/A

Table 5-141 amsr::uds_transport::ServiceValidation::~ServiceValidation

5.12.5 amsr::uds_transport::ServiceValidation::Confirmation

Prototype				
`void amsr::uds_transport::ServiceValidation::Confirmation ( enum amsr::uds_transport::ServiceValidation::ConfirmationStatusType status, const class amsr::uds_transport::MetaInfo & meta_info ) noexcept`				
Description				
Handles a service confirmation				
Parameter				
status	Status/outcome of the service processing			
meta_info	MetaInfo of the request			
Returns				
void	N/A			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 5-142 amsr::uds_transport::ServiceValidation::Confirmation

5.12.6 amsr::uds_transport::ServiceValidation::Validate

Prototype	
`ara::core::Future<void, ValidationError> amsr::uds_transport::ServiceValidation::Validate ( ara::core::Span<const std::uint8_t> request_data, const class amsr::uds_transport::MetaInfo & meta_info, amsr::uds_transport::CancellationHandler cancellation_handler ) noexcept`	
Description	
Handles a service validation	
Parameter	
request_data	Diagnostic request data (including SID)
meta_info	MetaInfo of the request

cancellation_handler	Informs if the current validation operation is canceled. This object shall not be stored or accessed beyond the lifetime of the validation operation. Once a cancellation is communicated by the DM, the returned future must be set to ready as soon as possible.
Returns	
ara::core::Future<void, Validation Error>	A future of type void or an UDS NRC value (for a negative response message)
Errors	
amsr::uds_transport::UdsNegativeResponseCode Uds NRC set by the user	
ara::core::ErrorCode Errorcode containing a raw error code value and an error domain. Note: assigning a non UDS NRC will result in service validation returning an NRC (refer to Technical Reference for details).	

Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
FALSE	ANY	FALSE	TRUE	FALSE
Constraints				
N/A				

Table 5-143 amsr::uds_transport::ServiceValidation::Validate

based on template version 1.00.00

5.13 amsr::uds_transport::ServiceValidation::ValidationError

Description	
Wrapper for the UDS negative response code that can be used as the error type of a Future	
Include	**Generalize**
amsr/uds_transport/service_validation.h	
Constructors	
ValidationError	Creates a validation error from a UDS NRC
ValidationError	Creates a validation error from an error code
ValidationError	Move constructor
ValidationError	Copy constructor
Operators	
operator=	Move assignment
operator=	Copy assignment
MemberFunctions	
GetError	Retrieves the contained error

Table 5-144 amsr::uds_transport::ServiceValidation::ValidationError

5.13.1 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

Prototype

```
amsr::uds_transport::ServiceValidation::ValidationError::ValidationError (
   amsr::uds_transport::UdsNegativeResponseCode uds_nrc
) noexcept
```

Description

Creates a validation error from a UDS NRC

Parameter

uds_nrc	Uds Negative Response Code

Detailed Description

Constructor shall not be explicit to enable implicit assignment of errors by the user.

Errors

N/A

Particularities and Limitations

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE

Constraints

N/A

Table 5-145 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

5.13.2 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

Prototype

```
amsr::uds_transport::ServiceValidation::ValidationError::ValidationError (
  ara::core::ErrorCode error_code
) noexcept
```

Description	
Creates a validation error from an error code	
Parameter	
error_code	Errorcode containing a raw error code value and an error domain.
Detailed Description	
Constructor shall not be explicit to enable implicit assignment of errors by the user. Assigning a non Uds NRC will result in service validation returning an NRC (refer to Technical Reference for details).	
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-146 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

5.13.3 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

Prototype	

```
amsr::uds_transport::ServiceValidation::ValidationError::ValidationError (
  amsr::uds_transport::ServiceValidation::ValidationError && other
) noexcept
```

Description	
Move constructor	
Parameter	
other	The object used for move-constructor
Errors	
N/A	
Particularities and Limitations	

Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-147 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

5.13.4 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

Prototype				
`amsr::uds_transport::ServiceValidation::ValidationError::ValidationError ( const class amsr::uds_transport::ServiceValidation::ValidationError & other ) noexcept`				
Description				
Copy constructor				
Parameter				
other	The object used for copy-constructor			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-148 amsr::uds_transport::ServiceValidation::ValidationError::ValidationError

5.13.5 amsr::uds_transport::ServiceValidation::ValidationError::~ValidationError

Prototype				
`amsr::uds_transport::ServiceValidation::ValidationError::~ValidationError ( ) noexcept`				
Description				
Destructor				
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-149 amsr::uds_transport::ServiceValidation::ValidationError::~ValidationError

5.13.6 amsr::uds_transport::ServiceValidation::ValidationError::operator=

Prototype
`amsr::uds_transport::ServiceValidation::ValidationError & amsr::uds_transport::ServiceValidation::ValidationError::operator= ( amsr::uds_transport::ServiceValidation::ValidationError && other ) & noexcept`
Description

Move assignment				
Parameter				
other		The object used for move-assignment		
Returns				
amsr::uds_transport::Service Validation::ValidationError &		Reference to object		
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-150 amsr::uds_transport::ServiceValidation::ValidationError::operator=

5.13.7 amsr::uds_transport::ServiceValidation::ValidationError::operator=

Prototype	
`amsr::uds_transport::ServiceValidation::ValidationError & amsr::uds_transport::ServiceValidation::ValidationError::operator= ( const class amsr::uds_transport::ServiceValidation::ValidationError & other ) & noexcept`	
Description	
Copy assignment	
Parameter	
other	The object used for copy-assignment
Returns	
amsr::uds_transport::Service Validation::ValidationError &	Reference to object
Errors	
N/A	
Particularities and Limitations	
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints	
N/A	

Table 5-151 amsr::uds_transport::ServiceValidation::ValidationError::operator=

5.13.8 amsr::uds_transport::ServiceValidation::ValidationError::GetError

Prototype

`amsr::uds_transport::ServiceValidation::ValidationError::Error amsr::uds_transport::ServiceValidation::ValidationError::GetError () const noexcept`				
Description				
Retrieves the contained error				
Parameter				
N/A	N/A			
Returns				
amsr::uds_transport::Service Validation::ValidationError::Error	Variant containing either UDS negative response code or error code			
Errors				
N/A				
Particularities and Limitations				
Synchronous	Call Context	Reentrant	ThreadSafe	Can Throw
TRUE	ANY	FALSE	FALSE	FALSE
Constraints				
N/A				

Table 5-152 amsr::uds_transport::ServiceValidation::ValidationError::GetError

6 Configuration

According to the component design described in section 1.1.2, the DM consists of multiple parts: The Diagnostics Daemon executable composed of multiple Diagnostic Server and Transport Protocols (e.g. DoIP), the Diagnostics Library (ara::diag) which is part of the custom diagnostic application executable and the Intrusion Detection System Manager Library (ara::idsm) which is part of the custom Idsm application.

This chapter describes the diagnostic specific compile-time and the runtime configurations for Diagnostics Daemon executable and custom diagnostic application executable. The configuration of custom Idsm application is described in the technical reference of Idsm [23]. The configuration of components used by the Diagnostics Daemon according to the Architectural Overview section 1.1.2 is described in the technical references of the respective components.

6.1 Used Generators

The DM uses the following generators:

- `amsr_aradiag_api`: Provides the diagnostic interfaces for the custom diagnostic application. For more details, please refer to [23].
- `amsr_diagnosticmanager_daemon`: Provides the Diagnostics Daemon JSON file.
- `amsr_aradidsm_api`: Provides the json configuration for the custom Idsm application.
- `amsr_modelleddatatypes_api`: Provides data types necessary for the diagnostic interfaces.
- `amsr_persistency_config`: Provides NV storage to persist information over reboot.
- `amsr_socal`: Provides service interface proxies / skeletons.
- `amsr_em_executionmanager_execconfig`: Provides configuration for the Execution Manager.
- `amsr_applicationbase`: Provides code to initialize the adaptive platform core functionality.
- `amsr_ipcbinding`: Provides IPC binding for service interface deployment.
- `amsr_someipprotocol`: Provides SOME/IP binding for service interface deployment.
- `amsr_logapi_config`: Provides the logging configuration file.

The trigger for the generators to generate output files is the `Executable` model element: an `Executable` that refers to some `DiagnosticPortInterface` triggers the `amsr_aradiag_api` to generate diagnostic interfaces, and the `Executable` marked as `DIAGNOSTIC_MANAGER` triggers the `amsr_diagnosticmanager_daemon` to generate the Diagnostics Daemon configuration files, see section 3.3 for details on how to model such a marker.

The following chapters provide detailed information about the DM configuration.

6.2 MEX Configuration

This chapter gives an overview about the relevant MEX parameter definitions.

based on template version 1.00.00

6.2.1 MEX parameter in Diagnostics Library (ara::diag)

MEX Parameter	Description	Refer to
`numberOfParallelRequestExecutions`	The number of requests that can be handled in parallel.	6.11.1
`requestQueueSize`	The maximum queue size for requests from the Diagnostics Daemon.	6.11.1
`interfaceType`	Interface type of an extended `P-PORT` or extended `R-PORT`.	e. g. 6.18.2

Table 6-1 Description of Diagnostics Library (ara::diag) executable's MEX parameter

Logging of request queue size upon shutdown

Upon graceful shutdown, the overall maximum queue size is printed via an info log message: "The overall maximum job queue utilization was...". The associated logging context is "vdal".

MEX Parameter	Description	Refer to
`memoryResourceProps`	The properties to configure the memory resources per executable.	6.29.1
`memoryResourceManagement`	The memory resource management configuration of the executable.	6.29.2

Table 6-2 Description of Diagnostics Library (ara::diag) executable's memory management MEX parameter

6.2.2 MEX parameter in Diagnostics Daemon

MEX Parameter	Description	Refer to
`subServiceKind`	Describes the custom service instance type, either `SUB_FUNCTION_ID`, or sub-service, or `NONE`.	2.1.5.12
`timeout`	The authentication timeout value.	6.12
`timerMode`	The authentication timer mode value.	6.12
`evaluationId`	The evaluation identifier.	6.12
`subServiceId`	Custom service instance extension. Must be configured if subServiceKind has a value different than `NONE`.	2.1.5.12
`sourceAddressCode`	The Diagnostic Client Authentication source address code.	6.13.1
`sourceAddressMask`	The Diagnostic Client Authentication source address mask.	6.13.1

MEX Parameter	Description	Refer to
functionalityType	An extension of DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS. It describes the type of swc dependency in case of mapping is impossible to achieve such situations as *ServiceProcessingNotification*, *SecurityEventLog*.	6.15.4, 6.17
serviceValidation	The service validation configuration.	6.5.1
maxPayloadBytes	The maximum number of bytes in payload.	6.6
isDefault	A default authentication role can be set by setting this parameter.	6.13
maxNumChangeOfDataIdentifierEvents	Maximum number of change of data identifier event.	2.1.5.11.1
fileHandlingMode	File handling mode for Fault Memory. Can either be Multifile (default) or Singlefile.	6.8
freezeOnCount	Represents a seizing threshold for the event counter increment.	6.15
eventId	Represents the event counter ID.	6.15
persistentStorage	Indicates whether the data related to an event type shall be persisted.	6.15
numberOfBufferEntries	Represents the number of buffer entries in an event type.	6.15
numberOfBytesPerEntry	Represents the max number of bytes per entry in a buffer of an event type.	6.15
logType	Represents the type of security event log.	6.15, 6.16
numberOfAsynchronousJobs	Represents the maximum number of asynchronous jobs for Idsm functionality.	6.16
fullyQualifiedCreatorFunction	The transport protocol creator function configuration.	3.3.1.3
creatorHeaderFilePath	The transport protocol creator header file path.	3.3.1.3
creatorFunctionType	Transport protocol creator function type. Either CUSTOM_FUNCTION or CONSTRUCTOR.	3.3.1.3
persistencyUpdateRate	The frequency of checks for a pending persistency update (milliseconds). A background task will write out changes in DTC data with this frequency. Data is also written out on shutdown.	6.8
numberOfConversations	The number of parallel testers supported by a software cluster (positive number).	2.1.3
udsMessageLength	The maximum length of a UDS request or response message supported by a software cluster (positive number).	N/A

MEX Parameter	Description	Refer to
`p2ServerAdjustTime`	Time offset, subtracted from the P2-time to compensate internal thread scheduling effects (milliseconds).	N/A
`p2StarServerAdjustTime`	Time offset, subtracted from the P2*-time to compensate internal thread scheduling effects (milliseconds).	N/A
`s3ServerTimeoutEnabled`	Switch to Default session after S3-timeout (true/-false). Default, when missing: true.	N/A
`s3ServerTimeout`	Set S3-timeout to duration specified (in seconds). Default, 5s.	2.1.5.1.1
`maxNumberOfPeriodicMessages`	The number of periodic messages of the transport protocol that has the highest number of periodic messages among all the subscribed transport protocols.	2.1.5.4.3
`maxPeriodicMessageLength`	The maximum periodic message payload length of the transport protocol that has the largest maximum periodic message payload length amon all the subscribed transport protocols.	2.1.5.4.3
`maxNumberOfJobs`	The number of parallel RoutineControl or DID DataElement read/write jobs (positive number). Should be at least `numberOfConversations` +1.	2.1.3
`numberOfZeroCopySlots`	The number of zero copy slots available for pipelined data transfer to the ECU. Defaults to zero when not configured.	2.1.5.9.6
`numberOfDataCollecting ThreadsPerEventMemory`	The number of threads that collect snapshot data (positive number).	2.1.9
`dataCollectingTimeout`	The timeout for reading snapshot data from an application (seconds).	2.1.9
`useSharedAttemptCounter`	The number of threads that collect snapshot data (positive number).	2.1.5.5
`useSharedTimer`	Use shared security attempt delay timer (true/-false).	2.1.5.5
`delayTimeOnFailedNvDataRecovery`	Time to prevent security access on persistent data read failure (seconds).	2.1.5.5
`useMemoryIdentifier`	Need `MemorySelection` dataparameter from ISO14229-1:2020 in ClearDiagnosticInformation `0x14` request message (true/false). Default, when missing: false.	2.1.5.3.1
`schedulerRate`	The scheduler rate in microseconds If not defined the fastest rate of the available modes (Fast,Slow;medium) is used.	2.1.5.4.3

MEX Parameter	Description	Refer to
`pdidStopOnStateChange`	Automatical stopping of a PDID's responses upon ECU transition to a session or a security level where the PDID is not supported (true/-false).	2.1.5.4.3
`pdidReadMode`	Determines readability of a PDID according to the alteration in the cached data. Has a value of: Any if a periodic response for a PDID is required regardless whether cached data has been updated or already read NewOnly if a periodic response for a PDID is required only when cached data has been updated.	2.1.5.4.3
`ignorePreconditions`	Determines suppression of authentication preconditions evaluation.	2.1.4.4.3
`ethernetPort`	Indicates ethernet port class.	6.16.3, 6.26.2
`sovdModuleInstantiation`	Indicates SOVD module initialization class.	6.26.2
`udsNrcMappingTable`	The table of NRCs to be mapped in the context of diagnostic service processing	6.30.2

Table 6-3 Description of MEX parameter in Diagnostics Daemon

6.3 Compile Time Configuration

The complete documentation of the DEXT input format is already provided by AUTOSAR (see [3, 10]). This document targets specific configuration items that are of general impact or involve extension of [3, 10].

For each SoftwareCluster, the Diagnostics Daemon reads a JSON configuration file. This file basically consists of the content of the DEXT and is loaded during initialization of the Diagnostics Daemon. For details, refer to section 6.4.

The JSON files created for a SoftwareCluster must match in order to correctly process DIDs and RIDs, so always update both artifacts in parallel.

6.3.1 Diagnostic Server configuration: Software Cluster

A Diagnostic Server insider the Diagnostics Daemon is configured by a `SoftwareCluster` which provides a reference to its corresponding DEXT along with the TargetAddress(es) needed in DoIP communication.

The ContributionSet referenced by the SoftwareCluster has to provide a proper `SERVICE-TABLE` listing all supported UDS services, along with complete containment references `DIAGNOSTIC-COMMON-ELEMENT-REF`. Services must be modeled completely.

Multiple Software Clusters

The DM does not support multiple software clusters. If there are multiple `SOFTWARE-CLUSTER`s in the model, the generator will not keep the diagnostic contents of these `SOFTWARE-CLUSTER`s separate, thereby yielding an inconsistent result. Only use configurations containing a single cluster, and a single DEXT contribution set.

based on template version 1.00.00

Software Cluster Name

The SHORT-NAME of the SoftwareCluster is used to build unique interface names. Make sure to provide a strong name here.

Software Cluster

```
<SOFTWARE-CLUSTER>
  <SHORT-NAME>Name_of_the_SoftwareCluster</SHORT-NAME>
  <DIAGNOSTIC-ADDRESSS>
    <SOFTWARE-CLUSTER-DOIP-DIAGNOSTIC-ADDRESS>
      <ADDRESS-SEMANTICS>FUNCTIONAL-ADDRESS</ADDRESS-SEMANTICS>
      <DIAGNOSTIC-ADDRESS>0xE400</DIAGNOSTIC-ADDRESS>
    </SOFTWARE-CLUSTER-DOIP-DIAGNOSTIC-ADDRESS>
    <SOFTWARE-CLUSTER-DOIP-DIAGNOSTIC-ADDRESS>
      <ADDRESS-SEMANTICS>PHYSICAL-ADDRESS</ADDRESS-SEMANTICS>
      <DIAGNOSTIC-ADDRESS>0x0010</DIAGNOSTIC-ADDRESS>
    </SOFTWARE-CLUSTER-DOIP-DIAGNOSTIC-ADDRESS>
  </DIAGNOSTIC-ADDRESSS>
  <DIAGNOSTIC-EXTRACT-REF DEST="DIAGNOSTIC-CONTRIBUTION-SET">Path to ContributionSet</
      DIAGNOSTIC-EXTRACT-REF>
</SOFTWARE-CLUSTER>
```

Listing 6-1 Software Cluster

6.3.1.1 Configuration Extensions

Global settings of a software cluster that are not covered by model elements in the DEXT can be configured as MEX data element within the software cluster. Details about the configuration of global settings of a software cluster as MEX data elements can be found in listing 6-2 and listing 6-3, and a description of the parameters in table 6-3.

based on template version 1.00.00

MEX data on Software Cluster

```
<SOFTWARE-CLUSTER>
  <SHORT-NAME>Name of the SoftwareCluster</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SDG GID="diagnostics">
          <SDG GID="Diagnostics">
            <SDG-CAPTION>
              <SHORT-NAME>testName</SHORT-NAME>
            </SDG-CAPTION>
            <SDG GID="clearDiagnosticInformation">
              <SDG GID="ClearDiagnosticInformation">
                <SD GID="useMemoryIdentifier">true</SD>
              </SDG>
            </SDG>
            <SDG GID="dmIpc">
              <SDG GID="DmIpc">
                <SD GID="maxNumberOfJobs">4</SD>
              </SDG>
            </SDG>
            <SDG GID="faultMemory">
              <SDG GID="FaultMemory">
                <SD GID="dataCollectingTimeout">5.000</SD>
                <SD GID="numberOfDataCollectingThreadsPerEventMemory">2</SD>
                <SD GID="fileHandlingMode">Multifile</SD>
              </SDG>
            </SDG>
            <SD GID="maxNumberOfPeriodicMessages">20</SD>
            <SD GID="maxPeriodicMessageLength">10</SD>
            <SD GID="numberOfConversations">2</SD>
            <SD GID="p2ServerAdjustTime">20</SD>
            <SD GID="p2StarServerAdjustTime">20</SD>
            <SD GID="s3ServerTimeoutEnabled">false</SD>
            <SD GID="udsMessageLength">500</SD>
            <SDG GID="periodicData">
              <SDG GID="PeriodicData">
                <SD GID="pdidReadMode">Any</SD>
                <SD GID="pdidStopOnStateChange">true</SD>
                <SD GID="schedulerRate">25000</SD>
              </SDG>
            </SDG>
            <SDG GID="securityAccess">
              <SDG GID="SecurityAccess">
                <SD GID="useSharedAttemptCounter">false</SD>
                <SD GID="useSharedTimer">true</SD>
                <SD GID="delayTimeOnFailedNvDataRecovery">0.1</SD>
              </SDG>
            </SDG>
            ...
```

Listing 6-2 MEX data in SoftwareCluster

based on template version 1.00.00

MEX data on Software Cluster (continued)

```
            ...
            <SDG GID="securityEventLog">
              <SDG GID="SecurityEventLog">
                <SD GID="logType">IdsM</SD>
                <SD GID="numberOfAsynchronousJobs">10</SD>
              </SDG>
            </SDG>
            <SDG GID="transferData">
              <SDG GID="TransferData">
                <SD GID="numberOfZeroCopySlots">5</SD>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-3 MEX data in SoftwareCluster (continued)

6.3.2 Modeling of Data Access

The DM and the Diagnostics Library (ara::diag) currently only support the following representations of the CPP standard data types:

Type	Variants	Examples of equivalents
Integral	(un-)signed, 8-64bit sizes	`std::uint8_t`, `std::int32_t` etc.
Boolean	boolean	`bool`
Arrays	Byte arrays	`std::array<std::uint8_t, SIZE>` or `ara::core::Vector<std::uint8_t>`

Table 6-4 Supported Data Types

Depending on the diagnostic processing level (UDS server or application), we distingush the following configurations of data types, which shall represent the set listed in 6-4.

6.3.2.1 Application Interface Data Types

Since the generator needs to create `DiagnosticPortInterface` classes and (de-)serialization code for all `DiagnosticPortInterface`s defined by ARXML modeling such as for DIDs and RIDs, the provided type information in the ARXML must be consistent and complete.

For each `ARGUMENT-DATA-PROTOTYPE` a type reference `TYPE-TREF` with `DEST="STD-CPP-IMPLEMEN↲TATION-DATA-TYPE"` must be defined. It can either refer to any AUTOSAR standardized data type e.g. `/AUTOSAR/StdTypes/uint16_t` or custom CPP-implementation ones. In any case, the equivalent data type shall be one of the ones listed in 6-4.

Application Data Types

Currently application data types i.e. `TYPE-TREF` with `DEST="APPLICATION-DATA-TYPE"` are not supported.

6.3.2.2 DEXT Data Types

The data types used by DM are fully determined by the DEXT. There are the following use cases distinguished from DM point of view:

- Typed reading a single DataElement value: In case of EDR or DID, the DM accesses a physical value represented by any of the supported data types in 6-4. This is performed over stand-alone DataElement-related `DiagnosticPortInterface` (see 6-5).
- Untyped reading/writing a complete byte stream: In case of RID or DID, the DM accesses the packaged data only as a whole without interpreting any single value in it. The byte stream is forwarded to the corresponding application `DiagnosticPortInterface` as opaque data.

To be able to interpret the typed values and serialize them to the UDS-client, the representative data types used within the DEXT need to provide a complete description. The resulted `DataElementInterface` implementation data type shall be equivalent to a data type listed in 6-4.

- `DIAGNOSTIC-DATA-ELEMENT`s must be part of `DIAGNOSTIC-PARAMETER`s with the following constraints:
  - `BIT-OFFSET` shall always be a multiple of 8 i.e. all data elements have byte alignment.
  - The correponding `SW-DATA-DEF-PROPS` refers to `BASE-TYPE` that includes
    - size: must be a multiple of 8 i.e. byte bounded
    - encoding: Any encoding is accepted, but will be treated as opaque, except for boolean. As per "constr_1163" in [11], boolean data will have the following values when acquired through a typed reading access:
      - 0 - for false
      - 1 - for true
    - byte order information: only `MOST-SIGNIFICANT-BYTE-FIRST` are supported for DataElement `DiagnosticPortInterface`. If any of the "generic" port interfaces is used within the Diagnostics Library (ara::diag), then any byte order can be chosen
- If the `DIAGNOSTIC-DATA-ELEMENT` has array semantics, `MAX-NUMBER-OF-ELEMENTS` has to be specified as well
- Both Arrays of `VARIABLE-SIZE` and `FIXED-SIZE` are supported.

Position of DataElements with variable size in DIDs and RIDs within DEXT

If a DataElement of variable size is used within DIDs or RIDs, it must be the last one in the list of DataElements in order to be able to detect its end i.e. by end-of-data.

6.3.2.3 Naming

Generated source files and ServiceInterfaces are distinguished based on the `SHORT-NAME` of the respective DID or RID. While the AUTOSAR model allows to place DIDs in different packages, multiple DIDs or RIDs that have the same `SHORT-NAME` will lead to name duplication.

Name clashes for DIDs and RIDs

The DM relies on all DIDs, RIDs, etc. having different `SHORT-NAME`s

6.3.2.4 Interface Selection

The generator evaluates provided `DIAGNOSTIC-SERVICE-SW-MAPPING`s to identify the type of used service interfaces and to create respective ports. If no such reference is given, the code defaults to `InstanceIdentifier::Any`, which might not result in the intended connection. Service dependencies not referenced by a `DIAGNOSTIC-SERVICE-SW-MAPPING` are not evaluated.

The model and code generator evaluate the diagnostic mappings as defined in AUTOSAR to select the specific interface type used to access a DID or RID.

6.3.2.4.1 Interface DataIdentifier

While it is possible and supported to specifically select the DataIdentifier interface for a DID, if no other definition is given, the DM falls back to this interface. So, for the sake of simplicity, you can just rely on the default.

DataIdentifier Mapping

```
<DIAGNOSTIC-SERVICE-SW-MAPPING>
  <SHORT-NAME>SwMapping_ExampleDid</SHORT-NAME>
  <CATEGORY>DATA_IDENTIFIER</CATEGORY>
  <MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF>
    <CONTEXT-ROOT-COMPONENT-REF DEST="ROOT-SW-COMPONENT-PROTOTYPE">/SystemModel/ExampleExe/
        ExampleRootSwc</CONTEXT-ROOT-COMPONENT-REF>
    <TARGET-SWC-SERVICE-DEPENDENCY-REF DEST="SWC-SERVICE-DEPENDENCY">/SystemModel/
        ExampleSwc/InternalBehavior/SwcDependency_ReadDid</TARGET-SWC-SERVICE-DEPENDENCY-
        REF>
  </MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF>
  <SERVICE-INSTANCE-REF DEST="DIAGNOSTIC-READ-DATA-BY-IDENTIFIER">/DEXT/ReadExampleDid</
      SERVICE-INSTANCE-REF>
</DIAGNOSTIC-SERVICE-SW-MAPPING>
```

Listing 6-4 DataIdentifier Mapping

6.3.2.4.2 Interface GenericUDS

If an application has to provide access to many DIDs or Routines, it can be more convenient to use the generic UDS interface instead of the dedicated DID or Routine Interfaces. The DM will still validate the DID/RID preconditions, but the response data must follow the rules of the generic UDS interface (see section 2.1.5.13).

based on template version 1.00.00

GenericUDS Mapping

```
<DIAGNOSTIC-SERVICE-SW-MAPPING>
  <SHORT-NAME>SwMapping_GenericDid</SHORT-NAME>
  <CATEGORY>GENERIC_UDS_SERVICE</CATEGORY>
  <MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF>
    <CONTEXT-ROOT-COMPONENT-REF DEST="ROOT-SW-COMPONENT-PROTOTYPE">/SystemModel/ExampleExe/
        ExampleRootSwc</CONTEXT-ROOT-COMPONENT-REF>
    <TARGET-SWC-SERVICE-DEPENDENCY-REF DEST="SWC-SERVICE-DEPENDENCY">/SystemModel/
        ExampleSwc/InternalBehavior/SwcDependency_GenericUDS</TARGET-SWC-SERVICE-DEPENDENCY
        -REF>
  </MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF>
  <SERVICE-INSTANCE-REF DEST="DIAGNOSTIC-READ-DATA-BY-IDENTIFIER">/DEXT/ReadGenericDid</
      SERVICE-INSTANCE-REF>
</DIAGNOSTIC-SERVICE-SW-MAPPING>
```

Listing 6-5 GenericUDS Mapping

6.3.2.4.3 Interface DataElement

Instead of accessing a DID as a whole, it is also possible to create access points for individual elements within a DID or for data in Extended Data Records. It also allows data provided by the DM to be included in a DID or Extended Data Record instead of data provided by an application. For details, please refer to [3]. This interface is not available for RIDs.

Internal DataElement Mapping

```
<DIAGNOSTIC-PROVIDED-DATA-MAPPING>
  <SHORT-NAME>DataMapping_ActiveSession</SHORT-NAME>
  <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/DEXT/ActiveSession/ActiveSessionData</
      DATA-ELEMENT-REF>
  <DATA-PROVIDER>DCM_SESSION</DATA-PROVIDER>
</DIAGNOSTIC-PROVIDED-DATA-MAPPING>
```

Listing 6-6 Internal DataElement Mapping

6.3.2.5 Port Mapping

The number of ports required for all but one interfaces can be derived directly from the DEXT elements. For the generic UDS interface, the Diagnostics Daemon has to provide a number of ports depending on the number of applications providing a port of this type. The generator evaluates the reference `MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF` of `DIAGNOSTIC-SERVICE-SW-MAPPING`s to identify the providing application.

The port names created for generic UDS interfaces will follow the application port name.

GenericUDS port *Any*

If no `MAPPED-SWC-SERVICE-DEPENDENCY-IN-EXECUTABLE-IREF` can be resolved to identify a providing application, the DM creates a port GenericUDSService_*SWCL_Other*. Also, the DM assumes that all unmapped generic UDS services are provided by an application connected to that port.

6.4 Runtime Configuration

6.4.1 Global Configuration

The `diagnosticmanager.json` file contains global configuration for Diagnostics Daemon and is genererated using the `amsr_diagnosticmanager_daemon` generator. It contains configuration for:

- Service Validation behavior (See section 6.5.1)
- Logging (See section 6.6)
- Non-volatile Data (See section 6.7)
- Remote Procedure Call (internally deduced)
- DoIP Group Identification (See section 6.23)
- Vehicle Identification Number (See section 6.24)
- Service Oriented Vehicle Diagnostics (See section 6.26)
- Different Diagnostic Clusters (See section 6.10)

The location of the file `diagnosticmanager.json` among the different files to be configured for Diagnostics Daemon can be found in section 3.8.1.

6.4.2 Port Mappings

The link between diagnostic elements defined in the Diagnostic Extract, and related application ports is configured via `DiagnosticSwMappings`, see AUTOSAR [10, Section 4.1]. There are multiple realizations of `DiagnosticSwMappings` defined, each for specific types of diagnostic elements and specific types of application ports. Table 6-5 provides an overview of the supported mapping elements.

Caution

Every instantiation of an application port without a corresponding defined `DiagnosticSwMappings` will lead to an immediate abnormal application process termination.

6.4.2.1 Event to Port Mapping

Since the `DiagnosticEventPortMapping` is used to map a `DiagnosticEvent` to either a `DiagnosticEventInterface` or a `DiagnosticMonitorInterface`, the generator can not directly differentiate between the two. Therefore, the generator scans the Application model to determine the concrete interface type that is mapped to a specific `DiagnosticEvent`. To also support use-cases in which the Application model is not available during generation, the `CATEGORY` field of the `DiagnosticEventPortMapping` can be used to explicitly specify the interface type. If the `CATEGORY` of a `DiagnosticEventPortMapping` is set, the generator uses this information instead of scanning the Application model. In case the configured value is not valid, the generator prints a warning and falls back to scanning the Application model after all. Valid values are `EVENT` or `MONITOR`.

based on template version 1.00.00

CommonElement (Diagnostic...)	SwMapping (Diagnostic...)	PortInterface (Diagnostic...)
DataIdentifier	ServiceDataIdentifierMapping	DataIdentifierInterface, DataIdentifierGenericInterface
DataElement	ServiceSwMapping	DataElementInterface
RoutineControl	ServiceSwMapping	RoutineInterface, RoutineGenericInterface
RequestDownload, DataTransfer, TransferExit	UploadDownloadPortMapping	DownloadInterface
RequestUpload, DataTransfer, TransferExit	UploadDownloadPortMapping	UploadInterface
CustomServiceInstance, ComControl, EcuReset	GenericUdsPortMapping, ServiceSwMapping	GenericUdsInterface
SecurityLevel	SecurityLevelMapping	SecurityLevelInterface
MemoryDestination	MemoryDestinationMapping	DTCInformationInterface
ClearCondition	ClearConditionPortMapping	ConditionInterface
EnableCondition	EnableConditionPortMapping	ConditionInterface
Event	EventPortMapping	EventInterface, MonitorInterface
Indicator	IndicatorPortMapping	IndicatorInterface
OperationCycle	OperationCyclePortMapping	OperationCycleInterface

Table 6-5 Supported DiagnosticSwMappings

based on template version 1.00.00

6.5 Service Validation Configuration

The service validation configuration can be done as illustrated in 6.5.

Service Validation Configuration

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcDependency_ServiceValidation_Manufacturer</SHORT-NAME>
  <ASSIGNED-PORTS>
    <ROLE-BASED-PORT-ASSIGNMENT>
      <PORT-PROTOTYPE-REF DEST="P-PORT-PROTOTYPE">/vector/component_test_app/
          component_test_app_swc/ServiceValidation_Manufacturer</PORT-PROTOTYPE-REF>
      <ROLE>DiagnosticServiceValidation</ROLE>
    </ROLE-BASED-PORT-ASSIGNMENT>
  </ASSIGNED-PORTS>
  <SERVICE-NEEDS>
    <DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
      <SHORT-NAME>Manufacturer</SHORT-NAME>
      <SERVICE-REQUEST-CALLBACK-TYPE>REQUEST-CALLBACK-TYPE-MANUFACTURER</SERVICE-
          REQUEST-CALLBACK-TYPE>
    </DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
  </SERVICE-NEEDS>
</SWC-SERVICE-DEPENDENCY>
```

6.5.1 Service Validation Behavior Configuration

The ExecutionMode for service validation can be configured using MEX parameters and have to be added to the ARXML model. A description of the parameter can be found in table 6-6.

Parameter	JSON key	Description
Service validation	`ServiceValidation`	The service validation execution mode. It can be either `All` (service validation returns NRC `0x10` if the responsible application is not connected yet) or `OnceConnected` (service validation is skipped if the responsible application is not connected yet).

Table 6-6 ExecutionMode JSON parameter description

Caution

Once the adaptive application responsible for service validation is connected to Diagnostics Daemon, the latter will assume it stays connected.

An example how to configure ExecutionMode is shown in listing 6-7. If the configuration cannot be found in the model, `All` is used as default value.

ExecutionMode configuration for service validation

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <AR-PACKAGES>
      <AR-PACKAGE>
        <SHORT-NAME>component_test_daemon</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:DiagnosticManagerArPackageExtensions">
              <SDG GID="diagnosticExecutionMode">
                <SDG GID="DiagnosticExecutionMode">
                  <SDG-CAPTION>
                    <SHORT-NAME>execMode</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="serviceValidation">All</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
      </AR-PACKAGE>
    </AR-PACKAGES>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 6-7 Example definition of ExecutionMode where service validation will return NRC `0x10` if the responsible application is not connected yet

Then the `Executable` shall refer to the configuration using an `Executable` extension as shown in section 6.5.1.

based on template version 1.00.00

Reference to ExecutionMode configuration

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <AR-PACKAGES>
      <AR-PACKAGE>
        <SHORT-NAME>component_test_daemon</SHORT-NAME>
        <ELEMENTS>
          <EXECUTABLE>
            <SHORT-NAME>component_test_daemon</SHORT-NAME>
            <CATEGORY>PLATFORM_LEVEL</CATEGORY>
            <ADMIN-DATA>
              <SDGS>
                <SDG GID="DvMex:DiagnosticManagerExecutableExtensions">
                  <SDG GID="diagnosticExecutionModeRef">
                    <SDX-REF DEST="SDG-CAPTION">/vector/component_test_daemon/execMode</SDX
                        -REF>
                  </SDG>
                </SDG>
              </SDGS>
            </ADMIN-DATA>
            <VERSION>1.0.0</VERSION>
          </EXECUTABLE>
        </ELEMENTS>
      </AR-PACKAGE>
    </AR-PACKAGES>
  </AR-PACKAGE>
</AR-PACKAGES>
```

6.6 Logging Configuration

The Diagnostics Daemon logging can be configured using MEX parameters and have to be added to the ARXML model. A description of the parameters can be found in table 6-7.

Parameter	JSON key	Description
Maximum Payload Bytes	`MaxPayloadBytes`	The maximum number of bytes in payload logs

Table 6-7 Logging JSON parameter description

An example how to configure Diagnostics Daemon logging is shown in listing 6-8. If the configuration cannot be found in the model, `13` is used as default value.

based on template version 1.00.00

Logging configuration for Diagnostics Daemon

```
{
    <AR-PACKAGES>
      <AR-PACKAGE>
        <SHORT-NAME>vector</SHORT-NAME>
        <AR-PACKAGES>
          <AR-PACKAGE>
            <SHORT-NAME>component_test_daemon</SHORT-NAME>
            <ADMIN-DATA>
              <SDGS>
                <SDG GID="DvMex:DiagnosticManagerArPackageExtensions">
                  <SDG GID="diagnosticLogging">
                    <SDG GID="DiagnosticLogging">
                      <SDG-CAPTION>
                        <SHORT-NAME>diagLoggingConfig</SHORT-NAME>
                      </SDG-CAPTION>
                      <SD GID="maxPayloadBytes">11</SD>
                    </SDG>
                  </SDG>
                </SDG>
              </SDGS>
            </ADMIN-DATA>
          </AR-PACKAGE>
        </AR-PACKAGES>
      </AR-PACKAGE>
    </AR-PACKAGES>
}
```

Listing 6-8 Example definition of logging for Diagnostics Daemon

Then the `Executable` shall refer to the configuration using an `Executable` extension as shown in section 6.6.

based on template version 1.00.00

Reference to ExecutionMode configuration

```
{
  <AR-PACKAGES>
    <AR-PACKAGE>
      <SHORT-NAME>vector</SHORT-NAME>
      <AR-PACKAGES>
        <AR-PACKAGE>
          <SHORT-NAME>component_test_daemon</SHORT-NAME>
          <ELEMENTS>
            <EXECUTABLE>
              <SHORT-NAME>component_test_daemon</SHORT-NAME>
              <CATEGORY>PLATFORM_LEVEL</CATEGORY>
              <ADMIN-DATA>
                <SDGS>
                  <SDG GID="DvMex:DiagnosticManagerExecutableExtensions">
                    <SDG GID="diagnosticLoggingRef">
                      <SDX-REF DEST="SDG-CAPTION">/vector/component_test_daemon/
                          diagLoggingConfig</SDX-REF>
                    </SDG>
                  </SDG>
                </SDGS>
              </ADMIN-DATA>
              <VERSION>1.0.0</VERSION>
            </EXECUTABLE>
          </ELEMENTS>
        </AR-PACKAGE>
      </AR-PACKAGES>
    </AR-PACKAGE>
  </AR-PACKAGES>
}
```

6.7 NvData Configuration

The Diagnostics Daemon requires a persistency port of file storage type to be configured according to the AUTOSAR specification. The details on how to configure a persistency port can be found in [28] and section 3.3.

If the ARXML model is valid, the `amsr_diagnosticmanager_daemon` generator is automatically generating the NV data configuration described in table 6-8.

Parameter	JSON key	Description
Persistency instance specifier	`InstanceSpecifier`	The instance specifier string for the persistency port

Table 6-8 NvData JSON parameter description

based on template version 1.00.00

Note

In order to determine what is the maximum needed NvData storage place, please perform the following steps when the Diagnostics Daemon is freshly deployed onto the target ECU, i.e. its NvData storage is still empty:

> Let the Diagnostics Daemon start.
> Make sure it is running normally and did not abnormally terminate e.g. due to an invalid configuration detection.
> Gracefully terminate the Diagnostics Daemon process e.g. send SIGTERM or via Execution Manager (ara::em).
> Make sure the DiagnosticManager log file does not contain any error messages regarding inability to store NvData.
> All the NvData files required by Diagnostics Daemon will be now created within the path specified in the configuration.
> Now you can add the file sizes or use a disk usage tool for this path to determine the total amount of NvData room to reserve for Diagnostics Daemon proper function.

6.8 Fault Memory File Handling Configuration

The MEX parameters `persistencyUpdateRate` and `fileHandlingMode` can be used to configure the file handling behavior of the Fault Memory as shown in table 6-9.

During runtime, the persistency update thread `vDMSPerUp` periodically synchronizes updates of the Fault Memory with persistency. The MEX parameter `persistencyUpdateRate` can be used to configure the update interval in milliseconds, balancing data update delays and the maximum frequency of write operations. A low value ensures that changes in the Fault Memory are immediately persisted and cannot be lost in case of an unexpected termination of the Diagnostics Daemon, such as a hard reset. However, this will also result in a potentially high write load on persistency, which can impact the lifetime of the storage. If the write load on persistency needs to be lowered, the persistency update interval can be increased. Note that on shutdown of the DiagDaemon, the Fault Memory is always synchronized with persistency.

The file handling mode can be chosen using the MEX parameter `fileHandlingMode`. The allowed values are listed here and explained further down:

> `Multifile`: Each DTC is stored in a seperate file (default)
> `Singlefile`: All DTCs are stored in one single file which can provide better I/O performance, but brings also some drawbacks. For details see below comparison.

Parameter	JSON key	Description
File Handling Mode	`FileHandlingMode`	The file handling mode to use for the fault memory.
Persistency Update Rate	`UpdateTimeout`	Update rate for persistency in milliseconds.

Table 6-9 Fault Memory File Handling JSON parameter description

Caution

The `fileHandlingMode` parameter should be set before the first run of the Diagnostics Daemon and should not be changed afterwards. Changing the `fileHandlingMode` later will result in the data written using the old mode being discarded. However, files created by the old mode will not be deleted automatically and should be manually removed if desired.

6.8.1 File Handling Mode Multifile

The default value of `FileHandlingMode` is `Multifile`. In this mode, each DTCs is stored in a seperate file. The file name consists of the type of fault memory (i.e. PrimaryMemory or name of UserMemory) and a consecutive number.

The design of this file handling mode results in an increase in file open and close operations proportional to the number of configured DTCs. This behavior can be particularly noticeable during bulk reading or writing scenarios, such as the startup or shutdown of the Diagnostics Daemon. If this results in noticable performance degradation, the alternative file handling mode, `Singlefile`, described below, can be utilized.

An unexpected termination of the Diagnostics Daemon while updating a DTC's status may result in the loss of that DTC's data, as the pre-update state is compromised and a valid post-update state is not achieved.

6.8.2 File Handling Mode Singlefile

With the `fileHandlingMode` configured to `Singlefile`, all DTCs of an Event Memory are stored in a single file. The name of the file will be determined by the name of the Event Memory (i.e. PrimaryMemory or name of UserMemory) followed by the extension `.singlefile`.

This file handling mode will provide better scalability for a large number of DTCs since less file open and close operations are performed resulting in an improved I/O performance.

An unexpected termination of the Diagnostics Daemon while updating a DTC's status may result in the loss of that DTC's data, as the pre-update state is compromised and a valid post-update state is not achieved. If the Diagnostics Daemon is started with a changed configuration (i.e. new DTC configured or DTC configuration changed), it will restructure the single file to reflect the updated configuration. An unexpected termination of the Diagnostics Daemon during restructuring may result in the loss of the content of the event memory stored in that single file. If protection of event memory content even in case of unexpected termination of the Diagnostics Daemon is required, the redundancy feature of ara::per shall be used (see section 6.9).

6.9 Persistency Redundancy Configuration

A file storage of ara::per [28] is used to store NV data of the Diagnostics Daemon. An unexpected termination of the Diagnostics Daemon while data is being written might result in loss or corruption of that data since the pre-update was compromised, but a valid post-update state might not have been achieved yet.

To protect against this, the redundancy feature of ara::per can be used, such as the M-out-of-N redundancy feature in a 1-out-of-2 configuration. This will always preserve a valid backup of the data even in case when a data update is prematurely terminated. For details on configuring the redundancy, please refer to the documentation of ara::per in [28].

Note
The redundancy configuration will be used for all non-volatile data of the Diagnostics Daemon, including Fault Memory, Security Event Log etc.

Note
Configuring redundancy will increase the required storage space and the amount of data being written by at least the configured factor of redundancy.

Caution
Changing the redundancy configuration (e.g. enabling or disabling redundancy or the number of redundant copies) requires manual deletion of all previously created files in the storage.

6.10 Dext Files Configuration

The Diagnostics Daemon requires the list of DextFiles. If the ARXML model is valid, the `amsr_diagnosticmanager_` generator is automatically generating this list described in table 6-10.

Parameter	JSON key	Description
DextFiles Array	`DextFiles`	List containing Dext configurations that Diagnostics Daemon loads on runtime

Table 6-10 DextFiles JSON parameter description

6.11 AraDiag Configuration

6.11.1 Request Execution Configuration

During runtime, the Diagnostics Daemon requests the Diagnostics Library (ara::diag) to process some diagnostic functionality such as providing diagnostic data or starting some routines. The Diagnostics Library (ara::diag) library can be configured to process such requests in parallel. The number of requests that can be handled in parallel by the Diagnostics Library (ara::diag) can be specified using the model extension parameter `numberOfParallelRequestExecutions`.

If they cannot be handled immediately (because other requests are still in execution), the incoming requests are stored inside a queue. If this queue runs full, requests might get lost, which might degrade the functionality of the DM. Therefore, the queue size can be configured using the model extension parameter `requestQueueSize`. Note that a larger queue size leads to increased memory consumption. The exact amount of additional memory needed varies and depends not only on the size of the queue, but also on the kinds of the stored requests.

Both model extension parameters are defined under
`/MICROSAR/ADAPTIVE/AraDiagConfigModelExtension/AraDiagConfig/`.

An example on how to configure these parameters is shown in listing 6-9. Both parameters are optional and can be configured independently of each other. For constraints on the configurable values refer to table 6-11.

MEX Parameter	Valid range (inclusive)	Default value
numberOfParallelRequestExecutions	1...65535	1
requestQueueSize	3...65535	100

Table 6-11 Value constraints of request execution related parameters

Request execution related AdminData on AraDiag Executable

```
<EXECUTABLE>
  <SHORT-NAME>amsr_diag_example_app</SHORT-NAME>
  ...
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagExecutableExtensions">
        <SDG GID="araDiagConfig">
          <SDG GID="AraDiagConfig">
            <SD GID="numberOfParallelRequestExecutions">4</SD>
            <SD GID="requestQueueSize">250</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 6-9 Request execution related AdminData on AraDiag Executable

Info

If numberOfParallelRequestExecutions is set to one, all the requests to the Diagnostics Library (ara::diag) will be processed cooperatively.

If numberOfParallelRequestExecutions is set to a value greater than one, the sequential order of processing requests by Diagnostics Library (ara::diag) is guaranteed only for the same diagnostic service port instances. This leads to the following considerations:

> For example, reading or writing a DID issued by the DM will always be executed sequentially for the same DID diagnostic port instance, regardless of the number of parallel request executors. In this case, no synchronization implementation is needed in the application code for concurrent calls to this specific port instance.

> However, for two different diagnostic service port instances (e.g., two different DID diagnostic service ports) that share access to resources, having numberOfParallelRequestExecutions set to a value greater than one will require synchronization in the application code.

6.12 Authentication Configuration

6.12.1 Service

6.12.1.1 Authentication Service

AUTOSAR custom service is used to simulate authentication service. DIAGNOSTIC-CUSTOM-SERVICE-CLASS and DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE were extended with admin parameters to achieve this purpose. The Service configuration is illustrated in 6-10

and 6-11.

Extending DIAGNOSTIC-CUSTOM-SERVICE-CLASS with Authentication parameters and subServiceKind

```
<DIAGNOSTIC-CUSTOM-SERVICE-CLASS>
  <SHORT-NAME>AuthenticationClass</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticCustomServiceClassExtensions">
        <SDG GID="authenticationCustomParameters">
          <SDG GID="AuthenticationCustomParameters">
            <SD GID="timeout">10</SD>
            <SD GID="timerMode">REFRESH_BY_USER</SD>
            <SD GID="evaluationId">2</SD>
          </SDG>
        </SDG>
        <SD GID="subServiceKind">SUB_FUNCTION_ID</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <ACCESS-PERMISSION-REF DEST="DIAGNOSTIC-ACCESS-PERMISSION">/
      Diagnostics_Component_Test_SWCL/Dcm/AP_All</ACCESS-PERMISSION-REF>
  <ACCESS-PERMISSION-VALIDITY>ACCESS-PERMISSION-SERVICE-CLASS</ACCESS-PERMISSION-
      VALIDITY>
  <CUSTOM-SERVICE-ID>0x29</CUSTOM-SERVICE-ID>
</DIAGNOSTIC-CUSTOM-SERVICE-CLASS>
```

Listing 6-10 `DIAGNOSTIC-CUSTOM-SERVICE-CLASS` as Authentication class

`subServiceKind`

Authentication is a service with sub-functions, therefore, `subServiceKind` must be configured as `SUB_FUNCTION_ID`. Each sub-function which shall be available to the customer has to be modeled explicitly! (see 6-11 for an example DeAuthenticate `0x00` sub-function.)

The extension parameter `subServiceId` in `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` is required to be configured for each `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE`.

Extending DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE with subServiceId

```
<DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE>
  <SHORT-NAME>DeAuthentication</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticCustomServiceInstanceExtensions">
        <SD GID="subServiceId">0</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <CUSTOM-SERVICE-CLASS-REF DEST="DIAGNOSTIC-CUSTOM-SERVICE-CLASS">/Project/Dcm/
      ServiceClasses/AuthenticationClass</CUSTOM-SERVICE-CLASS-REF>
</DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE>
```

Listing 6-11 `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` as Authentication sub-function

based on template version 1.00.00

6.12.1.2 Client Authorization

The required configuration parameters for the client authorization (see 2.1.20) are globally configured inside the Authentication `DIAGNOSTIC-CUSTOM-SERVICE-CLASS`. Besides these parameters the 6.13

The `timeout` configures in milliseconds how long a client shall be authenticated, this additionally depends on the used `timerMode`

The `timerMode` configuration parameter can be either `ISO14229_1_2020` or `REFRESH_BY_USER`. If not configured, `timerMode` with value `ISO14229_1_2020` will be used as default mode.

`timerMode=ISO14229_1_2020`

Authentication timer handling is as per ISO 14229-1 2020, DM will restart the authentication time period on every request message and keep the authenticated state alive. Authentication Timer can also be restarted from Application by calling `Refresh` 4.25.5

`timerMode=REFRESH_BY_USER`

Authentication timer will continue running towards expiry on completion of every request message. Application holds the responsibility to restart the authentication timer explicitly by calling `Refresh` 4.25.5.

6.12.2 Ports

6.12.2.1 Application

On the application side, `ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE` is extended to hold `P-PORT`s configuration for authentication service as illustrated in 6-12.

Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with `P-PORT`s for authentication service

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
  <SHORT-NAME>AdaptiveApplicationSwComponentType</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
        <SDG GID="diagPPort">
          <SDG GID="DiagPPort">
            <SDG-CAPTION>
              <SHORT-NAME>AuthenticationPPort</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="interfaceType">Authentication</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-12 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with `P-PORT`s for authentication service

Caution
Only one authentication port is allowed.

6.12.2.2 Daemon

SWC-SERVICE-DEPENDENCY is extended with a reference to P-Port extension as illustrated in 6-13.

Extending SWC-SERVICE-DEPENDENCY with reference to P-Port extension

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcDependency_Authentication</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="pPortRef">
          <SDX-REF DEST="SDG-CAPTION">/Project/AdaptiveApplicationSwComponentType/
              AuthenticationPPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-13 Extending SWC-SERVICE-DEPENDENCY with reference to P-Port extension

Caution
Only one SWC-SERVICE-DEPENDENCY is required for all the configured Authentication sub-service mappings.

6.12.3 Service Mappings

Each service instance (service sub-function, DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE), would require a service mapping of its own. Based on the service mapping type, the service shall be considered as internally or externally implemented.

6.12.3.1 Internal Service

Internal implementation of a service instance requires service instance mapping of a type DIAGNOSTIC-SERVICE-GENERIC-MAPPING

6.12.3.2 External Service

External implementation of a service instance requires service instance mapping of a type DIAGNOSTIC-SERVICE-SW-MAPPING

Caution
Regardless the type of the service implementation: internally or externally, The service mappings: DIAGNOSTIC-SERVICE-GENERIC-MAPPING or DIAGNOSTIC-SERVICE-SW-MAPPING, respectively, of all service instances shall reference the same service dependency (SWC-SERVICE-DEPENDENCY).

Caution

Service instances (DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE) shall be either all internally or all externally implemented. Mixing internally and externally implemented service instances is not permitted.

6.13 Client Authentication Configuration

6.13.1 Diagnostic Client Authentication Identification

SOFTWARE-CLUSTER is extended to hold configuration of diagnostic client authentication identification as illustrated in 6-14.

Extending SOFTWARE-CLUSTER with client authentication identifications

```
<SOFTWARE-CLUSTER>
 <SHORT-NAME>SoftwareCluster</SHORT-NAME>
 <ADMIN-DATA>
   <SDGS>
     <SDG GID="DvMex:SoftwareClusterExtensions">
       <SDG GID="diagnosticClientAuthenticationIdentification">
         <SDG GID="DiagnosticClientAuthenticationIdentification">
           <SDG-CAPTION>
             <SHORT-NAME>client</SHORT-NAME>
           </SDG-CAPTION>
           <SD GID="sourceAddressCode">122</SD>
           <SD GID="sourceAddressMask">50</SD>
           </SDG>
         </SDG>
       </SDG>
     </SDG>
   </SDGS>
 </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-14 Extending SOFTWARE-CLUSTER with client authentication identifications

DiagnosticClientAuthenticationIdentification parameters specify all the clients diagnostic addresses that is supported for client authentication. It is possible to group more than one diagnostic client to share the same authentication state. This may be useful when the source address of a client encodes not only the client address but also additional information." DiagnosticClientAuthenticationIdentification contains the parameters sourceAddressCode and sourceAddressMask. In a DiagnosticClientAuthenticationIdentification, All source Addresses that bit-AND-operated with sourceAddressMask where the operations results are identical and equal to sourceAddressCode that resides the very same DiagnosticClientAuthenticationIdentification, share the same authentication state. Hence, a single client authentication identification may represent a group of source addresses "clients".

This bears the possibility to model ranges of clients which represent the same DiagnosticClientAuthenticationIdentification and thus the same authentication state. For example by setting the sourceAddressMask to 0xFF00 and the corresponding sourceAddressCode to 0x2500. This means every client in the range of 0x2500 - 0x25FF will be considered as the same DiagnosticClientAuthenticationIdentification. If this behaviour is not desired, then the sourceAddressMask has to be configured to consider each client on its own by setting the value to 0xFFFF and the selected sourceAddressCode to the corresponding value.

based on template version 1.00.00

Caution

The source-address groups of different `DiagnosticClientAuthenticationIdentification` must be disjoint. In other words, two `DiagnosticClientAuthenticationIdentification` must not match the same source address of a diagnostic client.

Caution

`sourceAddressMask` must not be zero.

Caution

Authentication is treated as activated if and only if both `AuthenticationRole` and `DiagnosticClientAuthenticationIdentification` are not empty.

6.13.2 Suppress Precondition Evaluation

6-15 illustrates the configuration of MEX parameter `ignorePreconditions`. A boolean true value of this parameter indicates a suppression of authentication preconditions evaluation globally for all services. Otherwise, the authentication precondtions are evaluated. If the parameter is not configured, it is defaulted to a boolean false value.

Suppress precondition evaluation configuration

```
<SOFTWARE-CLUSTER>
  <SHORT-NAME>BaseComponentTestModel</SHORT-NAME>
  <CATEGORY>PLATFORM_CORE</CATEGORY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SDG GID="diagnostics">
          <SDG GID="Diagnostics">
            <SDG GID="authentication">
              <SDG GID="Authentication">
                <SD GID="ignorePreconditions">true</SD>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-15 Suppress precondition evaluation configuration

6.13.3 Ports

6.13.3.1 Application

On the application side, `ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE` is extended to hold RPorts configuration for external client authentication as illustrated in 6-16.

Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPorts for external client authentication

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
        <SHORT-NAME>project</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
              <SDG GID="diagRPort">
                <SDG GID="DiagRPort">
                  <SDG-CAPTION>
                    <SHORT-NAME>ClientAuthentication</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="interfaceType">ExternalAuthentication</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-16 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPorts for external client authentication

In addition, `SWC-SERVICE-DEPENDENCY` is extended to hold rPortRef configuration to reference external client authentication port as illustrated in 6-17.

SWC-SERVICE-DEPENDENCY with rPortRef

```
<SWC-SERVICE-DEPENDENCY UUID="34bc57a9-f0c4-4458-99d2-4b02d0fcb555">
  <SHORT-NAME>SwcDependency_ExternalAuthentication</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="rPortRef">
          <SDX-REF DEST="SDG-CAPTION">/vector/ARPackage/
              AdaptiveApplicationSwComponentType/ClientAuthenticationPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-17 Extending SWC-SERVICE-DEPENDENCY with rPortRef for external client authentication

Caution

Only one External authentication port is allowed.

6.13.3.2 Daemon

No configuration is required on the Daemon side as only one external authentication port is allowed on the application side.

based on template version 1.00.00

6.14 Failure event associated identification configuration

6.14.1 Failure Associated Identifier

Failure event logging reporting is based on the associated event identification for each event. Therefore, it is essential that each event is configured with an associated event identification as illustrated in listing 6-18.

Failure event associated identification

```
<DIAGNOSTIC-EVENT>
  <SHORT-NAME>Event</SHORT-NAME>
  <ASSOCIATED-EVENT-IDENTIFICATION>XXXXX</ASSOCIATED-EVENT-IDENTIFICATION>
</DIAGNOSTIC-EVENT>
```

Listing 6-18 Failure event associated identification

6.14.2 Provided Data Mapping

In order to activate the failure event associated identification, the corresponding provided data mapping must be configured as illustrated in listing 6-19.

Failure associated event identification mapping

```
<DIAGNOSTIC-DEM-PROVIDED-DATA-MAPPING>
  <SHORT-NAME>FailureEventAssociatedIdentification_Mapping</SHORT-NAME>
  <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/DEXT/ExtendedDataRecords/
      ExtendedDataRecord0x07/FailureEventElementEDR</DATA-ELEMENT-REF>
  <DATA-PROVIDER>DEM_FAILURE_EVENT_ASSOCIATED_IDENTIFICATION</DATA-PROVIDER>
</DIAGNOSTIC-DEM-PROVIDED-DATA-MAPPING>
```

Listing 6-19 Failure event associated identification mapping

Caution

Upon configuration of DTC with extended data record or snapshot record that includes data element serves as storage for `DATA-PROVIDER` of type `DEM_FAILURE_EVENT_ASSOCIATED_IDENTIFICATION`, `ASSOCIATED-EVENT-IDENTIFICATION` of the related events must be configured.

6.14.3 Data Element Configuration

`DIAGNOSTIC-DATA-ELEMENT` Failure event associated identification data must be stored in a fixed size array with configurable size as illustrated in listing 6-20.

based on template version 1.00.00

Failure associated identification data element type

```
<DIAGNOSTIC-DATA-ELEMENT>
  <SHORT-NAME>FailureEventElementEDR</SHORT-NAME>
  <ARRAY-SIZE-SEMANTICS>FIXED-SIZE</ARRAY-SIZE-SEMANTICS>
  <MAX-NUMBER-OF-ELEMENTS>15</MAX-NUMBER-OF-ELEMENTS>
  <SW-DATA-DEF-PROPS>
    <SW-DATA-DEF-PROPS-VARIANTS>
      <SW-DATA-DEF-PROPS-CONDITIONAL>
        <BASE-TYPE-REF DEST="SW-BASE-TYPE">/ComponentTest/BaseTypes/uint8_opaq</BASE-TYPE-
            REF>
        <COMPU-METHOD-REF DEST="COMPU-METHOD">/ComponentTest/CompuMethods/HexDump1Byte</
            COMPU-METHOD-REF>
      </SW-DATA-DEF-PROPS-CONDITIONAL>
    </SW-DATA-DEF-PROPS-VARIANTS>
  </SW-DATA-DEF-PROPS>
</DIAGNOSTIC-DATA-ELEMENT>
```

Listing 6-20 Failure associated identification data element type

`MAX-NUMBER-OF-ELEMENTS` is configured based on the total number of events taking part in Failure Event Logging multiplied by 5 byte (Associated Event Id(4 byte) + Event Status(1 byte)).

6.15 Security Event Log Configuration

Security event log consists of group of security event types. Each security event type consists of group of security events and a buffer containing logged data. Each security event consists of a counter with a predetermined upper limit. Ports are provided on the security event level and on the security event log level to communicate with the application. In the following subsections, the configuration of security event log elements, in DEXT, are illustrated.

6.15.1 Security Event Log type

As Idsm and security event log features are not supported at the same time, the activation of the feature is done via the MEX parameter logType 6.2. If this parameter is not set or set to `SEvL`, the Security Event Log feature is activated.

6.15.2 Security Event type

Figure 6-21 illustrates a typical configuration of a security event type. `SECURITY-EVENT-DEFINITION` represents a security event type. ID of a security event type must be unique among the IDs of all security event types in a log.

`numberOfBufferEntries` represents the number of buffer entries in an event type. `numberOfBytes↲PerEntry` represents the max number of bytes per entry in a buffer. `persistentStorage` Indicates whether the buffer shall be persisted.

Note

`persistentStorage` shall be set to true. Currently, event type data are always persisted.

based on template version 1.00.00

Using SECURITY-EVENT-DEFINITION to simulate security event type

```
<SECURITY-EVENT-DEFINITION>
  <SHORT-NAME>SecurityEventDefinition</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SecurityEventDefinitionExtensions">
        <SD GID="numberOfBufferEntries">10</SD>
        <SD GID="numberOfBytesPerEntry">5</SD>
        <SD GID="persistentStorage">true</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <ID>0</ID>
</SECURITY-EVENT-DEFINITION>
```

Listing 6-21 `SECURITY-EVENT-DEFINITION` as security event type

6.15.3 Security Event

Figure 6-22 illustrates a typical configuration of a security event. `SECURITY-EVENT-CONTEXT-PROPS` in `SECURITY-EVENT-CONTEXT-MAPPING-APPLICATION` represents a security event. All `SECURITY-EVENT-CONTEXT-PROPS`s can be configured in one `SECURITY-EVENT-CONTEXT-MAPPING-APPLICATION` or `SECURITY-EVENT-CONTEXT-PROPS`s can be divided in groups each configured in `SECURITY-EVENT-CONTEXT-MAPPING-APPLICATION`. A security event (`SECURITY-EVENT-CONTEXT-PROPS`) must reference one and only one security event type. However, multiple security events can reference the same security event type.

`SECURITY-EVENT-DEFINITION` is used to map the security event to the corresponding security event type as mentioned above in 6.15.2

`freezeOnCount` represents the upper limit at which the security event counter will stop to increment even when new events get reported. When reporting a security event more often than `freezeOnCount`, the counter will not increase anymore, but the report itself will still be stored in the buffer. `eventId` represents the ID of the security event, which must be unique among all security events belonging to the same security event type.

Note

`PERSISTENT-STORAGE` shall be set to true. Currently, event data are always persisted.

based on template version 1.00.00

Using SECURITY-EVENT-CONTEXT-PROPS to simulate security event

```
<SECURITY-EVENT-CONTEXT-MAPPING-APPLICATION>
  <SHORT-NAME>SecurityEventContextMappingApplication</SHORT-NAME>
  <MAPPED-SECURITY-EVENTS>
    <SECURITY-EVENT-CONTEXT-PROPS>
      <SHORT-NAME>SecurityEventContextProps</SHORT-NAME>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="DvMex:SecurityEventContextPropsExtensions">
            <SD GID="freezeOnCount">10</SD>
            <SD GID="eventId">11</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
      <PERSISTENT-STORAGE>true</PERSISTENT-STORAGE>
      <SECURITY-EVENTS>
        <SECURITY-EVENT-DEFINITION-REF-CONDITIONAL>
          <SECURITY-EVENT-DEFINITION-REF DEST="SECURITY-EVENT-DEFINITION">/Project/
              ArPackage/SecurityEventDefinition</SECURITY-EVENT-DEFINITION-REF>
        </SECURITY-EVENT-DEFINITION-REF-CONDITIONAL>
      </SECURITY-EVENTS>
    </SECURITY-EVENT-CONTEXT-PROPS>
  </MAPPED-SECURITY-EVENTS>
</SECURITY-EVENT-CONTEXT-MAPPING-APPLICATION>
```

Listing 6-22 `SECURITY-EVENT-CONTEXT-PROPS` as security event

6.15.4 Security Event Log Port

Note

No mappings are required for security event log ports.

Figure 6-23 illustrates a configuration of a typical `SWC-SERVICE-DEPENDENCY` used for security event log.

based on template version 1.00.00

Configuration of SWC-SERVICE-DEPENDENCY for security event log

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependency</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="rPortRef">
          <SDX-REF DEST="SDG-CAPTION">/Project/ArPackage/
              AdaptiveApplicationSwComponentType/DiagRPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SERVICE-NEEDS>
    <DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
      <SHORT-NAME>SecureEventLogNeeds</SHORT-NAME>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="DvMex:DiagnosticCommunicationManagerNeedsExtensions">
            <SD GID="functionalityType">SecurityEventLog</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
    </DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
  </SERVICE-NEEDS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-23 Secuirty event log `SWC-SERVICE-DEPENDENCY` configuration

Figure 6-24 illustrates a configuration of a typical `R-PORT-PROTOTYPE` used for security event log.

Configuration of R-PORT-PROTOTYPE for security event log

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
  <SHORT-NAME>AdaptiveApplicationSwComponentType</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
        <SDG GID="diagRPort">
          <SDG GID="DiagRPort">
            <SDG-CAPTION>
              <SHORT-NAME>SecurityEventLog</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="interfaceType">SecurityEventLog</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-24 Security event log `R-PORT-PROTOTYPE` configuration

6.15.5 Security Event Port

Figure 6-25 illustrates mapping of a security event to `SWC-SERVICE-DEPENDENCY`

based on template version 1.00.00

Using SECURITY-EVENT-MAPPING to map a security event to a port

```
<SECURITY-EVENT-MAPPING>
  <SHORT-NAME>SecurityEventMapping</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SecurityEventMappingExtensions">
        <SDG GID="securityEventContextPropsRef">
          <SDX-REF DEST="SECURITY-EVENT-CONTEXT-PROPS">/Project/
              SecurityEventContextMappingApplication/SecurityEventContextProps</SDX-
              REF>
        </SDG>
        <SDG GID="swcServiceDependencyRef">
          <SDX-REF DEST="SWC-SERVICE-DEPENDENCY">/Project/ArPackage/
              AdaptiveApplicationSwComponentType/AdaptiveSwcInternalBehavior/
              SwcServiceDependency</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <PROCESS-REF DEST="PROCESS">/Project/ArPackage/Process</PROCESS-REF>
  <REPORTING-PORT-PROTOTYPE-IREF>
    <CONTEXT-ROOT-SW-COMPONENT-PROTOTYPE-REF DEST="ROOT-SW-COMPONENT-PROTOTYPE">/
        Project/ArPackage/Executable/rootSwComponentType</CONTEXT-ROOT-SW-COMPONENT-
        PROTOTYPE-REF>
  </REPORTING-PORT-PROTOTYPE-IREF>
</SECURITY-EVENT-MAPPING>
```

Listing 6-25 Security event `SECURITY-EVENT-MAPPING` configuration

Figure 6-26 illustrates a configuration of a typical `SWC-SERVICE-DEPENDENCY` used in `SECURITY-EVENT-MAPPING`.

Configuration of SWC-SERVICE-DEPENDENCY for security event

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependency</SHORT-NAME>
  <ASSIGNED-PORTS>
    <ROLE-BASED-PORT-ASSIGNMENT>
      <PORT-PROTOTYPE-REF DEST="R-PORT-PROTOTYPE">/Project/ArPackage/
          AdaptiveApplicationSwComponentType/RPortPrototype</PORT-PROTOTYPE-REF>
      <ROLE>SecurityEvent</ROLE>
    </ROLE-BASED-PORT-ASSIGNMENT>
  </ASSIGNED-PORTS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-26 Security event `SWC-SERVICE-DEPENDENCY` configuration

Figure 6-27 illustrates a configuration of a typical `R-PORT-PROTOTYPE` used for security event. The interface referenced by ths port prototype must be of a type of `SECURITY-EVENT-REPORT-INTERFACE`

based on template version 1.00.00

Configuration of R-PORT-PROTOTYPE for security event

```
<R-PORT-PROTOTYPE>
  <SHORT-NAME>RPortPrototype</SHORT-NAME>
  <REQUIRED-INTERFACE-TREF DEST="SECURITY-EVENT-REPORT-INTERFACE">/Project/ArPackage/
      AdaptiveApplicationSwComponentType/SecurityEventReportInterface</REQUIRED-
      INTERFACE-TREF>
</R-PORT-PROTOTYPE>
```

Listing 6-27 Security event `R-PORT-PROTOTYPE` configuration

6.16 Idsm Configuration

AUTOSAR already provides the configuration model for Idsm security events AUTOSAR (see [10]). Some of the Idsm features supported by the DM also require an extension of the configuration model, for more details, please refer to [23].

6.16.1 Feature activation

As Idsm and security event log features are not supported at the same time, the activation of the feature is done via the MEX parameter logType 6.2. If this parameter is set to `IdsM`, the Idsm feature is activated.

6.16.2 Number Of Asynchronous Jobs

The Diagnostics Daemon asynchronously notifies the qualified event to the Idsm application. Once an event is qualified, the notification job is added into a queue. The queue has a configurable size of NumberOfAsynchronousJobs 6.2, which has a default value of 10 if not configured.

6.16.3 IDSR Configuration

6-28, 6-29, and 6-30 illustate the configuration of the local and the remote endpoints of Idsr communication.

The MEX aggregation `Udp` was added to configure the remote UDP port number.

Communication protocol limitation

Currently, Idsr communications are limited to UDP.

IDSR local IP address credential configuration

```
<PLATFORM-MODULE-ETHERNET-ENDPOINT-CONFIGURATION>
  <SHORT-NAME>PlatformModuleEthernetEndpointConfiguration</SHORT-NAME>
  <COMMUNICATION-CONNECTOR-REF DEST="ETHERNET-COMMUNICATION-CONNECTOR">/DEXT/
      MachineDesign/EthernetCommunicationConnector</COMMUNICATION-CONNECTOR-REF>
  <IPV-4-MULTICAST-IP-ADDRESS>192.168.154.128</IPV-4-MULTICAST-IP-ADDRESS>
  <UDP-PORT>55</UDP-PORT>
</PLATFORM-MODULE-ETHERNET-ENDPOINT-CONFIGURATION>
```

Listing 6-28 IDSR local IP address credential configuration

based on template version 1.00.00

IDSR remote endpoint configuration - Communication connector

```
<MACHINE-DESIGN>
  <COMMUNICATION-CONNECTORS>
    <ETHERNET-COMMUNICATION-CONNECTOR>
      <SHORT-NAME>EthernetCommunicationConnector</SHORT-NAME>
      <NETWORK-ENDPOINT-REFS>
        <NETWORK-ENDPOINT-REF DEST="NETWORK-ENDPOINT">/DEXT/Ethernet/Channel/
            IdsRNetworkEndpoint</NETWORK-ENDPOINT-REF>
      </NETWORK-ENDPOINT-REFS>
    </ETHERNET-COMMUNICATION-CONNECTOR>
  </COMMUNICATION-CONNECTORS>
</MACHINE-DESIGN>
```

Listing 6-29 Extending SOFTWARE-CLUSTER with client authentication identifications

IDSR remote endpoint configuration - Network endpoint

```
<ETHERNET-CLUSTER>
  <ETHERNET-CLUSTER-VARIANTS>
    <ETHERNET-CLUSTER-CONDITIONAL>
      <PHYSICAL-CHANNELS>
        <ETHERNET-PHYSICAL-CHANNEL>
          <NETWORK-ENDPOINTS>
            <NETWORK-ENDPOINT>
              <SHORT-NAME>IdsRNetworkEndpoint</SHORT-NAME>
              <ADMIN-DATA>
                <SDGS>
                  <SDG GID="DvMex:NetworkEndpointExtensions">
                    <SDG GID="ethernetPort">
                      <SDG GID="EthernetPort">
                        <SDG GID="udp">
                          <SDG GID="Udp">
                            <SD GID="number">50</SD>
                          </SDG>
                        </SDG>
                      </SDG>
                    </SDG>
                  </SDG>
                </SDGS>
              </ADMIN-DATA>
              <NETWORK-ENDPOINT-ADDRESSES>
                <IPV-4-CONFIGURATION>
                  <IPV-4-ADDRESS>192.168.154.132</IPV-4-ADDRESS>
                </IPV-4-CONFIGURATION>
              </NETWORK-ENDPOINT-ADDRESSES>
            </NETWORK-ENDPOINT>
          </NETWORK-ENDPOINTS>
        </ETHERNET-PHYSICAL-CHANNEL>
      </PHYSICAL-CHANNELS>
    </ETHERNET-CLUSTER-CONDITIONAL>
  </ETHERNET-CLUSTER-VARIANTS>
</ETHERNET-CLUSTER>
```

Listing 6-30 IDSR remote endpoint configuration - Network endpoint

based on template version 1.00.00

Note
Idsr is activated by the correct configuration of the local and the remote communication endpoints.

6.17 Service Processing Notification

Note
No mappings are required for Service processing notification ports.

Figure 6-31 illustrates a configuration of a typical SWC-SERVICE-DEPENDENCY used for service processing notification.

Configuration of SWC-SERVICE-DEPENDENCY for Service processing notification

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependency</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="pPortRef">
          <SDX-REF DEST="SDG-CAPTION">/Project/ArPackage/
              AdaptiveApplicationSwComponentType/DiagPPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SERVICE-NEEDS>
    <DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS UUID="867c4f0b-5527-4d41-9e30-
        e4975e8eab3e">
      <SHORT-NAME>ServiceProcessingNotificationNeeds</SHORT-NAME>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="DvMex:DiagnosticCommunicationManagerNeedsExtensions">
            <SD GID="functionalityType">ServiceProcessingNotification</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
    </DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
  </SERVICE-NEEDS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-31 Service processing notification SWC-SERVICE-DEPENDENCY configuration

Figure 6-32 illustrates a configuration of a typical P-PORT-PROTOTYPE used for service processing notification.

based on template version 1.00.00

Configuration of P-PORT-PROTOTYPE for Service processing notification

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
  <SHORT-NAME>AdaptiveApplicationSwComponentType</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
        <SDG GID="diagPPort">
          <SDG GID="DiagPPort">
            <SDG-CAPTION>
              <SHORT-NAME>ServiceProcessingNotification</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="interfaceType">ServiceProcessingNotification</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-32 Service processing notification `P-PORT-PROTOTYPE` configuration

6.18 Request File Transfer Configuration

6.18.1 Service

A typical configuration for request file transfer class and instance, `DIAGNOSTIC-CUSTOM-SERVICE-CLASS` and `DIAGNOSTIC-CUSTOM-SERVICE-INSTANCE` are illustrated in 6-33 and 6-34, respectively.

DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS configuration

```
<DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS>
  <SHORT-NAME>RequestFileTransferClass</SHORT-NAME>
  <DESC>
    <L-2 L="FOR-ALL">;Request_File_Transfer</L-2>
  </DESC>
  <ACCESS-PERMISSION-VALIDITY>ACCESS-PERMISSION-SERVICE-INSTANCE</ACCESS-PERMISSION-
      VALIDITY>
</DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS>
```

Listing 6-33 `DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS` configuration

DIAGNOSTIC-REQUEST-FILE-TRANSFER configuration

```
<DIAGNOSTIC-REQUEST-FILE-TRANSFER>
  <SHORT-NAME>RequestFileTransfer</SHORT-NAME>
  <ACCESS-PERMISSION-REF DEST="DIAGNOSTIC-ACCESS-PERMISSION">/Project/ARPackage/
      DiagnosticAccessPermission</ACCESS-PERMISSION-REF>
  <REQUEST-FILE-TRANSFER-CLASS-REF DEST="DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS">/
      Project/ARPackage/ServiceClasses/RequestFileTransferClass</REQUEST-FILE-
      TRANSFER-CLASS-REF>
</DIAGNOSTIC-REQUEST-FILE-TRANSFER>
```

Listing 6-34 `DIAGNOSTIC-REQUEST-FILE-TRANSFER` configuration

based on template version 1.00.00

Caution

DIAGNOSTIC-REQUEST-FILE-TRANSFER-CLASS allows a configuration of only one DIAGNOSTIC-REQUEST-FILE-TRANSFER.

6.18.2 Ports

6.18.2.1 Application

On the application side, ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE is extended to hold P-PORTs configuration for request file transfer service as illustrated in 6-35.

Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with P-PORTs for DIAGNOSTIC-REQUEST-FILE-TRANSFER

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
  <SHORT-NAME>AdaptiveApplicationSwComponentType</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
        <SDG GID="diagPPort">
          <SDG GID="DiagPPort">
            <SDG-CAPTION>
              <SHORT-NAME>RequestFileTransferPPort</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="interfaceType">RequestFileTransfer</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-35 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with P-PORTs for DIAGNOSTIC-REQUEST-FILE-TRANSFER

Caution

Only one request file transfer port is allowed.

6.18.2.2 Daemon

SWC-SERVICE-DEPENDENCY is extended with a reference to P-Port extension as illustrated in 6-36.

based on template version 1.00.00

SWC-SERVICE-DEPENDENCY configuration for DIAGNOSTIC-REQUEST-FILE-TRANSFER

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcDependency_RequestFileTransfer</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="pPortRef">
          <SDX-REF DEST="SDG-CAPTION">/Project/AdaptiveApplicationSwComponentType/
              RequestFileTransferPPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-36 SWC-SERVICE-DEPENDENCY configuration for DIAGNOSTIC-REQUEST-FILE-TRANSFER

Caution

Only one SWC-SERVICE-DEPENDENCY is required as only one port is allowed for request file transfer service.

6.18.3 Service Mappings

DIAGNOSTIC-REQUEST-FILE-TRANSFER, requires a service mapping of its own. Based on the service mapping type, the service shall be considered as internally or externally implemented.

6.18.3.1 Internal Service

Internal implementation of a service instance requires service instance mapping of a type DIAGNOSTIC-SERVICE-GENERIC-MAPPING

6.18.3.2 External Service

External implementation of a service instance requires service instance mapping of a type DIAGNOSTIC-SERVICE-SW-MAPPING

6.19 Root cause event associated identification configuration

6.19.1 Root Cause Associated Identifier

The root cause associated identifier is configured for an event as illustrated in listing 6-37.

Root cause associated event identification

```
<DIAGNOSTIC-EVENT>
  <SHORT-NAME>Event</SHORT-NAME>
  <ASSOCIATED-EVENT-IDENTIFICATION>0</ASSOCIATED-EVENT-IDENTIFICATION>
</DIAGNOSTIC-EVENT>
```

Listing 6-37 Root cause event associated identification

based on template version 1.00.00

6.19.2 Provided Data Mapping

In order to activate the root cause event associated identification, the corresponding provided data mapping must be configured as illustrated in listing 6-38.

Root cause associated event identification mapping

```
<DIAGNOSTIC-PROVIDED-DATA-MAPPING>
  <SHORT-NAME>EventAssociatedIdentification</SHORT-NAME>
  <DATA-ELEMENT-REF DEST="DIAGNOSTIC-DATA-ELEMENT">/Diagnostics_Component_Test_SWCL/Dem/
      ExtendedDataRecords/RootCauseExtendedDataRecord/RootCauseDataElement</DATA-ELEMENT-
      REF>
  <DATA-PROVIDER>DEM_ROOT_CAUSE_EVENT_ASSOCIATED_IDENTIFICATION</DATA-PROVIDER>
</DIAGNOSTIC-PROVIDED-DATA-MAPPING>
```

Listing 6-38 Root cause event associated identification mapping

Caution

Upon configuration of DTC with extended data record or snapshot record that includes data element serves as storage for `DATA-PROVIDER` of type `DEM_ROOT_CAUSE_EVENT_ASSOCIATED_IDENTIFICATION`, `ASSOCIATED-EVENT-IDENTIFICATION` of the related events must be configured.

6.19.3 Base Type Configuration

`SW-BASE-TYPE` of the data element storing the root cause event associated identification must be 4 bytes with a Motorola byte order as illustrated in listing 6-39. In addition `BASE-TYPE-ENCODING` have to be `NONE` for unsigned integers (`2C` only valid for two's complement).

Root cause event associated identification base type

```
<SW-BASE-TYPE>
  <SHORT-NAME>uint32_motorola</SHORT-NAME>
  <CATEGORY>FIXED_LENGTH</CATEGORY>
  <BASE-TYPE-SIZE>32</BASE-TYPE-SIZE>
  <BASE-TYPE-ENCODING>NONE</BASE-TYPE-ENCODING>
  <BYTE-ORDER>MOST-SIGNIFICANT-BYTE-FIRST</BYTE-ORDER>
</SW-BASE-TYPE>
```

Listing 6-39 Root cause event associated identification base type

6.20 DTC confirmation configuration

The confirmation threshold is configured for each DiagnosticEvent.

Indicator

```
<DIAGNOSTIC-EVENT>
  ...
  <CONFIRMATION-THRESHOLD>3</CONFIRMATION-THRESHOLD>
  ...
</DIAGNOSTIC-EVENT>
```

Listing 6-40 Indicator Configuration

The following limitations apply:

- Default CONFIRMATION-THRESHOLD value is 1
- CONFIRMATION-THRESHOLD must be within the range 1 and 255
- For DTC combined events the CONFIRMATION-THRESHOLD values shall be identical

6.21 Indicator configuration

Indicators can be connected to DiagnosticEvents and the parameters are configured seperate for each DiagnosticEvent. For combined events the connected indicators must be identical with the same parameters.

Indicator

```
<DIAGNOSTIC-INDICATOR>
  <SHORT-NAME>...</SHORT-NAME>
</DIAGNOSTIC-INDICATOR>


<DIAGNOSTIC-EVENT>
  ...
  <CONNECTED-INDICATORS>
    <DIAGNOSTIC-CONNECTED-INDICATOR>
      <SHORT-NAME>...</SHORT-NAME>
      <BEHAVIOR>...</BEHAVIOR>
      <HEALING-CYCLE-COUNTER-THRESHOLD>...</HEALING-CYCLE-COUNTER-THRESHOLD>
      <INDICATOR-REF DEST="DIAGNOSTIC-INDICATOR">...</INDICATOR-REF>
    </DIAGNOSTIC-CONNECTED-INDICATOR>
    ...
  </CONNECTED-INDICATORS>
  ...
</DIAGNOSTIC-EVENT>
```

Listing 6-41 Indicator Configuration

The following limitations apply:

- BEHAVIOR parameter must be one of the following:
  - CONTINUOUS-ON-MODE
  - BLINK-MODE
  - BLINK-OR-CONTINUOUS-ON-MODE
- HEALING-CYCLE-COUNTER-THRESHOLD must be between 0 and 255

based on template version 1.00.00

6.22 DoIP activation line configuration

6.22.1 DoIP Activation Line Interface

An interface of type `DIAGNOSTIC-DO-IP-ACTIVATION-LINE-INTERFACE` shall be configured.

6.22.2 DoIP Activation Line Port

DoIP activation line requires a configuration of an R-port as illustrated in listing 6-42

DoIP activation line port

```
<R-PORT-PROTOTYPE>
  <SHORT-NAME>DoIPActivationLine</SHORT-NAME>
  <REQUIRED-INTERFACE-TREF DEST="DIAGNOSTIC-DO-IP-ACTIVATION-LINE-INTERFACE">
      port_interfaces/DoIPActivationLine_Interface</REQUIRED-INTERFACE-TREF>
</R-PORT-PROTOTYPE>
```

Listing 6-42 DoIP activation line port

6.22.3 DoIP activation line Mapping

DoIP activation line requires a configuration of a software service component mapping as illustrated in listing 6-43

DoIP activation line mapping

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcDependency_DoIPActivationLine</SHORT-NAME>
  <ASSIGNED-PORTS>
    <ROLE-BASED-PORT-ASSIGNMENT>
      <PORT-PROTOTYPE-REF DEST="R-PORT-PROTOTYPE">ports/DoIPActivationLine</PORT-PROTOTYPE-
          REF>
      <ROLE>DoIPActivationLine</ROLE>
    </ROLE-BASED-PORT-ASSIGNMENT>
  </ASSIGNED-PORTS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-43 DoIP activation line mapping

6.23 DoIP group identification configuration

6.23.1 DoIP Group Identification Interface

An interface of type `DIAGNOSTIC-DO-IP-GROUP-IDENTIFICATION-INTERFACE` shall be configured.

6.23.2 DoIP Group Identification Port

DoIP group identification (GID) requires a configuration of an R-port as illustrated in listing 6-44

based on template version 1.00.00

DoIP group identification port

```
<R-PORT-PROTOTYPE>
  <SHORT-NAME>DoIpGroupIdentificationPort</SHORT-NAME>
  <REQUIRED-INTERFACE-TREF DEST="DIAGNOSTIC-DO-IP-GROUP-IDENTIFICATION-INTERFACE">
      portInterfaces/DoIPGroupIdentificationInterface</REQUIRED-INTERFACE-TREF>
</R-PORT-PROTOTYPE>
```

Listing 6-44 DoIP group identification port

6.23.3 DoIP Group Identification Mapping

DoIP group identification (GID) requires a configuration of a software service component mapping as illustrated in listing 6-45

DoIP group identification mapping

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>DoIPGroupIdentificationSwcDependency</SHORT-NAME>
  <ASSIGNED-PORTS>
    <ROLE-BASED-PORT-ASSIGNMENT>
      <PORT-PROTOTYPE-REF DEST="R-PORT-PROTOTYPE">ports/DoIpGroupIdentificationPort</PORT-
          PROTOTYPE-REF>
      <ROLE>DoIPGroupIdentification</ROLE>
    </ROLE-BASED-PORT-ASSIGNMENT>
  </ASSIGNED-PORTS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-45 DoIP group identification mapping

6.24 DoIP vehicle identification number configuration

6.24.1 Data Identifier

DoIP VIN requires a data identifier configuration as illustrated in listing 6-46

based on template version 1.00.00

DoIP vehicle group identification data identifier

```
<DIAGNOSTIC-DATA-IDENTIFIER>
  <SHORT-NAME>Vehicle_Identification_DoIP</SHORT-NAME>
  <!-- Either explicitly declare the DID as VIN over the ADMIN-DATA "usage" parameter -->
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticDataIdentifierExtensions">
        <SD GID="usage">DOIP_VIN</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <!-- ------------------------------------------------------------------------------ -->
  <ID>61845</ID>
  <DATA-ELEMENTS>
    <DIAGNOSTIC-PARAMETER>
      <BIT-OFFSET>0</BIT-OFFSET>
      <DATA-ELEMENTS>
        <DIAGNOSTIC-DATA-ELEMENT>
          <SHORT-NAME>VIN</SHORT-NAME>
          <ARRAY-SIZE-SEMANTICS>FIXED-SIZE</ARRAY-SIZE-SEMANTICS>
          <MAX-NUMBER-OF-ELEMENTS>17</MAX-NUMBER-OF-ELEMENTS>
          <SW-DATA-DEF-PROPS>
            <SW-DATA-DEF-PROPS-VARIANTS>
              <SW-DATA-DEF-PROPS-CONDITIONAL>
                <BASE-TYPE-REF DEST="SW-BASE-TYPE">/BaseTypes/uint8_opaque</BASE-TYPE-REF>
                <COMPU-METHOD-REF DEST="COMPU-METHOD">/CompuMethods/VIN_17_Byte</COMPU-
                    METHOD-REF>
              </SW-DATA-DEF-PROPS-CONDITIONAL>
            </SW-DATA-DEF-PROPS-VARIANTS>
          </SW-DATA-DEF-PROPS>
        </DIAGNOSTIC-DATA-ELEMENT>
      </DATA-ELEMENTS>
    </DIAGNOSTIC-PARAMETER>
  </DATA-ELEMENTS>
  <!-- Or define the VIN over the REPRESENTS-VIN parameter -->
  <REPRESENTS-VIN>true</REPRESENTS-VIN>
  <!-- ------------------------------------------------------ -->
</DIAGNOSTIC-DATA-IDENTIFIER>
```

Listing 6-46 DoIP vehicle group identification data identifier

There are two different ways to represent a DID as VIN. Define a VIN via the `REPRESENTS-VIN` parameter. Or define a DoIP VIN data identifier by adding the `usage` `DOIP_VIN` as `ADMIN-DATA` to the DID. This means that the DoIP will interpret this DID as VIN even when another DID has `REPRESENTS-VIN` set to true.

Note

`usage` has priority over `REPRESENTS-VIN`. In other words, if both `usage` and `REPRESENTS-VIN` are configured, `DIAGNOSTIC-DATA-IDENTIFIER` shall be considered as DoIP VIN data identifier

6.24.2 Service Instance Mapping

A service instance mapping is not required for DoIP VIN data identifier.

based on template version 1.00.00

6.24.3 Port Mapping

A port mapping of type `DIAGNOSTIC-SERVICE-DATA-IDENTIFIER-PORT-MAPPING` is required for DoIP VIN data identifier.

Note

If DoIP VIN data identifier shall be used for update and read/write operations as well, then service instance mappings are required for read and write options.
In this case, port mapping can be either configured via data identifier mapping (`DIAGNOSTIC-SERVICE-DATA-IDENTIFIER-PORT-MAPPING`) or service instance port mapping (`DIAGNOSTIC-SERVICE-SW-MAPPING`)

6.25 Secured Data Transmission Configuration

6.25.1 Secured Data Transmission properties configuration

`DIAGNOSTIC-CONTRIBUTION-SET` in DEXT is extended to hold the configuration of secured data transmission properties as illustrated in 6-47.

Secured Data Transmission properties

```
<DIAGNOSTIC-CONTRIBUTION-SET>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticContributionSetExtensions">
        <SD GID="securedDataTransmissionOverSecuredLink">true</SD>
        <SD GID="sdtUdsMessageLength">550</SD>
        <SDG GID="securedDataTransmissionProps">
          <SDG GID="SecuredDataTransmissionProps">
            <SDG-CAPTION>
              <SHORT-NAME>AdministrativeParameters</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="preSharedKey">REQUIRED</SD>
            <SD GID="encryption">NONE</SD>
            <SD GID="signing">OPTIONAL</SD>
            <SD GID="signatureOnResponse">NONE</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-CONTRIBUTION-SET>
```

Listing 6-47 Secured Data Transmission properties

The `securedDataTransmissionOverSecuredLink` parameter is currently unused. In the future it will be used to enable secured transmission via underlying transport layer TLS instead of service `0x84`.

The `sdtUdsMessageLength` parameter indicates the maximum length of a transmitted secured message. `sdtUdsMessageLength` must be greater than or equal to the summation of `UdsMessageLength` and the secured data transmission header length (8 bytes). If `sdtUdsMessageLength` is not configured, the minimum length (the sum of `UdsMessageLength` and the secured data transmission header length) will be generated.

Service `0x84` itself is enabled only if `securedDataTransmissionProps` is configured.

It is possible for the user to define constraints on the administrative parameter of incoming service

based on template version 1.00.00

`0x84` requests. These constraints will be verified by the Diagnostics Daemon before the SDT plug-in is invoked. The configuration of `preSharedKey`, `encryption`, `signing` and `signatureOnResponse` bit values in the AdministrativeParameter for a valid SDT request message can be one of the following:

> `REQUIRED` - indicates that the flag must be set on incoming messages.
> `NONE` - indicates that the flag must not be set on incoming messages.
> `OPTIONAL` - indicates that the flag may or may not be set on incoming messages.

If any of the parameters `preSharedKey`, `encryption`, `signing` or `signatureOnResponse` are not configured, they will be implicitly set to `OPTIONAL`.

Note that all other, not listed here, `0x84` UDS request message AdministrativeParameter bits are not configurable since they are handled internally by Diagnostics Daemon or are specified as reserved in the UDS. If desired, they still can be evaluated from within the SDT plug-in (see 2.1.5.10.2).

Caution

It is the responsibility of the user to make sure that `sdtUdsMessageLength` is large enough to hold the largest possible UDS request and response message that shall be enclosed within a SDT message, including all additionally needed mandatory and optional payload parameters as defined by [40] for service `0x84`. The absolute maximum can be calculated with the formula: `sdtUdsMessageLength` = 8Byte SDT header + LargestEnclosedUdsMessage (refer to the UDS message length related option in listing 6-2) + LargestSignatureLength handled by the SDT plug-in (user defined implementation).

6.25.2 Secured Data Transmission creator configuration

The configuration and integration aspects of a custom implementated SDT plug-in are explained in 3.3.2.2.

6.26 SOVD Configuration

The currently supported AUTOSAR model doesn't contain the configuration elements for SOVD. Thus MEX parameters are used to configure SOVD related parameters.

6.26.1 Feature activation

The feature is activated if SOVD configuration is available in the ARXML model and cmake variable `AMSR_ENABLE_SOVD` is set to ON (see table 3-12). The SOVD configuration contains:

> One SOVD module instantiation

6.26.2 SOVD module instantiation

To identify the Diagnostics Daemon within the URI of a SOVD request, the names of the SOVD server and the Diagnostics Daemon component have to be configured in the model. This is done using an extension of the `GenericModuleInstantiation` object as described in the example illustrated in listing 6-48, listing 6-50, and listing 6-53. MEX parameters are described in table 6-12.

Communication protocol limitation

Currently, SOVD communications are limited to TCP.

Parameter Name	Description
nodeIdentifier	This attribute represents the local hostname of the SOVD server to be used during the execution of the DNS-SD protocol.
componentQualifier	This attributes described the component qualifier used to compose an SOVD query.

Table 6-12 SOVD module instantiation MEX parameters

Configuration for SOVD module instantiation

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <ELEMENTS>
      <MACHINE>
        <SHORT-NAME>DiagnosticManagerMachine</SHORT-NAME>
        <MACHINE-DESIGN-REF DEST="MACHINE-DESIGN">/vector/DiagnosticManagerMachineDesign</
            MACHINE-DESIGN-REF>
        <MODULE-INSTANTIATIONS>
          <GENERIC-MODULE-INSTANTIATION UUID="7bb1bc9e-603c-4190-a869-85c0a95fad2e">
            <SHORT-NAME>GenericModuleInstantiationForSovd</SHORT-NAME>
            <ADMIN-DATA>
              <SDGS>
                <SDG GID="DvMex:GenericModuleInstantiationExtensions">
                  <SDG GID="sovdModuleInstantiation">
                    <SDG GID="SovdModuleInstantiation">
                      <SDG-CAPTION>
                        <SHORT-NAME>SovdModuleInstantiation</SHORT-NAME>
                      </SDG-CAPTION>
                      <SD GID="nodeIdentifier">SovdServer</SD>
                      <SD GID="componentQualifier">DiagDaemon</SD>
                      <SD GID="numberOfWorkers">5</SD>
                      <SD GID="maxNumberOfConnections">7</SD>
                      <SD GID="maxNumberOfParallelRequestsPerConnection">3</SD>
                      <SD GID="maxHttpBodyBufferSize">1200</SD>
                      <SDG GID="communicationConnectorRef">
                        <SDX-REF DEST="COMMUNICATION-CONNECTOR">/DEXT/MachineDesign/
                            ethernetCommunicationConnector</SDX-REF>
                      </SDG>
                    </SDG>
                  </SDG>
                </SDG>
              </SDGS>
            </ADMIN-DATA>
          </GENERIC-MODULE-INSTANTIATION>
        </MODULE-INSTANTIATIONS>
      </MACHINE>
    </ELEMENTS>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 6-48 Example of SOVD module instantiation

listing 6-49, listing 6-51, and listing 6-52 illustrate the configuration of TLS.

For the list of the supported TLS cipher suits, please refer to [18]. All defined TLS cipher suits must share the same key and the same certificate.

To activate TLS, `TransportLayerSecurity` container must be configured as showed in listing 6-49. Remote (server) endpoint configuration is illustrated in listing 6-53.

Configuration for transport layer security

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <ELEMENTS>
      <MACHINE>
        <SHORT-NAME>DiagnosticManagerMachine</SHORT-NAME>
        <MACHINE-DESIGN-REF DEST="MACHINE-DESIGN">/vector/DiagnosticManagerMachineDesign</
            MACHINE-DESIGN-REF>
        <MODULE-INSTANTIATIONS>
          <GENERIC-MODULE-INSTANTIATION UUID="7bb1bc9e-603c-4190-a869-85c0a95fad2e">
            <SHORT-NAME>GenericModuleInstantiationForSovd</SHORT-NAME>
            <ADMIN-DATA>
              <SDGS>
                <SDG GID="DvMex:GenericModuleInstantiationExtensions">
                  <SDG GID="sovdModuleInstantiation">
                    <SDG GID="SovdModuleInstantiation">
                      <SDG-CAPTION>
                        <SHORT-NAME>SovdModuleInstantiation</SHORT-NAME>
                      </SDG-CAPTION>
                      <SDG GID="transportLayerSecurity">
                        <SDG GID="TransportLayerSecurity">
                          <SD GID="tlsMaximumNumberOfClients">12</SD>
                          <SD GID="tlsMaximumNumberOfServers">9</SD>
                          <SD GID="tlshandshakeMessageTimeout">2000</SD>
                          <SD GID="tlsMaximumTransmissionUnit">16384</SD>
                          <SD GID="trustzoneCryptoProviderUuid">12re76tf-h66y-76yy-89uj-
                              iu78o9u7jy65ju98</SD>
                          <SDG GID="secureComPropsRef">
                            <SDX-REF DEST="SECURE-COM-PROPS">/DEXT/SecureComPropsSet/
                                TlsSecureComProps</SDX-REF>
                          </SDG>
                        </SDG>
                      </SDG>
                    </SDG>
                  </SDG>
                </SDG>
              </SDGS>
            </ADMIN-DATA>
          </GENERIC-MODULE-INSTANTIATION>
        </MODULE-INSTANTIATIONS>
      </MACHINE>
    </ELEMENTS>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 6-49 Example of transport layer security configuration

based on template version 1.00.00

SOVD endpoint configuration - Communication connector

```
<MACHINE-DESIGN>
  <COMMUNICATION-CONNECTORS>
    <ETHERNET-COMMUNICATION-CONNECTOR>
      <SHORT-NAME>EthernetCommunicationConnector</SHORT-NAME>
      <NETWORK-ENDPOINT-REFS>
        <NETWORK-ENDPOINT-REF DEST="NETWORK-ENDPOINT">/DEXT/Ethernet/Channel/
            IdsRNetworkEndpoint</NETWORK-ENDPOINT-REF>
      </NETWORK-ENDPOINT-REFS>
    </ETHERNET-COMMUNICATION-CONNECTOR>
  </COMMUNICATION-CONNECTORS>
</MACHINE-DESIGN>
```

Listing 6-50 Extending SOFTWARE-CLUSTER with client authentication identifications

Configuration for `TlsSecureComProps`

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <ELEMENTS>
      <SECURE-COM-PROPS-SET>
        <SHORT-NAME>SecureComPropsSet</SHORT-NAME>
        <SECURE-COM-PROPSS>
          <TLS-SECURE-COM-PROPS>
            <SHORT-NAME>TlsSecureComProps</SHORT-NAME>
            <TLS-CIPHER-SUITES>
              <TLS-CRYPTO-CIPHER-SUITE>
                <SHORT-NAME>TLS_PSK_WITH_AES_128_GCM_SHA256</SHORT-NAME>
                <ADMIN-DATA>
                  <SDGS>
                    <SDG GID="DvMex:TlsCryptoCipherSuiteExtension">
                      <SD GID="cipherSuiteIdentifier">168</SD>
                    </SDG>
                  </SDGS>
                </ADMIN-DATA>
                <CERTIFICATE-REF DEST="CRYPTO-SERVICE-CERTIFICATE">/DEXT/
                    CryptoServiceCertificateLocal</CERTIFICATE-REF>
                <PRE-SHARED-KEY-REF DEST="CRYPTO-SERVICE-KEY">/DEXT/CryptoServiceKey</PRE-
                    SHARED-KEY-REF>
                <VERSION>TLS-13</VERSION>
              </TLS-CRYPTO-CIPHER-SUITE>
            </TLS-CIPHER-SUITES>
          </TLS-SECURE-COM-PROPS>
        </SECURE-COM-PROPSS>
      </SECURE-COM-PROPS-SET>
    </ELEMENTS>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 6-51 Example of `TlsSecureComProps` configuration

based on template version 1.00.00

Configuration for key/certificate

```
<AR-PACKAGES>
  <AR-PACKAGE>
    <SHORT-NAME>vector</SHORT-NAME>
    <ELEMENTS>
      <CRYPTO-SERVICE-CERTIFICATE>
        <SHORT-NAME>CryptoServiceCertificateRoot01</SHORT-NAME>
        <LONG-NAME>
          <L-4>./etc/certs/root01.cert</L-4>
        </LONG-NAME>
      </CRYPTO-SERVICE-CERTIFICATE>
      <CRYPTO-SERVICE-CERTIFICATE>
        <SHORT-NAME>CryptoServiceCertificateLocal</SHORT-NAME>
        <LONG-NAME>
          <L-4>./etc/certs/local.cert</L-4>
        </LONG-NAME>
        <NEXT-HIGHER-CERTIFICATE-REF DEST="CRYPTO-SERVICE-CERTIFICATE">/DEXT/
            CryptoServiceCertificateIntermediate</NEXT-HIGHER-CERTIFICATE-REF>
      </CRYPTO-SERVICE-CERTIFICATE>
      <CRYPTO-SERVICE-CERTIFICATE>
        <SHORT-NAME>CryptoServiceCertificateIntermediate</SHORT-NAME>
        <LONG-NAME>
          <L-4>./etc/certs/intermediate.cert</L-4>
        </LONG-NAME>
        <NEXT-HIGHER-CERTIFICATE-REF DEST="CRYPTO-SERVICE-CERTIFICATE">/DEXT/
            CryptoServiceCertificateRoot01</NEXT-HIGHER-CERTIFICATE-REF>
      </CRYPTO-SERVICE-CERTIFICATE>
      <CRYPTO-SERVICE-KEY>
        <SHORT-NAME>CryptoServiceKey</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:CryptoServiceKeyExtensions">
              <SD GID="keyUuid">4e3d33et-1jh3-3kkq-9bh3-wd6332ds6y7bkj3e</SD>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
      </CRYPTO-SERVICE-KEY>
      <TLS-IAM-REMOTE-SUBJECT>
        <SHORT-NAME>TlsIamRemoteSubject</SHORT-NAME>
        <ACCEPTED-REMOTE-CERTIFICATE-REFS>
          <ACCEPTED-REMOTE-CERTIFICATE-REF DEST="CRYPTO-SERVICE-CERTIFICATE">/DEXT/
              CryptoServiceCertificateRoot01</ACCEPTED-REMOTE-CERTIFICATE-REF>
        </ACCEPTED-REMOTE-CERTIFICATE-REFS>
        <IAM-RELEVANT-TLS-SECURE-COM-PROPS-REFS>
          <IAM-RELEVANT-TLS-SECURE-COM-PROPS-REF DEST="TLS-SECURE-COM-PROPS">/DEXT/
              SecureComPropsSet/TlsSecureComProps</IAM-RELEVANT-TLS-SECURE-COM-PROPS-REF>
        </IAM-RELEVANT-TLS-SECURE-COM-PROPS-REFS>
      </TLS-IAM-REMOTE-SUBJECT>
    </ELEMENTS>
  </AR-PACKAGE>
</AR-PACKAGES>
```

Listing 6-52 Example of key/certificate configuration

based on template version 1.00.00

SOVD endpoint configuration - Network endpoint

```
<ETHERNET-CLUSTER>
  <ETHERNET-CLUSTER-VARIANTS>
    <ETHERNET-CLUSTER-CONDITIONAL>
      <PHYSICAL-CHANNELS>
        <ETHERNET-PHYSICAL-CHANNEL>
          <NETWORK-ENDPOINTS>
            <NETWORK-ENDPOINT>
              <SHORT-NAME>SovdNetworkEndpoint</SHORT-NAME>
              <ADMIN-DATA>
                <SDGS>
                  <SDG GID="DvMex:NetworkEndpointExtensions">
                    <SDG GID="ethernetPort">
                      <SDG GID="EthernetPort">
                        <SDG GID="tcp">
                          <SDG GID="Tcp">
                            <SD GID="number">50</SD>
                            <SD GID="keepAliveTimeout">60000</SD>
                          </SDG>
                        </SDG>
                      </SDG>
                    </SDG>
                  </SDG>
                </SDGS>
              </ADMIN-DATA>
              <NETWORK-ENDPOINT-ADDRESSES>
                <IPV-4-CONFIGURATION>
                  <IPV-4-ADDRESS>192.168.154.132</IPV-4-ADDRESS>
                </IPV-4-CONFIGURATION>
              </NETWORK-ENDPOINT-ADDRESSES>
            </NETWORK-ENDPOINT>
          </NETWORK-ENDPOINTS>
        </ETHERNET-PHYSICAL-CHANNEL>
      </PHYSICAL-CHANNELS>
    </ETHERNET-CLUSTER-CONDITIONAL>
  </ETHERNET-CLUSTER-VARIANTS>
</ETHERNET-CLUSTER>
```

Listing 6-53 IDSR remote endpoint configuration - Network endpoint

6.26.3 SOVD server parameters

listing 6-54 illustrates the SOVD server parameters.

based on template version 1.00.00

Configuration for SOVD server parameters

```
<SOFTWARE-CLUSTER>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SDG GID="diagnostics">
          <SDG GID="Diagnostics">
            <SDG GID="sovdAddress">
              <SDG GID="SovdAddress">
                <SD GID="componentQualifier">sovdServerQualifier</SD>
                <SD GID="name">sovd Name</SD>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-54 Example of SOVD server parameters

6.26.4 SOVD resource collection

6.26.4.1 SOVD method

listing 6-55 illustrates the configuration of a method.

Configuration for SOVD method

```
<AR-PACKAGE>
  <SHORT-NAME>SOVD</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSovdArPackageExtensions">
        <SDG GID="diagnosticSovdMethod">
          <SDG GID="DiagnosticSovdMethod">
            <SDG-CAPTION>
              <SHORT-NAME>FaultMemoryAccess</SHORT-NAME>
            </SDG-CAPTION>
            <SDG GID="diagnosticSovdMethodPrimitive">
              <SDG GID="DiagnosticSovdMethodPrimitive">
                <SD GID="category">get</SD>
                <SDG GID="diagnosticAccessPermissionRef">
                  <SDX-REF DEST="DIAGNOSTIC-ACCESS-PERMISSION">/DEXT/Permissions/
                      Restrictions</SDX-REF>
                </SDG>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-55 Example of SOVD method

based on template version 1.00.00

6.26.4.2 SOVD generic method

listing 6-56 illustrates the configuration of a generic method.

Configuration for SOVD generic method

```
<AR-PACKAGE>
  <SHORT-NAME>SOVD</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSovdArPackageExtensions">
        <SDG GID="diagnosticSovdGenericMethod">
          <SDG GID="DiagnosticSovdGenericMethod">
            <SDG-CAPTION>
              <SHORT-NAME>sovdGenericMethod</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="path">app/data</SD>
            <SD GID="allowSubpaths">true</SD>
            <SD GID="contentType">json</SD>
            <SDG GID="diagnosticSovdMethodPrimitive">
              <SDG GID="DiagnosticSovdMethodPrimitive">
                <SD GID="category">get</SD>
                <SDG GID="diagnosticAccessPermissionRef">
                  <SDX-REF DEST="DIAGNOSTIC-ACCESS-PERMISSION">/DEXT/Permissions/
                      Restrictions</SDX-REF>
                </SDG>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-56 Example of SOVD generic method

listing 6-57, listing 6-58, and listing 6-59 illustrate the configuration of a generic method port instance.

based on template version 1.00.00

Configuration for SOVD generic method mapping

```
<AR-PACKAGE>
  <SHORT-NAME>SOVD</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSovdArPackageExtensions">
        <SDG GID="diagnosticSovdGenericMethodMapping">
          <SDG GID="DiagnosticSovdGenericMethodMapping">
            <SDG-CAPTION>
              <SHORT-NAME>SovdGenericMethodMapping</SHORT-NAME>
            </SDG-CAPTION>
            <SDG GID="diagnosticSovdGenericMethodRef">
              <SDX-REF DEST="SDG-CAPTION">/DEXT/sovdGenericMethod</SDX-REF>
            </SDG>
            <SDG GID="swcServiceDependencyRef">
              <SDX-REF DEST="SWC-SERVICE-DEPENDENCY">/DEXT/AdaptiveInternalBehavior/
                  SwcServiceDependencySovdGenericMethod</SDX-REF>
            </SDG>
            <SDG GID="processRef">
              <SDX-REF DEST="PROCESS-DESIGN">/DEXT/ProcessDesign</SDX-REF>
            </SDG>
            <SDG GID="rootSwComponentPrototypeRef">
              <SDX-REF DEST="ROOT-SW-COMPONENT-PROTOTYPE">/DEXT/Root</SDX-REF>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-57 Example of SOVD generic method mapping

Configuration for SOVD generic method SWC service dependency

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependencySovdGenericMethod</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="sovdPPortRef">
          <SDX-REF DEST="SDG-CAPTION">/DEXT/SovdGenericPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-58 Example of SOVD generic method SWC service dependency

based on template version 1.00.00

Configuration for SOVD generic method port

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
  <SHORT-NAME>sovd_integration_app_swc</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
        <SDG GID="sovdPPort">
          <SDG GID="SovdPPort">
            <SDG-CAPTION>
              <SHORT-NAME>SovdGenericPort</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="interfaceType">Generic</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-59 Example of SOVD generic method port

6.26.4.3 SOVD service instance

listing 6-60 illustrates the configuration of a service instance while listing 6-61 illustrates the configuration of a generic service instance

Note

`diagnosticSovdMethodRef` and `diagnosticSovdGenericMethodRef` are optional. Either one of them must be configured for a SOVD instance. It is not allowed to configure both of them in the same service instance.

based on template version 1.00.00

Configuration for SOVD service instance

```
<AR-PACKAGE>
  <SHORT-NAME>SOVD</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSovdArPackageExtensions">
        <SDG GID="diagnosticSovdServiceInstance">
          <SDG GID="DiagnosticSovdServiceInstance">
            <SDG-CAPTION>
              <SHORT-NAME>SovdFaultMemory</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="category">Faults</SD>
            <SDG GID="diagnosticSovdMethodRef">
              <SDX-REF DEST="SDG-CAPTION">/SOVD/FaultMemoryAccess</SDX-REF>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-60 Example of SOVD service instance

Configuration for SOVD generic service instance

```
<AR-PACKAGE>
  <SHORT-NAME>SOVD</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticSovdArPackageExtensions">
        <SDG GID="diagnosticSovdServiceInstance">
          <SDG GID="DiagnosticSovdServiceInstance">
            <SDG-CAPTION>
              <SHORT-NAME>SovdFaultMemory</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="category">Generic</SD>
            <SDG GID="diagnosticSovdGenericMethodRef">
              <SDX-REF DEST="SDG-CAPTION">/SOVD/SovdGenericMethod</SDX-REF>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-61 Example of SOVD generic service instance

Table 6-13 lists the supported resource collections.

6.26.4.4 SOVD authorization

The configuration of `SOVD` authorization is illustrated in 6-62.

based on template version 1.00.00

Resource Collection	Primitives	Method
Faults	Read, Delete	diagnosticSovdMethod
Generic	Get, Delete, Put, Post	diagnosticSovdGenericMethod

Table 6-13 Supported SOVD Resource Collections

Configuration for SOVD authorization

```
<SOFTWARE-CLUSTER>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SDG GID="diagnostics">
         <SDG GID="Diagnostics">
           <SDG GID="sovdAddress">
             <SDG GID="SovdAddress">
               <SDG GID="authorization">
                 <SDG GID="Authorization">
                   <SD GID="maxNumberOfValidTokens">10</SD>
                 </SDG>
               </SDG>
             </SDG>
           </SDG>
         </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-62 Example of SOVD authorization

6.26.4.5 SOVD authorization port

`ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE` is extended to hold PPort configuration for `SOVD` authorization as illustrated in 6-63.

based on template version 1.00.00

Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with PPort for SOVD authorization

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
        <SHORT-NAME>project</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
              <SDG GID="sovdPPort">
                <SDG GID="SovdPPort">
                  <SDG-CAPTION>
                    <SHORT-NAME>SovdAuthorizationPort</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="interfaceType">Authorization</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-63 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with PPort for SOVD authorization

In addition, `SWC-SERVICE-DEPENDENCY` is extended to hold sovdPPortRef configuration to reference `SOVD` authorization port as illustrated in 6-64.

SWC-SERVICE-DEPENDENCY with sovdPPortRef

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependency_SovdAuthorization</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="sovdPPortRef">
          <SDX-REF DEST="SDG-CAPTION">/vector/ARPackage/
              AdaptiveApplicationSwComponentType/SovdAuthorizationPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SERVICE-NEEDS>
    <DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
      <SHORT-NAME>SovdAuthorizationNeeds</SHORT-NAME>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="DvMex:DiagnosticCommunicationManagerNeedsExtensions">
            <SD GID="functionalityType">Authorization</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
    </DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
  </SERVICE-NEEDS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-64 Extending SWC-SERVICE-DEPENDENCY with sovdPPortRef for SOVD Authorization

based on template version 1.00.00

Caution

Only one SOVD authorization port is allowed.

Note

SOVD authorization is activated by configuring SOVD authorization port.

6.26.4.6 SOVD lock

The configuration of SOVD lock is illustrated in 6-65.

Configuration for SOVD lock

```
<SOFTWARE-CLUSTER>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SD GID="alwaysOnOperationCycle">false</SD>
        <SDG GID="diagnostics">
          <SDG GID="Diagnostics">
            <SDG GID="sovdAddress">
              <SDG GID="SovdAddress">
                <SDG GID="lock">
                  <SDG GID="Lock">
                    <SD GID="maxExpirationTime">3</SD>
                  </SDG>
                </SDG>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-65 Example of SOVD lock

Note

maxExpirationTime in seconds represents the maximum lock expiration time. This prameter is not allowed to have fraction of milliseconds

6.26.4.7 SOVD lock port

ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE is extended to hold RPort configuration for SOVD lock as illustrated in 6-66.

based on template version 1.00.00

Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPort for SOVD lock

```
<ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
        <SHORT-NAME>project</SHORT-NAME>
        <ADMIN-DATA>
          <SDGS>
            <SDG GID="DvMex:AraDiagAdaptiveApplicationSwComponentTypeExtension">
              <SDG GID="sovdRPort">
                <SDG GID="SovdRPort">
                  <SDG-CAPTION>
                    <SHORT-NAME>SovdLockPort</SHORT-NAME>
                  </SDG-CAPTION>
                  <SD GID="interfaceType">DiagnosticSovdLock</SD>
                </SDG>
              </SDG>
            </SDG>
          </SDGS>
        </ADMIN-DATA>
</ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE>
```

Listing 6-66 Extending ADAPTIVE-APPLICATION-SW-COMPONENT-TYPE with RPort for SOVD lock

In addition, `SWC-SERVICE-DEPENDENCY` is extended to hold sovdRPortRef configuration to reference `SOVD` lock port as illustrated in 6-67.

SWC-SERVICE-DEPENDENCY with sovdRPortRef

```
<SWC-SERVICE-DEPENDENCY>
  <SHORT-NAME>SwcServiceDependency_SovdLock</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SwcServiceDependencyExtensions">
        <SDG GID="sovdRPortRef">
          <SDX-REF DEST="SDG-CAPTION">/vector/ARPackage/
              AdaptiveApplicationSwComponentType/SovdLockPort</SDX-REF>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  <SERVICE-NEEDS>
    <DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
      <SHORT-NAME>SovdLockNeeds</SHORT-NAME>
      <ADMIN-DATA>
        <SDGS>
          <SDG GID="DvMex:DiagnosticCommunicationManagerNeedsExtensions">
            <SD GID="functionalityType">DiagnosticSovdLock</SD>
          </SDG>
        </SDGS>
      </ADMIN-DATA>
    </DIAGNOSTIC-COMMUNICATION-MANAGER-NEEDS>
  </SERVICE-NEEDS>
</SWC-SERVICE-DEPENDENCY>
```

Listing 6-67 Extending SWC-SERVICE-DEPENDENCY with sovdRPortRef for SOVD lock

based on template version 1.00.00

Note

Daemon can handle Multiple `SOVD` lock ports. However, each application ia allowed to configure only one lock port.

6.27 Data identifier configuration

In order to use Data identifiers in a diagnostic context with SOVD, some extra parameters must be configured within the data identifier. listing 6-68 illustrates those parameters. Data category can be one of four values, namely: `StoredData`, `SysInfo`, `CurrentData`, and `IdentData`.

Extened data identifier parameters

```
<DIAGNOSTIC-DATA-IDENTIFIER>
  <LONG-NAME>
    <L-4>ActiveDiagnosticSessionHolder HolderOfSessionId</L-4>
  </LONG-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticDataIdentifierExtensions">
        <SD GID="dataCategory">StoredData</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-DATA-IDENTIFIER>
```

Listing 6-68 Extened data identifier parameters

6.28 Long Name Configuration

listing 6-69 illustrates long name configuration. Only one L-4 label and within that label, only one description in this label shall be used as a long name.

LongNameConfiguration

```
  <LONG-NAME>
    <L-4>Long name</L-4>
  </LONG-NAME>
```

Listing 6-69 Long name configuration

6.29 Steady Mode Configuration

Non-deterministic timing of interactions with the operating system for on-demand memory allocations and deallocations might be problematic at runtime. To avoid (de)allocations in the system memory by diagnostic cluster functionalities to the extent feasible, diagnostic memory resources have to be configured. The supported AUTOSAR model does not contain a configuration for memory resources, which is why MEX parameters are used. An extension is configured to describe the memory resources of the respective executable.

The Steady Mode configuration of the diagnostic cluster is optional, meaning that the Default Memory Resource will be used if a diagnostic memory resource is not configured.

Diagnostic Manager memory resource support

The Diagnostics Daemon uses the Default Memory Resource. Therefore, no configuration of diagnostic memory resources is required for the Diagnostics Daemon.

6.29.1 Memory Resource Properties Configuration

A memory resource references exactly one `MemoryResourceProps` to specify the associated properties. A `MemoryResourceProps` can be referenced by multiple memory resources. A `MemoryResourceProps` supports two parameters: the size of the reserved memory in bytes and the property to expand the memory if the reserved size is exceeded.

Reserved memory is exhausted

The program aborts if the reserved memory size is exceeded, unless the resource is configured to acquire additional memory and the request can be served.

Memory Resource Properties configuration

```
<EXECUTABLE>
  <SHORT-NAME>amsr_diag_example_app</SHORT-NAME>
  ...
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:MemoryResourceManagementModelExecutableExtensions">
        <SDG GID="memoryResourceProps">
          <SDG GID="MemoryResourceProps">
            <SDG-CAPTION>
              <SHORT-NAME>MemoryResourceProps</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="size">1024</SD>
            <SD GID="expanding">true</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 6-70 Memory Resource Props configuration

6.29.2 Diagnostic Application Steady Mode Configuration

A memory resource is required for the memory needs of the Diagnostics Library (ara::diag). Additionally, a resource is needed for the communication of an application with the Diagnostics Daemon (IPC). These two resources are used exclusively for diagnostic functionalities and are therefore grouped, as shown in the following figure Figure 6-71.

Memory Resource configuration

```
<EXECUTABLE>
  <SHORT-NAME>amsr_diag_example_app</SHORT-NAME>
  ...
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:MemoryResourceManagementModelExecutableExtensions">
        <SDG GID="memoryResourceManagement">
          <SDG GID="MemoryResourceManagement">
            <SDG-CAPTION>
              <SHORT-NAME>Memory resource management</SHORT-NAME>
            </SDG-CAPTION>
            <SDG GID="memoryResourceProps">
              <SDG GID="MemoryResourceProps">
                <SDG-CAPTION>
                  <SHORT-NAME>MemoryResourceProps</SHORT-NAME>
                </SDG-CAPTION>
                <SD GID="size">2048</SD>
                <SD GID="expanding">false</SD>
              </SDG>
            </SDG>
            <SDG GID="diagnostics">
              <SDG GID="Diagnostics">
                <SDG GID="araDiag">
                  <SDG GID="AraDiag">
                    <SDG GID="resource">
                      <SDX-REF DEST="SDG-CAPTION">/example/executable/amsr_diag_example_app/
                          MemoryResourceProps</SDX-REF>
                    </SDG>
                  </SDG>
                </SDG>
                <SDG GID="diagCommunication">
                  <SDG GID="DiagCommunication">
                    <SDG GID="resource">
                      <SDX-REF DEST="SDG-CAPTION">/example/executable/amsr_diag_example_app/
                          MemoryResourceProps</SDX-REF>
                    </SDG>
                  </SDG>
                </SDG>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
  ...
</EXECUTABLE>
```

Listing 6-71 Memory Resource configuration

6.29.3 Diagnostics Daemon Steady Mode Configuration

The Diagnostics Daemon uses the Default Memory Resource. Please refer to [15] for further documentation.

based on template version 1.00.00

6.30 UDS NRC Mapping Configuration

6.30.1 UDS NRC Mapping

Each instance of the `UdsNrcMapping` class specifies a mapping that defines one source NRC and one target NRC. See 2.1.5.15 for more details.

6.30.2 UDS NRC Mapping Table

Each instance of the `udsNrcMappingTable` class contains at least one `UdsNrcMapping` instance. Only one mapping table is allowed per extension. See 2.1.5.15 for more details.

UDS NRC Mapping Configuration of two mappings

```
<SOFTWARE-CLUSTER>
  <SHORT-NAME>Name of the SoftwareCluster</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:SoftwareClusterExtensions">
        <SDG GID="diagnostics">
          <SDG GID="Diagnostics">
            <SDG GID="udsNrcMappingTable">
              <SDG GID="UdsNrcMapping">
                <SD GID="sourceNrc">148</SD>
                <SD GID="targetNrc">16</SD>
              </SDG>
              <SDG GID="UdsNrcMapping">
                <SD GID="sourceNrc">130</SD>
                <SD GID="targetNrc">97</SD>
              </SDG>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</SOFTWARE-CLUSTER>
```

Listing 6-72 UDS NRC Mapping Configuration

6.31 TSSR Configuration

6.31.1 Profile

TSSR profiles can be configured as illustrated in 6-73.

based on template version 1.00.00

TSSR profile configuration

```
<AR-PACKAGE>
  <SHORT-NAME>tssrSamplingProfiles</SHORT-NAME>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:TssrArPackageExtensions">
        <SDG GID="tssrSamplingProfile">
          <SDG GID="TssrSamplingProfile">
            <SDG-CAPTION>
              <SHORT-NAME>tssrSamplingProfile</SHORT-NAME>
            </SDG-CAPTION>
            <SD GID="numberOfSamplesBeforeTrigger">2</SD>
            <SD GID="numberOfSamplesAfterTrigger">3</SD>
            <SD GID="samplingInterval">2.31</SD>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</AR-PACKAGE>
```

Listing 6-73 TSSR profile configuration

Note

- `numberOfSamplesBeforeTrigger` represents the number collected snapshots before the trigger
- `numberOfSamplesAfterTrigger` represents the number of collected snapshots after the trigger
- `samplingInterval` in seconds, represents the time interval between two samples. Time intervals should not contain fraction of millisecond.

6.31.2 DTC props configuration

The configuration of `DiagnosticTroulbeCodeProps` is extended to include TSSR as illustrated in 6-74.

based on template version 1.00.00

TSSR configuration of DTC

```
<DIAGNOSTIC-TROUBLE-CODE-PROPS>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticTroubleCodePropsExtensions">
        <SDG GID="tssr">
          <SDG GID="Tssr">
            <SDG GID="setProps">
              <SDG GID="SetProps">
                <SD GID="startRecordNumber">128</SD>
                <SDG GID="diagnosticFreezeFrameRef">
                  <SDX-REF DEST="DIAGNOSTIC-FREEZE-FRAME">/Diagnostics/
                      diagnosticFreezeFrames/freezeFrame</SDX-REF>
                </SDG>
              </SDG>
            </SDG>
            <SDG GID="tssrSamplingProfileRef">
              <SDX-REF DEST="SDG-CAPTION">/Diagnostics/tssrSamplingProfiles/
                  tssrSamplingProfile</SDX-REF>
            </SDG>
          </SDG>
        </SDG>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-TROUBLE-CODE-PROPS>
```

Listing 6-74 TSSR configuration of DTC

Note

- Number of `SetProps` must be less than or equal to the number of trigger snapshots. The number of trigger snapshots is equal to `MaxNumberFreezeFrameRecords` in case of calculated snapshots and is equal to number of referenced `FreezeFrames` in case of configured snapshots.
- `diagnosticFreezeFrameRef` is optional. The parameter is required only for configured snapshots. However, it is not needed for calculated snapshots.
- `tssrSamplingProfileRef` is mandatory in all cases.
- At least a configuration of one `SetProps` is required.
- Each `SetProps` represents a set of snapshot samples where the corresponding record numbers range from `startRecordNumber` to `startRecordNumber + numberOfSamplesBeforeTrigger + numberOfSamplesAfterTrigger - 1`. In DTC, snapshot record numbers are unique. Therefore, The ranges of snapshot record numbers of `SetPropss` shall not intersect. In addition, the record number of triggered snapshots (Whether calculated or configured) shall not lay in any of the snapshot record number ranges of `SetPropss`.
- `update` and `trigger` of triggered snapshots are applied for the corresponding TSSR samples as well
- Triggered snapshots are prerequisite for TSSR. I.e, if DTC does not configure a triggered snapshot, TSSR is not generated for this DTC

based on template version 1.00.00

6.32 Event memory configuration

`DiagnosticMemoryDestinationPrimary` and `DiagnosticMemoryDestinationUserDefined` are extended to configure the snapshot update overflow strategy of the related DTCs as illustrated in listing 6-75 and as illustrated in listing 6-76, respectively.

The available snapshot overlfow update strategies are `UPDATE_OLDEST` and `UPDATE_RECENT`.

Configuration of snapshot overflow update strategy in primary memory

```
<DIAGNOSTIC-MEMORY-DESTINATION-PRIMARY>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticMemoryDestinationPrimaryExtensions">
        <SD GID="snapshotOverflowUpdateStrategy">UPDATE_RECENT</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-MEMORY-DESTINATION-PRIMARY>
```

Listing 6-75 Configuration of snapshot overflow update strategy in primary memory

Configuration of snapshot overflow update strategy in user defined memory

```
<DIAGNOSTIC-MEMORY-DESTINATION-USER-DEFINED>
  <ADMIN-DATA>
    <SDGS>
      <SDG GID="DvMex:DiagnosticMemoryDestinationUserDefinedExtensions">
        <SD GID="snapshotOverflowUpdateStrategy">UPDATE_RECENT</SD>
      </SDG>
    </SDGS>
  </ADMIN-DATA>
</DIAGNOSTIC-MEMORY-DESTINATION-USER-DEFINED>
```

Listing 6-76 Configuration of snapshot overflow update strategy in user defined memory

based on template version 1.00.00

7 Cybersecurity

Caution
This chapter is under construction and written in parts. It does not yet cover Cybersecurity topics for the entirety of Diagnostic Manager.

This chapter describes relevant information for a secure integration and configuration, so-called Cybersecurity Manual Items (CMIs), of this component to fulfill the identified Technical Cybersecurity Requirements (TCRs). Additionally, functional security dependencies to other components are described.

7.1 DoIP Transport Protocol

This section describes cybersecurity-relevant information specifically relating to the DoIP Transport Protocol component.

The following TCRs are allocated to this component:

> TCR-MSRA-SecCom

7.1.1 Integration

No cybersecurity-relevant aspects for integration have been identified.

7.1.2 Configuration

7.1.2.1 CMI-DoIP-TlsConfiguration

Technical Cybersecurity Requirements: TCR-MSRA-SecCom

The user of MICROSAR shall configure DoIP (see 2.1.1.3.8) according to the user's cybersecurity concept.

Rationale:
The level of security provided by TLS and whether TLS is required depends on how DoIP is configured.

7.1.2.2 CMI-DoIP-UseTcpIfCertificatesAreUnavailable

Technical Cybersecurity Requirements: TCR-MSRA-SecCom

The user of MICROSAR shall only enable `UseTcpIfCertificatesAreUnavailable` according to the user's cybersecurity concept.

Rationale:
When the `UseTcpIfCertificatesAreUnavailable` option is enabled, TLS requirements will be ignored under specific circumstances.

7.1.3 Runtime Interfaces: Application

No cybersecurity-relevant aspects for runtime interfaces to application software have been identified.

7.1.4 Runtime Interfaces: BSW

Table 7-1 shows which TCRs allocated to this component depend on the fulfillment of TCRs provided by other components.

Allocated TCR	Required Component	Required TCR	Comment
TCR-MSRA-SecCom	SecCom	TCR-MSRA-SecCom	Provides the implementation of the TLS protocol

Table 7-1 Cybersecurity-relevant Dependencies for Runtime Interfaces of DoIP

based on template version 1.00.00

8 Glossary and Abbreviations

8.1 Glossary

Term	Description
MICROSAR Adaptive	The Vector Adaptive AUTOSAR solution.
AUTOSAR Adaptive Platform	The AUTOSAR Adaptive Platform is the standardized platform for microprocessor- based ECUs supporting use cases like highly automated driving as well as high speed on-board and off-board communication [2].
Basic Communication Protocol	AUTOSAR Adaptive Platform Library for basic inter-process communication
Crypostack daemon	Background process for handling ara::crypto functions
Diagnostic Client	Entity that acts as a client requesting diagnostic functionality from DM. There is no distinction between vehicle internal or external Diagnostic Clients.
Diagnostic Conversation	Abstract term for a one-to-one communication between a Diagnostic Client and a Diagnostic Server.
Diagnostic Manager	AUTOSAR Adaptive Platform system for handling diagnostic communication and fault memory
Diagnostic Server	Transport layer independent subcomponent of the Diagnostics Daemon responsible for handling UDS protocol and fault management.
Diagnostics Daemon	Daemon process providing the UDS protocol handling, and a fault manager conforming to UDS.
Diagnostics Library (ara::diag)	Library that abstracts the interaction between an Adaptive Application and the Diagnostics Daemon. Implements the standardized AUTOSAR API towards an Adaptive Application.
Execution Manager (ara::em)	AUTOSAR Adaptive Platform Execution manager
ExecutionMode	DM execution mode configuration
Inter-Process Communication	Means to exchange data between separate processes on the same machine.
Intrusion Detection System Manager Library (ara::idsm)	Library that abstracts the interaction between an Adaptive Application and the Diagnostics Daemon. Implements the standardized AUTOSAR API towards an Adaptive Application. (see [9] and [23])
LibVAC	Vector base type library
LibVaJson	Vector library for parsing JSON files
Logging Library (ara::log)	AUTOSAR Adaptive Platform library for logging and tracing
Microcontroller Open System Architecture	Microcontroller Open System Architecture (the Vector AUTOSAR solution).
OS Abstraction	Vector platform abstraction library
PDU Header TP	Diagnostics Daemon extension to provide communication using the AUTOSAR PduId Header TP (for details see [27])

based on template version 1.00.00

Persistency Library (ara::per)	AUTOSAR Adaptive Platform library for data persistence
Remote Procedure Call	Medium to exchange data between two separate processes.
Software Cluster	Set of vehicle diagnostics provided by one Diagnostic Server. A Software Cluster is identified by a unique TA.
SOME/IP POSIX daemon	Background process for handling SOME/IP protocol messages on POSIX system

8.2 Abbreviations

Abbreviation	Description
ALFID	Address and Length Format Identifier
AMSR	MICROSAR Adaptive
API	Application Programming Interface
ARA	AUTOSAR Runtime for Adaptive Applications
ara::per	Adaptive AUTOSAR Persistency
ARXML	AUTOSAR XML
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software
CMI	Cybersecurity Manual Item
DDDID	Dynamically Defined Data Identifier
Dem	Diagnostic Event Manager
DEXT	Diagnostic Extract
DFI	Data Format Identifier
DHCP	Dynamic host control protocol
DID	Data Identifier
DM	Diagnostic Manager
DoIP	Diagnostics over IP
DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
EDR	Extended Data Record
GID	Group Identification
Idsm	Intrusion detection system manager
Idsr	Intrusion detection system reporter
IP	Internet Protocol
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
MEX	Model Extension
MICROSAR	Microcontroller Open System Architecture

MOOP	Mode Of Operation
NRC	Negative Response Code
PDID	Periodic Data Identifier
REST	Representational State Transfer
RID	Routine Identifier
RoE	Response on Event
RPC	Remote Procedure Call
SDT	UDS Service SecuredDataTransmission (SID 0x84)
SID	UDS Service Identifier
SOVD	Service-oriented vehicle diagnostics
SPRMIB	Suppress Positive Response Message Indication Bit
STRT	Service to respond to
SWC	Software component
SWCL	Software Cluster
SWS	Software Specification
TA	Target Address
TCP	Transmission Control Protocol
TCR	Technical Cybersecurity Requirement
TLS	Transport layer security
TP	Transport Protocol
TSSR	Time series snapshot records
TTL	Time To Live
UDP	User Datagram Protocol
UDS	Unified Diagnostics Services
URI	Uniform Resource Identifier
VIN	Vehicle Identification Number

based on template version 1.00.00

9 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

based on template version 1.00.00